

COMPUTER SYSTEMS

A Programmer's Perspective

深入理解计算机系统



主讲人：张延松

中国人民大学 信息学院

zhangys_ruc@hotmail.com





第6章 存储器层次结构

教学目标：

- ◆ 了解现代计算主流存储技术及其基本工作原理
- ◆ 掌握数据局部性原理及局部性优化技术
- ◆ 理解现代计算机存储器的层次结构
- ◆ 了解cache的结构和工作原理
- ◆ 编写cache友好的代码
- ◆ 了解cache对程序性能的影响及如何提高程序的cache局部性

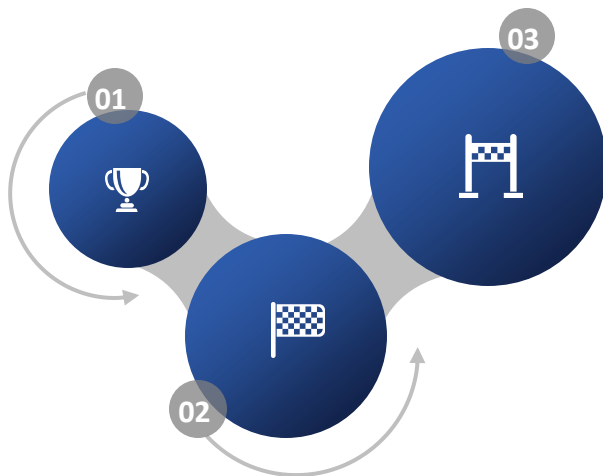
本章学习要求

深入了解存储硬件

掌握存储器硬件结构与工作原理，了解当前硬件发展趋势。

实践

通过实验和测试深入掌握存储器硬件特性能程序性能的影响因素，学习hardware-conscious的程序设计方法



扩展

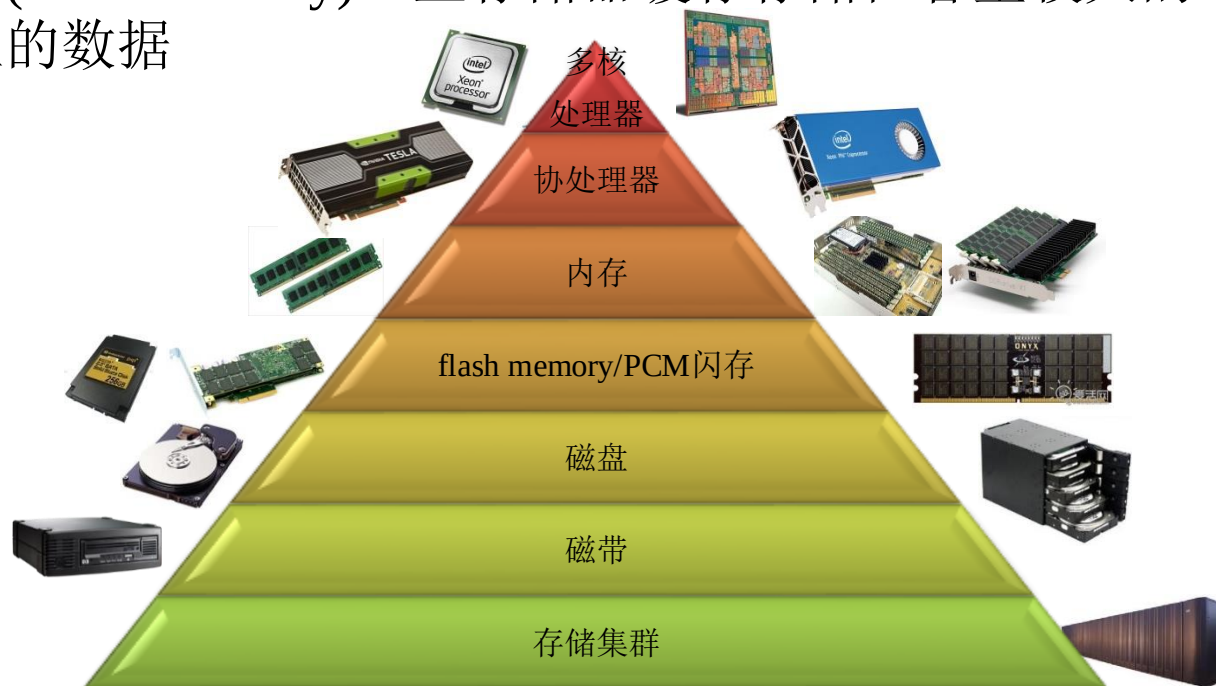
通过扩展论文研读与分析，理解当前数据库hardware-conscious优化技术的原理和基本方法

目标：知其然，知其所以然

引言

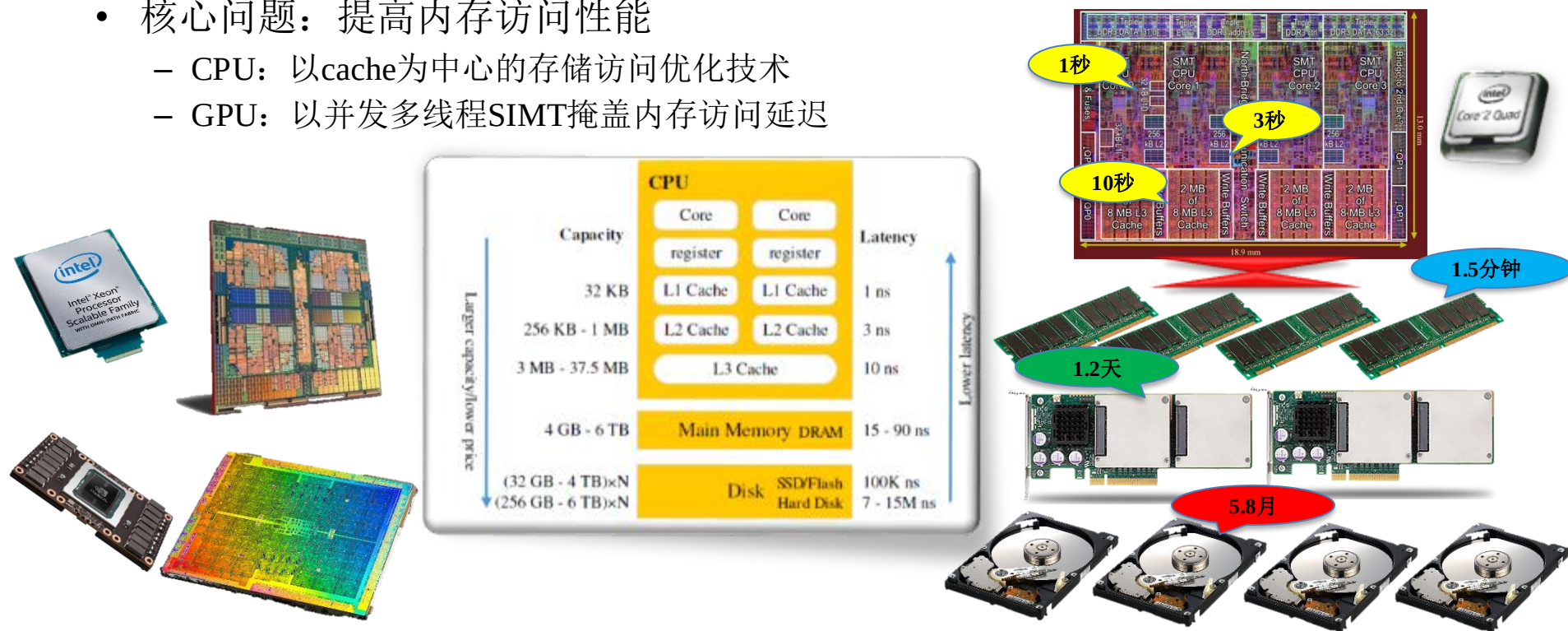
- 基本概念:

- 存储器系统(memory system): 具有不同容量、成本和访问时间的层次结构
- 高速缓存存储器(cache memory): 靠近CPU的小的、快速的高速缓存存储器作为一部分存储在相对慢速的主存储器中数据和指令的缓冲区
- 主存储器(main memory): 主存储器缓存存储在容量较大的、慢速磁盘上的数据



引言

- 存储墙(memory wall)成为处理器性能的重要瓶颈, 提高存储访问性能的主要技术是缓存优化
- 局部性(locality):
 - 良好局部性的程序倾向于一次又一次地访问相同的数据项集合, 或倾向于访问邻近的数据项集合
 - 具有良好局部性的程序比局部性差的程序更多地倾向于从存储器层次结构中较高层次处访问数据项, 因此运行更快
- 核心问题: 提高内存访问性能
 - CPU: 以cache为中心的存储访问优化技术
 - GPU: 以并发多线程SIMT掩盖内存访问延迟



存储技术的发展



Built in Serviceability

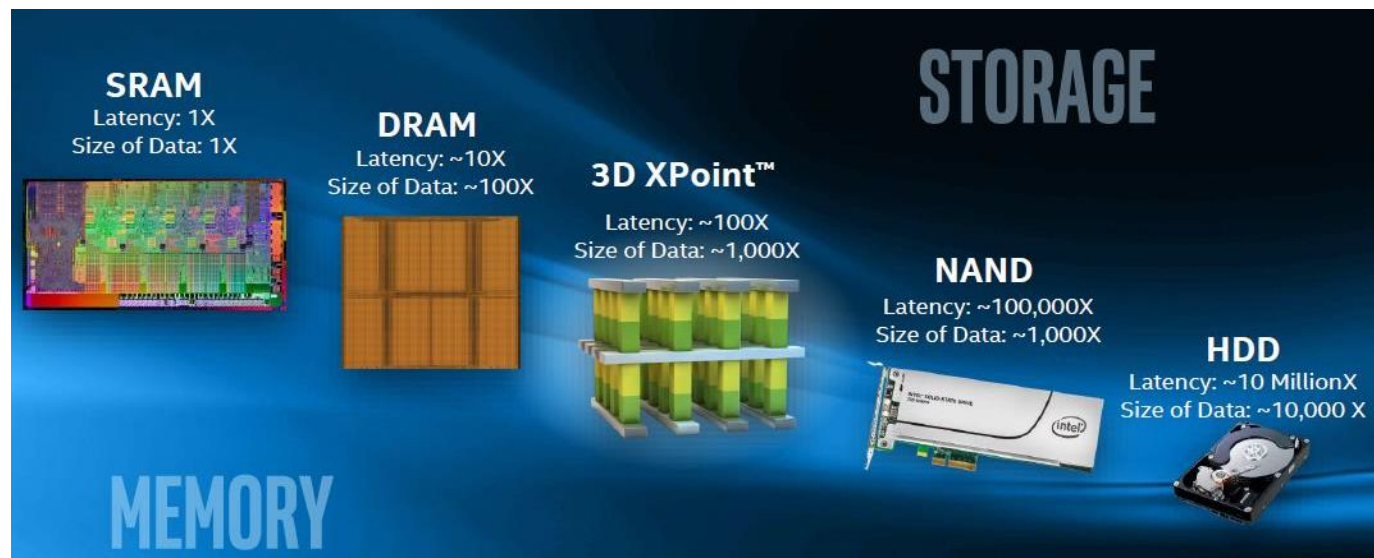
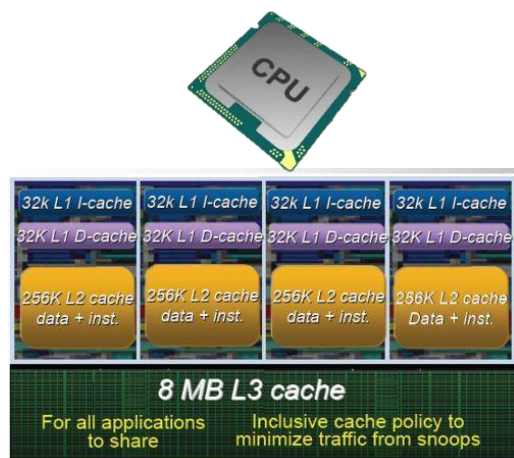
Programmable LEDs to quickly locate failed drives, offline drives, and unpopulated slots

Carrier-less design with integrated pull tab removes need for drive carriers

- 各级存储性能不断提升，但各级存储访问延迟性能差距仍然较大
- Memory is the new disk(相对于CPU仍然较慢)

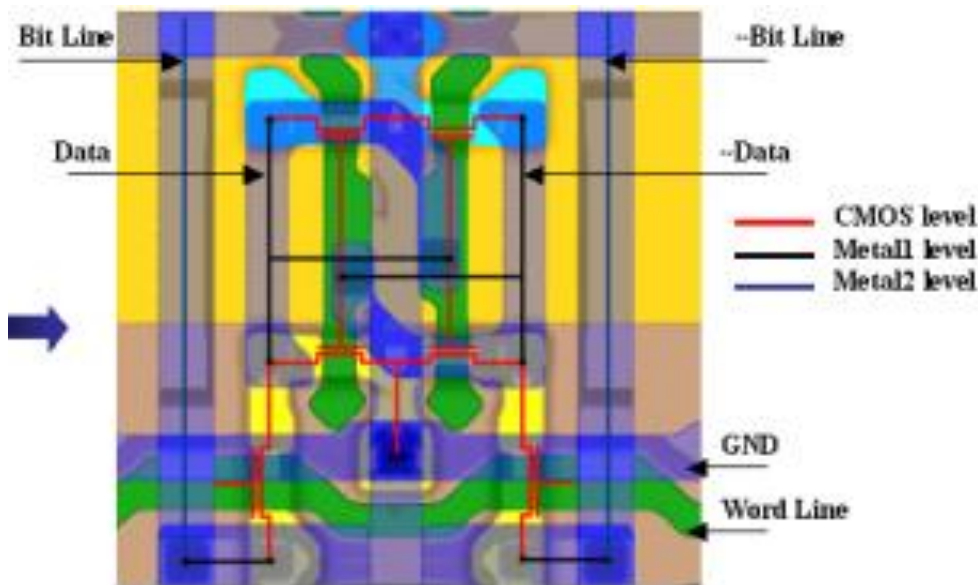
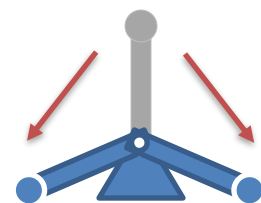
6.1.1 随机访问存储器

- 随机访问存储器(Random-Access Memory, RAM):
 - 静态RAM(SRAM): 更快, 更贵, 主要用作高速缓存存储器, 可以在CPU芯片上, 也可以在片下, 容量从几MB到几十MB
 - 动态RAM(DRAM): 用作主存及外部存储设备缓存, 相对SRAM较慢, 价值相对更低, 容量达到GB甚至TB级
 - **非易失内存NVDIMM**: 以3D Xpoint DIMM为代表, 当前单条最大容量512GB, 较DRAM慢, 但存储密度高, 读写不对称



1. 静态RAM

- 工作原理：
 - SRAM将每个位存储在一个双稳态的(bistable)存储单元里，每个单元用一个六晶体管电路实现
 - 特性：可以无限期保持在两个不同的电压配置(configuration)或状态(state)之一，其他任何状态都是不稳定的，从不稳定状态开始，电路会迅速地转移到两个稳定状态中的一个
- 优点：
 - 只要有电，可以永远保持它的值
 - 外界干扰扰乱电压情况下，干扰消除时电路就会恢复到稳定值
 - 不需要刷新电路
 - 速度快
- 缺点：
 - 价格贵
 - 晶体管数量多，集成度低
 - 耗电高
- 主要用途：CPU各级cache



2. 动态RAM

- 工作原理

- DRAM将每个位存储为对一个电容的充电
- 每个单元由一个电容和一个访问晶体管组成
- 特性：DRAM存储器单元对干扰非常敏感，当电容电压被扰乱永远不会恢复
- 需要刷新：周期性读出，重写刷新内存每一位

- 优点：

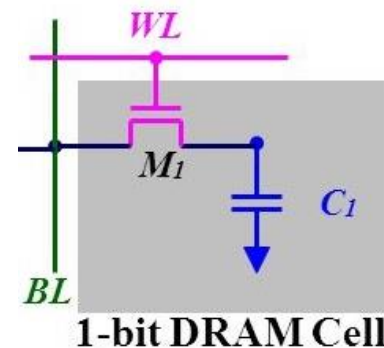
- 需要晶体管数量少
- 结构简单
- 存储密度大
- 价格较SRAM低

- 缺点：

- 速度慢
- 需要刷新

- 用途：

- 用作主存储main memory



1



0



SRAM vs. DRAM

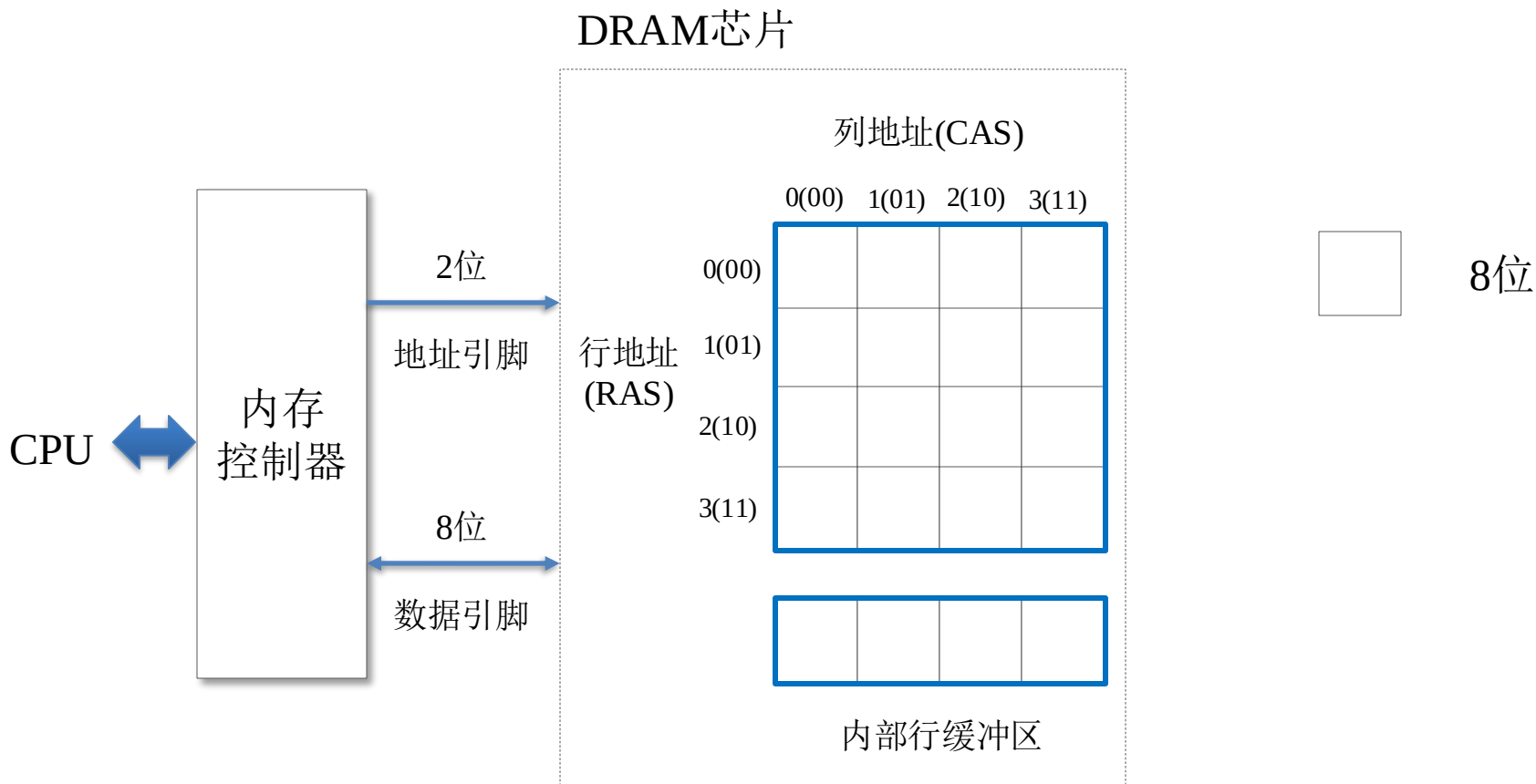
- SRAM与DRAM的主要区别

	每位晶体管数量	相对访问时间	持续的？	敏感的？	相对成本	应用
SRAM	6	1X	是	否	1000x	高速缓存存储器
DRAM	1	10X	否	是	1x	主存，帧缓冲区

- SRAM与DRAM共同的缺点：
 - 易失性存储，断电后数据丢失
- 用途：
 - 各级缓存：
 - SRAM(cache)↔DRAM(main memory)
 - L1 cache ↔ L2 cache
 - L2 cache ↔ L3 cache
 - L3 cache ↔ DRAM(main memory)
 - DRAM(main memory)↔Hard Disk

3 传统的DRAM

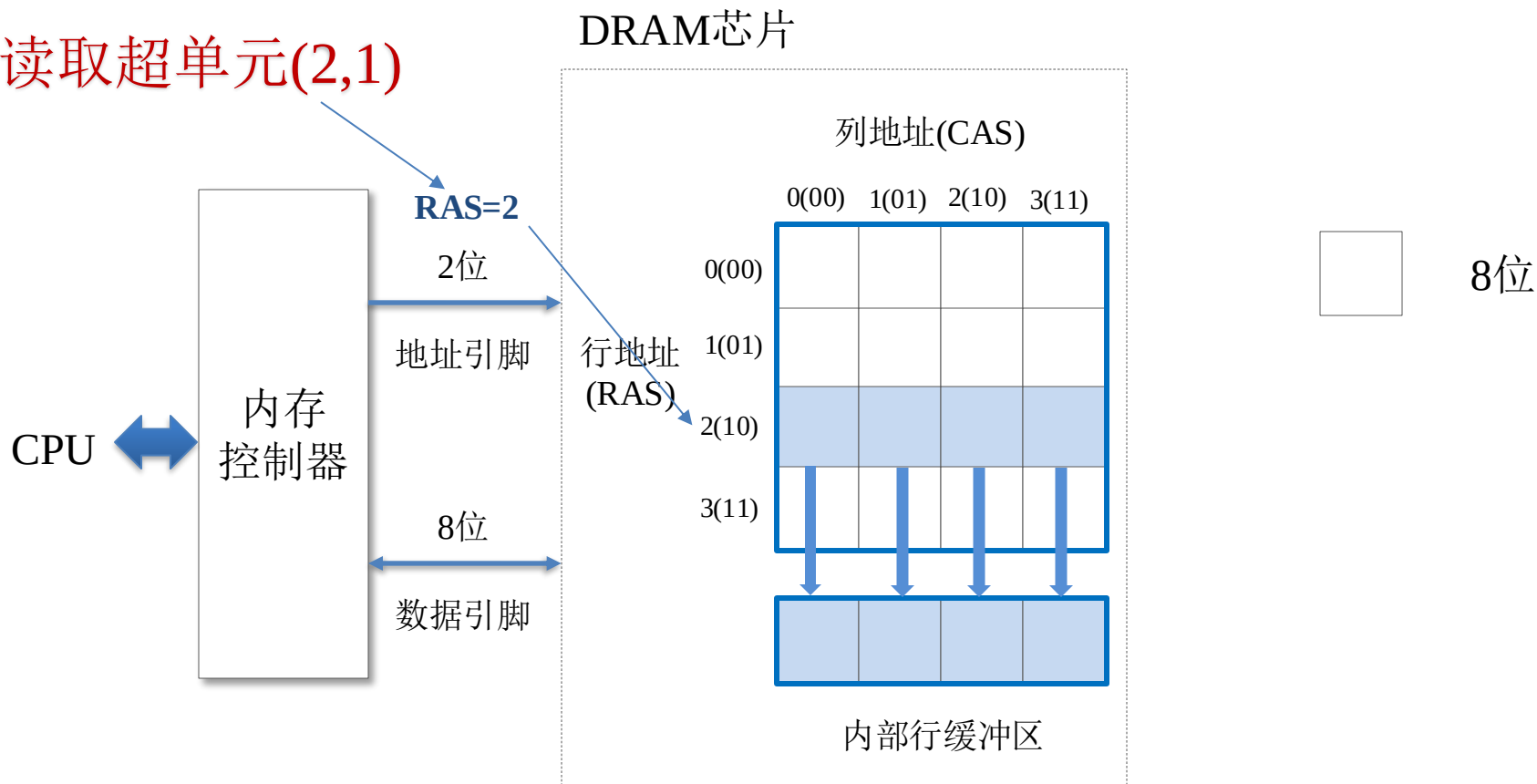
- 128位 16×8 的DRAM芯片视图
 - 芯片由 r 行 $\times c$ 列= d 个超单元（supercell）组成
 - 超单元由 w 位组成，芯片共有 $d \times w$ 位存储信息
 - 示例：4行、4列、8位存储单元总存储容量为： $4 \times 4 \times 8 = 128$ 位



3 传统的DRAM

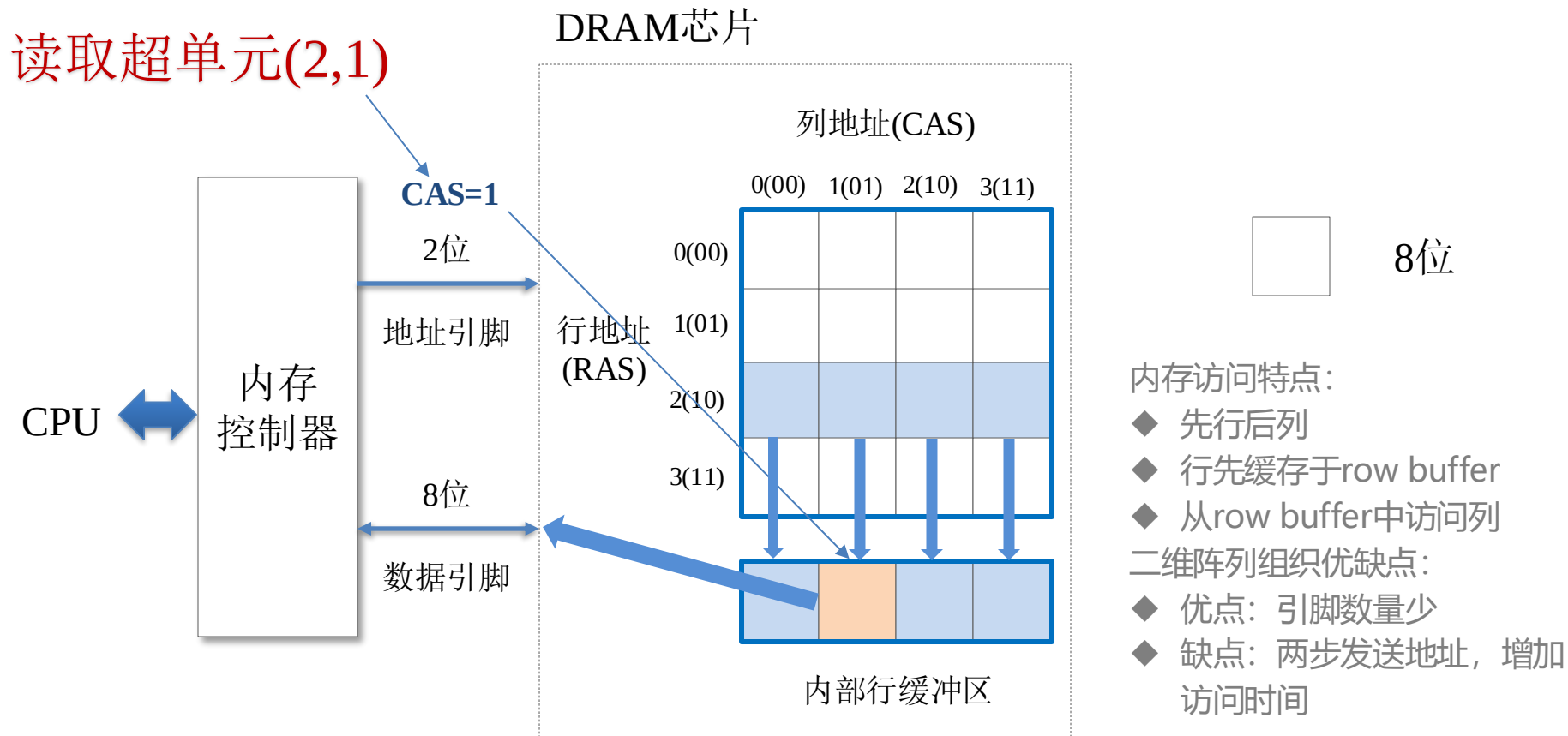
- 128位 16×8 的DRAM芯片视图
 - 芯片由 r 行 $\times c$ 列= d 个超单元（supercell）组成
 - 超单元由 w 位组成，芯片共有 $d \times w$ 位存储信息
 - 示例：4行、4列、8位存储单元总存储容量为： $4 \times 4 \times 8 = 128$ 位

读取超单元(2,1)

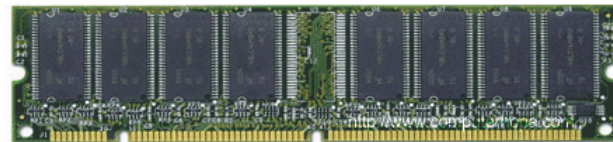


3 传统的DRAM

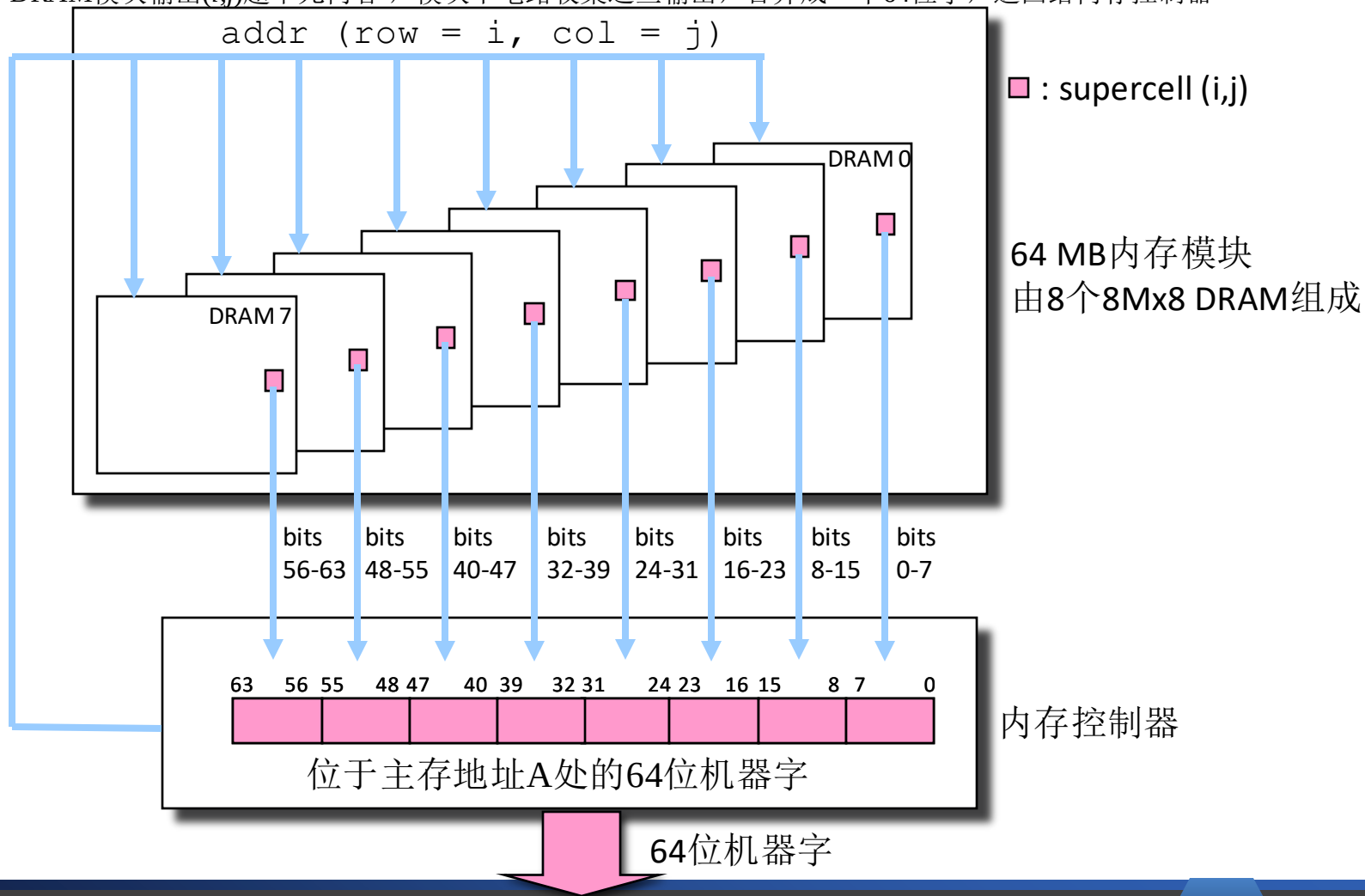
- 128位 16×8 的DRAM芯片视图
 - 芯片由 r 行 $\times c$ 列= d 个超单元（supercell）组成
 - 超单元由 w 位组成，芯片共有 $d \times w$ 位存储信息
 - 示例：4行、4列、8位存储单元总存储容量为： $4 \times 4 \times 8 = 128$ 位



4. 内存模块

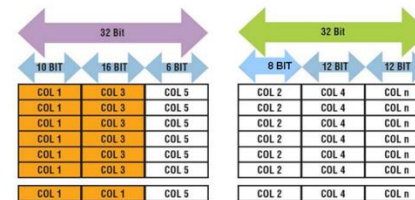


- DIMM: 双列直插内存模块(Dual Inline Memory Module, DIMM), 以64位为块传送数据到内存控制器和从内存控制器传出数据
- 访问地址A处机器字: 内存控制器将地址A转换为超单元地址(i, j), 发送到内存模块, 内存模块将(i, j)广播到每个DRAM模块, 每个DRAM模块输出(i, j)超单元内容, 模块中电路收集这些输出, 合并成一个64位字, 返回给内存控制器



扩展知识

- Instead, Blink vertically partitions the bit-aligned, encoded columns of each horizontal partition into a family of fixed-size, byte-aligned **banks** of 8, 16, 32, or 64 bits, to allow efficient ALU operations.



The Informix Warehouse Accelerator stores data in **columns groups**, or vertical partitions of the table, called “banks”. The fraction of a row (or tuple) that fits in each bank is called a “tuplelet”. The assignment of columns to banks is cell-specific, because the column lengths vary from cell to cell.

IBM Information Management software

IBM Informix Warehouse Accelerator Performance is everything

<http://www.iug.org/library/warehouse/technical/InformixWarehouseAcceleratorPaper.pdf>

March 2011

Technical White Paper

by Keshava Murthy

Senior Technical Staff Member

IBM Informix Development

Business Analytics in (a) Blink

Ronald Barber[†] Peter Bendel[‡] Marco Czech^{••} Oliver Draese[‡] Frederick Ho[#] Namik Hrle[‡]
Stratos Idreos^{§*} Min-Soo Kim^{◊*} Oliver Koeth[‡] Jae-Gil Lee^{◊*} Tianchao Tim Li[‡] Guy Lohman[†]
Konstantinos Morfonios^{△*} Rene Mueller[‡] Keshava Murthy[#] Ippokratis Pandis[‡] Lin Qiao^{◊*}
Vijayshankar Raman[‡] Richard Sidle[‡] Knut Stolze[‡] Sandor Szabo[‡]

[†]IBM Almaden Research Center [‡]IBM Germany [•]SIX Group, Ltd. [#]IBM

[§]CWI Amsterdam [◊]DGIST, Korea [◊]KAIST, Korea [△]Oracle [◊]LinkedIn

<http://sites.computer.org/debull/A12mar/blink.pdf>

练习题6.1

- r 表示一个DRAM阵列中的行数， c 表示列数， b_r 表示行寻址需要的位数， b_c 表示列寻址需要的位数。对于下面每个DRAM，确定2的幂数的阵列维数，使得 $\max(b_r, b_c)$ 最小， $\max(b_r, b_c)$ 是对阵列的行寻址或列寻址所需的位数中较大的值
- 目标：使纵横比最小，使地址位置最小，数组越接近正方形，地址位数越少

组织	r	c	b_r	b_c	$Max(b_r, b_c)$
16×1	4	4	2	2	2
16×4	4	4	2	2	2
128×8	16	8	4	3	4
512×4	32	16	5	4	5
1024×4	32	32	5	5	5

5. 增强的DRAM

• DRAM存储器的主要类型和发展:

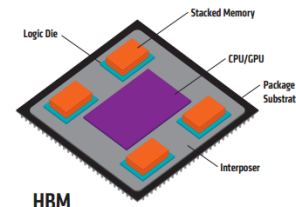
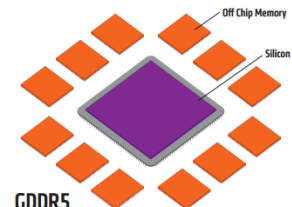
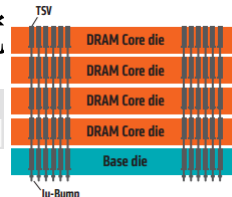
- 快页模式DRAM, FPM DRAM: 对同一行连续访问可以直接从行缓冲区得到服务
- 扩展数据输出DRAM, EDO DRAM: FPM DRAM的增加形式, 允许各CAS信号在时间上靠得更紧密
- 同步DRAM, SDRAM: 与内存控制器通信采用同步信号
- 双倍数据速率同步DRAM, DDR SDRAM: 使用两个时钟沿作为控制信号, 使DRAM速度翻倍
- 视频RAM, VRAM: 用于图形系统帧缓冲区, 允许并行内存读和写

• 当前主流内存:

- DDR4: 128GB DIMM
- GDDR5/6(Graphics Double Data Rate, version 5/6): 高性能显卡使用的同步动态随机存取存储器, 专为高带宽需求计算机应用所设计, 最高带宽768 GB/s
- High-Bandwidth Memory (HBM2): 采用垂直堆叠方式和高速信息传输, 实现高带宽、小尺寸
TESLA V100达到900 GB/秒最大带宽



型号	计算单元	流处理器	峰值半精度 (FP16) 性能	峰值单精度 (FP32) 性能	峰值双精度 (FP64) 性能	显存大小	显存类型	显存带宽
AMD Instinct™ MI250X	220	14,080	383 TFLOPS	47.9 TFLOPS	47.9 TFLOPS	128 GB	HBM2e	3276.8 GB/s



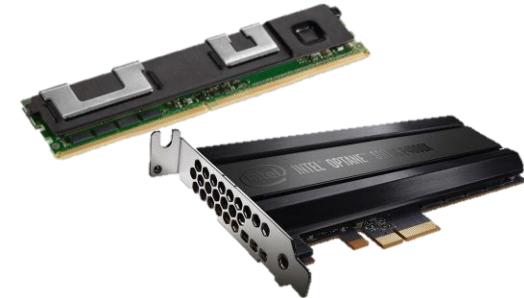
6. 非易失存储器(memory/storage)

- 易失的存储器(volatile memory)：断电后丢失信息的存储器，如SRAM和DRAM
- 非易失的存储器(novolatile memory)：关电后仍然保持信息的存储器
 - PROM, ROM：只能被编程一次
 - 可擦写可编程ROM, EPROM：通过紫外线擦除数据，通过特殊设备写数据
 - 电子可擦除ROM, PROM：直接在印制电路卡上编程
 - Intel optane persistent memory持久内存
 - 闪存, flash：非易失存储器，基于闪存的磁盘驱动器称为SSD(Solid State Disk, 固态硬盘)

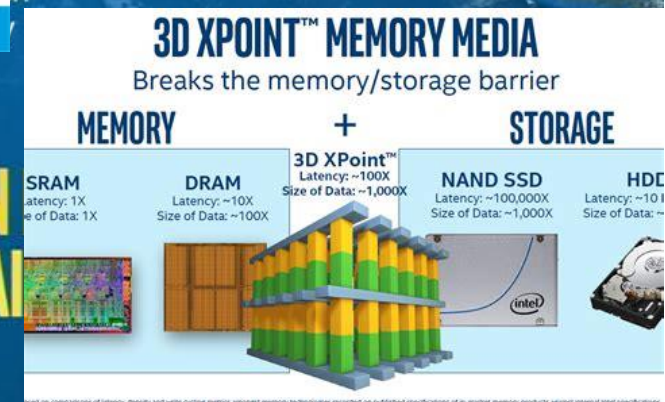
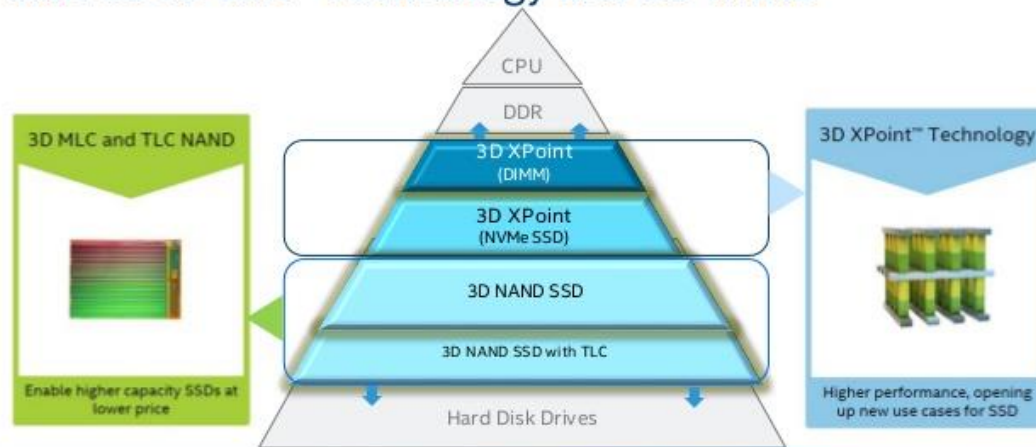


新型非易失存储：3D XPoint

- 3D Xpoint: 新型内存和存储
 - 3D Xpoint DIMM: 当前最大容量单条512GB
 - 3D Xpoint SSD: 大容量, 字节级访问



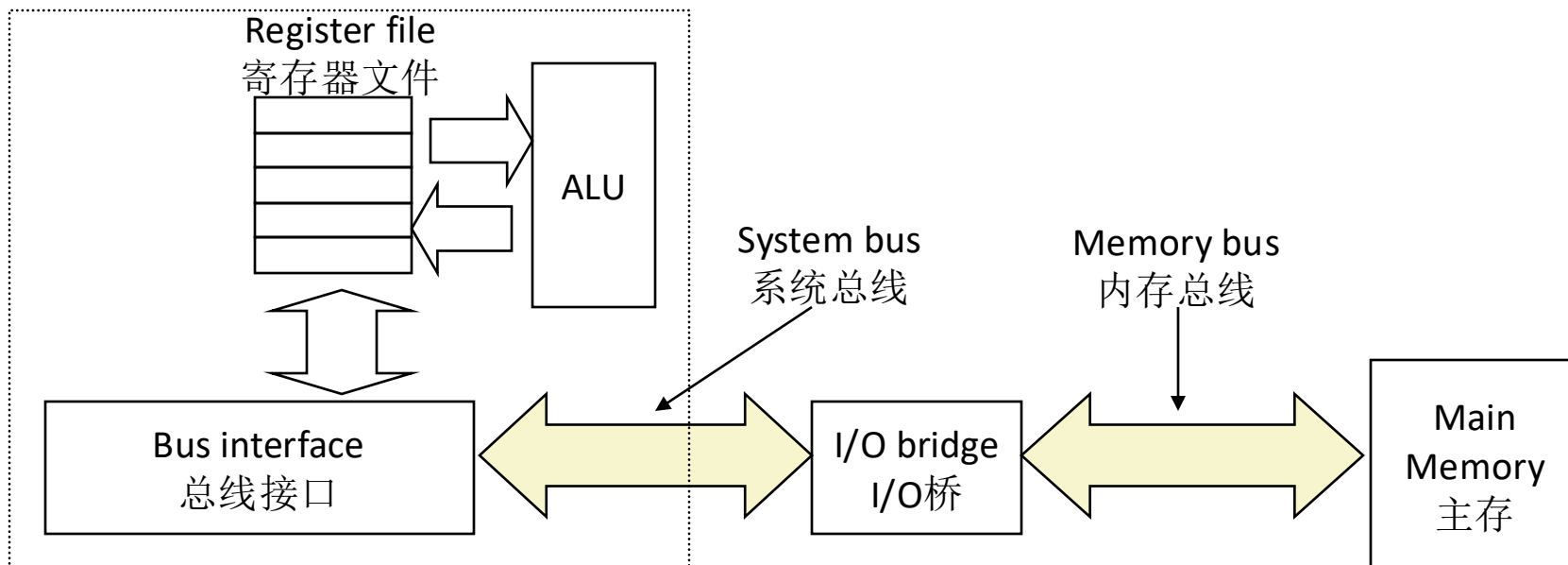
Re-Architecting the Memory-Storage Hierarchy with 3D XPoint™ Technology and 3D NAND



7. 访问主存

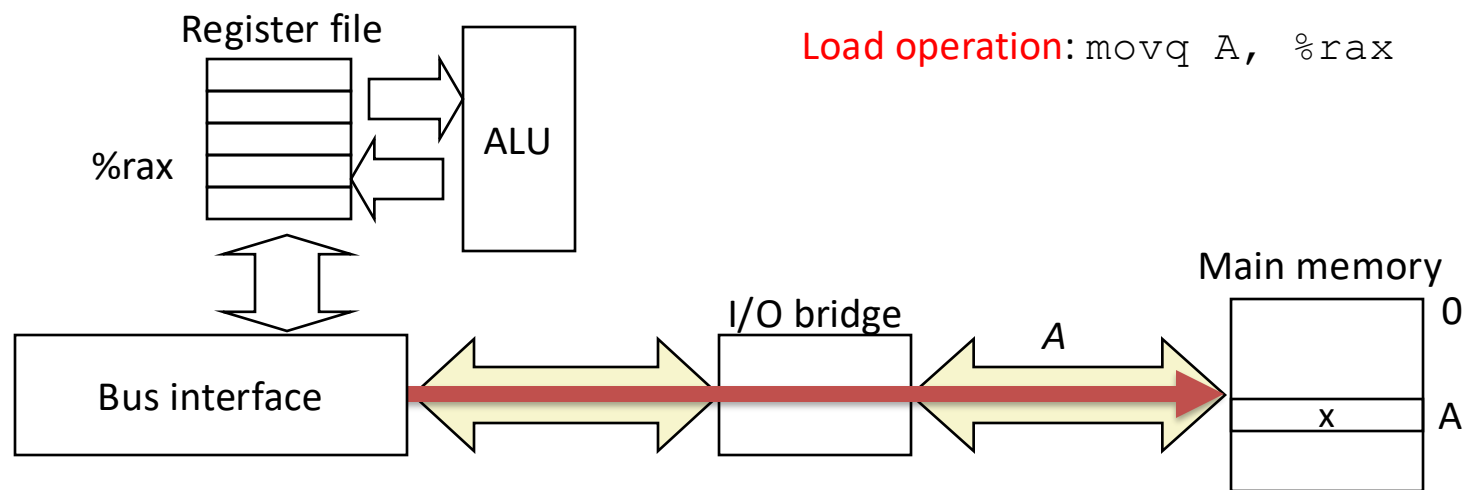
- 总线(bus): 在处理器和DRAM主存之间传输数据流的共享电子电路
- 总线事务(bus transaction): CPU和主存之间传送数据的一系列步骤
 - 读事务(read transaction): 从主存传送数据到CPU
 - 写事务(write transaction): 从CPU传送数据到主存
- 总线是一组并行的导线, 能够携带地址、数据和控制信号; 两个以上的设备也能共享同一总线; 控制线携带的信号会同步事务, 并标识出当前正在被执行事务的类型
- 系统总线: 连接CPU和I/O桥接器(I/O桥接器芯片组包括内存控制器)
- 内存总线: 连接I/O桥接器和主存
- I/O桥接器: 将系统总线的电子信号翻译成内存总线的电子信号; 将系统总线和内存总线连接到I/O总线

CPU芯片



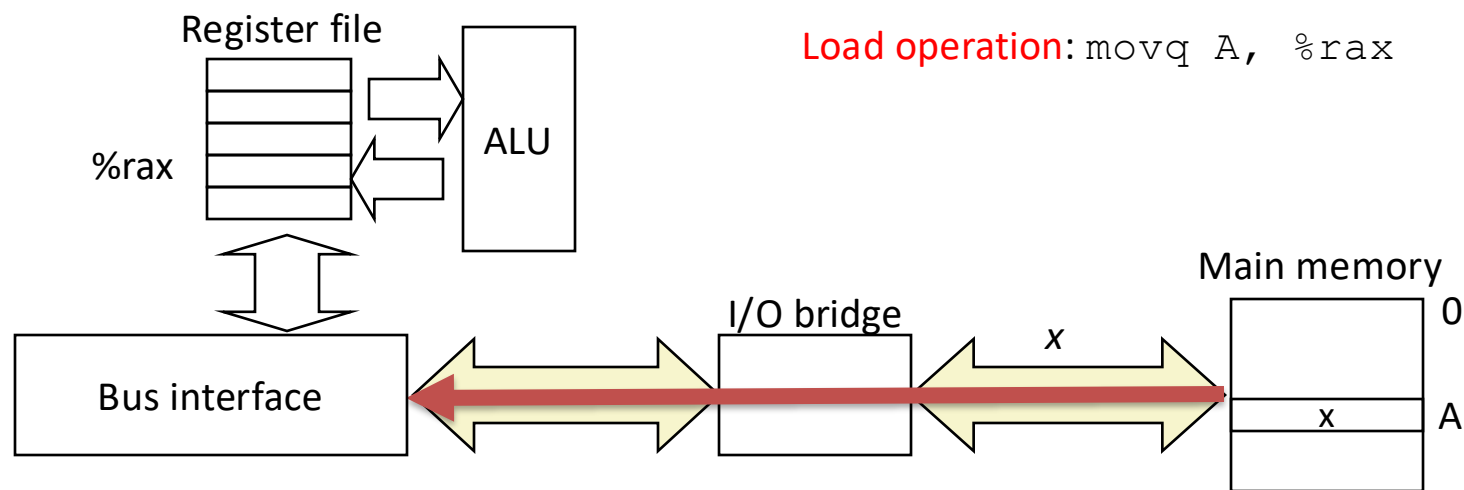
内存加载操作

- `Movq A, %rax`
- 读事务的三个操作步骤：
 - CPU将地址A放到系统总线上



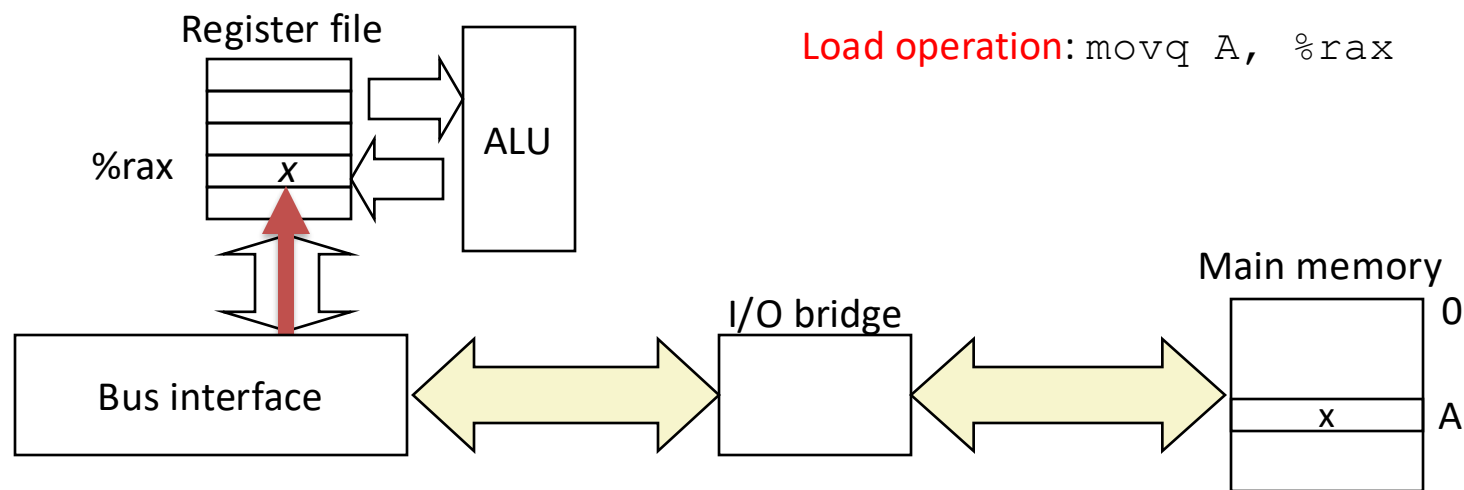
内存加载操作

- `Movq A, %rax`
- 读事务的三个操作步骤：
 - CPU将地址A放到系统总线上
 - 主存从内存总线读地址，从DRAM取出数据，并写到内存总线



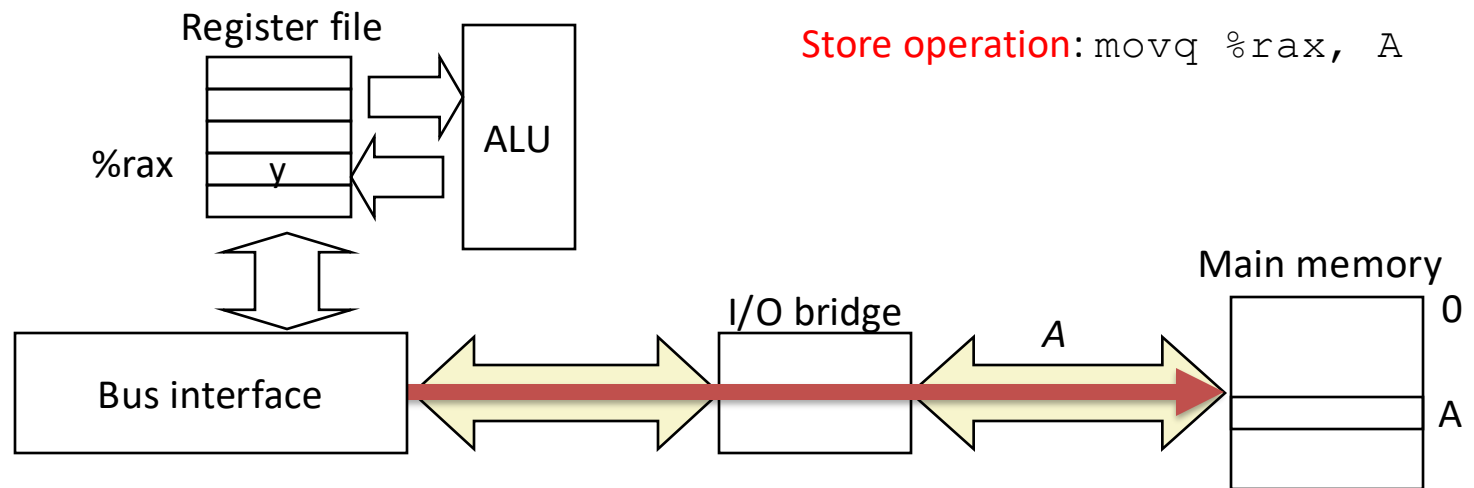
内存加载操作

- `Movq A, %rax`
- 读事务的三个操作步骤：
 - CPU将地址A放到系统总线上
 - 主存从内存总线读地址，从DRAM取出数据，并写到内存总线
 - CPU从系统总线上读数据，并复制到寄存器%rax中



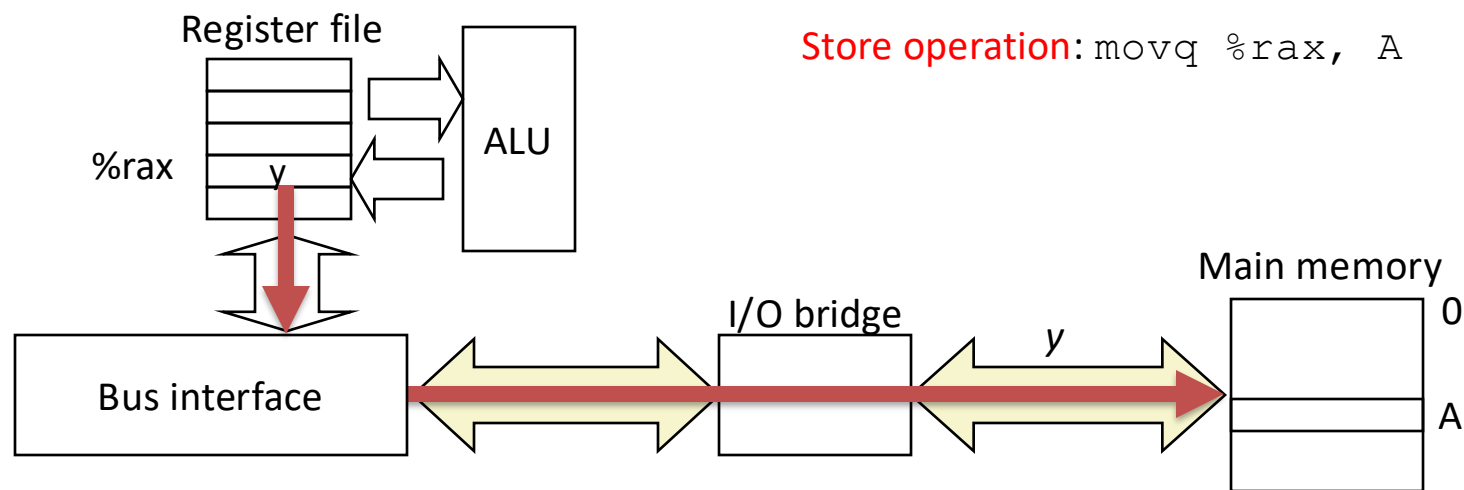
内存存储操作

- `Movq %rax, A`
- 写事务的三个操作步骤：
 - CPU将地址A放到系统总线上



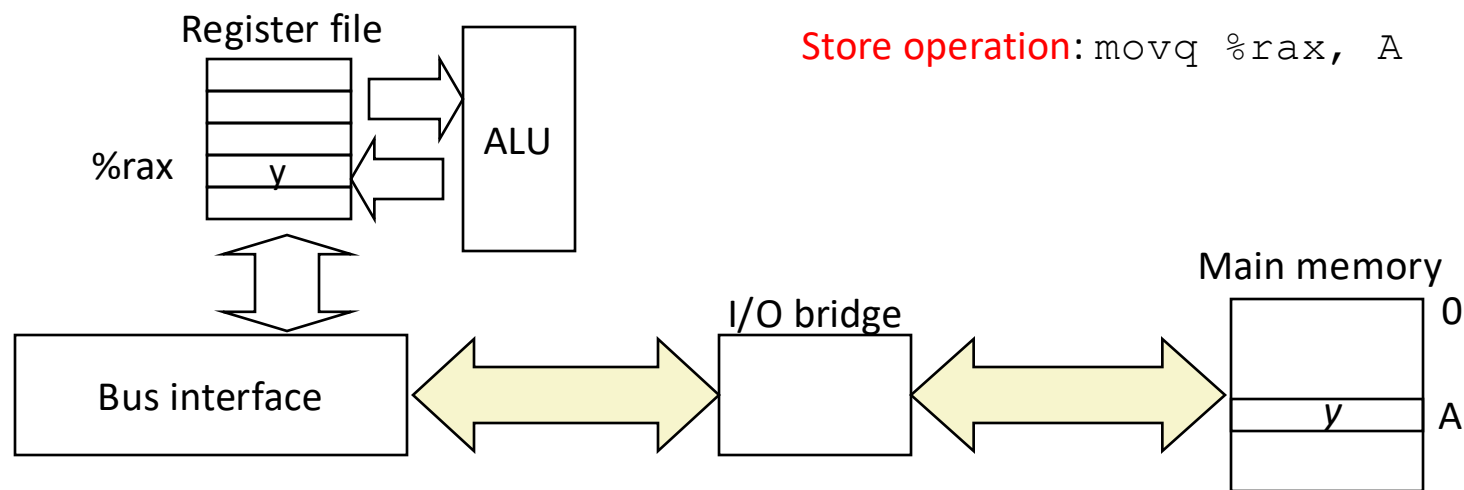
内存存储操作

- `Movq %rax, A`
- 写事务的三个操作步骤：
 - CPU将地址A放到系统总线上
 - CPU将%rax中的数据复制到系统总线

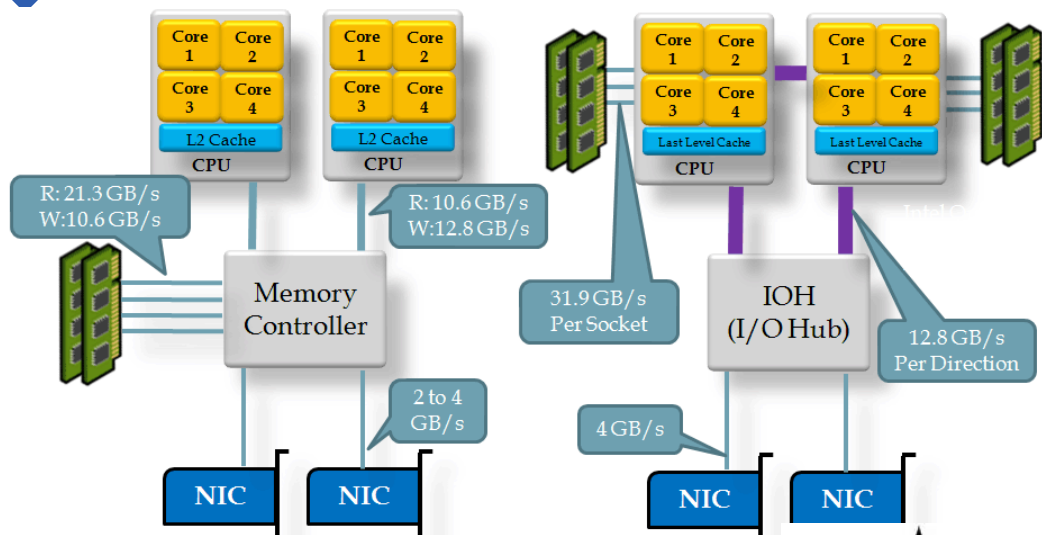


内存存储操作

- `Movq %rax, A`
- 写事务的三个操作步骤：
 - CPU将地址A放到系统总线上
 - CPU将%rax中的数据复制到系统总线
 - 主存从内存总线读出数据，并将这些位存储到DRAM中

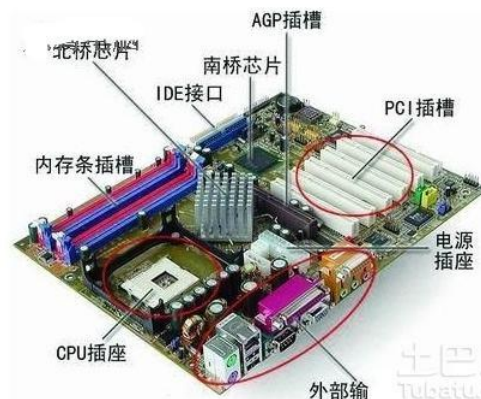


总线类型



前端总线(Front Side Bus, FSB):

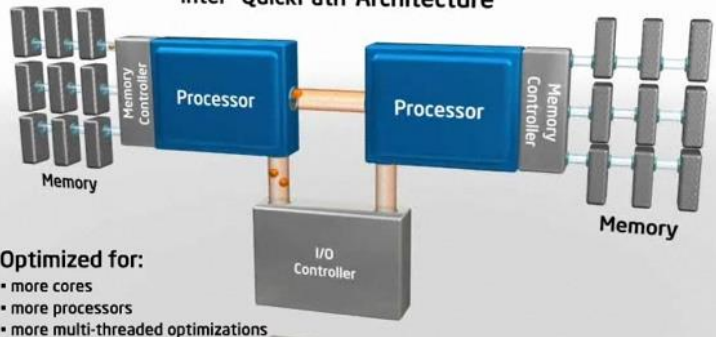
- ◆ 北桥芯片负责与CPU的联系并控制内存
- ◆ 南桥芯片负责I/O总线之间的通信
- ◆ FSB将CPU连接到北桥



快速通道互联(Quick Path Interconnect, QPI):

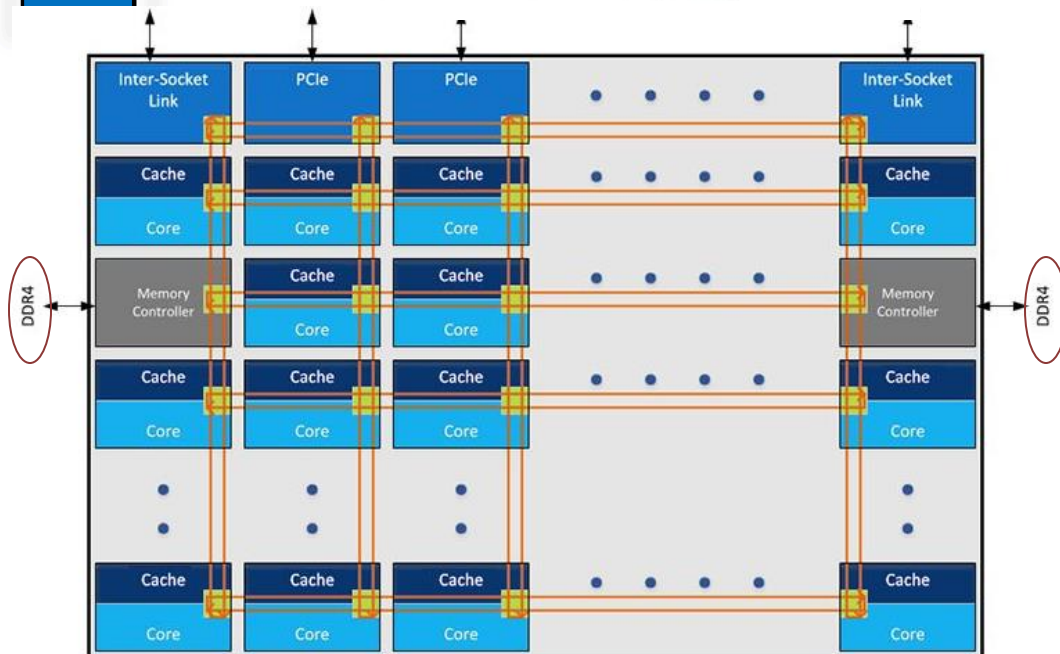
- ◆ 一种点对点互联结构
- ◆ CPU可直接通过内存控制器访问内存资源
- ◆ 实现多核处理器内部的直接互联

Intel® QuickPath Architecture



Optimized for:

- more cores
- more processors
- more multi-threaded optimizations



<https://www.intel.cn/content/www/cn/zh/io/quickpath-technology/quick-path-interconnect-introduction-paper.html>

<https://www.intel.com/content/dam/www/public/us/en/documents/product-briefs/xeon-scalable-platform-brief.pdf>

胶水多核和原生多核

Content Under Embargo Until 9:15 AM PST July 11, 2017

Intel® Xeon® Scalable Processor Family: Architected for the Data Center

Implementation Matters

FULLY INTEGRATED

Intel® Xeon® Scalable Processor Family
Architected for the Data Center

- Single die implementation maximizes performance
- Optimized for data sharing between all threads in a socket
- Mesh architecture enables low and consistent latencies
- Data center-specific features:
 - Up to 8-socket support, AVX-512, QuickAssist Technology, Trusted Execution Technology, RAS, Innovation Engine

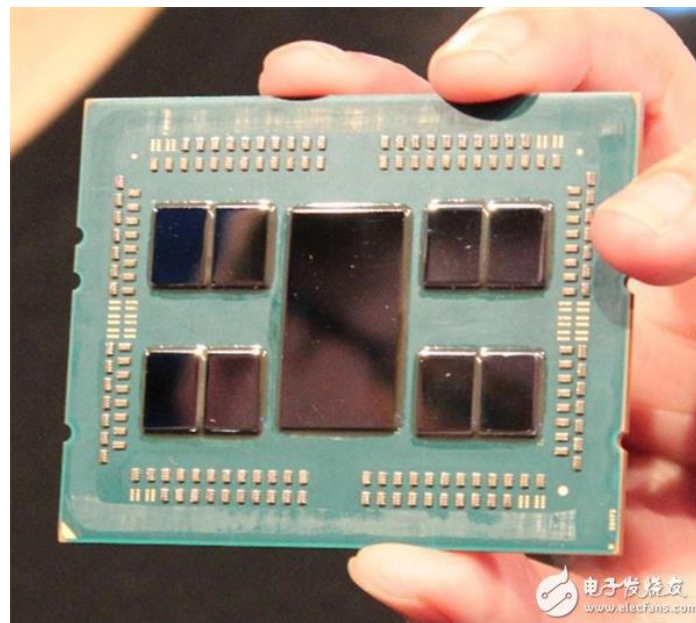
PARTITIONED AND DISJOINTED

AMD Naples
Repurposed Desktop Dies for Server

- 4 glued-together desktop dies
- Inconsistent performance and higher latencies due to die-to-die interconnect

Intel Press Workshops - June 2017

www.elecfans.com



Sapphire Rapids

Memory and Last Level Cache

Increased Shared Last Level Cache (LLC)

Up to >100 MB LLC shared across ALL cores

Increased bandwidth, security & reliability via DDR 5 Memory

4 memory controllers supporting 8 channels

Intel® Optane™ Persistent Memory 300 Series

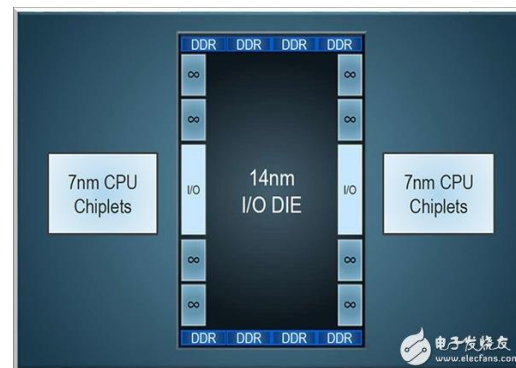
Architecture Day 2021

Under embargo until August 19th at 6:00 am Pacific time

intel 126

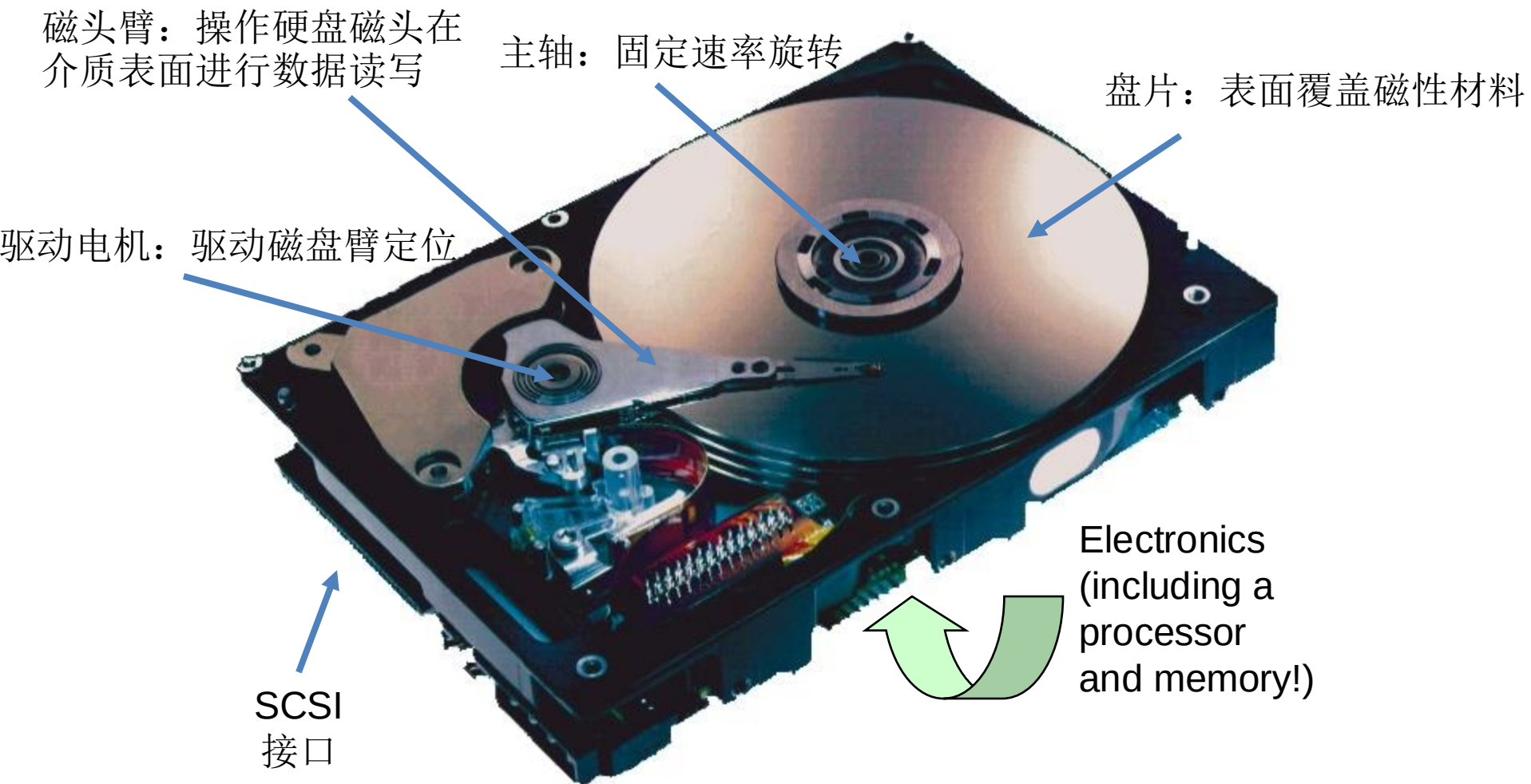
英特尔sapphire rapids多die结构

AMD 7nm罗马处理器的MCM是8+1架构，在MCM多芯片架构中，AMD将CPU内核与IO单元分离，四周的8个小核心是纯CPU内核，而DDR内存控制器、PCIe控制器、IF控制器等IO单元单独做成了一个核心。

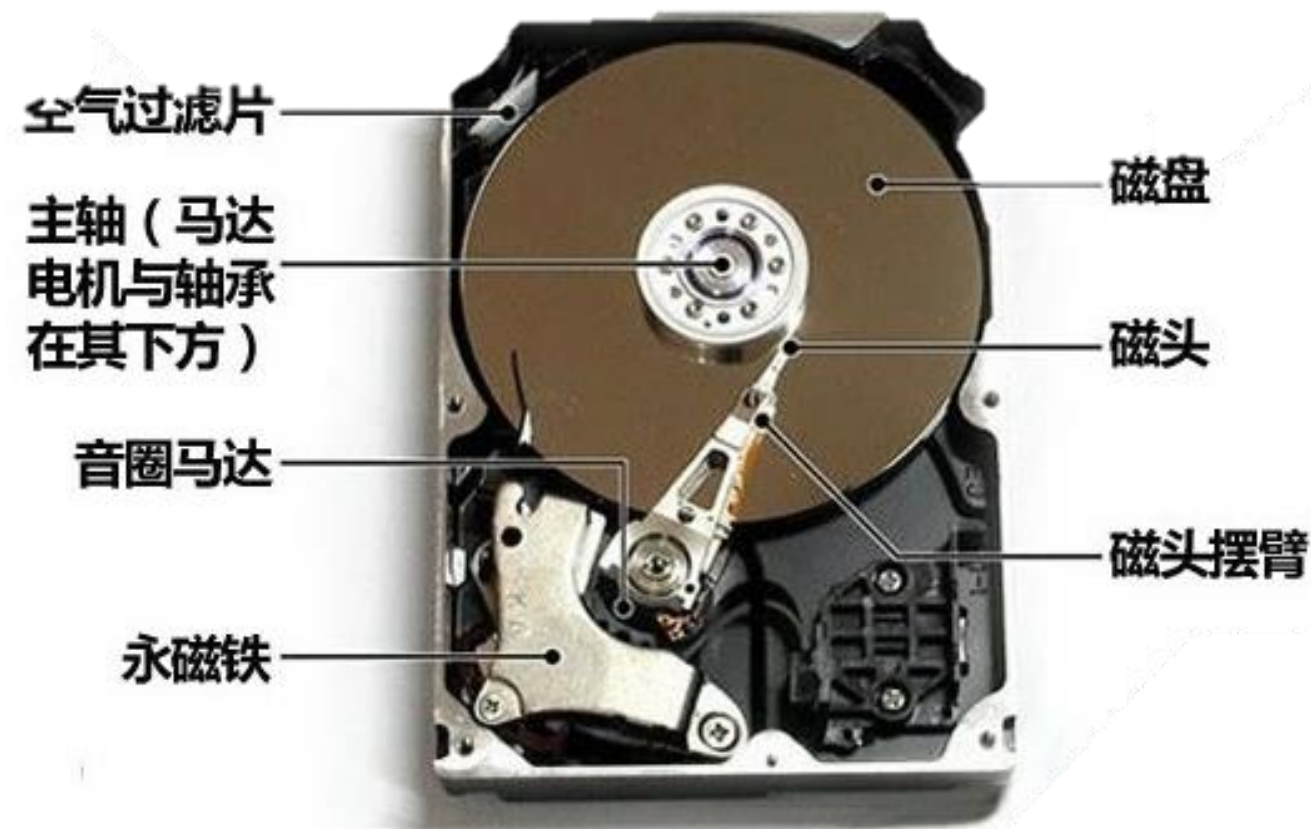


6.1.2 磁盘存储

- 1. 磁盘构造



硬件新技术：氦气填充硬盘



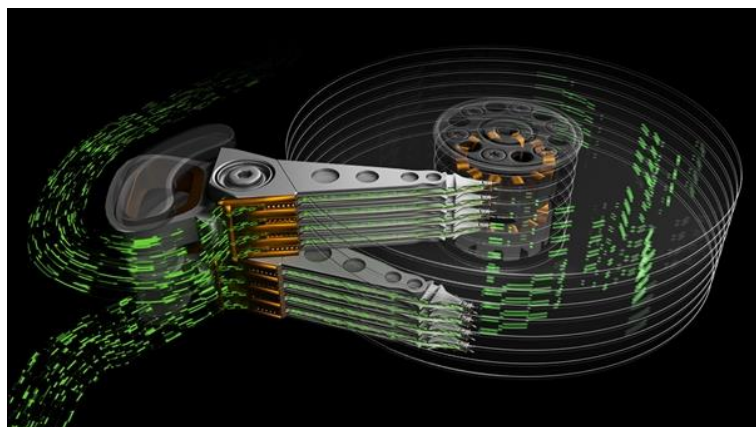
空气填充硬盘：

- ◆ 磁头依据空气动力学悬浮于磁盘碟片的表面
- ◆ 容易产生盘抖动，空气阻力大

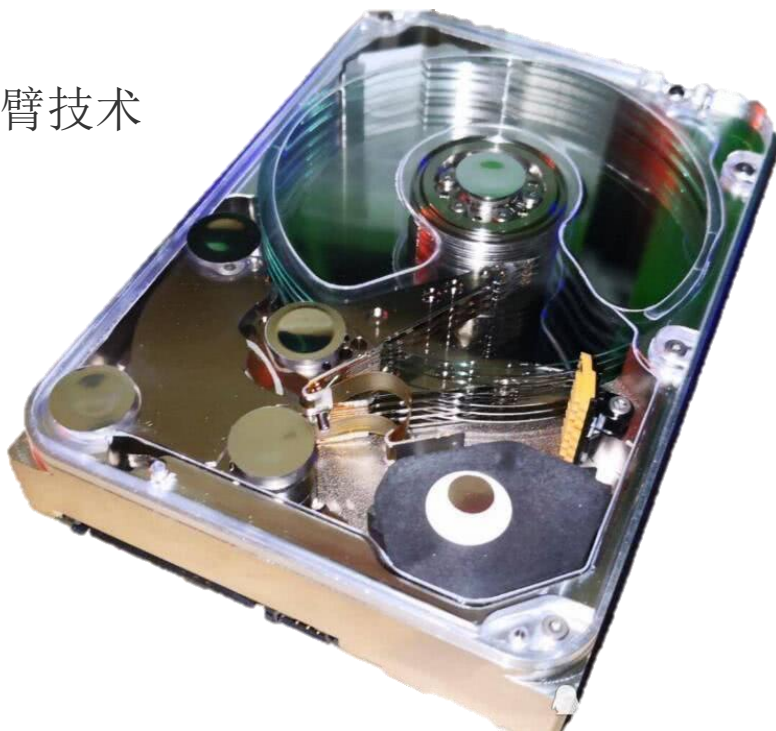
全封闭盘体氦气填充硬盘：

- ◆ 氦气密度较低，减小磁盘转轴上的阻力
- ◆ 流体流动力大幅减少，数据轨道也可以靠得更近
- ◆ 容纳更多盘片
- ◆ 运行温度更低，噪音更小

硬盘新技术：速度&容量



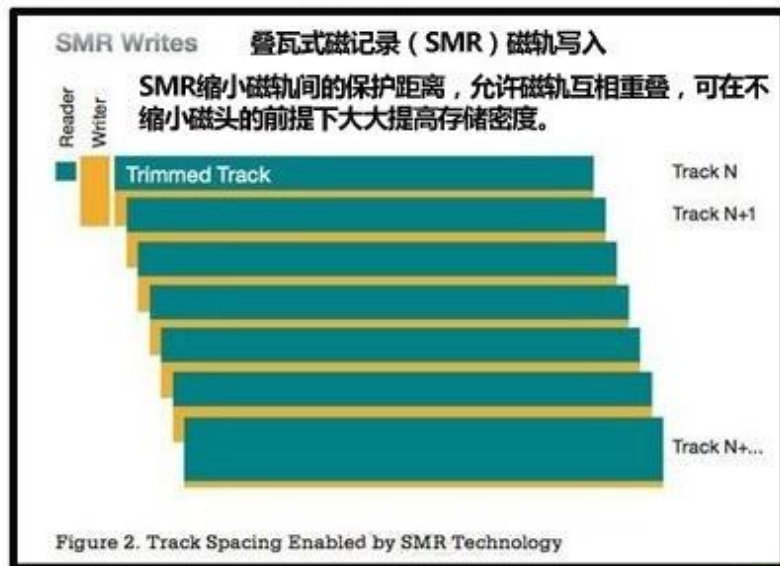
多读写臂技术



叠瓦式磁记录 (SMR) 技术

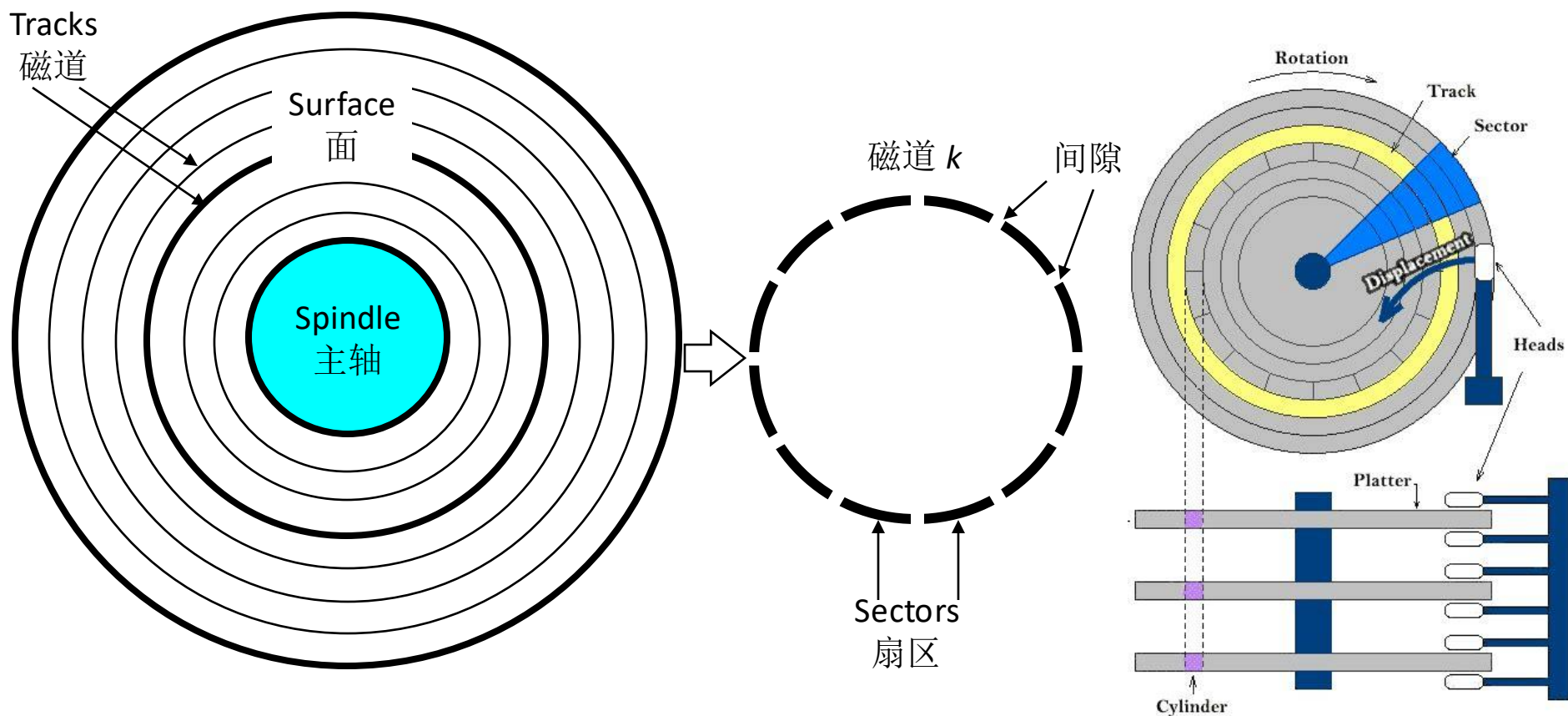


叠瓦式与传统磁轨写入对比



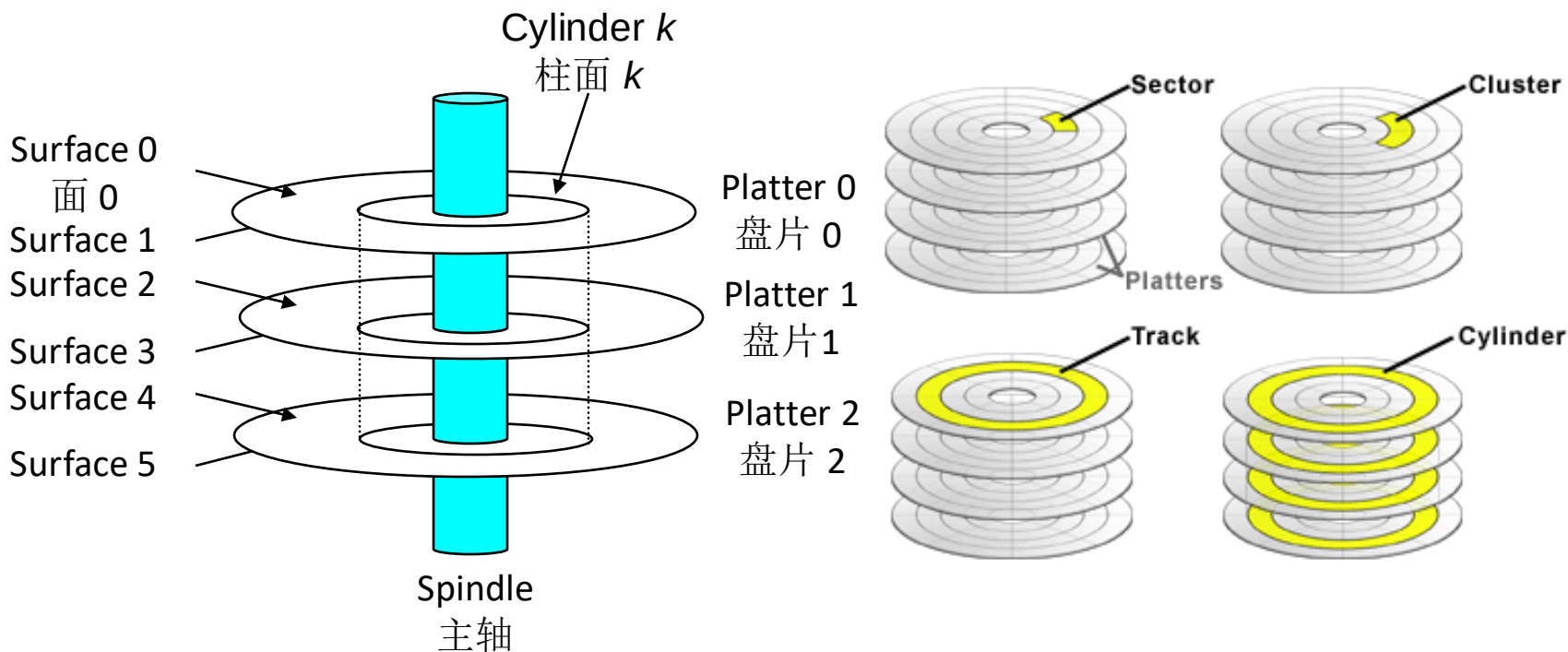
磁盘结构

- 磁盘由一组盘片构成，每个盘片有两个存储面
- 每个盘面由一系列同心圆磁道组成
- 每个磁道由一系列扇区组成，扇区之间由间隙分隔



磁盘结构

- 柱面(cylinder): 所有盘片表面上到主轴中心的距离相等的磁道的集合
 - 示例: 柱面 k 是3个盘片、6个面上6个磁道 k 的集合

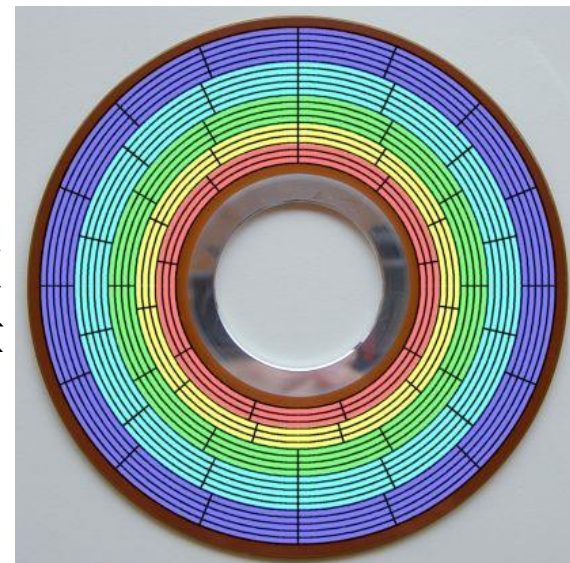


2. 磁盘容量

- 容量(Capacity): 磁盘上可以记录的最大位数
 - 示例: 10TB硬盘 = 10^{13} 字节
- 磁盘容量决定因素:
 - 记录密度(Recording density (位/英寸)): 磁道一英寸的段中可以放入的位数
 - 磁道密度(Track density (道/英寸)): 从盘片中心出发半径上一英寸的段内可以有的有磁道数
 - 面密度(Areal density (位/平方英寸)): 记录密度与磁道密度的乘积
- 多区记录(multiple zone recording):
 - 初期: 每个磁道分为数目相同的扇区, 扇区数量由最靠内的磁道能记录的扇区数决定
 - 缺点: 越靠外, 扇区间隔越大
 - 当前: 柱面集合分割为不相交的子集合—记录区(recording zone)
 - 每个记录区包含一组连续的柱面
 - 一个区中每个柱面中的每条磁道都有相同数量的扇区
 - 扇区数量由该区中最里面的磁道所能包含的扇区数量决定

GBASE®
南大通用

TERADATA



磁盘容量计算

$$\text{磁盘容量} = \frac{\text{字节数}}{\text{扇区}} \times \frac{\text{平均扇区数}}{\text{磁道}} \times \frac{\text{磁道数}}{\text{表面}} \times \frac{\text{表面数}}{\text{盘片}} \times \frac{\text{盘片数}}{\text{磁盘}}$$

- 示例:

- 512 bytes/sector
- 300 sectors/track (on average)
- 20,000 tracks/surface
- 2 surfaces/platter
- 5 platters/disk

$$\begin{aligned}\text{磁盘容量} &= \frac{512\text{字节}}{\text{扇区}} \times \frac{300\text{扇区}}{\text{磁道}} \times \frac{20000\text{磁道}}{\text{表面}} \times \frac{2\text{表面}}{\text{盘片}} \times \frac{5\text{盘片}}{\text{磁盘}} \\ &= 30,720,000,000 \\ &= 30.72 \text{ GB}\end{aligned}$$

练习题6.2 计算磁盘容量

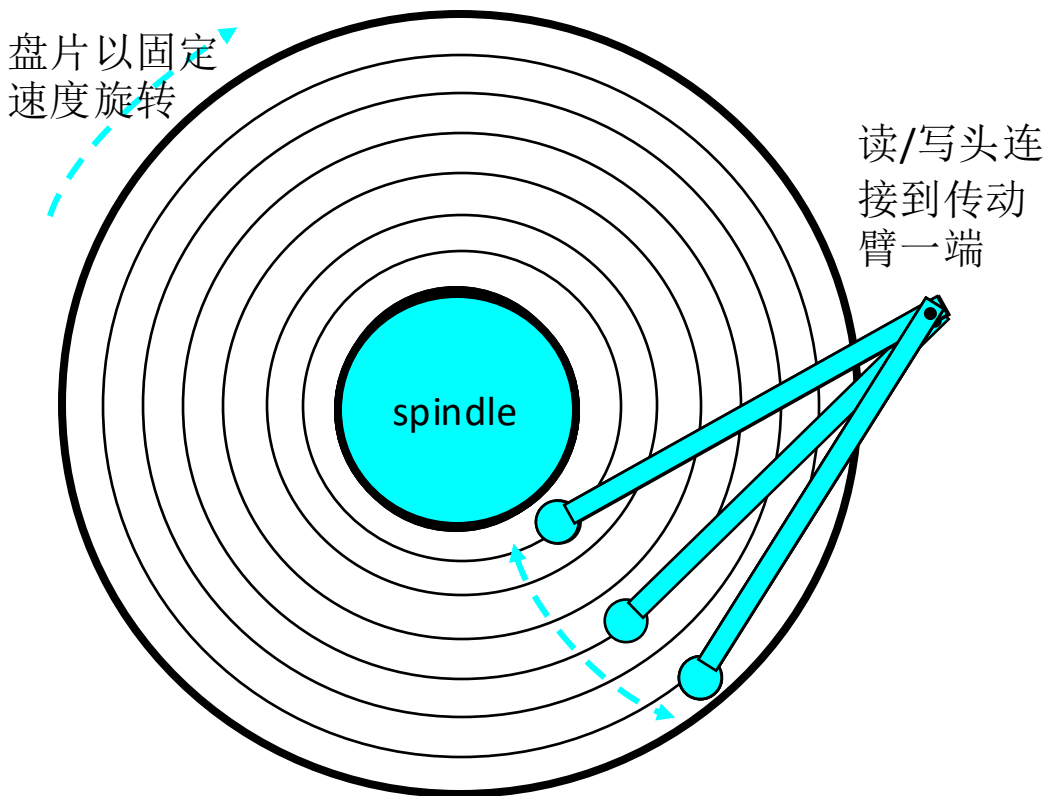
- 磁盘有2个盘片，10000个柱面，每条磁道平均400个扇区，每扇区512字节

$$\begin{aligned}\text{磁盘容量} &= \frac{512\text{字节}}{\text{扇区}} \times \frac{400\text{扇区}}{\text{磁道}} \times \frac{10000\text{磁道}}{\text{表面}} \times \frac{2\text{表面}}{\text{盘片}} \times \frac{2\text{盘片}}{\text{磁盘}} \\ &= 8,192,000,000 \\ &= 8.192 \text{ GB}\end{aligned}$$

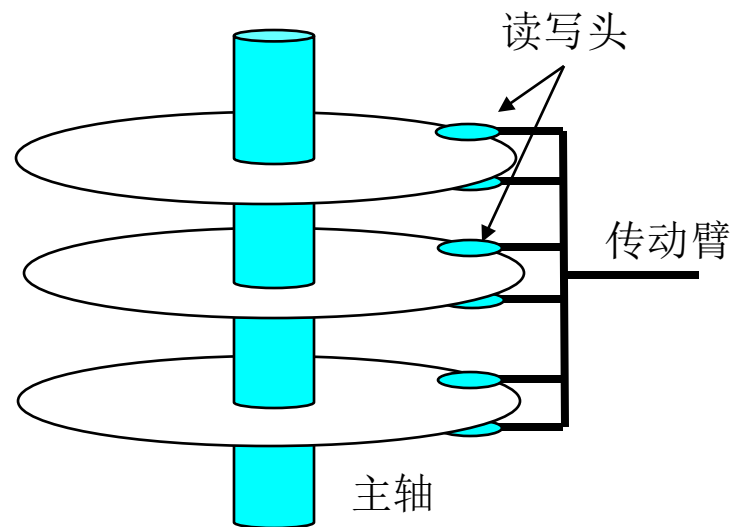


3. 磁盘操作

- 寻道：通过传动臂将磁头定位在指定磁道上
- 读/写：读写头感知(读该位)或修改(写该位)磁道上的位值

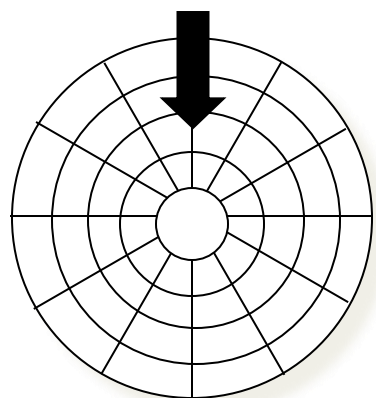


沿半径轴前后移动传动臂，驱动器可以将读/写头定位在盘面上的任何磁道上

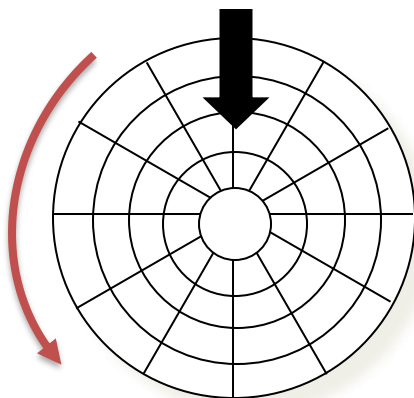


磁盘访问—读

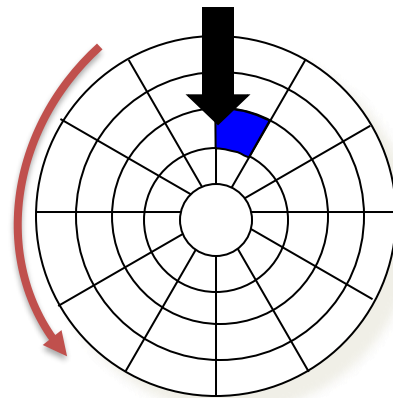
- 移动磁头臂到指定磁道
- 旋转至指定扇区
- 读取数据



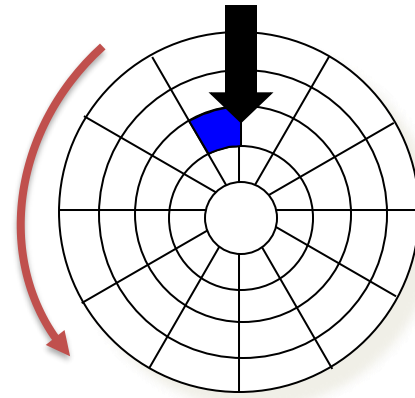
定位磁道



盘片旋转



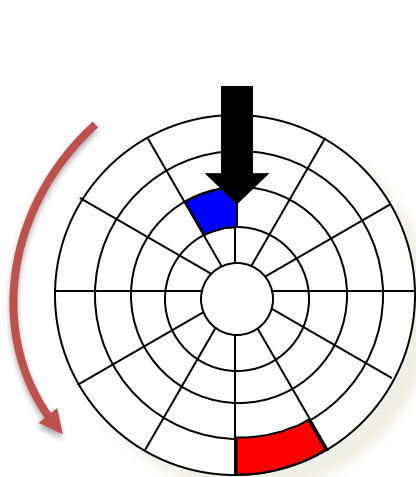
开始读取



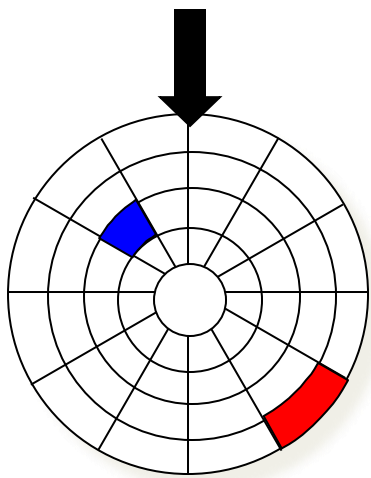
读取完毕

磁盘访问示例

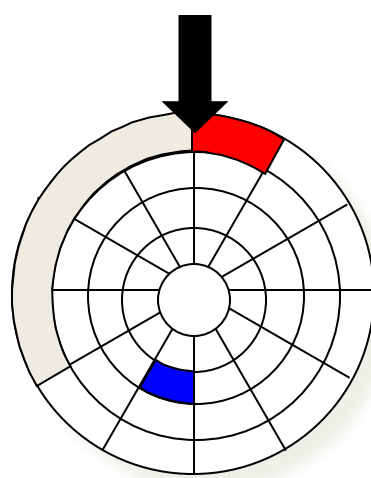
- 示例：访问蓝色扇区后访问红色扇区



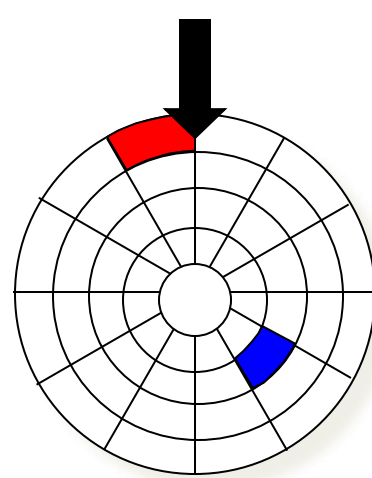
完成蓝色
扇区读取



磁头寻道
红色扇区



旋转等待至
红色扇区



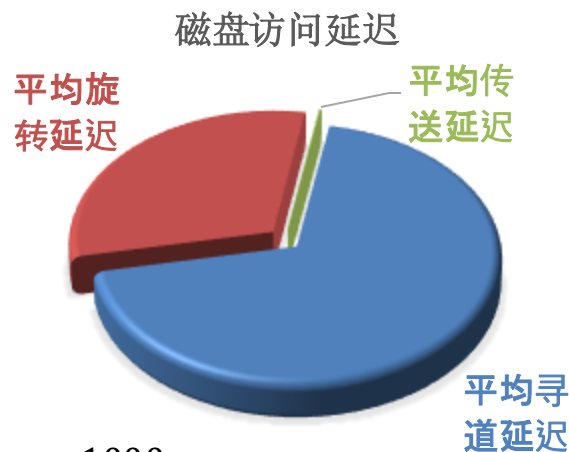
完成红色
扇区读取

扇区访问时间

- 扇区访问时间由三部分组成：寻道时间、旋转时间和传送时间
 - 寻道时间：移动传动臂将读/写头定位到包含目标扇区磁道上所需的时间
 - 寻道时间依赖于读/写头以前的位置和传动臂在盘面上移动的速度
 - 平均寻道时间为3~9ms，一次寻道最大时间可达20ms
 - 旋转时间：驱动器等待目标扇区的第一位旋转到读/写头下的时间
 - 旋转时间依赖于当读/写头到达目标扇区时盘面的位置以及磁盘的旋转速度
 - 最大旋转延迟为等待一整圈： $T_{MaxRotation} = \frac{1}{RPM} \times \frac{60s}{1min}$
 - 平均旋转延迟为最大旋转延迟一半： $T_{AvgRotation} = \frac{1}{2} \times \frac{1}{RPM} \times \frac{60s}{1min}$
 - 典型磁盘转速为7200RPM或15K RPM
 - 传送时间：当目标扇区的第一个位位于读/写头下时，驱动器读或写该扇区的时间
 - 一个扇区的传送时间依赖于旋转速度和每条磁道的扇区数目
 - 一个扇区以秒为单位的平均传送时间为： $T_{AvgTransfer} = \frac{1}{RPM} \times \frac{1}{\text{平均扇区数/磁道}} \times \frac{60s}{1min}$

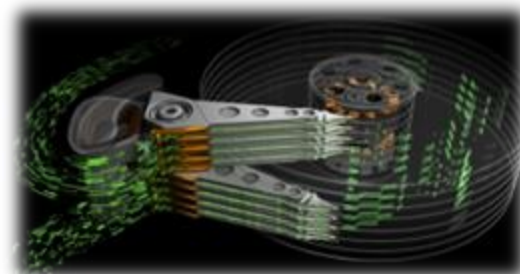
扇区访问时间示例

- 磁盘规格：
 - 旋转速率：7200RPM
 - 平均寻道时间：9ms
 - 每条磁道的平均扇区数：400
- 访问时间估算：
 - 平均旋转延迟： $T_{AvgRotation} = \frac{1}{2} \times \frac{1}{7200} \times \frac{60s}{1min} \times \frac{1000ms}{s} \approx 4ms$
 - 平均传送时间： $T_{AvgTransfer} = \frac{1}{7200} \times \frac{1}{400/1} \times \frac{60s}{1min} \times \frac{1000ms}{s} \approx 0.02ms$
 - 总访问延迟： $T_{Access} = T_{AvgSeek} + T_{AvgRotation} + T_{AvgTransfer} = 9ms + 4ms + 0.02ms = 13.02ms$
- 重要问题：
 - 访问延迟主要是寻道延迟和旋转延迟
 - 访问扇区第一个位用时很长，访问其他字节时间极少
 - SRAM中一个cache line(64字节)访问时间为4ns，DRAM访问时间为60ns；SRAM从内存读512个字节扇区大小的块时间为256ns，DRAM为4000ns
 - 磁盘访问时间为10ms，是SRAM的40000倍，是DRAM的2500倍



IOPS如何计算？

练习题6.3



数据在磁道上如何分布?

- 估算磁盘扇区访问时间(ms)

- 旋转速率: 15000RPM
- 平均寻道时间: 8ms
- 每条磁道的平均扇区数: 500

- 访问时间估算:

- 平均旋转延迟: $T_{AvgRotation} = \frac{1}{2} \times \frac{1}{15000} \times \frac{60s}{1min} \times \frac{1000ms}{s} \approx 2ms$

- 平均传送时间: $T_{AvgTransfer} = \frac{1}{15000} \times \frac{1}{500/1} \times \frac{60s}{1min} \times \frac{1000ms}{s} \approx 0.008ms$

- 总访问延迟: $T_{Access} = T_{AvgSeek} + T_{AvgRotation} + T_{AvgTransfer} = 8ms + 2ms + 0.008ms = 10.008ms$

- 磁盘带宽性能估算:

扇区数量	延迟(ms)	扇区大小(字节)	磁头通道	顺序访问带宽(MB/s)
500	4	512	1	61
3596	8	512	1	219
500	4	512	4	244

4. 磁盘逻辑块

- 现代磁盘使用复杂扇区的简单抽象视图：
 - 扇区大小的逻辑块序列 $0, 1, 2, \dots, B-1$ 对应 B 个扇区
- 逻辑块与物理块映射
 - 使用磁盘中固件设备——磁盘控制器
 - 维护逻辑块号和物理磁盘扇区(面-磁道-扇区)之间的映射关系
- 操作系统I/O操作(如读一个扇区到主存)过程：
 - 发送读某个逻辑块号命令到磁盘控制器
 - 控制器固件执行一个快速表查找，将一个逻辑块号翻译成一个(盘面-磁道-扇区)三元组，唯一地标识对应的物理扇区
 - 控制器解释此三元组，将读/写头移动到相应的柱面，等待扇区移动到读/写头下，将读/写头感知到的位放到控制器上的小缓冲区中，将缓冲区中的数据复制到主存



数据文件逻辑页示例

- 数据库中表文件存储页面逻辑链表结构:

SQLQuery1.sql -...LF32P\zhang (57))*

```
DBCC IND('TPCH','CUSTOMER',1)
```

100 %

结果 消息

	PagePID	PagePID	IAMFID	IAMFID	ObjectID	IndexID	Par...	PartitionID	iam_chain_type	PageType	IndexLevel	Nex...	NextPagePID	Fre...	PrevPagePID
1	1	80	NULL	NULL	1429580131	1	1	72057594044153856	In-row data	10	NULL	0	0	0	0
2	1	73880	1	80	1429580131	1	1	72057594044153856	In-row data	1	0	1	73881	0	0
3	1	73881	1	80	1429580131	1	1	72057594044153856	In-row data	1	0	1	73882	1	73880
4	1	73882	1	80	1429580131	1	1	72057594044153856	In-row data	1	0	1	73883	1	73881
5	1	73883	1	80	1429580131	1	1	72057594044153856	In-row data	1	0	1	73884	1	73882

SQLQuery1.sql -...LF32P\zhang (57))*

```
DBCC IND('TPCH','CUSTOMER',1)
DBCC TRACEON(3604)
DBCC PAGE(TPCH,1,73880,3)
```

100 %

消息

DBCC 执行完毕。如果 DBCC 输出了错误信息, 请与系统管理员联系。

PAGE: (1:73880)

BUFFER:

|

BUF @0x000001EEF5723D80

bpage = 0x000001EEA91EA000 bhash = 0x0000000000000000 bpageno = (1:73880)

bdbid = 7 bpreferences = 0 bcputicks = 0

bsampleCount = 0 bUserl = 61788 bstat = 0x109

blog = 0xad21c8a bnext = 0x0000000000000000 bDirtyContext = 0x0000000000000000

bstat2 = 0x0

PAGE HEADER:

Page @0x000001EEA91EA000

m_pageId = (1:73880) m_headerVersion = 1 m_type = 1

m_typeFlagBits = 0x0 m_level = 0 m_flagBits = 0x200

m_objId (AllocUnitId.idObj) = 193 m_indexId (AllocUnitId.idInd) = 256

Metadata: AllocUnitId = 72057594050576384

Metadata: PartitionId = 72057594044153856 Metadata: IndexId = 1

Metadata: ObjectId = 1429580131 m_prevPage = (0:0) m_nextPage = (1:73881)

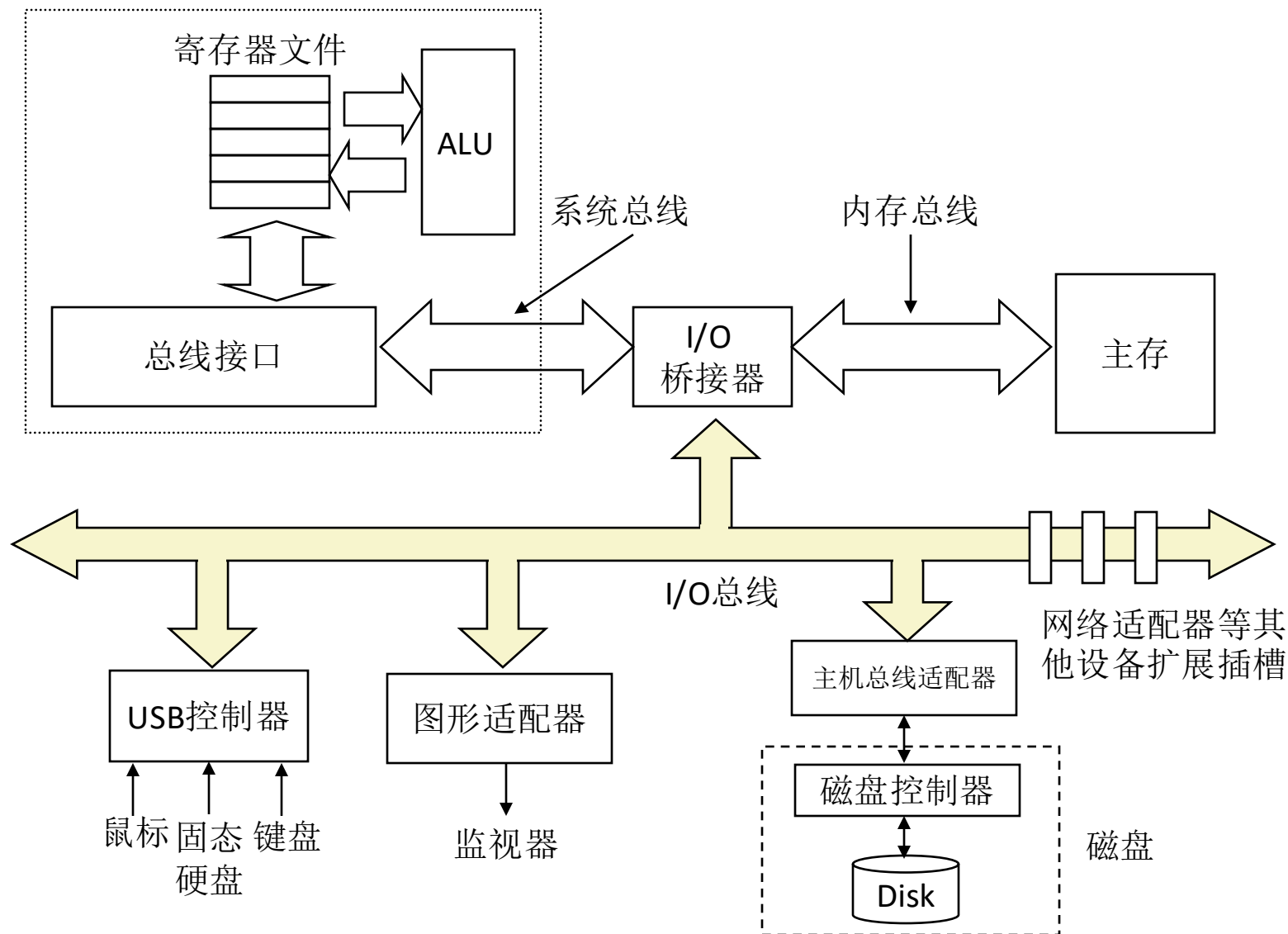
pminlen = 46 m_slotCnt = 45 m_freeCnt = 106

m_freeData = 7996 m_reservedCnt = 0 m_lsn = (113:80904:50)

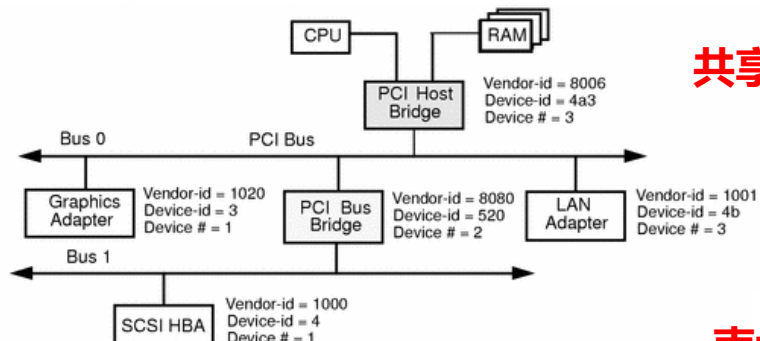
m_xactReserved = 0 m_xdesId = (0:0) m_ghostRecCnt = 0

m_tornBits = 383708996 DB Frag ID = 1

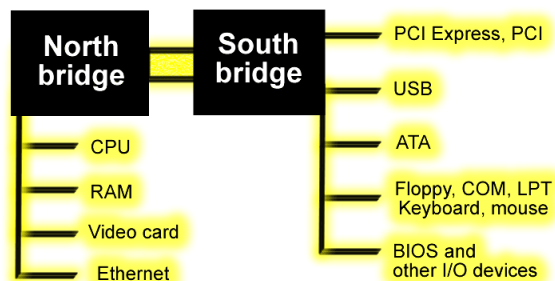
5. 连接I/O设备



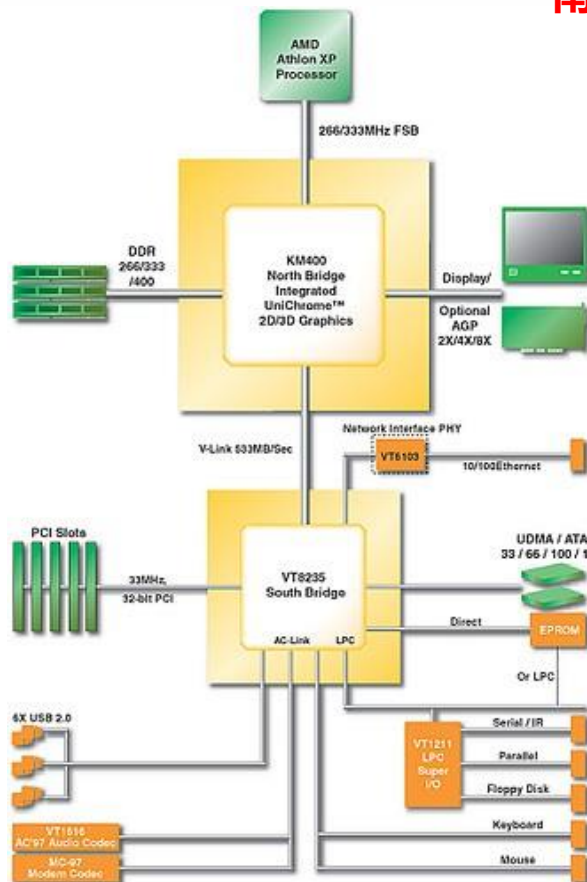
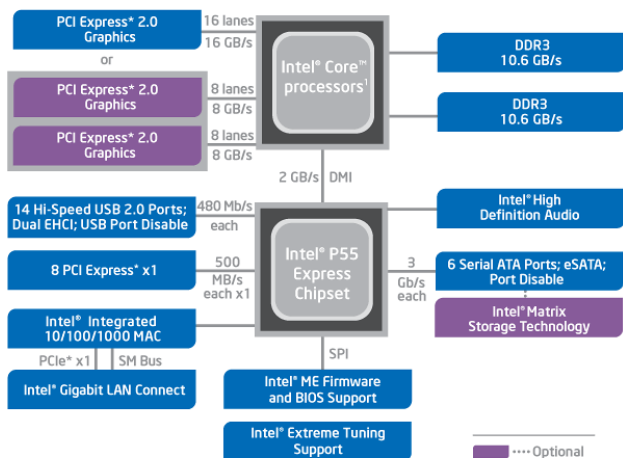
几种不同的I/O连接方式



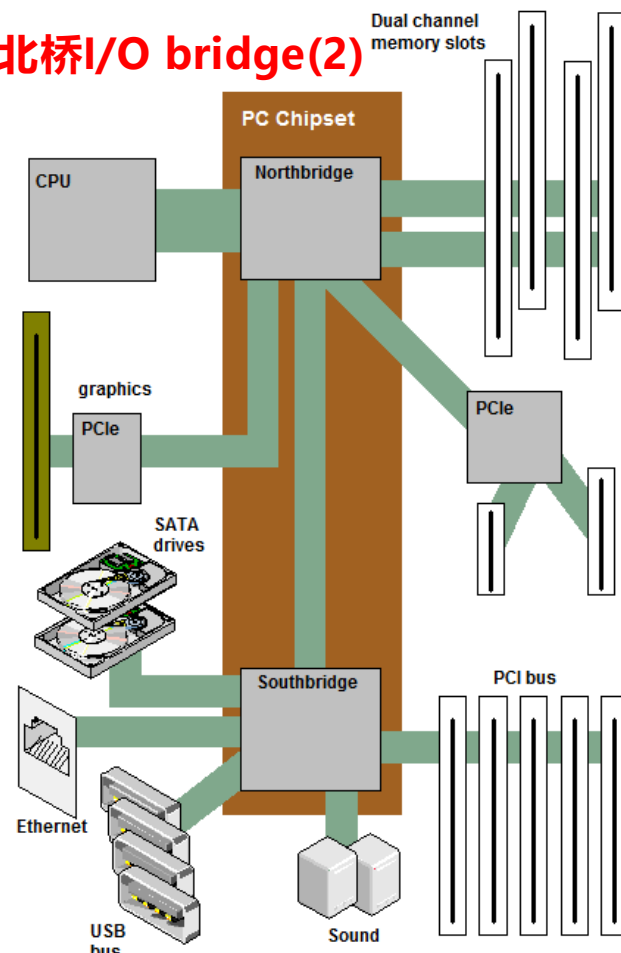
南北桥I/O bridge(1)



南桥I/O bridge only



南北桥I/O bridge(2)



练习题6.4

- 假设1MB文件由512字节的逻辑块组成，存储在具有如下特性的磁盘驱动器上：
 - 旋转速率：10,000RPM
 - 平均寻道时间：5ms
 - 每条磁道的平均扇区数：1000
 - 表面：4
 - 扇区大小：512字节
 - 假设程序顺序地读文件的逻辑块，一个接一个，将读/写头定位到第一块上的时间是 $T_{AvgSeek} + T_{AvgRotation}$
 - A. 最好的情况：给定逻辑块到磁盘扇区的最好的可能映射，估计读文件需要的最优时间(ms)
 - B. 随机的情况：如果块是随机地映射到磁盘扇区的，估计读文件需要的时间
- 文件由2000个512字节的逻辑块组成， $T_{AvgSeek}$ 为5ms， $T_{AvgRotation}$ 为 $\frac{1}{2} \times \frac{1}{10000} \times \frac{60s}{1min} = 3ms$
 - A. 最好的情况：块被映射到连续的扇区，在同一柱面，连接读取不用移动读/写头，定位到第一个扇区后，需要磁盘旋转2圈读取所有2000个块，读文件总时间为： $T_{AvgSeek} + T_{AvgRotation} + 2 \times T_{MaxRotation} = 5 + 3 + 12 = 20ms$
 - B. 随机情况：块被随机映射到扇区上，读2000个块中的每一块都需要 $T_{AvgSeek} + T_{AvgRotation}$ ms，读文件总时间为： $(T_{AvgSeek} + T_{AvgRotation}) \times 2000 = 16000ms$
 - 随机与最好情况总时间比为：16000：20=800：1

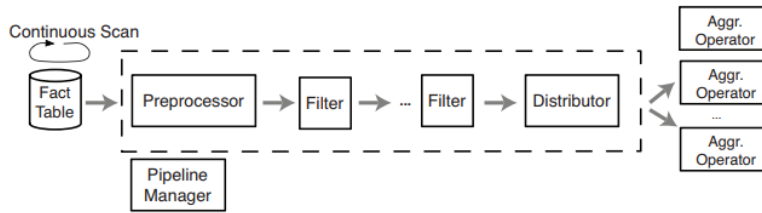
A Scalable, Predictable Join Operator for Highly Concurrent Data Warehouses

George Candea
EPFL & Aster Data
George.Candea@epfl.ch

Neoklis Polyzotis
U.C. Santa Cruz
alkis@ucsc.edu

Radek Vingralek
Aster Data
Radek.Vingralek@asterdata.com

<http://web.eecs.umich.edu/~mozafari/winter2014/eecs684/papers/predictable-join.pdf>



The DataPath System: A Data-Centric Analytic Processing Engine for Large Data Warehouses

Subi Arumugam, Alin Dobra
University of Florida, Gainesville, FL 32611
{sa2, adobra}@cise.ufl.edu

Christopher M. Jermaine,
Niketan Pansare, Luis Perez
Rice University, Houston, TX, 77005
{cmj4, np6, lp6}@rice.edu

<https://www.cise.ufl.edu/~cjermain/p519-arumugam.pdf>

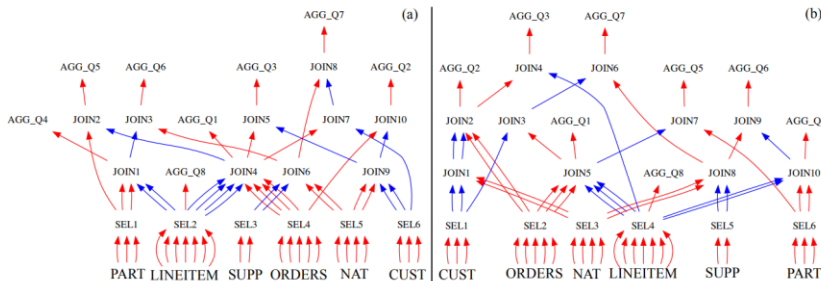


Figure 6: Path networks used for the slowest eight-query workload (a) and the fastest (b). Probing relations use blue links.

Workload-Aware Storage Layout for Database Systems

Oguzhan Ozmen, Kenneth Salem
University of Waterloo
Waterloo, ON Canada
{oozmen, kmsalem}@uwaterloo.ca

Jiri Schindler, Steve Daniel
NetApp, Inc.
{First.Last}@netapp.com

<https://cs.uwaterloo.ca/~kmsalem/pubs/OzmenSIGMOD10.pdf>

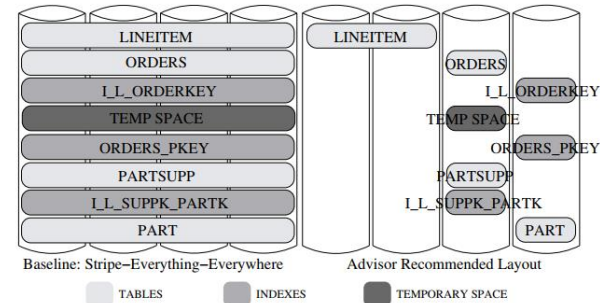


Figure 1: SEE and Optimized Layouts of TPC-H Database Objects.

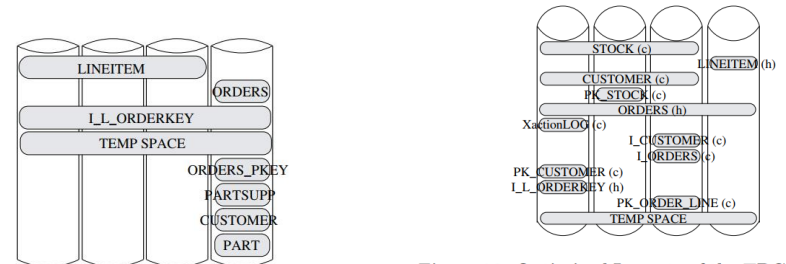
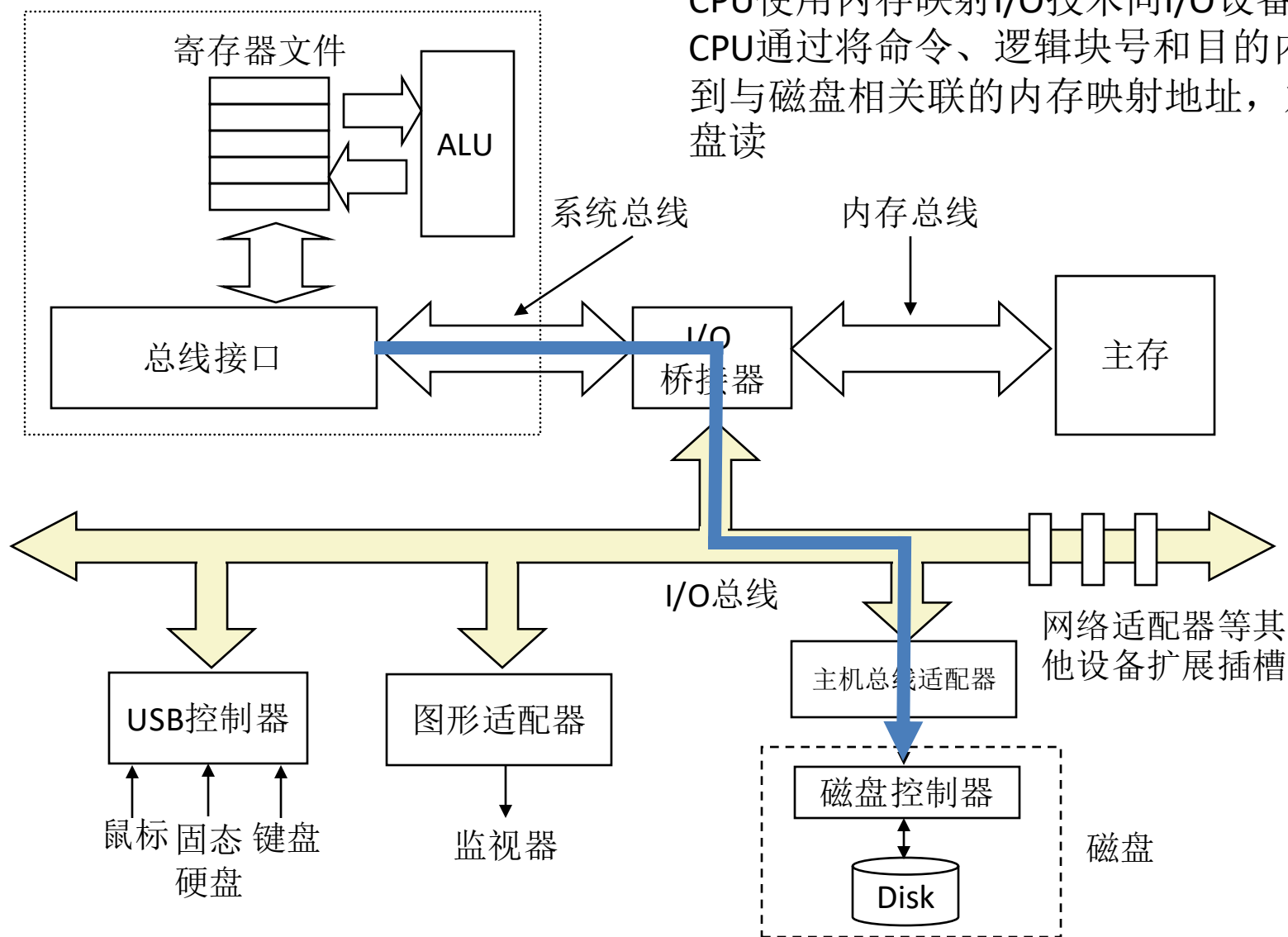


Figure 12: Optimized Layout for the OLAP8-63 Workload

Figure 16: Optimized Layouts of the TPC-H (h) and TPC-C (c) Objects for the Consolidated Workload on Homogeneous Storage Targets

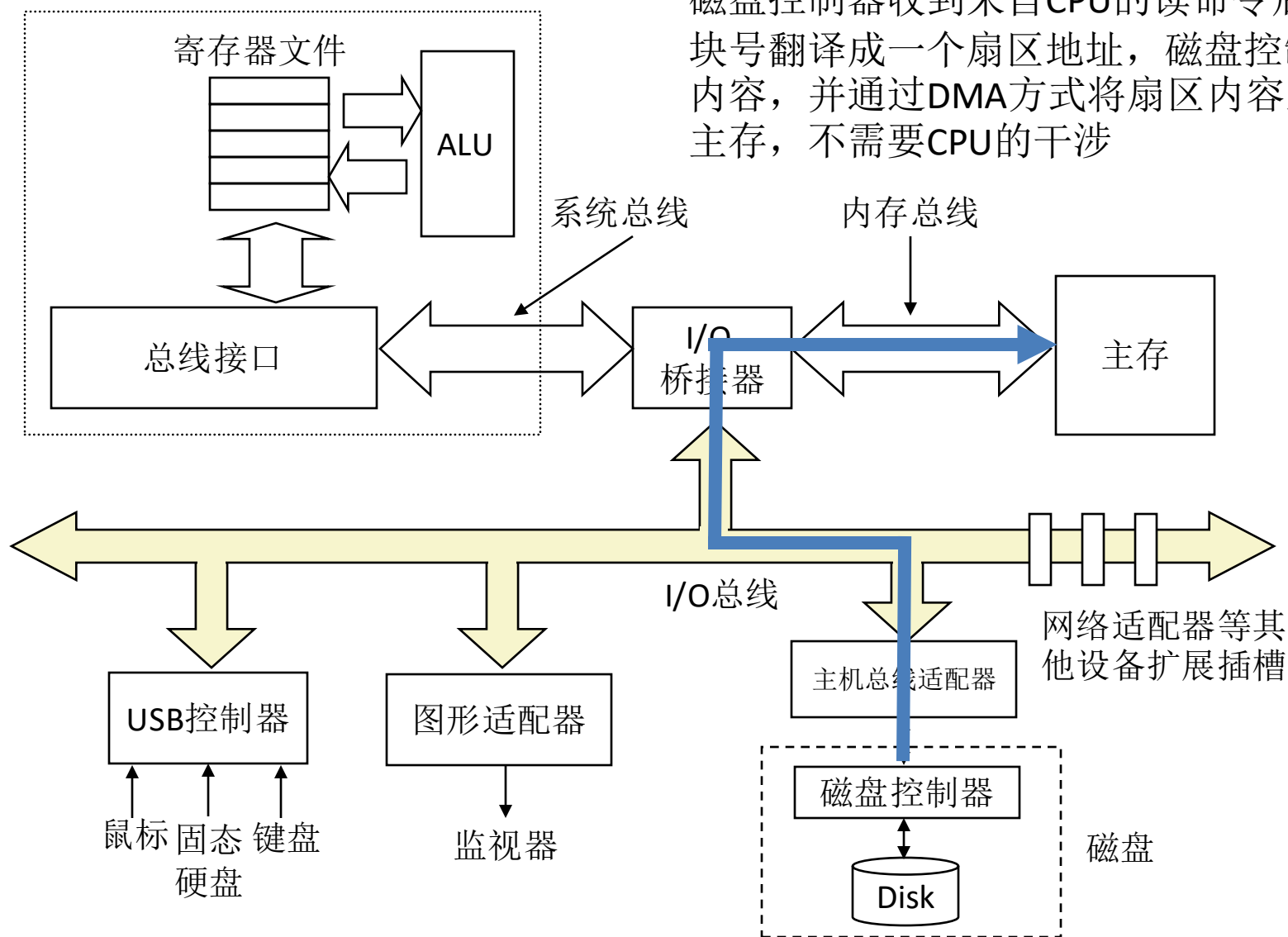
6. 访问磁盘——从磁盘读数据(1)

CPU使用内存映射I/O技术向I/O设备发射命令。
CPU通过将命令、逻辑块号和目的内存地址写到与磁盘相关联的内存映射地址，发起一个磁盘读



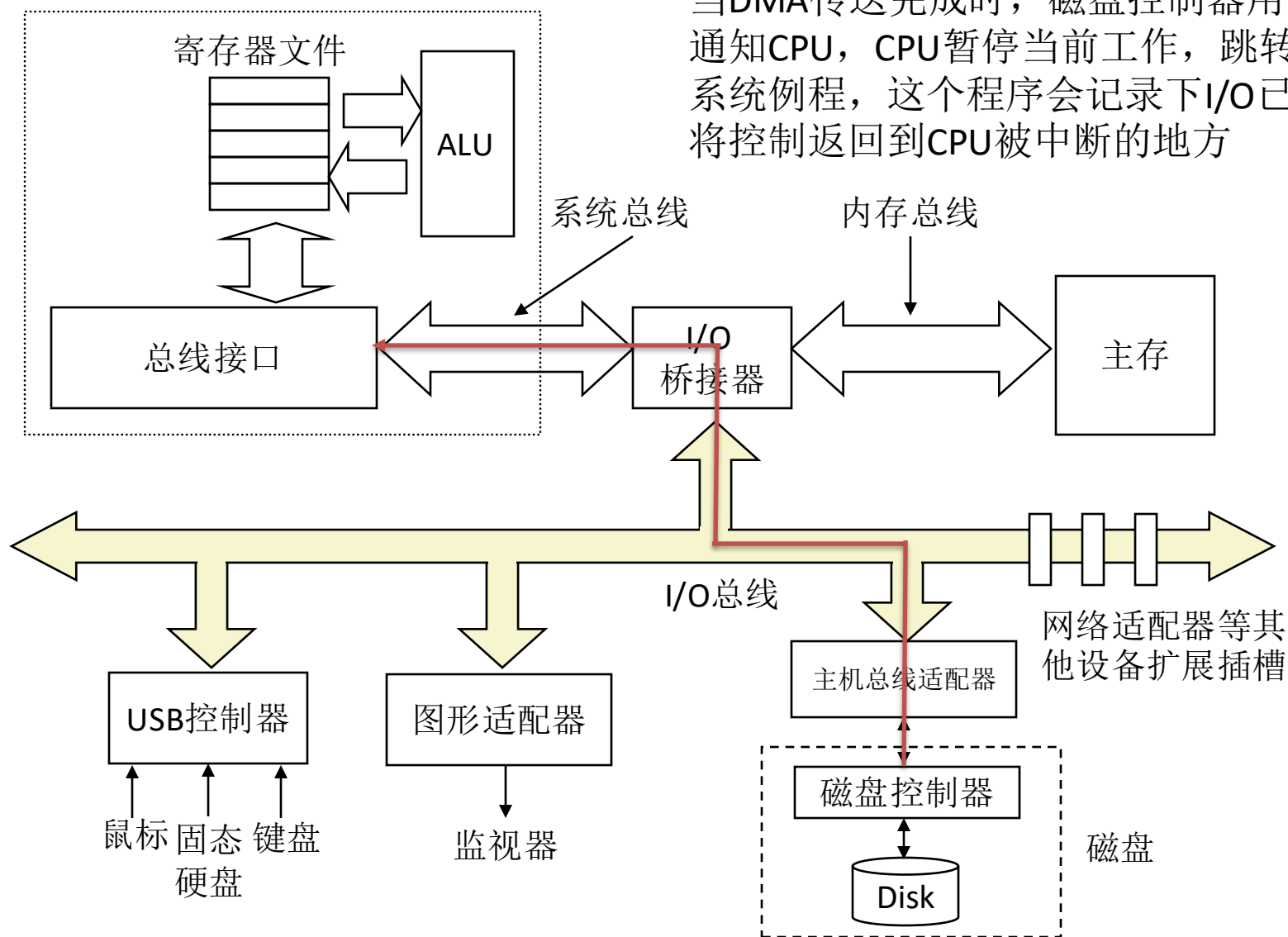
6. 访问磁盘——从磁盘读数据(2)

磁盘控制器收到来自CPU的读命令后，将逻辑块号翻译成一个扇区地址，磁盘控制器读扇区内容，并通过DMA方式将扇区内容直接传送到主存，不需要CPU的干涉



6. 访问磁盘——从磁盘读数据(3)

当DMA传送完成时，磁盘控制器用中断的方式通知CPU，CPU暂停当前工作，跳转到一个操作系统例程，这个程序会记录下I/O已完成，然后将控制返回到CPU被中断的地方



作业-1



- P451
- 6.23
 - 目标：掌握根据磁盘参数估算磁盘访问性能
- 6.24
 - 目标：掌握估算磁盘数据访问性能的方法，为磁盘访问优化算法设计提供决策依据

6.1.3 固态硬盘

固态硬盘是一种基于闪存的存储技术，SSD封装为M.2、PCIe、SATA、USB等接口插到I/O总线上



M.2接口SSD



PCIe接口SSD



Intel Optane系列SSD



SATA接口SSD



固态硬盘逻辑结构

闪存工作原理的核心是一种称为闪存单元的存储单元。闪存单元是由一个细小的晶体管和一個贮存电荷的栅极构成的。每个晶体管可以存储一个二进制位,也就是0或1。当电荷存储在栅极中时,晶体管的通路就被打开了,表示1。当栅极中没有电荷时,晶体管的通路就被关闭了,表示0。

I/O总线

读写逻辑磁盘块

固态硬盘(Solid State Disk, SSD)

闪存翻译层

闪存

Block 0

Page 0

Page 1

...

Page P-1

...

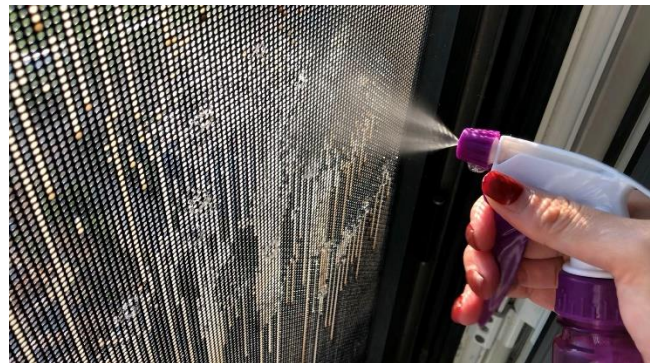
Block B-1

Page 0

Page 1

...

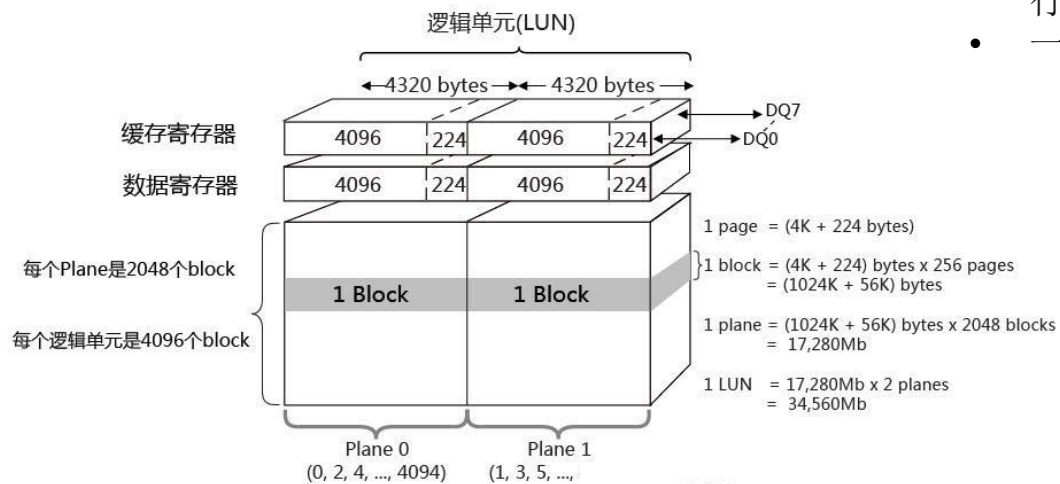
Page P-1



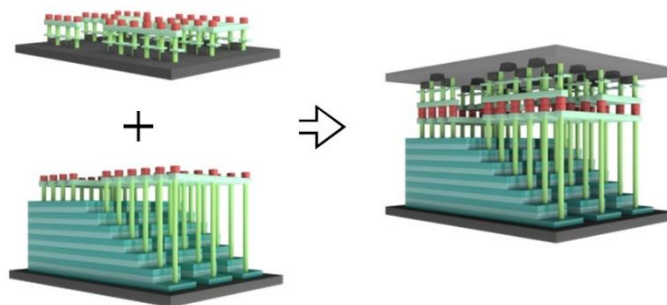
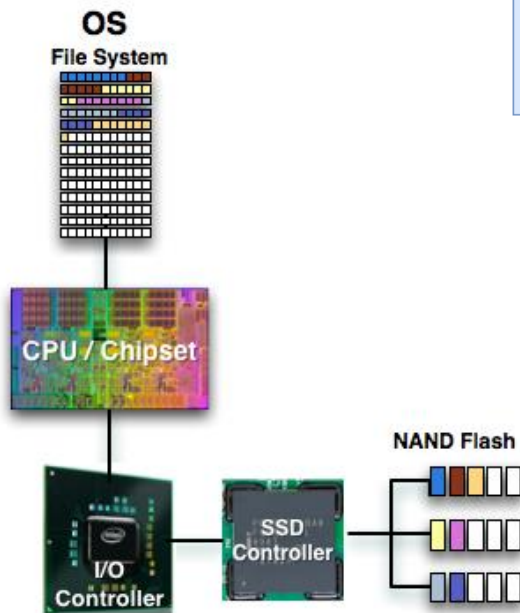
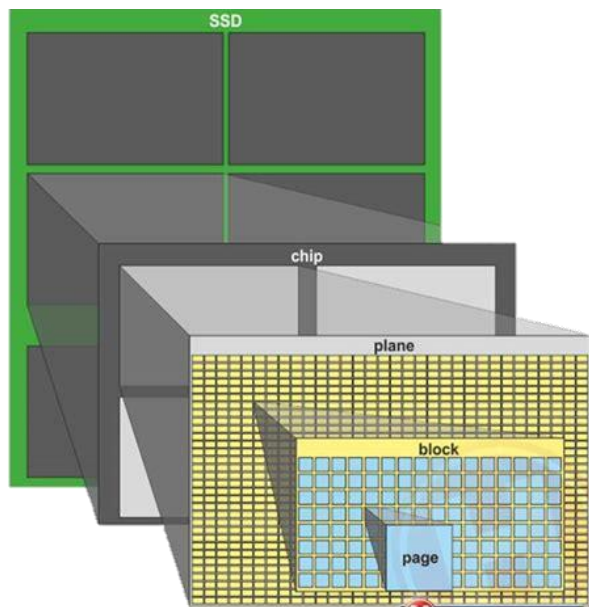
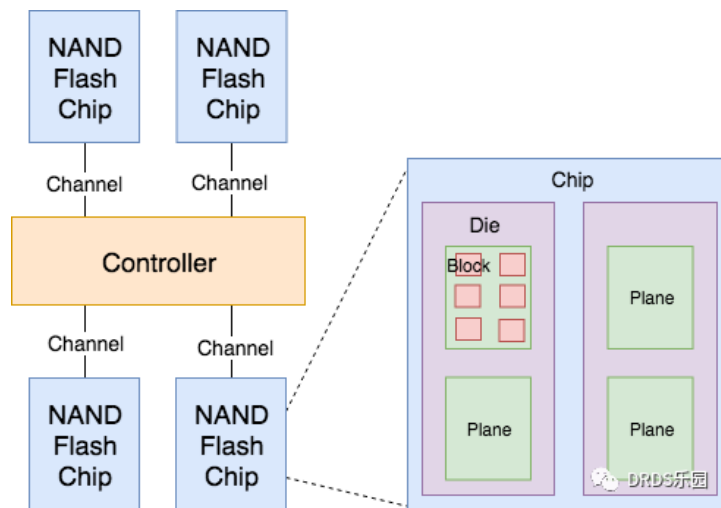
- 页大小: 512KB~4KB, 块: 32~128页
- 数据以页为单位读写
- 页所属的块被整个擦除后才能写该页(不支持原位置状态转换)
- 块磨损: 经过大约100000次重复写后, 块磨损坏, 不可用

SSD层次结构与访问路径

下图是个32Gb 25nm的MLC芯片：



- Die也被称作LUN（逻辑单元），也是闪存内可执行命令并返回自身状态的最小独立单元。
- 一个闪存Die可以拥有1到多个Plane面。



读		写	
顺序读吞吐量	550 MB/s	顺序写吞吐量	470 MB/s
随机读吞吐量(IOPS)	89,000 IOPS	随机写吞吐量(IOPS)	74,000 IOPS
随机读吞吐量(MB/s)	365 MB/s	随机写吞吐量(MB/s)	303 MB/s
平均顺序读访问时间	50 us	平均顺序写访问时间	60 us

A black Intel Optane SSD DC P4800X is shown. The drive is a 2.5-inch form factor with a black plastic casing. The Intel logo and "INTEL® OPTANE™ SSD DC P4800X" are printed on the top. A SATA-to-U.2 adapter is connected to the bottom, showing the U.2 interface pins.

<https://www.intel.cn/content/www/cn/zh/solid-state-drives/optane-ssd-dc-p4800x-brief.html>

SSD代表性产品

Intel® Optane™ SSD P5800X

What's New	
Next-gen Intel Optane Media	
PCIe 4.0	
Sequential Read	up to 7.2GB/s
Sequential Write	up to 6.2GB/s
Random 4k Read (IOPS)	up to 1.5M
Random 4k Write (IOPS)	up to 1.5M
Random 4k 70/30 (IOPS)	up to 1.8M
QoS (4KRR, QD=1, 99%)	<6us ²
QoS (4KRR, RW, Mixed QD=1, 99-999%)	<25us
4.6M 512B RR IOPS	
100 DDPD	

The World's Fastest Data Center SSD³



400/800GB
1.60/3.20TB
in U.2 form factor

Improvements vs Intel Optane SSD DC P4800X1 (PCIe gen 3)

3x	Greater Random 4k Mixed R/W IOPS
40%	Better QoS (4K Random Read, QD=1, 99%)
3x	Higher Seq 4K-128K Bandwidth (R/W)
67%	Higher endurance (1.6M writes per day)

The Micron 7500 SSD is the world's most advanced mainstream data center SSD



Micron 7500 NVMe SSD (U.3/U.2), 15mm

Micron 7500 SSD key specifications

Only Micron 232-layer NAND enables NV-LPDDR4

U.3		7500 PRO: U.3/U.2 Read-Intensive, 1 Drive Write per Day					7500 MAX: U.3/U.2 Mixed-Use, 3 Drive Writes per Day				
Capacity ¹²		960GB	1.92TB	3.84TB	7.68TB	15.36TB	800GB	1.6TB	3.2TB	6.4TB	12.8TB
Performance ¹³	Seq. Read (MB/s)	6,800	6,800	6,800	7,000	7,000	6,800	6,800	6,800	7,000	7,000
	Seq. Write (MB/s)	1,400	2,700	5,300	5,900	5,900	1,400	2,700	5,300	5,900	5,900
	Rand. Read (KIOPS)	800	1,000	1,100	1,100	1,100	800	1,000	1,100	1,100	1,100
	Rand. Write (KIOPS)	85	145	180	215	250	145	270	390	400	410
	70/30 Rand. Read/Write (KIOPS)	130	260	350	450	530	200	370	510	650	700
	Latency (TYP, µs) ¹⁴	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)	70 (read) 15 (write)
Endurance (total bytes written in TB) ¹⁵		1,752	3,504	7,008	14,016	28,032	4,380	8,760	17,520	35,040	70,080
Drive write cache enabled											

SSD的访问特性

- 随机写很慢：
 - 擦除块需要1ms级的时间，比访问页慢一个数量级
 - 写操作修改一个包含已经有数据的页 p ，这个块中所有带有数据的页必须被复制到一个新的(擦除过的)块，然后才能进行对页 p 的写——**株连法，为一个人修房子整个街道要异地搬迁**
 - 在闪存翻译层实现复杂逻辑，抵消擦除写块的代价，最小化内部写操作，但随机写性能低于读性能——**不得不搬，尽量少搬**
- SSD的优缺点：
 - 优点：由半导体存储器构成，没有移动部件，随机访问时间快于磁盘，能耗低，抗冲击能力强
 - 缺点：
 - 易磨损，通过闪存翻译层中的平均磨损逻辑将擦除平均分布在所有的块上最大化每个块的寿命
 - 成本高，每字节成本比磁盘贵约30倍，存储容量比磁盘小100倍
- 主要应用：便携存储设备、笔记本、高性能存储

Platform Storage Performance With 3D XPoint Technology

This paper reviews the potentialities on computing introduced by the 3-D XPoint technology in changing the memory-storage hierarchy.

By FRANK T. HADY, ANNIE FOONG, BRYAN VEAL, AND DAN WILLIAMS

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8003284>

Rethinking Database Algorithms for Phase Change Memory

Shimin Chen
Intel Labs Pittsburgh
shimin.chen@intel.com

Phillip B. Gibbons
Intel Labs Pittsburgh
phillip.b.gibbons@intel.com

Suman Nath
Microsoft Research
sumann@microsoft.com

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/pcm-db-algo-cidr11.pdf>

Table 1: Comparison of memory technologies.

	DRAM	PCM	NAND Flash	HDD
Read energy	0.8 J/GB	1 J/GB	1.5 J/GB [28]	65 J/GB
Write energy	1.2 J/GB	6 J/GB	17.5 J/GB [28]	65 J/GB
Idle power	~100 mW/GB	~1 mW/GB	1-10 mW/GB	~10 W/TB
Endurance	∞	$10^6 - 10^8$	$10^4 - 10^5$	∞
Page size	64B	64B	4KB	512B
Page read latency	20-50ns	~ 50ns	~ 25 μ s	~ 5 ms
Page write latency	20-50ns	~ 1 μ s	~ 500 μ s	~ 5 ms
Write bandwidth	~GB/s per die	50-100 MB/s per die	5-40 MB/s per die	~200MB/s per drive
Erase latency	N/A	N/A	~ 2 ms	N/A
Density	1x	2 - 4x	4x	N/A

Note: The table contents are based mainly on [10, 15, 22].

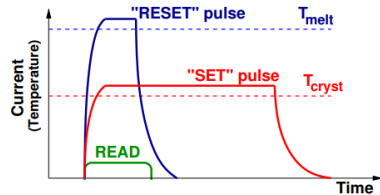


Figure 1: Currents and timings (not to scale) for SET, RESET, and READ operations on a PCM cell. For phase change material $Ge_2Sb_2Te_5$, $T_{melt} \approx 610^\circ C$ and $T_{cryst} \approx 350^\circ C$.

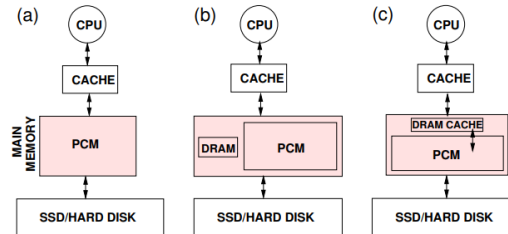


Figure 2: Candidate main memory organizations with PCM.

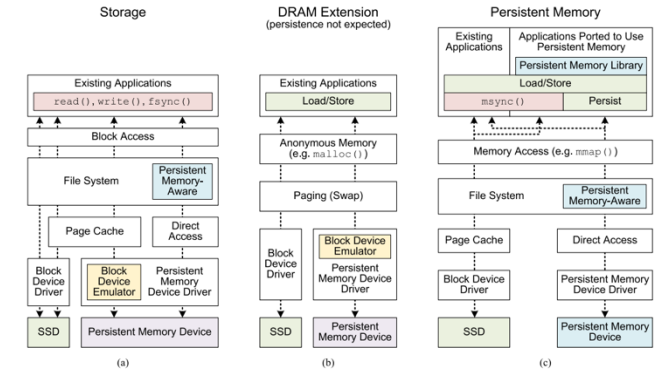


Fig. 1. Possible use models for SSDs and persistent memory devices: (a) as storage; (b) as DRAM extension; or (c) as persistent memory.

2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/ 12th IEEE International Conference On Big Data Science And Engineering

Reducing Relational Database Performance Bottlenecks Using 3D XPoint Storage Technology

Jinfeng Yang
University of Minnesota, Twin Cities
yang3116@umn.edu

David J. Lilja
University of Minnesota, Twin Cities
lilja@umn.edu

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8456140>

TABLE V: Read and Write Bandwidth (MB/s) at the Block Level.

	Seq Write	Rand Write	Seq Read	Rand Read
SSD	1865.3	1698.2	1066.1	92.3
Optane	2126.3	1892.9	2472.3	736.6

练习题6.5 估算SSD寿命

- 假定SSD能够经得起128PB的写，根据工作负载估算SSD寿命 *差距不大，但磁盘的差距大*

A. 顺序写最糟情况：以470MB/s速度持续写SSD

B. 随机写最糟情况：以303MB/s速度持续写SSD

C. 平均情况：以20GB/天速度写SSD

- 两种情况的估算寿命：

A. 顺序写最糟情况寿命：
$$\frac{128 \times 10^9}{470 \times 60 \times 60 \times 24 \times 365} \approx 8.64 \text{年}$$

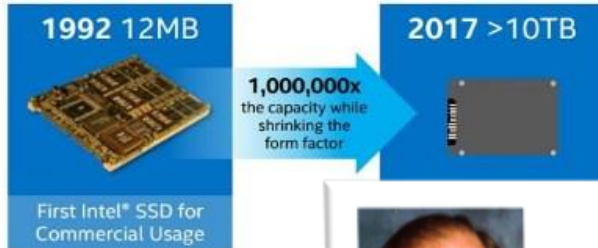
B. 随机写最糟情况寿命：
$$\frac{128 \times 10^9}{303 \times 60 \times 60 \times 24 \times 365} \approx 13.4 \text{年}$$

C. 平均情况寿命：
$$\frac{128 \times 10^9}{20000 \times 365} \approx 17534.25 \text{年}$$

– SSD的实际使用寿命通常都是大大超过官方标称值的

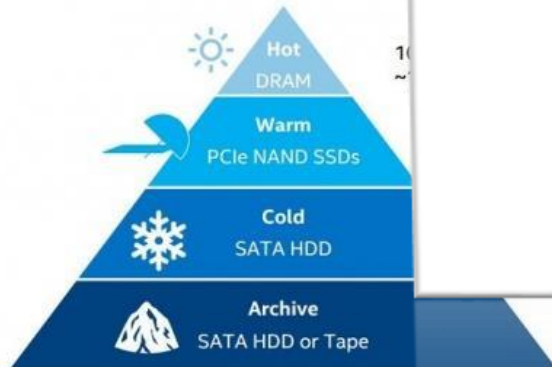
6.1.4 存储技术趋势

Moore's Law Continues to Disrupt the Computing Industry



24 Source: Intel projections on SSD capacity

Storage and Memory Hierarchy To

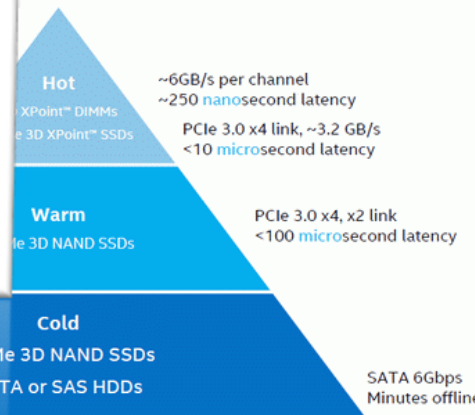


Tape is Dead
Disk is Tape
Flash is Disk
RAM Locality is King

Jim Gray
Microsoft
December 2006

Row

DRAM: 10GB/s per channel, ~100 nanosecond latency



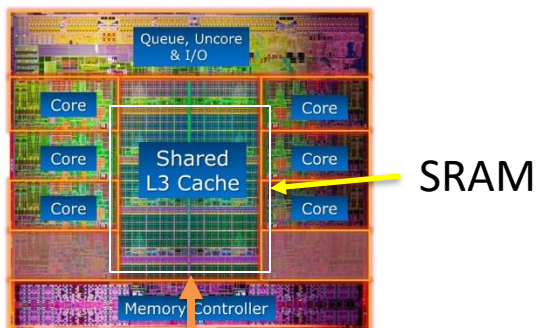
IDF15
INTEL DEVELOPER FORUM

Low cost archive

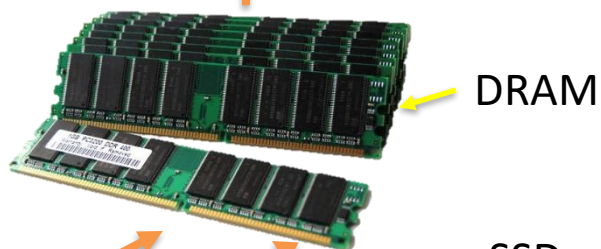
Comparisons between memory technologies based on in-market product specifications and internal Intel specifications.

IDF15
INTEL DEVELOPER FORUM

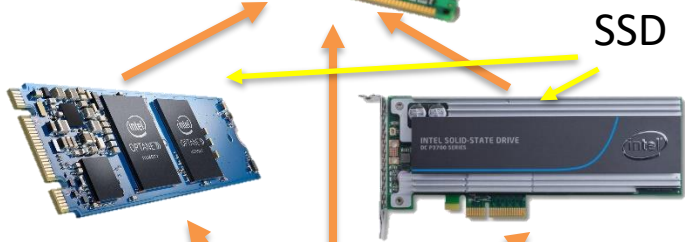
各级存储技术及各级缓存



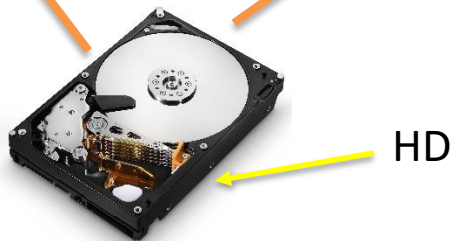
SRAM



DRAM



SSD



HD

各级存储设备作为当前存储和下一级缓存

Exadata Scale-Out Platform

Complete Platform for Running Database – Servers, Storage, Networking



Fully Redundant

Scale-Out Database Servers

- 8x 2-socket servers 256 GB DRAM each or
- 2x 8-socket servers 2 TB DRAM each

Scale-Out Intelligent Storage Servers

- 14x 2-socket storage servers
- 168 disk drives – 500TB
- 56 Flash PCI cards – 22 TB Flash

High Speed Network

- Unified InfiniBand internal connectivity (40 Gb/sec)
- 10Gb or 1Gb Data Center connectivity

ORACLE

© Copyright © 2013, Oracle and/or its affiliates. All rights reserved. |

Oracle Exadata采用大容量PCIe Flash作为高速缓存

存储技术发展趋势

SRAM

Metric	1985	1990	1995	2000	2005	2010	2015	2015:1985
\$/MB	2,900	320	256	100	75	60	25	116
access (ns)	150	35	15	3	2	1.5	1.3	115

性能和单位成本容量同步提升

DRAM

Metric	1985	1990	1995	2000	2005	2010	2015	2015:1985
\$/MB	880	100	30	1	0.1	0.06	0.02	44,000
access (ns)	200	100	70	60	50	40	20	10
typical size (MB)	0.256	4	16	64	2,000	8,000	16,000	62,500

单位成本容量极大提升，性能增长缓慢

Disk

Metric	1985	1990	1995	2000	2005	2010	2015	2015:1985
\$/GB	100,000	8,000	300	10	5	0.3	0.03	3,333,333
access (ms)	75	28	10	8	5	3	3	25
typical size (GB)	0.01	0.16	1	20	160	1,500	3,000	300,000

单位成本容量极大提升，性能增长缓慢

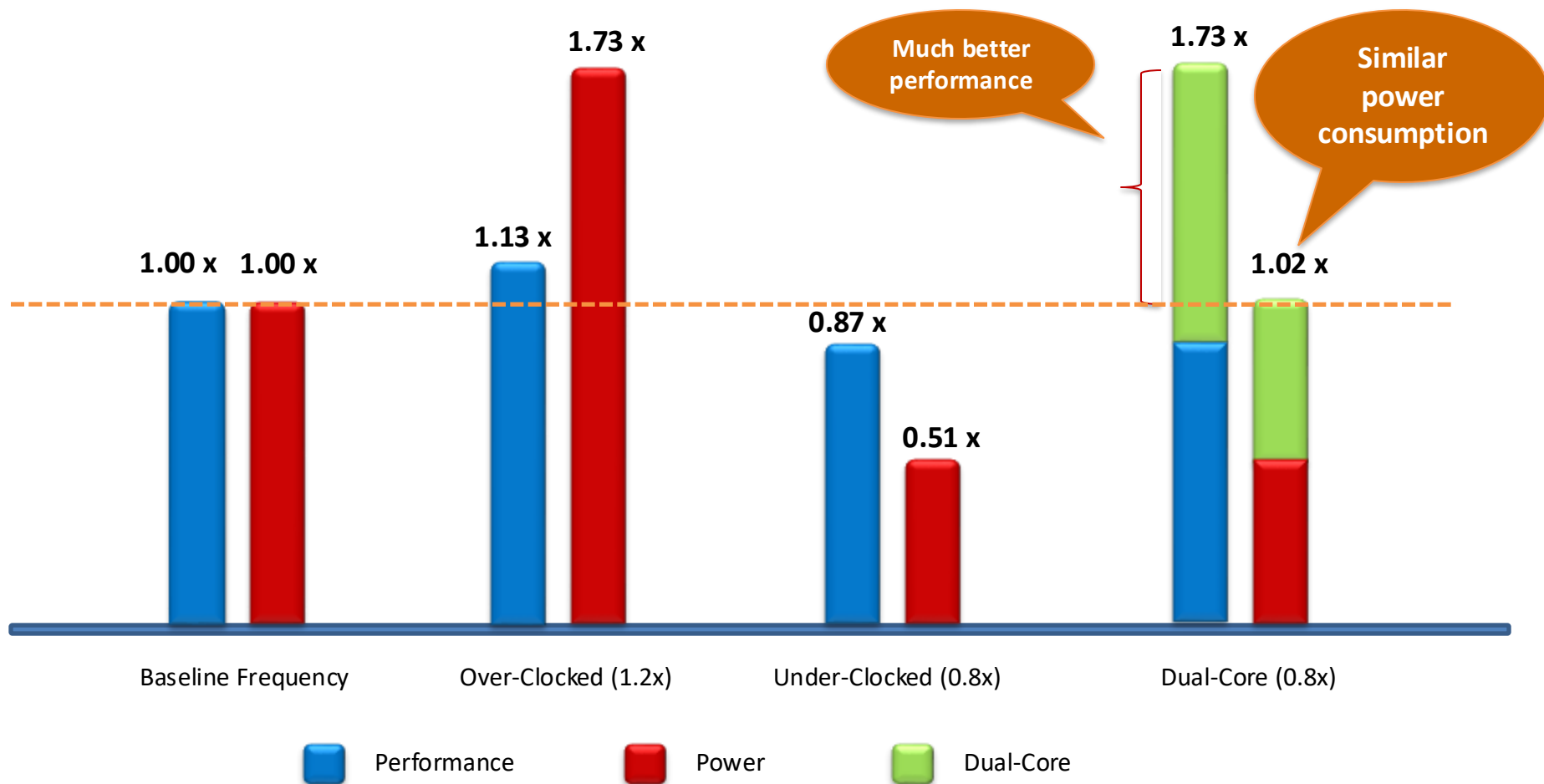
CPU Clock Rates

多核元年，处理器技术发展遇到“Power Wall”瓶颈

	1985	1990	1995	2003	2005	2010	2015	2015:1985
CPU	80286	80386	Pentium	P-4	Core 2	Core i7(n)		Core i7(h)
Clock rate (MHz)	6	20	150	3,300	2,000	2,500	3,000	500
Cycle time (ns)	166	50	6	0.30	0.50	0.4	0.33	500
Cores	1	1	1	1	2	4	4	4
Effective cycle time (ns)	166	50	6	0.30	0.25	0.10	0.08	2,075

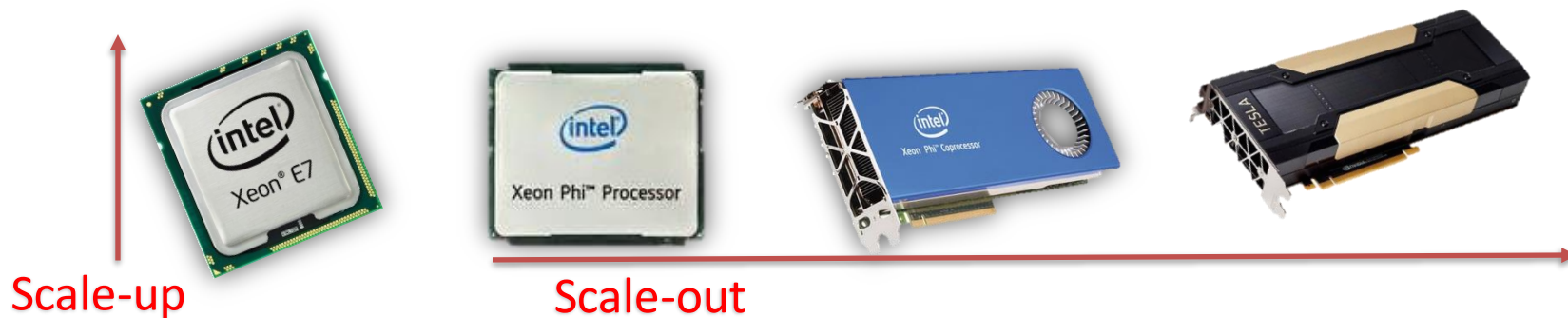
(n) Nehalem processor (h) Haswell processor

多核处理器出现的能耗因素



R.M. Ramanathan, *Intel Multi-Core Processors: Making the Move to Quad-Core and Beyond*, white paper

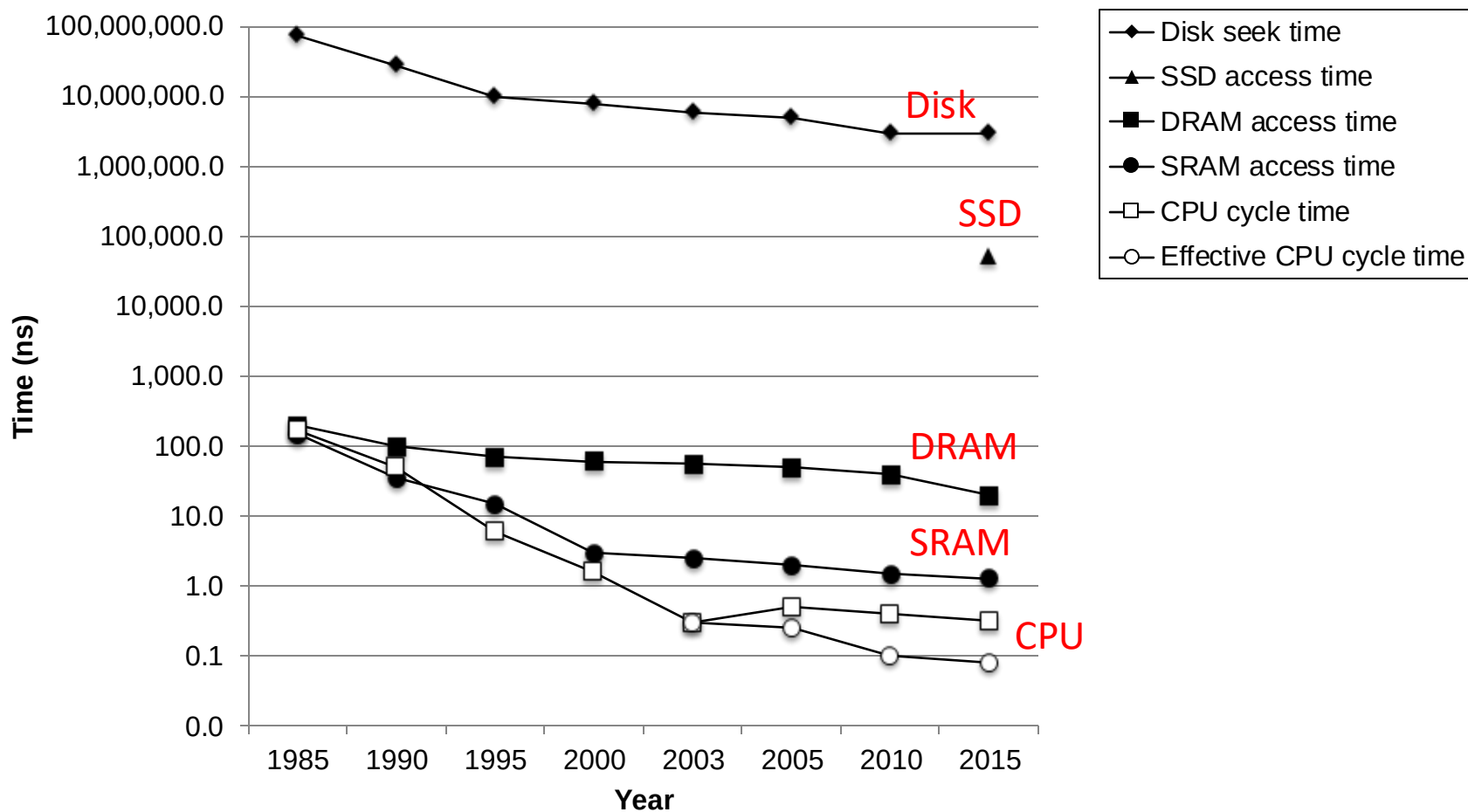
处理器技术发展



类型	Xeon Platinum 8180	Xeon Phi7290F	Xeon Phi 7250	NVIDIA Tesla V100
核心数量/线程数量	28/56	72/288	68/272	5120 CUDA Cores/640 Tensor Cores
主频	3.6 GHz	1.50 GHz	1.3 GHz	1.455 GHz
最大内存容量	1.5 TB	384GB	16GB	16 GB HBM2 VRAM
缓存容量	38.5 MB L3	36 MB L2/16 GB HBM	34 MB L2	6 MB L2
内存类型	DDR4-2666/6通道	DDR4-2400/6通道	MCDRAM/16通道	HBM2
最大内存带宽	85GB/s	115.2 GB/s	500 GB/s	900 GB/s
价格	\$10015.00	\$3368.00	\$3324.00	\$69,000

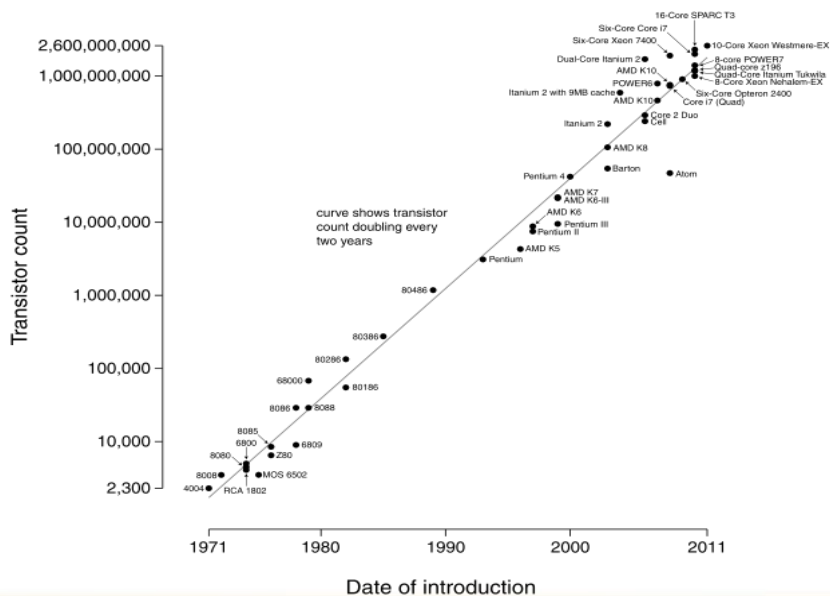
The CPU-Memory Gap

DRAM, 磁盘和CPU之间的性能差距不断加大，多核处理器并发向DRAM和磁盘发请求

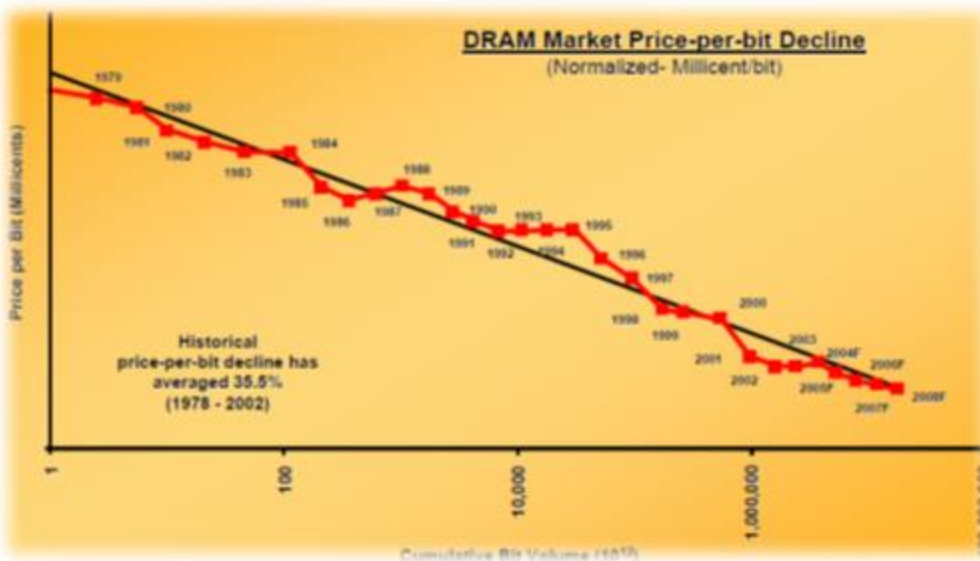
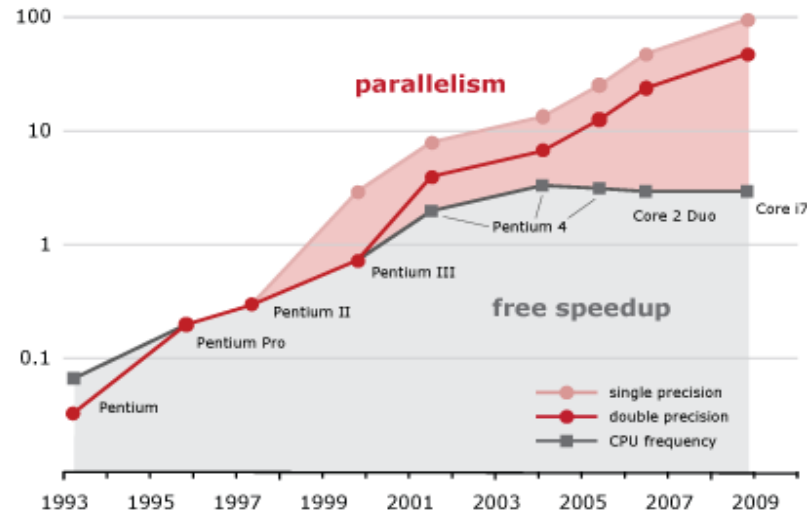


集成度、多核并行、内存墙

Microprocessor Transistor Counts 1971-2011 & Moore's Law



Floating point peak performance [Gflop/s]
CPU frequency [GHz]



练习题6.6 磁盘价格估算

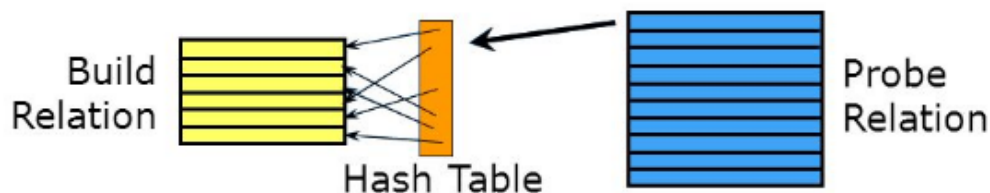
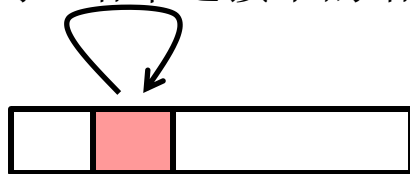
- 基于2005-2015年数据，估算哪一年可以以\$500的价格买到1PB的磁盘？
 - 1PB磁盘\$500对应的单位存储价格为0.0005\$/GB
 - 2005至2015年间价格变化相对稳定，作为基准参考价格
 - 取2010、2015两阶段平均价格递减倍数13.33
 - 估算后几个5年间磁盘单位容量价格，其中2025年约为0.00017\$/GB，即在2025年之前可能达到\$500的价格买1PB磁盘的需求

							价格	容量(GB)	\$/GB	
							500	1000000	0.0005	
Year	1985	1990	1995	2000	2005	2010	2015	2020	2025	2030
\$/GB	100000	8000	300	10	5	0.3	0.03	0.00225	0.00017	0.00001
递减倍数		12.50	26.67	30.00	2.00	16.67	10.00			
						平均倍数	13.33			

6.2 局部性

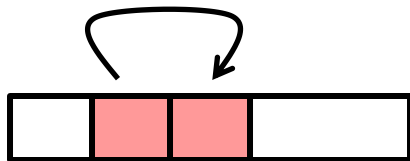
- 局部性(locality): 倾向于引用邻近于其他最近引用过的数据项的数据项, 或者最近引用过的数据项
- 局部性原理(principle of locality): 数据访问具有局部性原理
- 局部性的两种形式:
 - 时间局部性(temporal locality): 被引用过远的将来再被多次引用

- 示例: 哈希连接中的哈希表



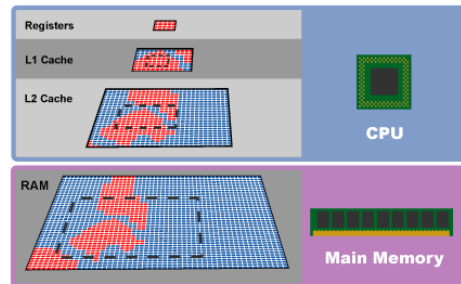
- 空间局部性(spatial locality): 一个内存位置被引用了一次, 程序很可能在不远的将来引用附近的一个内存位置

- 示例: 数组顺序扫描, CPU一次读取64字节cache line, prefetch预取操作

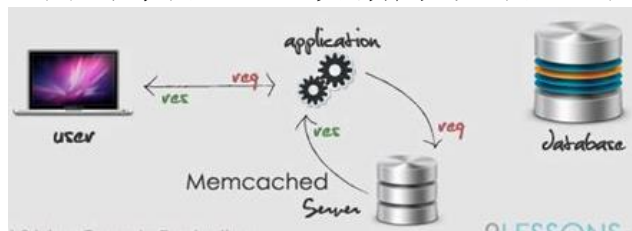
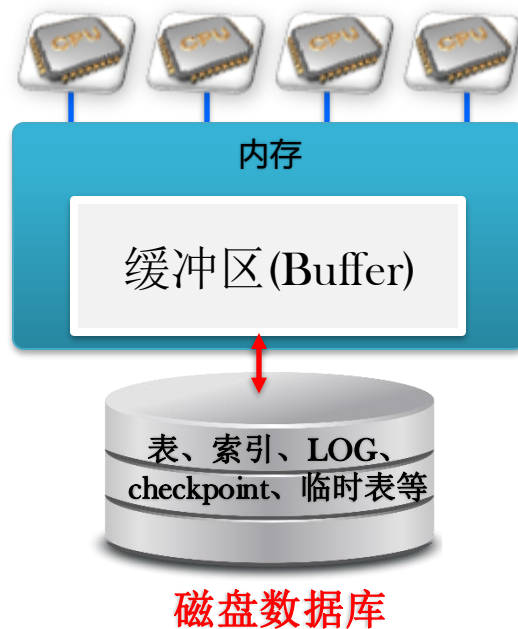


局部性原理的应用

- 硬件级：
 - Cache用于缓存最近被引用的指令和数据，提高对内存访问速度
 - 硬盘配置几十MB缓存，优化磁盘访问
 - SSD配置缓存，优化读写效率
- 操作系统级：
 - 使用主存缓存磁盘文件最近被使用的磁盘块
- 数据库级：
 - 数据库缓冲区(buffer pool)：缓存数据库数据页
- 系统级：
 - Memcached、Redis分布式缓存系统：分布式内存对象缓存系统，用于动态Web应用以减轻数据库负载。它通过在内存中缓存数据和对象来减少读取数据库的次数，从而提高动态、数据库驱动网站的速度



磁盘与SSD中的缓存芯片



6.2.1 对程序数据引用的局部性

- 示例：向量元素求和局部性分析

```
Int sumvec(int v[N]){  
    int i, sum=0;  
    for (i=0; i<N; i++)  
        sum+=v[i];  
    return sum;  
}
```

- 数据访问局部性：
 - 向量元素v[i]:
 - 步长为1的顺序访问，空间局部性好
 - 无重复访问，时间局部性差
 - 变量sum:
 - 每次循环迭代中引用一次，时间局部性好
 - Sum是标量，无空间局部性
- 指令访问局部性：
 - 顺序执行指令，空间局部性好
 - 循环执行指令，时间局部性好

多维数组引用局部性

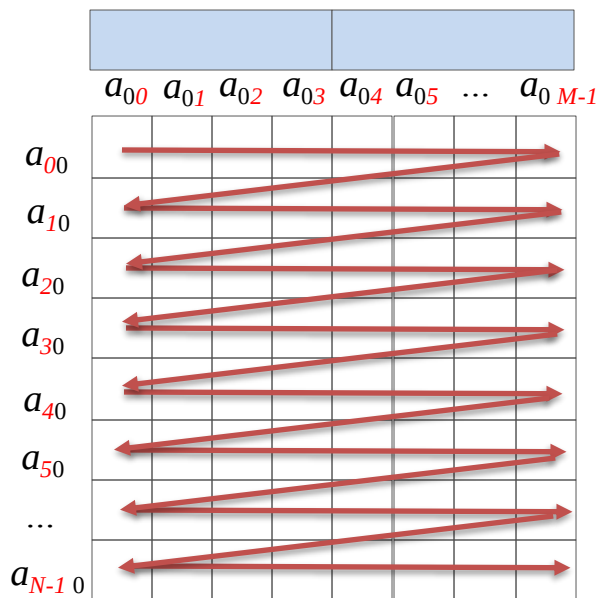
- 多维数组行优化顺序与列优化顺序具有不同的局部性

```
int sumarrayrows(int a[M][N]){
    int i,j,sum=0;
    for (i=0;i<M;i++)
        for (j=0;j<N;j++)
            sum+=a[i][j];
    return sum;
}
```

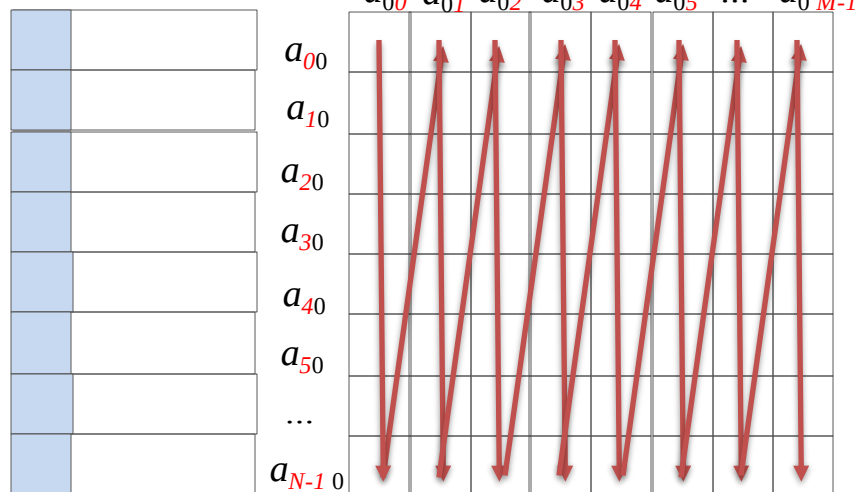
```
int sumarraycols(int a[M][N]){
    int i,j,sum=0;
    for (j=0;j<N;j++)
        for (i=0;i<M;i++)
            sum+=a[i][j];
    return sum;
}
```

```
[nsrc@localhost ~]$ gcc -o 6.18 6.18.c -O3
[nsrc@localhost ~]$ 6.18
sumarray by rows is 65536, cycles_per_elem: 1.0
sumarray by cols is 65536, cycles_per_elem: 2.1
```

Cache line Cache line

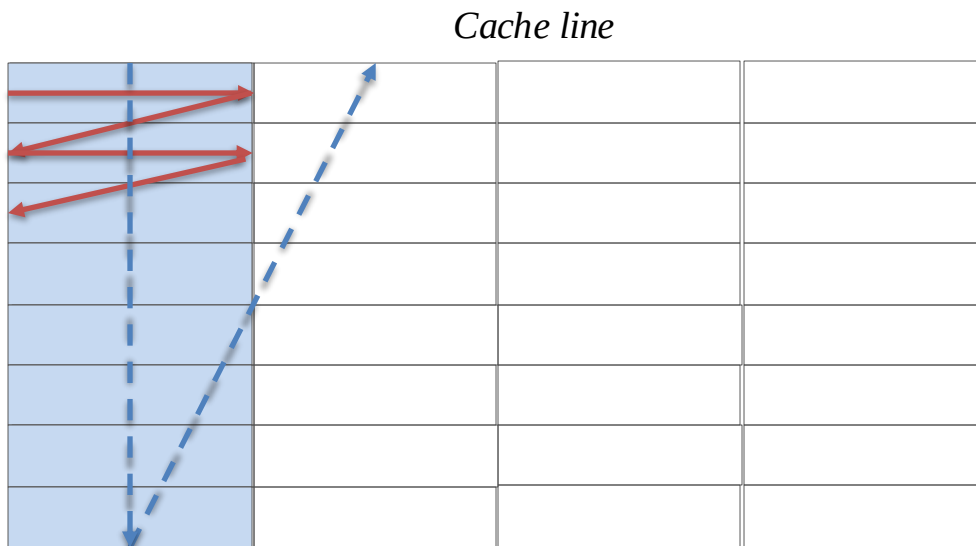


Cache line





- 多维数组引用局部性扩展技术研究
 - 基于cache line（64字节）粒度的列访问
 - 要求：
 - 数组可以使用不同宽度的数据类型
 - 以cache line为粒度水平方向访问，然后再垂直方向访问
 - 对比sumarrayrows(), sumarraycols(), sumarraycacheline()三种算法的性能



分析不同数据访问顺序数据局部性

- 设置M=N=1024，测试两种多维数组访问顺序cache性能
- 两种算法执行时间差异：

```
[nsrc@localhost ~]$ 6.18
Input R for row, C for col:R
sumarray by rows is 1048576, cycles_per_elem: 0.9
[nsrc@localhost ~]$ 6.18
Input R for row, C for col:C
sumarray by cols is 1048576, cycles_per_elem: 3.1
```

- 测试方案：
 - 增加输入控制语句，选择执行行访问或列访问求和
 - 通过perf命令测试L1-data-load-miss指标
 - perf stat -e L1-dcache-load-misses ./6.18

```
[nsrc@localhost ~]$ perf stat -e L1-dcache-load-misses ./6.18
Input R for row, C for col:R
sumarray by rows is 1048576, cycles_per_elem: 0.9

Performance counter stats for './6.18':

          188,773      L1-dcache-load-misses

          1.639574629 seconds time elapsed

[nsrc@localhost ~]$ perf stat -e L1-dcache-load-misses ./6.18
Input R for row, C for col:C
sumarray by cols is 1048576, cycles_per_elem: 2.9

Performance counter stats for './6.18':

          381,655      L1-dcache-load-misses

          1.908932360 seconds time elapsed
```

结论：
行访问比列访问方式的**L1-dcache-load-misses**值更低，程序性能更高

6.2.2 取指令的局部性

- 程序指令放在内存中，CPU需要读出指令
- For循环体内的指令按照连续的内存顺序执行
 - 循环有良好的空间局部性
 - 循环体被执行多次，循环有良好的时间局部性
 - 测试：通过L1 icache指令cache指标分析指令局部性
 - perf stat -e L1-icache-load-misses ./6.18

```
[nsrc@localhost ~]$ perf stat -e L1-icache-load-misses ./6.18
Input R for row, C for col:R
sumarray by rows is 1048576, cycles_per_elem: 0.9

Performance counter stats for './6.18':

      8,802      L1-icache-load-misses

      1.616458165 seconds time elapsed

[nsrc@localhost ~]$ perf stat -e L1-icache-load-misses ./6.18
Input R for row, C for col:C
sumarray by cols is 1048576, cycles_per_elem: 2.7

Performance counter stats for './6.18':

      9,614      L1-icache-load-misses

      1.374691916 seconds time elapsed
```

结论：
行访问比列访问方式的**L1-icache-load-misses**值更低，指令局部性略高

- 代码执行时不能被修改，程序执行时CPU只从内存中读指令，很少重写或修改指令

6.2.3 局部性小结

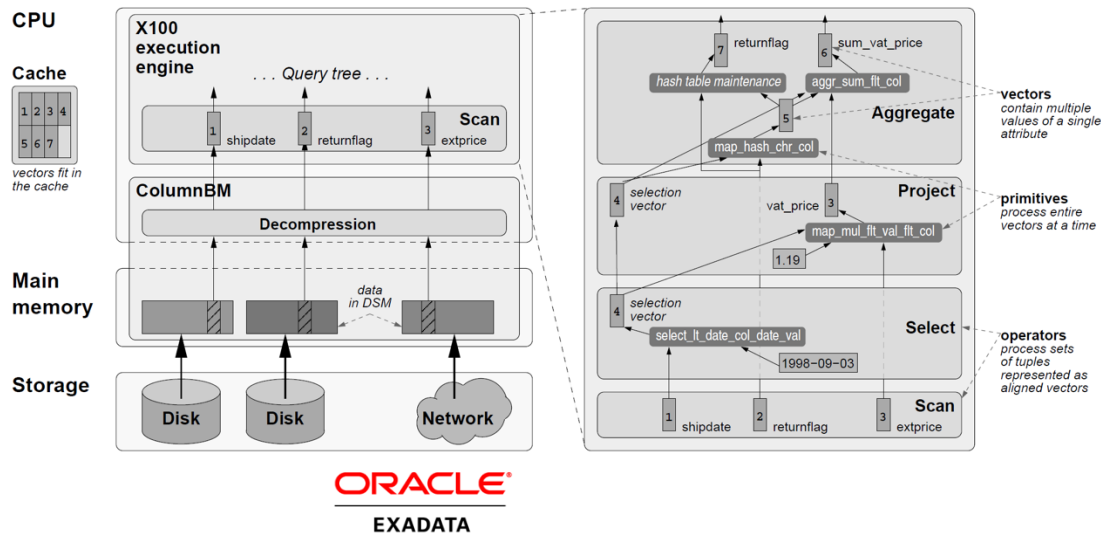
- 评价程序中局部性的基本原则：
 - 重复引用相同变量的程序有良好的时间局部性
 - 频繁访问数据集的缓存级别：寄存器>L1 cache>L2 cache>L3 cache>内存
 - 步长为k的引用模式程序，步长越小，空间局部性越好
 - CPU内存访问单位cache line(64字节)，步长越小，效率越高
 - 对于取指令而言，循环有好的时间和空间局部性，循环越小，循环迭代次数越多，局部性越好
 - 简化循环体内代码结构

Numbers Everyone Should Know

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns

扩展知识

- Vectorized execution
 - MonetDB/X100 - A DBMS In The CPU Cache
 - MonetDB/X100: Hyper-Pipelining Query Execution



Exadata Smart Flash Cache Hardware

Exadata systems use the latest PCI flash technology rather than flash disks. PCI flash greatly accelerates performance by placing flash directly on the high speed PCI bus rather than behind slow disk controllers and directors. Each Exadata Storage Server includes 4 PCI flash cards with a total capacity of 3.2 TB of flash memory. A Full Rack Exadata Database Machine includes 56 PCI flash cards providing 44.8 TB of flash memory. The flash modules used in Exadata X4 have an expected endurance of 10 years or more for typical database data. This solid state storage delivers dramatic performance advantages with Exadata storage. It implements automatic caching of database reads and writes and can do over 1,960,000 8K Flash Writes per second and 2,660,000 8K Flash Reads per second in a full rack X4 database machine.



An Oracle White Paper
December 2013

Exadata Smart Flash Cache Features and the Oracle Exadata Database Machine

<http://www.oracle.com/technetwork/database/exadata/exadata-smart-flash-cache-366203.pdf>

Sun Flash Accelerator F80 PCIe Card

练习题6.7

- 改变函数中循环的顺序，使得它以步长为1的引用模式扫描三维数组

```
int sum_array_3d(int a[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += a[k][i][j];
    return sum;
}
```



```
int sum_array_3d(int a[N][N][N])
{
    int i, j, k, sum = 0;

    for (k = 0; k < N; k++)
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                sum += a[k][i][j];
    return sum;
}
```

练习题6.8 分析三个函数的空间局部性



- Clear2>clear1>clear3

```
#define N 1024
#define M 256
typedef struct{
    int vel[M];
    int acc[M];
}point;
point p[N];
```

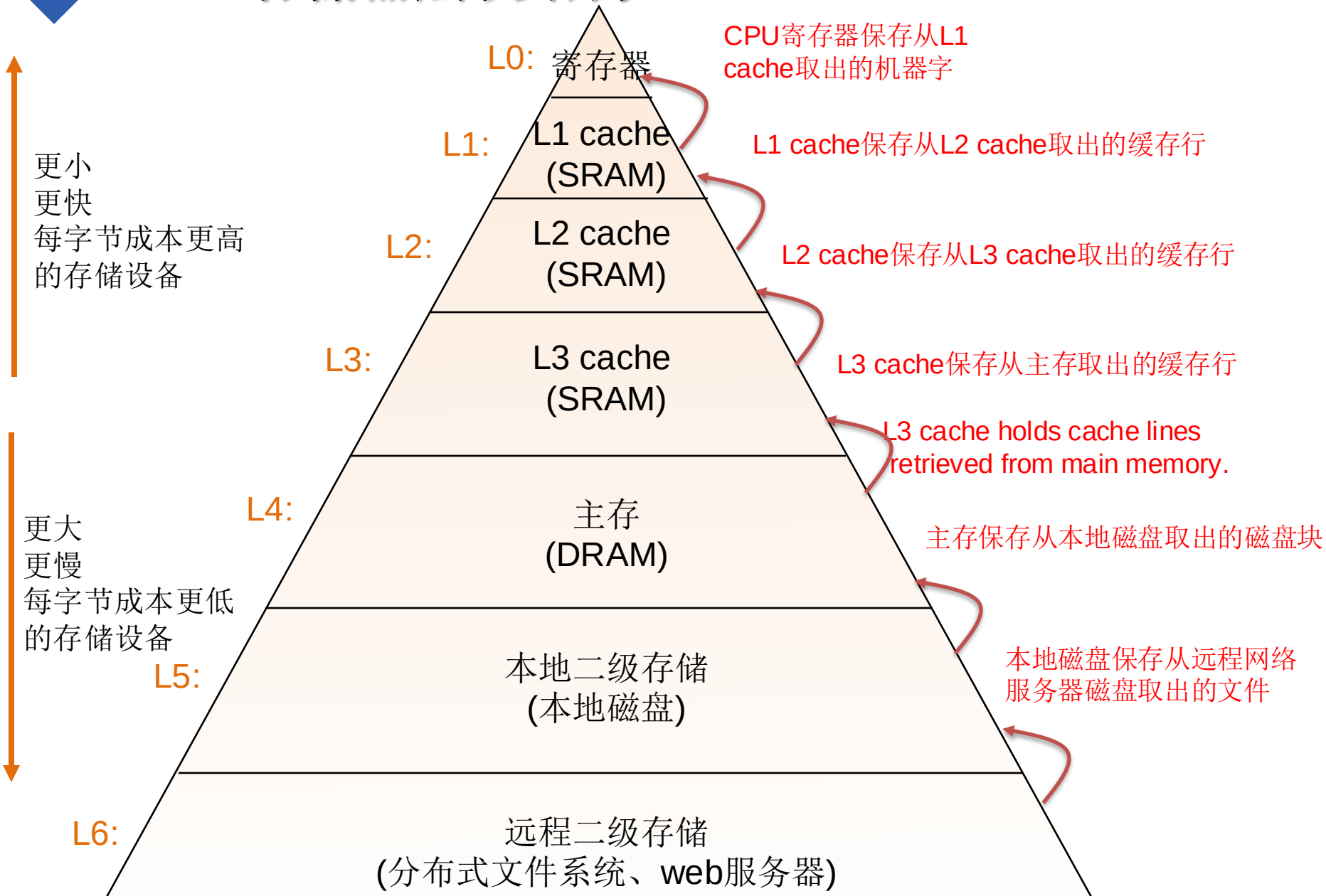
```
void clear1(point *p, int n){
    int i,j;
    for (i=0;i<n;i++){
        for (j=0;j<M;j++)
            p[i].vel[j]=0;
        for (j=0;j<M;j++)
            p[i].acc[j]=0;
    }
    return;
}
```

```
void clear2(point *p, int n){
    int i,j;
    for (i=0;i<n;i++){
        for (j=0;j<M;j++){
            p[i].vel[j]=0;
            p[i].acc[j]=0;
        }
    }
    return;
}
```

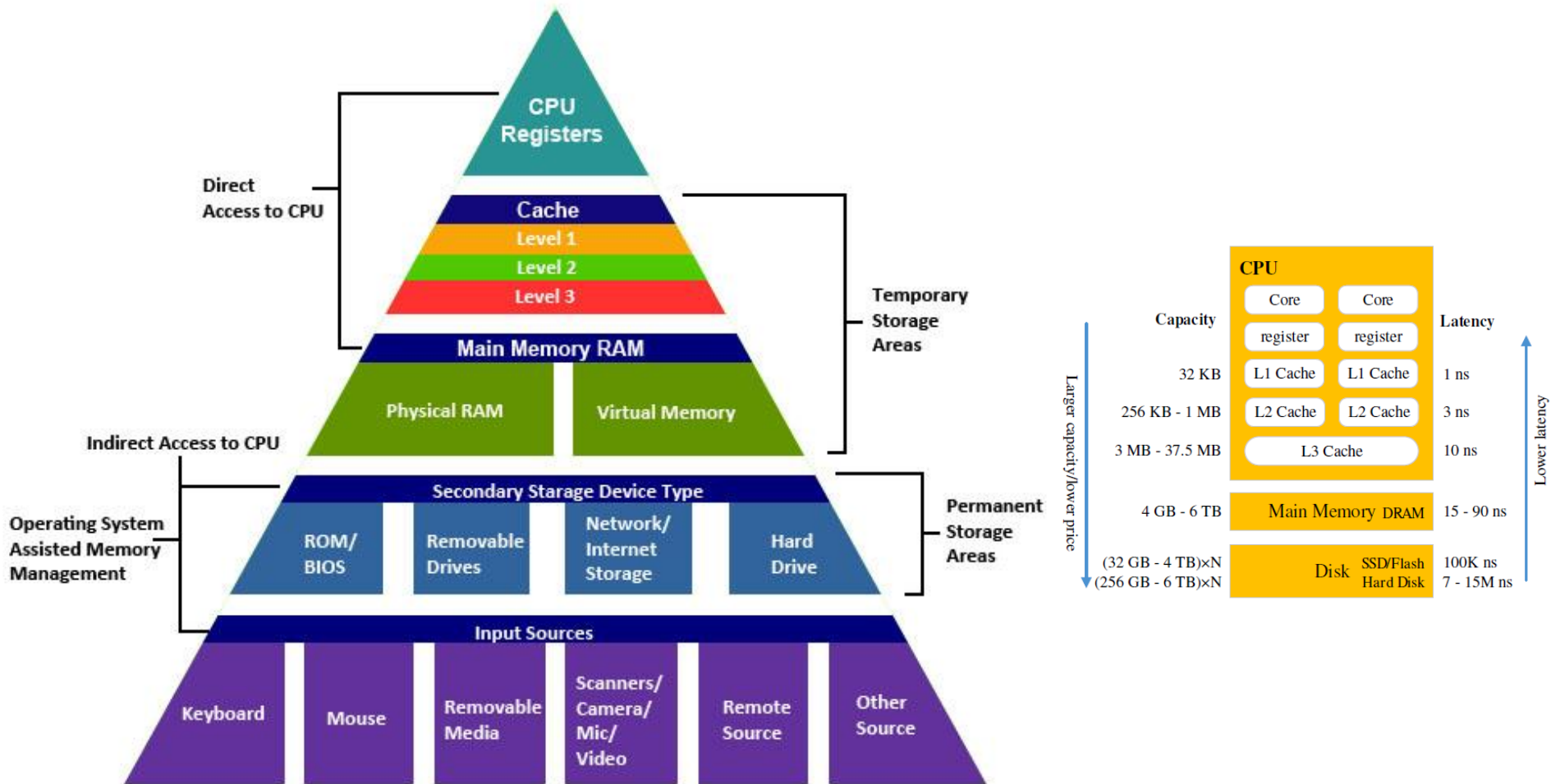
```
void clear3(point *p, int n){
    int i,j;
    for (j=0;j<M;j++){
        for (i=0;i<n;i++)
            p[i].vel[j]=0;
        for (i=0;i<n;i++)
            p[i].acc[j]=0;
    }
    return;
}
```

```
[nsrc@localhost ~]$ gcc -o test6.8 test6.8.c -O3
[nsrc@localhost ~]$ test6.8
clear1 function cycles_per_elem:12.8
clear2 function cycles_per_elem: 2.7
clear3 function cycles_per_elem:49.1
```

6.3 存储器层次结构



存储层访问特征



NVM存储变革: memory & storage

Performance

LATENCY IN: MILLISECONDS MICROSECONDS NANOSECONDS

DRAM

23

INTEL DIMMs WITH
3D XPOINT™ TECHNOLOGY

112

INTEL® OPTANE™ SSD

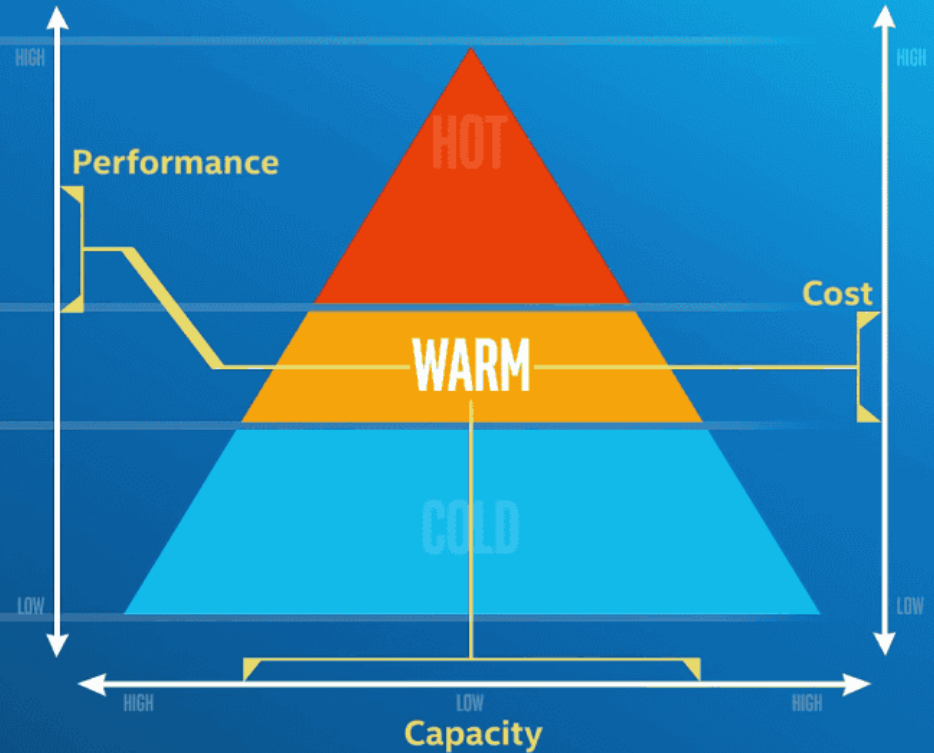
4,379

NAND SSD

78,291

HARD DRIVE

48,913,280



6.3.1 存储器多层次结构中的缓存

- 高速缓存(cache): 是一个小而快速的存储设备, 作为存储在更大、更慢下一级设备中的数据对象的缓存区域
- 缓存(caching): 使用高速缓存的过程
- 存储器层次的中心思想:
 - 对于每个 k , 位于 k 层的更快更小的存储设备作为位于 $k+1$ 层的更大更慢存储设备的缓存
- 存储层次的工作原理:
 - Power law(80/20原则), $k+1$ 层数据中部分数据属于频繁访问的热数据, 大部分数据属于不频繁访问的冷数据
 - 程序局部性特点倾向于访问 k 层的数据, 速度快于 $k+1$ 层数据访问
 - $k+1$ 层数据访问更慢, 但可以提供更大的容量
- **重要思想:** 存储层次创建一个大的存储池, 成本接近底层存储, 性能接近上层存储

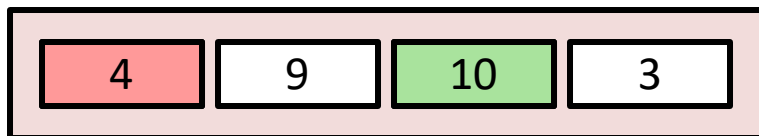
缓存的一般概念

- 第 $k+1$ 层存储器被划分为连续的数据对象组块，每个块都有一个唯一的地址或名字，块可以是固定大小，也可以可变大小
- 第 k 层被划分成较小的块的集合，**每个块的大小与第 $k+1$ 层一样**，第 k 层缓存包含第 $k+1$ 层块的一个子集的副本

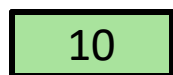


不同物理设备
向谁兼容？

第 k 层



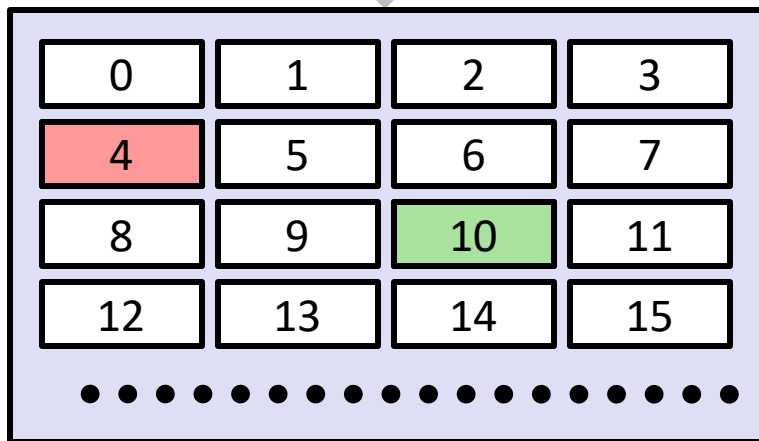
第 k 层更小、更快、更昂贵的设备缓存着第 $k+1$ 层块的一个子集



数据以块为大小传统单元
在层与层之间复制

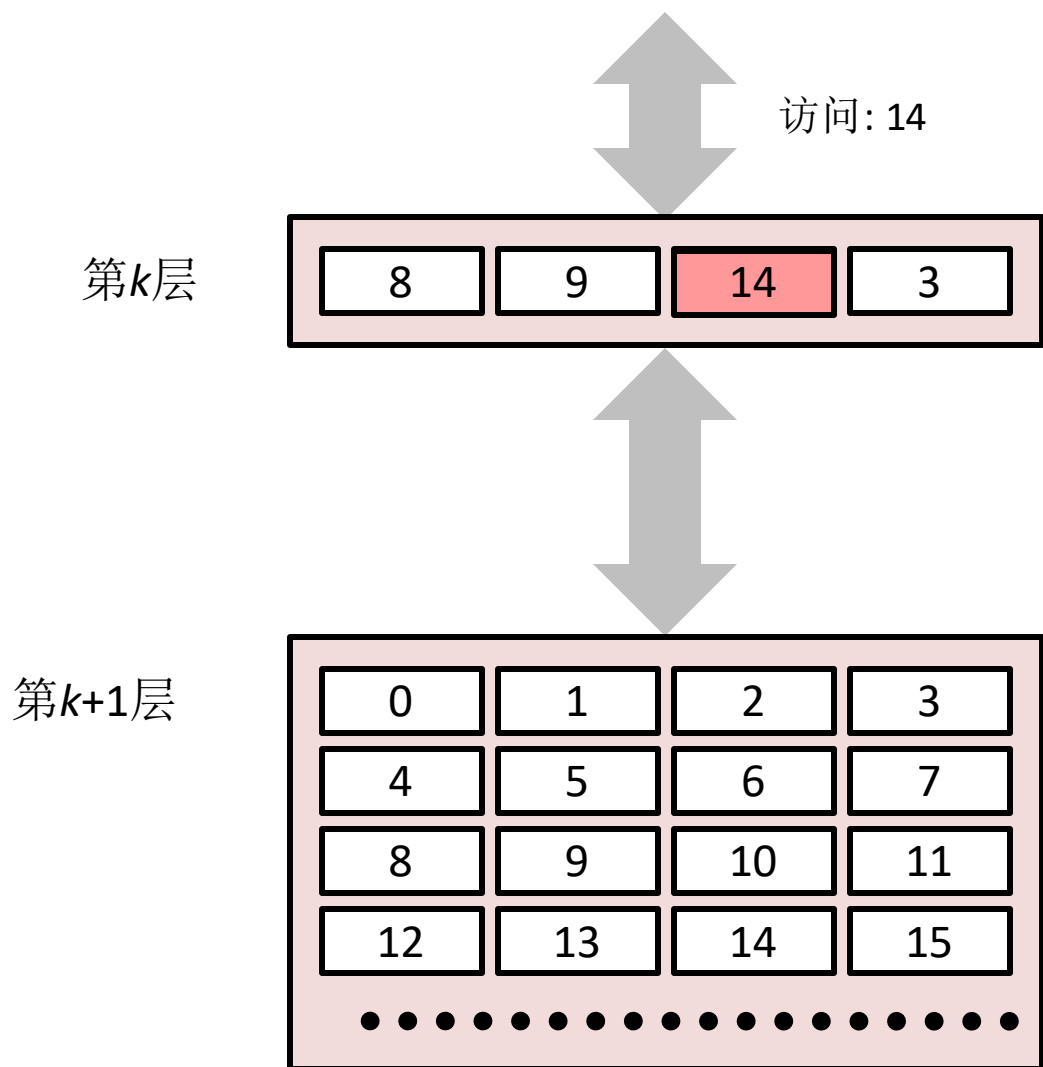
Anti-caching

第 $k+1$ 层



第 $k+1$ 层更大、更慢、更便宜的设备被划分成块

1. 缓存命中

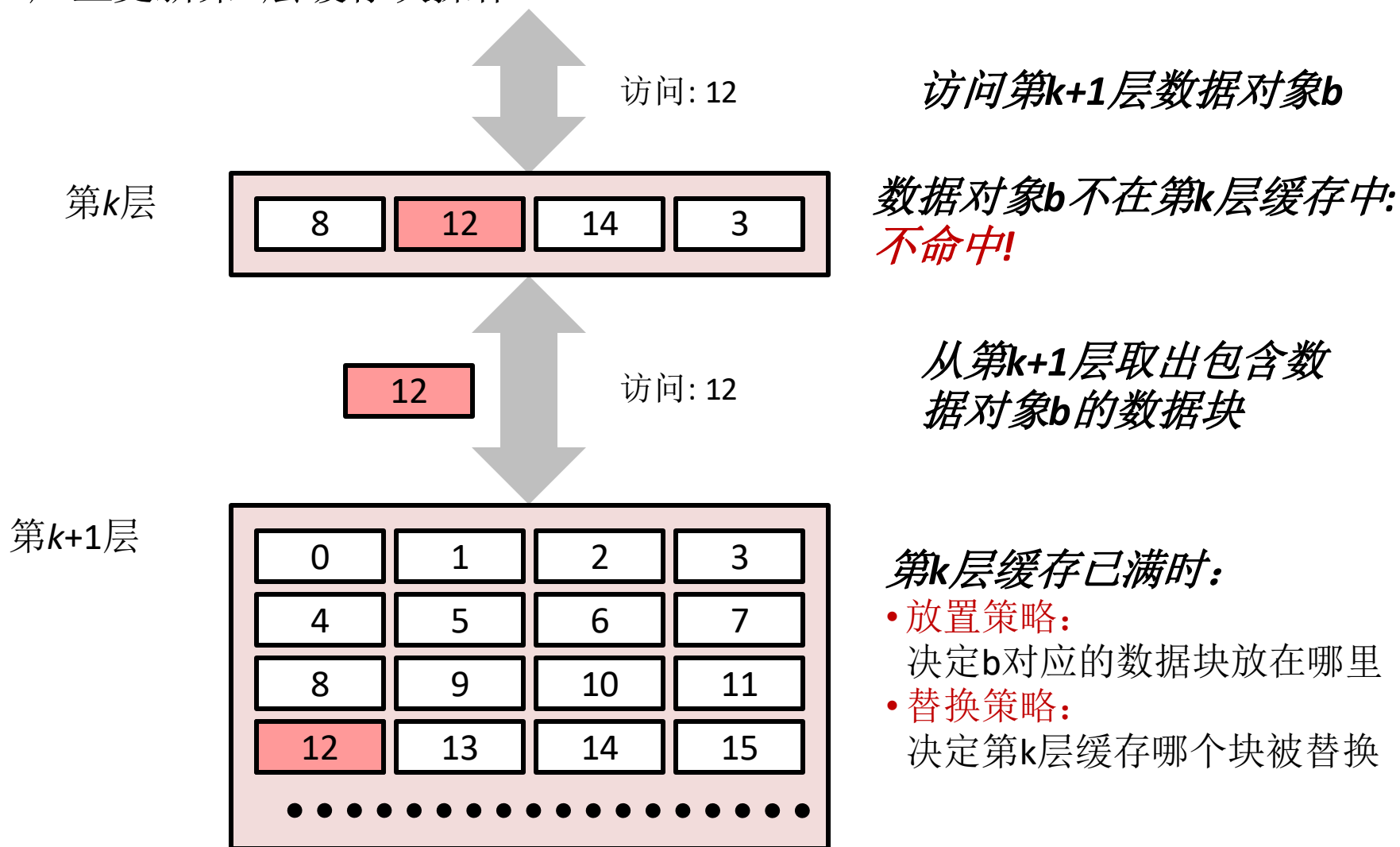


访问第k+1层数据块d

数据块缓存在第k层中:
命中!

2. 缓存不命中

- 在第k层产生cache miss，驱动从第k+1层取数据块并替换到第k层中
- 产生更新第k层缓存块操作

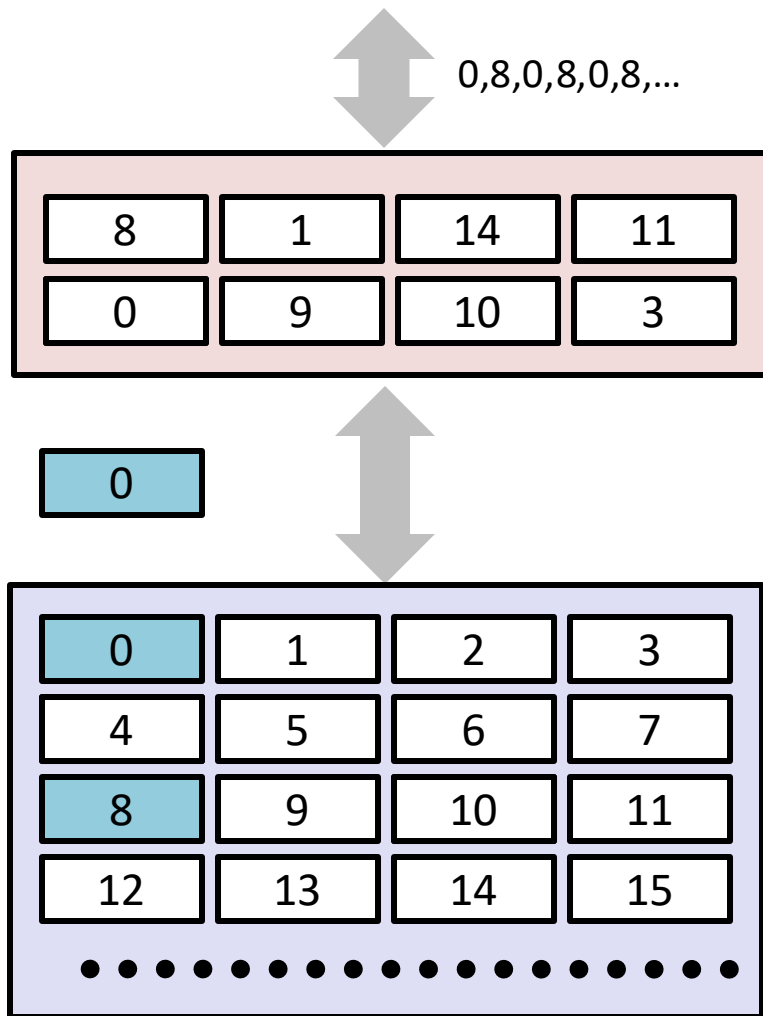


3. 缓存不命中的种类

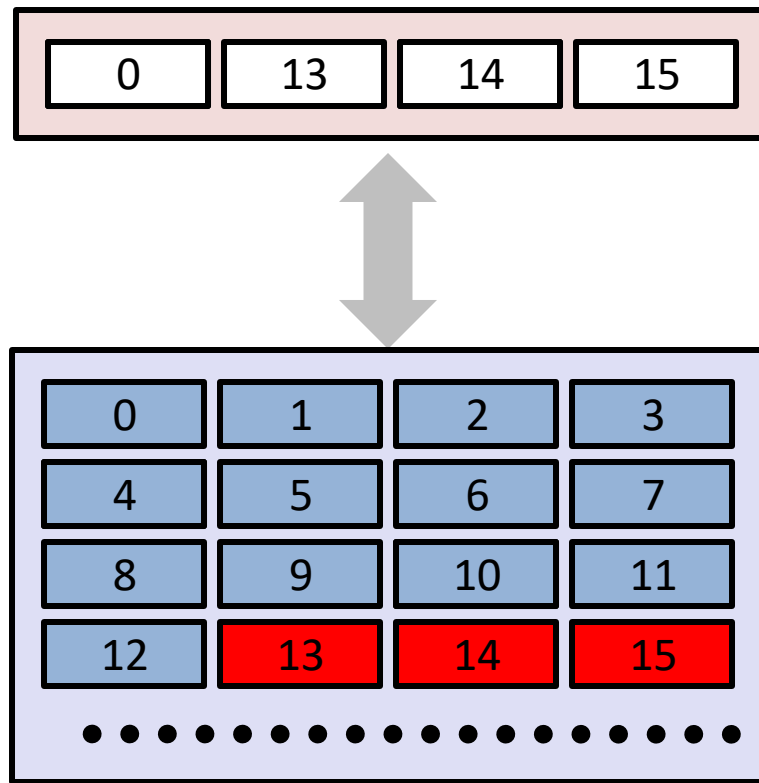
- 强制不命中(compulsory miss)或冷不命中(cold miss)
 - 缓存为空时产生的不命中，是暂时事件，缓存暖身(warm up)之后消失
- 冲突不命中(Conflict miss)
 - 缓存放置策略(placement policy): 通常将第k+1层的某个块限制放在第k层块的一个小的子集中(有时是一个块)
 - 例: 第k+1层的块i必须放在第k层的块($i \bmod 4$)中
 - 冲突不命中(Conflict misses)指在限制性的放置策略中缓存足够大，能够保存被引用的数据对象，但因为这些对象会映射到同一个缓存块，缓存会一直不命中
 - 例: 程序请求块0, 8, 0, 8, 0, 8, ...，在第k层的缓存中，对这两个块的每次引用都会不命中
- 容量(Capacity miss)
 - 当工作集大小超过缓存大小时产生的不命中

思考:

- 访问序列为0,8,0,8,0,8,...
– 一层vs. 多层



- 强局部性(13,14,15)与弱局部性(蓝色, 一次性访问)性混合数据集



4. 缓存管理

- 缓存管理：缓存划分成块，在不同层之间传送块，判定是否命中，并处理它们
 - 缓存管理可以是硬件、软件，或二者结合
- 不同层缓存管理方法
 - 寄存器文件：编译器管理，决定发生不命中时何时发射加载，以及确定哪个寄存器来存放数据
 - L1、L2、L3 cache：由内置在缓存中的硬件逻辑管理
 - DRAM主存：在有虚拟内存的系统中由操作系统软件和CPU上的地址翻译硬件共同管理
 - 本地磁盘：在分布式文件系统中，由运行在本地机器上的客户端进程管理
 - **大多情况下，缓存都是自动运行的，不需要程序采取特殊的或显式的行动**

6.3.2 存储器层次结构概念小结

- 基于缓存的存储器层次结构有效的基础是性价比(较慢的存储设备比较快的存储设备便宜)和程序的局部性
 - 时间局部性：同一数据对象可能被多次访问，将具有时间局部性的数据加载到缓存能够加速后续对该数据对象重复访问时的性能
 - 核心：确定hot数据集
 - 空间局部性：块通常包含多个数据对象，如cache line大小为64个字节，基于空间局部性，访问一个字节的的数据导致包含该字节的cache line加载到缓存，当访问后续63个字节时全部在缓存中命中
 - 核心：通过顺序访问提高cache效率
- 基本优化技术：
 - Cache-conscious算法设计：
 - 基于cache line大小划分数据存储结构：CSB+Tree, CSTree等
 - 基于cache访问特性算法设计：radix-partitioning hash join等
 - 硬件升级：
 - KNL Phi处理器16GB configurable cache

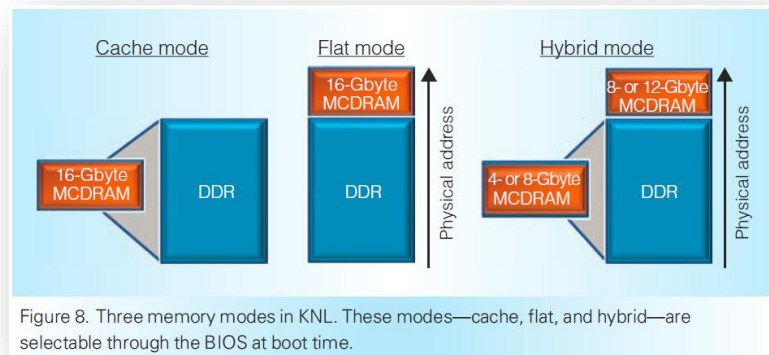
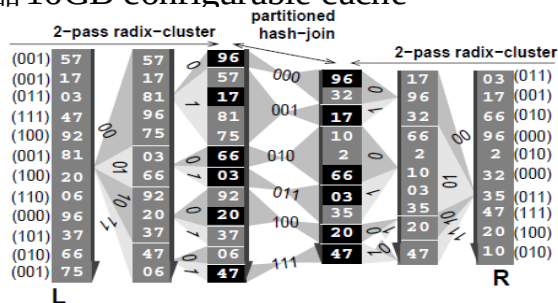
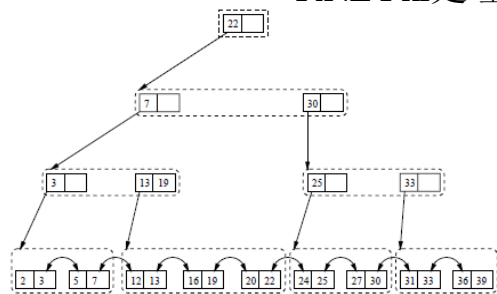


Figure 8. Three memory modes in KNL. These modes—cache, flat, and hybrid—are selectable through the BIOS at boot time.

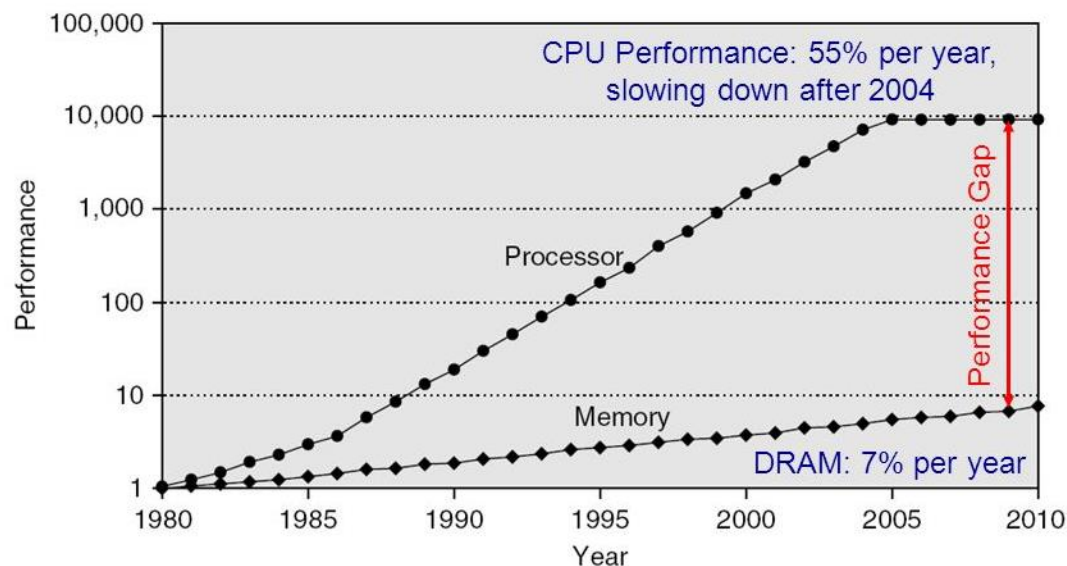
计算机系统中的缓存

Cache Type	What is Cached?	Where is it Cached?	Latency (cycles)	Managed By
Registers	4-8 bytes words	CPU core	0	Compiler
TLB	Address translations	On-Chip TLB	0	Hardware MMU
L1 cache	64-byte blocks	On-Chip L1	4	Hardware
L2 cache	64-byte blocks	On-Chip L2	10	Hardware
Virtual Memory	4-KB pages	Main memory	100	Hardware + OS
Buffer cache	Parts of files	Main memory	100	OS
Disk cache	Disk sectors	Disk controller	100,000	Disk firmware
Network buffer cache	Parts of files	Local disk	10,000,000	NFS client
Browser cache	Web pages	Local disk	10,000,000	Web browser
Web cache	Web pages	Remote server disks	1,000,000,000	Web proxy server

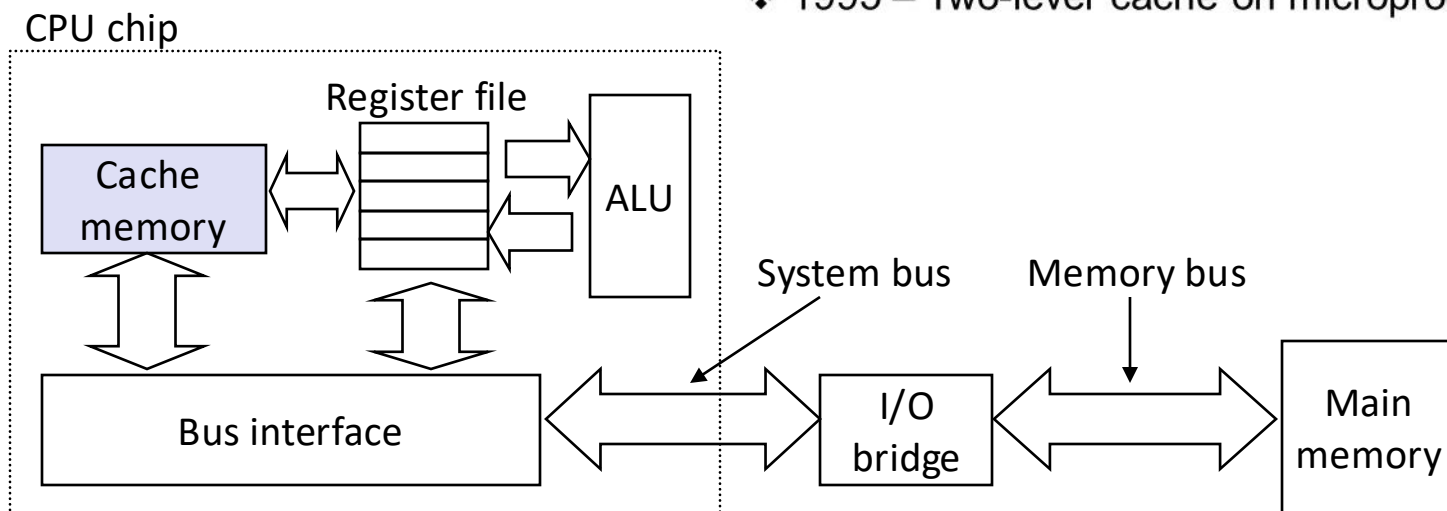
TLB: 翻译后备缓冲器, MMU: 内存管理单元, OS: 操作系统, AFS: 安备鲁文件系统, NFS: 网络文件系统

6.4 高速缓存存储器

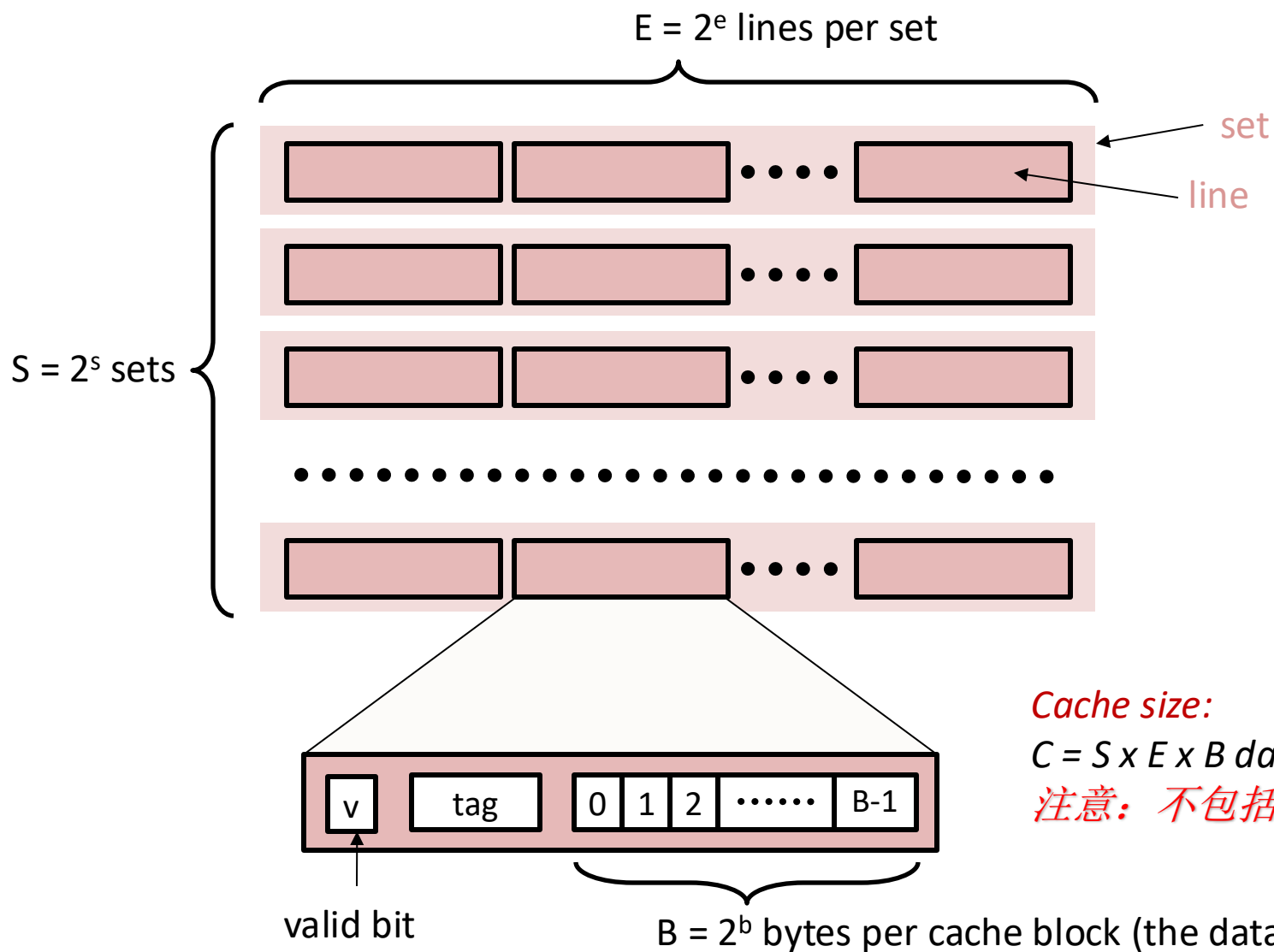
- Cache是在CPU寄存器和内存之间的一个SRAM高速缓存存储器
 - 硬件自动管理
 - 速度与寄存器一样快
 - 逐渐形成L1-L2-L3三级缓存结构
- 典型系统结构:



- ❖ 1980 – No cache in microprocessor
- ❖ 1995 – Two-level cache on microprocessor



6.4.1 通用的高速缓存存储器组织结构



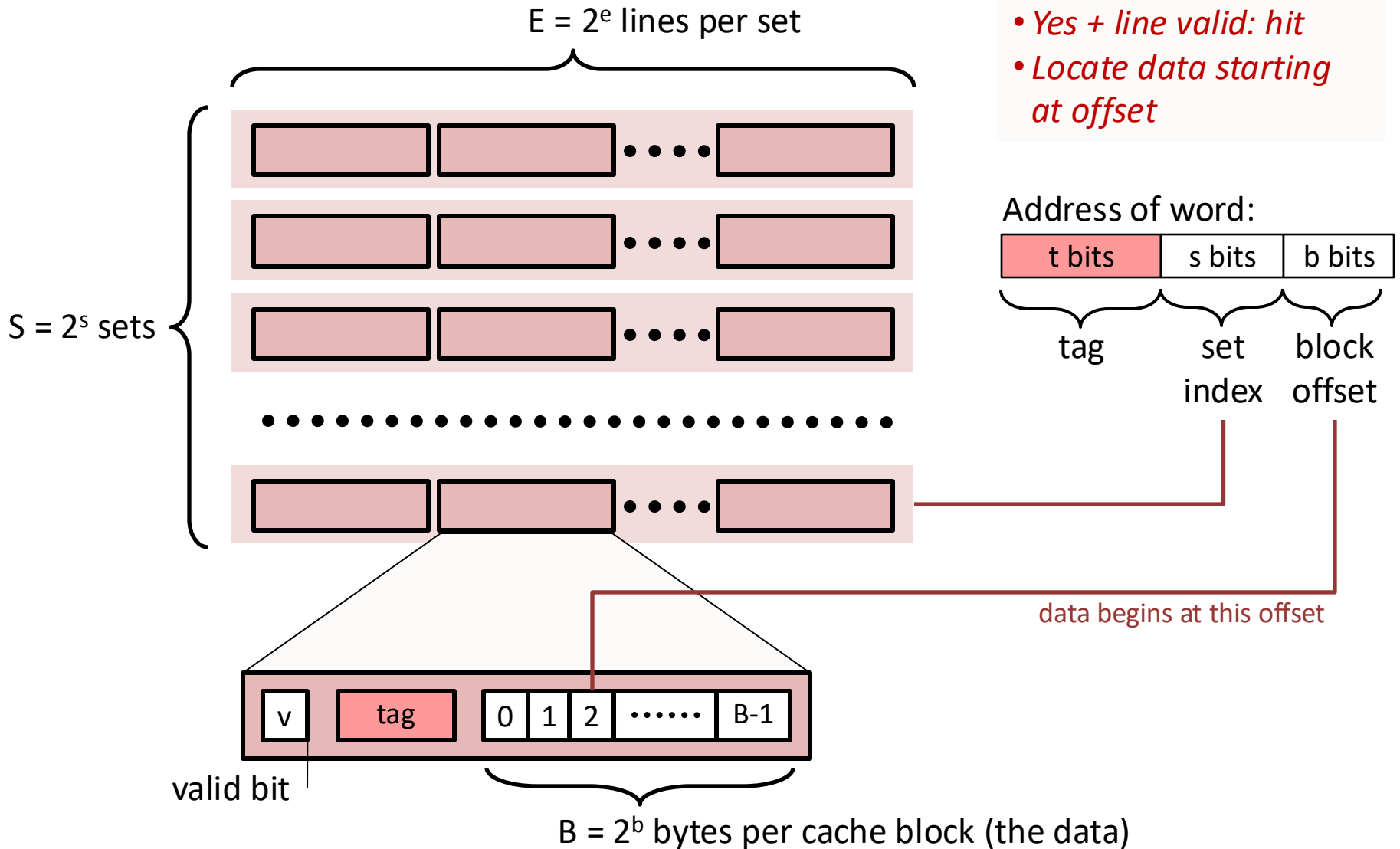
Cache size:

$C = S \times E \times B$ data bytes

注意：不包括tag大小

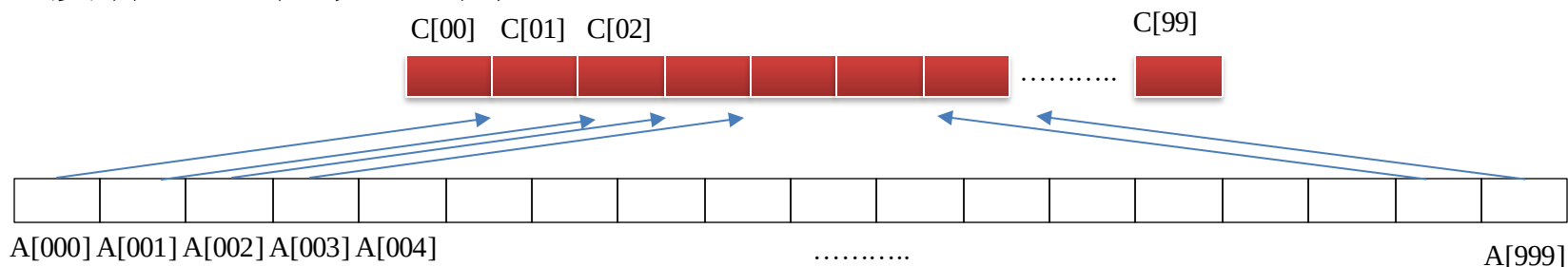
Cache读

- *Locate set*
- *Check if any line in set has matching tag*
- *Yes + line valid: hit*
- *Locate data starting at offset*



理解cache组织结构

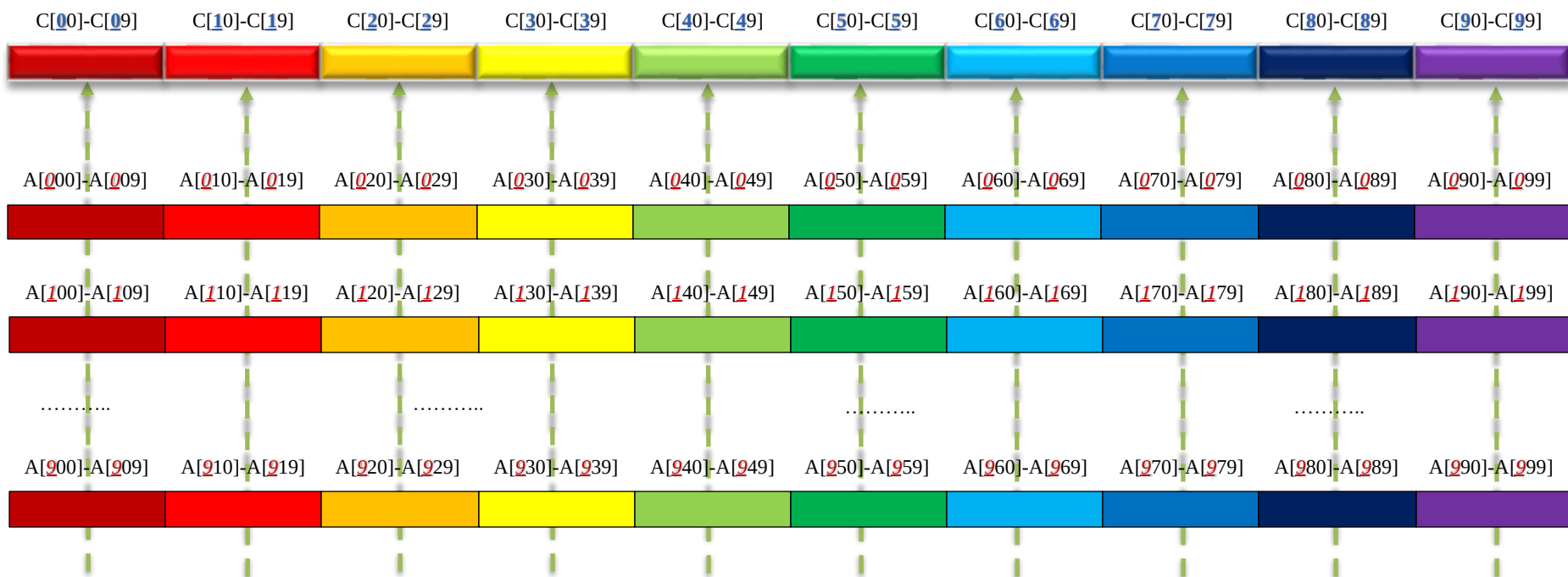
- 假设数组A[1000]，数据访问粒度为10个单元，cache能够缓存100个数组单元。



- 缓存存什么？数组单元值A[x]和数组地址x
 - 任意单元可缓存：线性查找空闲单元，记录每个单元访问频度，决定替换策略
 - 映射：通过物理地址映射，数组单元按地址映射到cache中特定地址区域

理解cache组织结构

- 假设数组A[0:999]，数据访问粒度为10个单元/块，cache能够缓存100个数组单元。



练习题6.9

- 根据参数确定每个高速缓存的高速缓存组数(S)、标记位数(t)、组索引位数(s)以及块偏移位数(b)

高速缓存	m	C	B	E	S	t	s	b
1	32	1024	4	1	256	22	8	2
2	32	1024	8	4	32	24	5	3
3	32	1024	32	32	1	27	0	5

- 参数计算(以1为例):
 - $S = C/B/E = 1024/4/1 = 256$
 - $s = \log_2(S) = \log_2 256 = 8$
 - $b = \log_2(B) = \log_2 4 = 2$
 - $t = m - (s + b) = 32 - (8 + 2) = 22$

作业-3



- P452
- 6.25
 - 目标：
 - 掌握根据cache参数计算相关参数的方法
 - 根据物理地址位数、cache大小、块大小、相联度计算cache组数
 - 计算标记位数、索引位数、块偏移位数的方法

作业-4



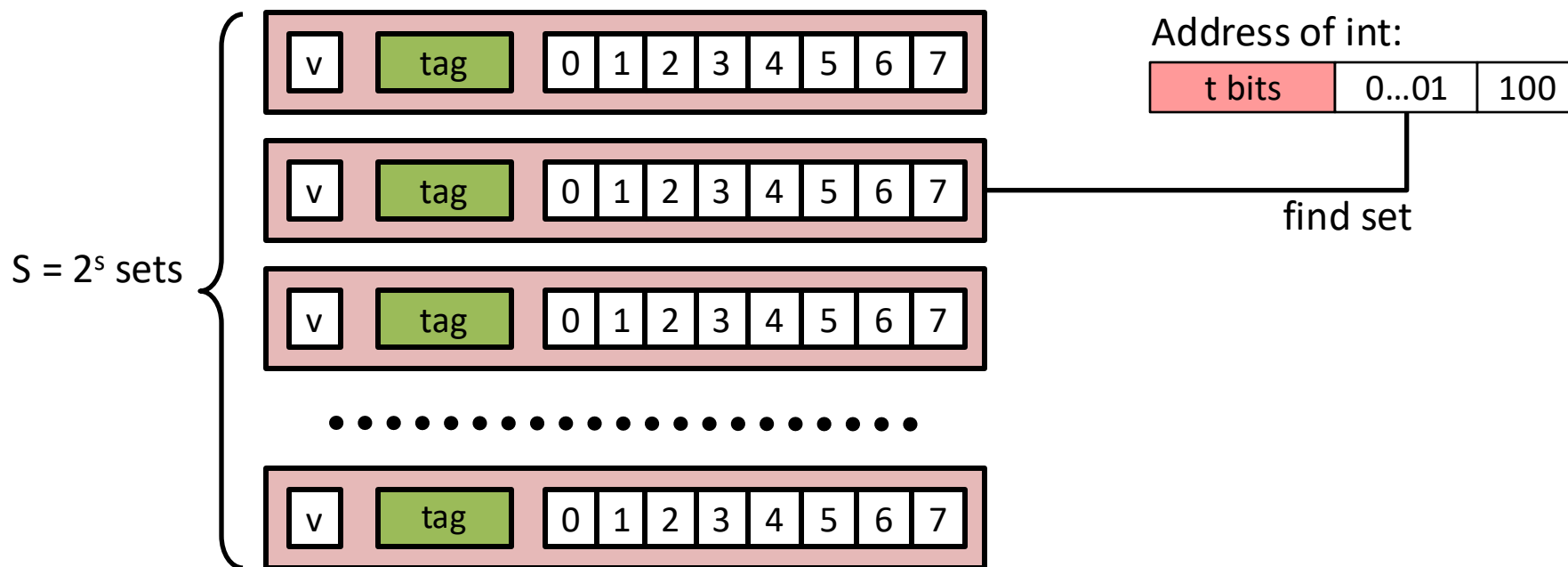
- P452
- 6.26
 - 目标：
 - 掌握根据cache参数计算相关参数的方法
 - 根据部分参数反推缺失参数的方法

6.4.2 直接映射高速缓存

- 每个组的高速缓存行数 $E=1$ 时称为直接映射高速缓存

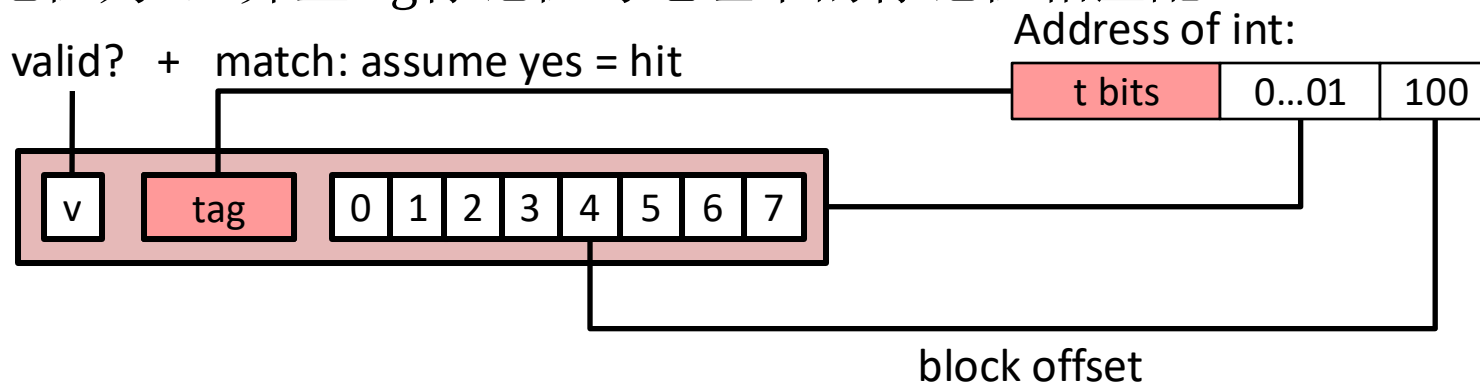
每组一行

Cache line大小为8字节



直接映射高速缓存访问

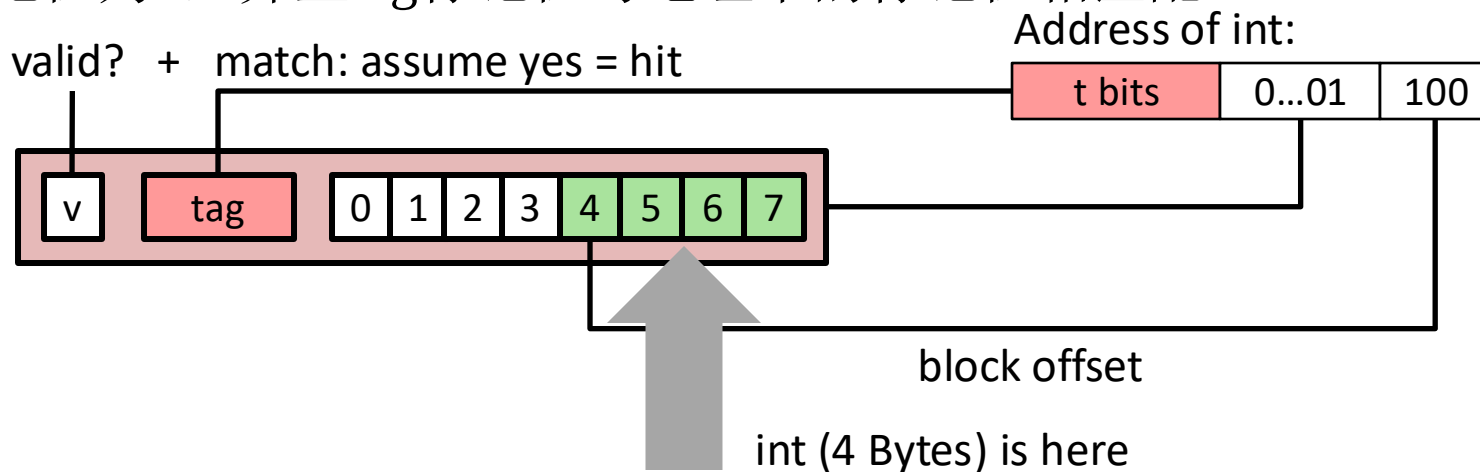
- 1.直接映射高速缓存中的组选择
 - 通过组索引位(0...01)选择高速缓存中的组
- 2.直接映射高速缓存中的行匹配
 - 标记位为1，并且tag标记位与地址中的标记位相匹配



- 3.直接映射高速缓存中的字选择
 - 块偏移位(100)提供了所需要的字的第一个字节在cache block中的偏移地址

直接映射高速缓存访问

- 1.直接映射高速缓存中的组选择
 - 通过组索引位(0...01)选择高速缓存中的组
- 2.直接映射高速缓存中的行匹配
 - 标记位为1，并且tag标记位与地址中的标记位相匹配



- 3.直接映射高速缓存中的字选择
 - 块偏移位(100)提供了所需要的字的第一个字节在cache block中的偏移地址
- 4.直接映射高速缓存不命中时的行替换
 - 如果不命中，从下一级存储器中取出请求的块，替换当前行

5.综合：运行中的直接映射高速缓存

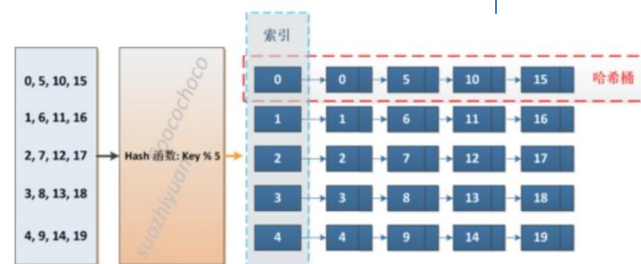
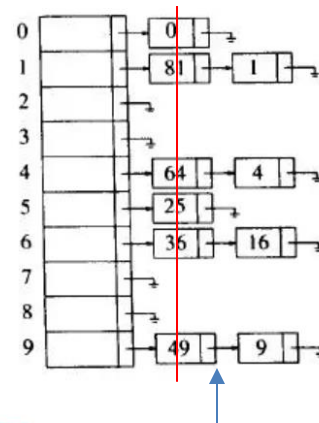
地址 (十进制)	地址位			块号 (十进制)
	标记位 (t=1)	索引位 (s=2)	偏移位 (b=1)	
0	0	00	0	0
1	0	00	1	0
2	0	01	0	1
3	0	01	1	1
4	0	10	0	2
5	0	10	1	2
6	0	11	0	3
7	0	11	1	3
8	1	00	0	4
9	1	00	1	4
10	1	01	0	5
11	1	01	1	5
12	1	10	0	6
13	1	10	1	6
14	1	11	0	7
15	1	11	1	7

- 假设直接映射高速缓存 $(S,E,B,m)=(4,1,2,4)$:

- 高速缓存有4个组
- 每组1行
- 每个块2字节
- 地址位是4位

- 地址映射

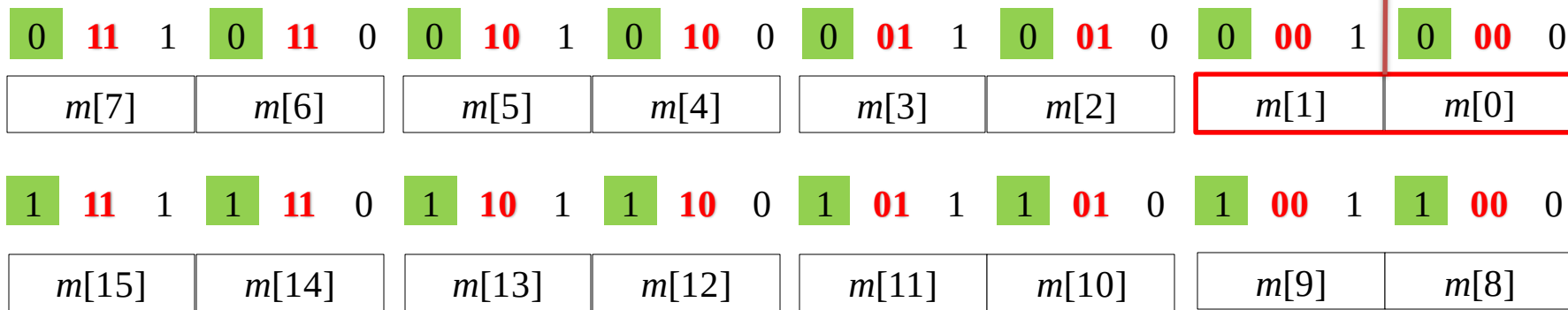
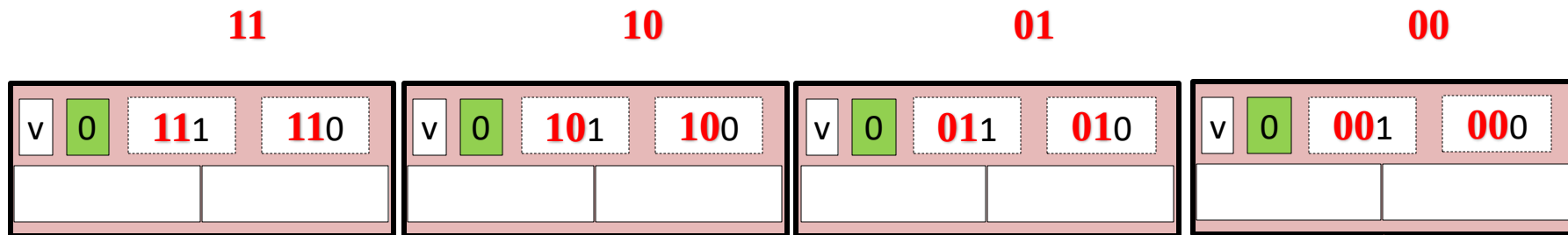
- 标记位和索引位连起来唯一标识内存中的每个块
- 有8个内存块，4个高速缓存组，有相同组索引的多个块映射到一个高速缓存组
- 映射到同一个高速缓存组的块由标记唯一地址标识



Cache缓存示例

1. 读地址0的字

Cache

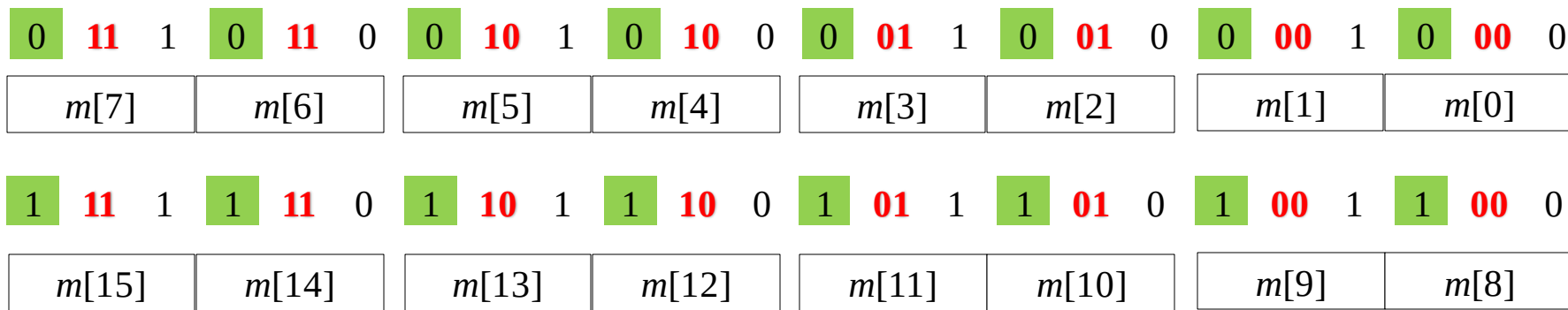
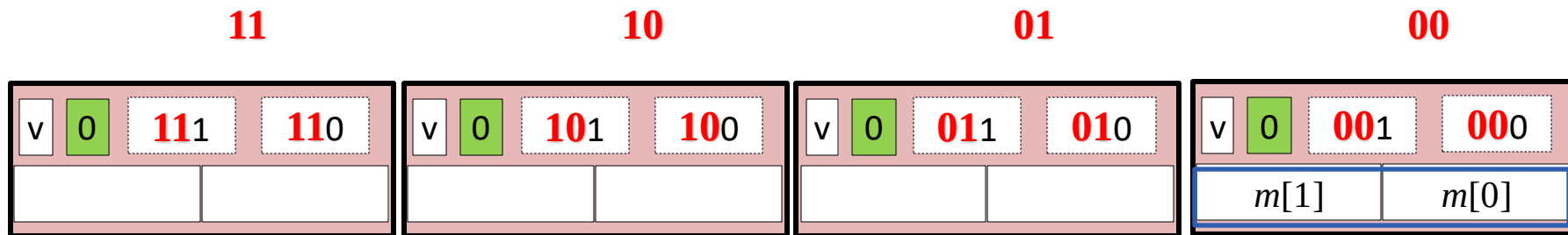


Main-Memory

Cache缓存示例

1. 读地址0的字

Cache

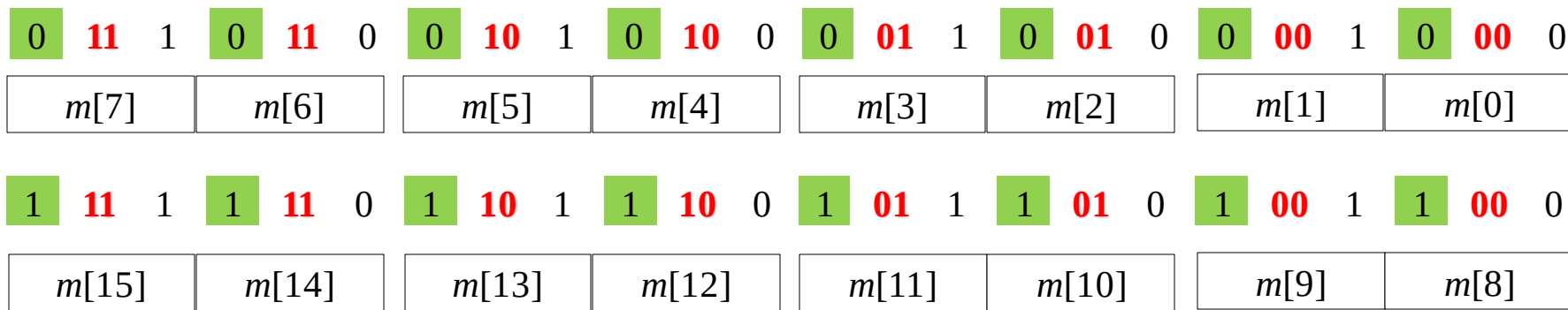
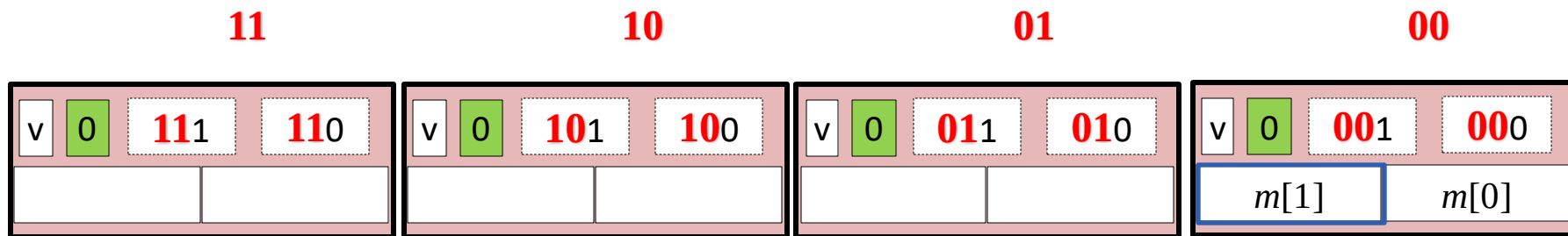


Main-Memory

Cache缓存示例

2.读地址1的字

Cache

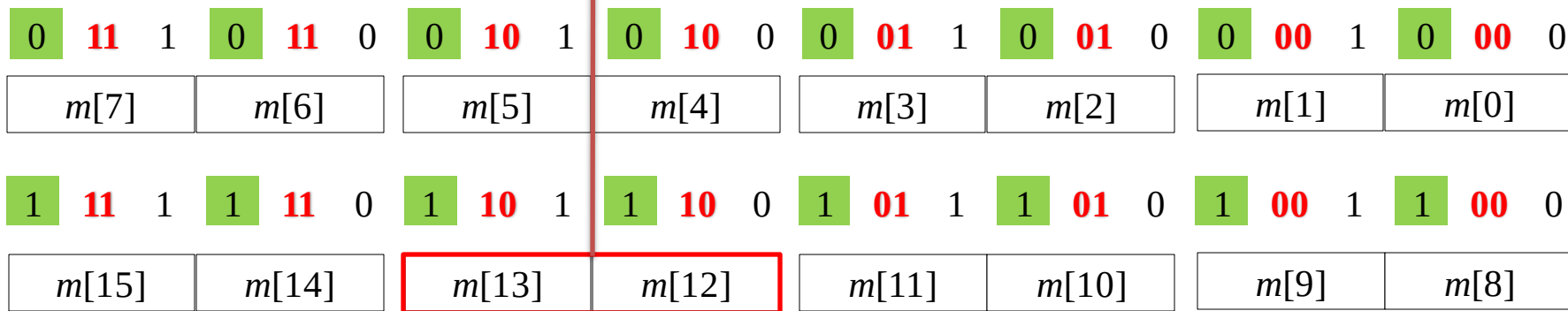
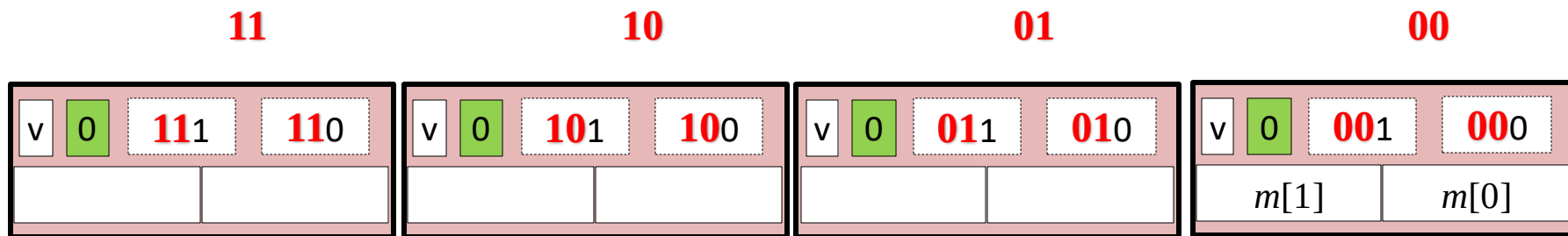


Main-Memory

Cache缓存示例

3.读地址13的字

Cache

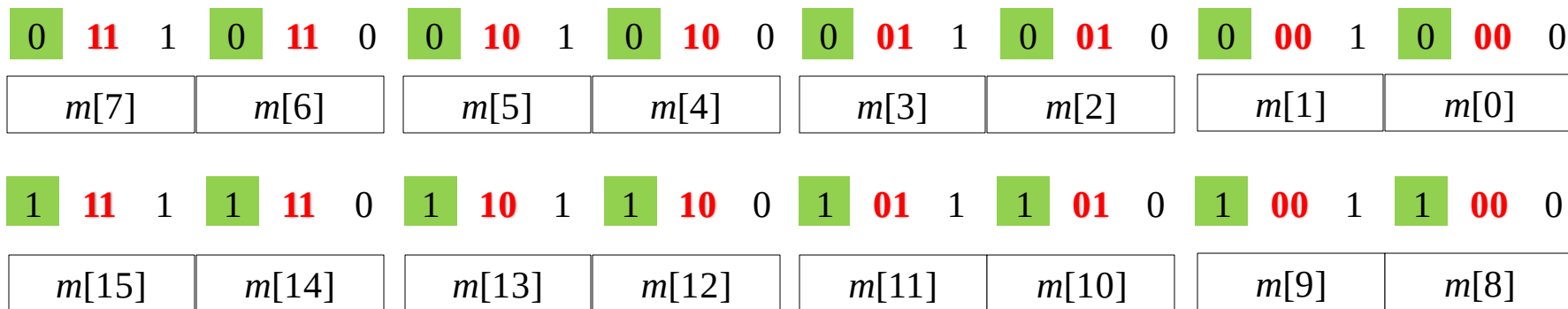
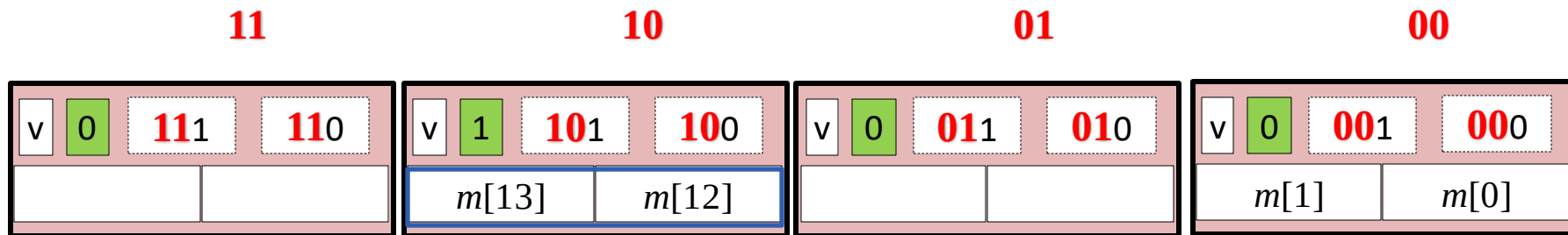


Main-Memory

Cache缓存示例

3.读地址13的字

Cache

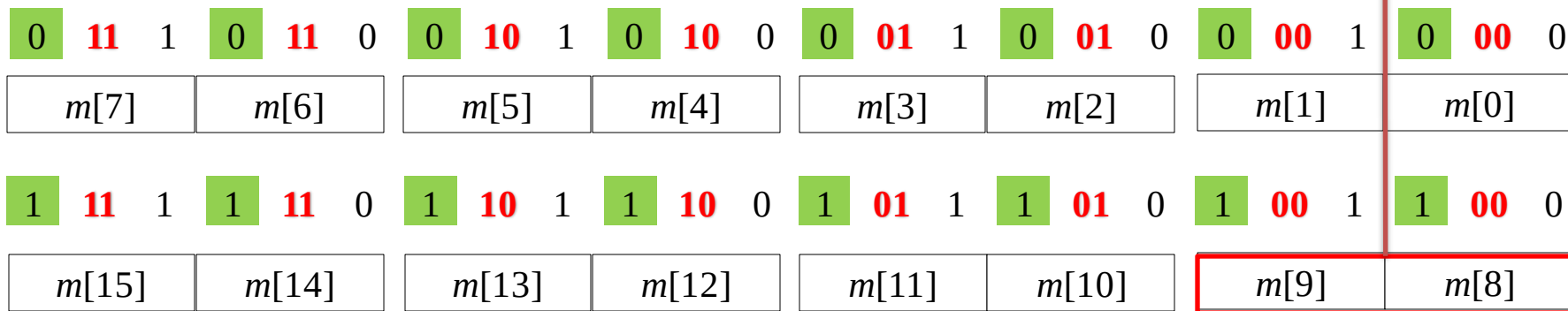
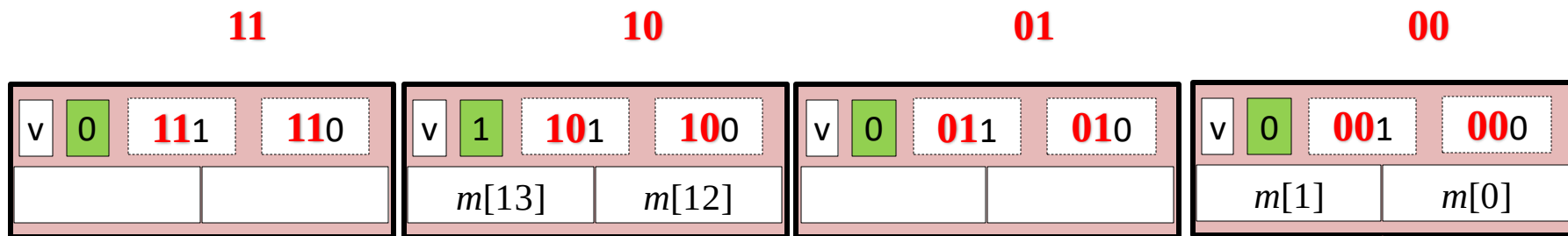


Main-Memory

Cache缓存示例

4. 读地址8的字

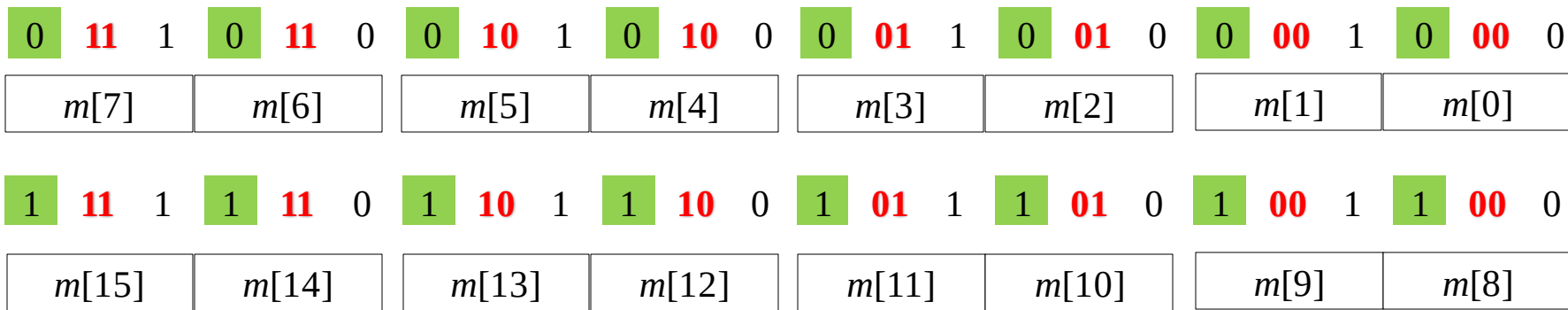
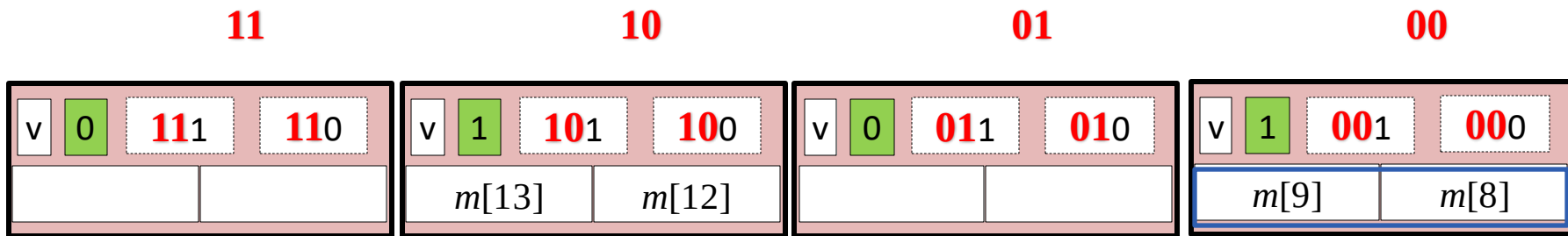
Cache



Main-Memory

4.读地址8的字

Cache

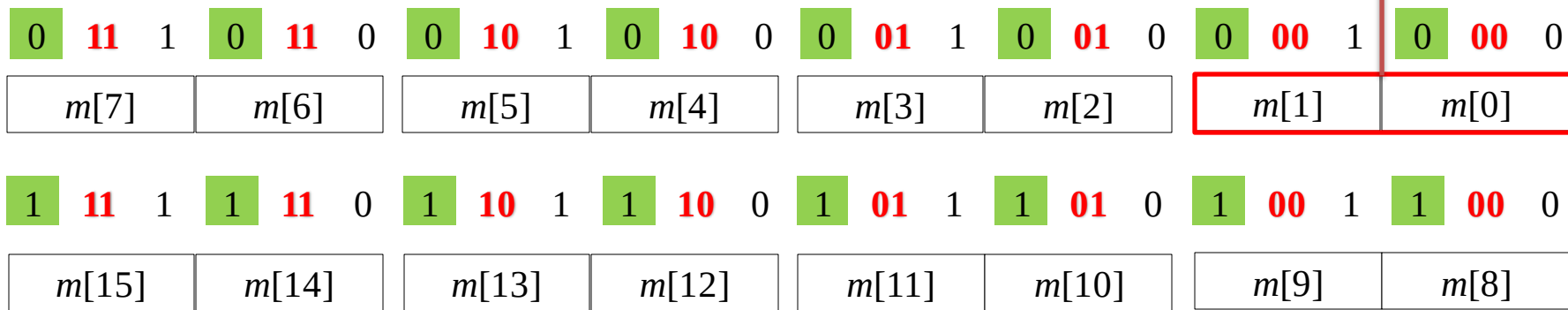
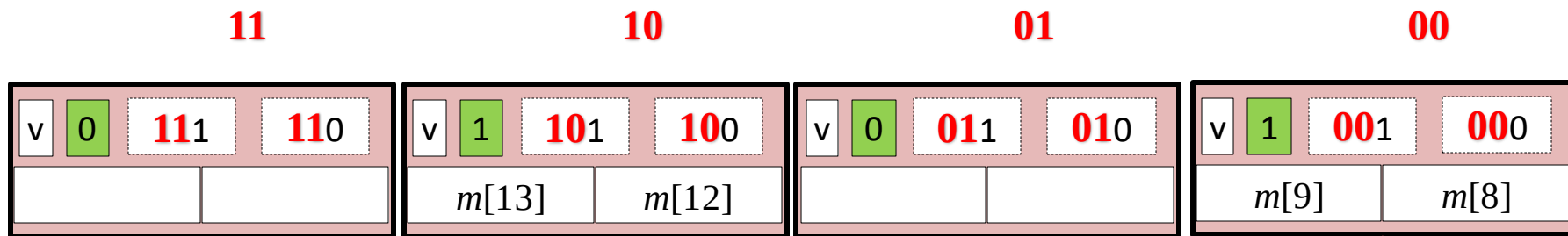


Main-Memory

Cache缓存示例

5. 读地址0的字

Cache

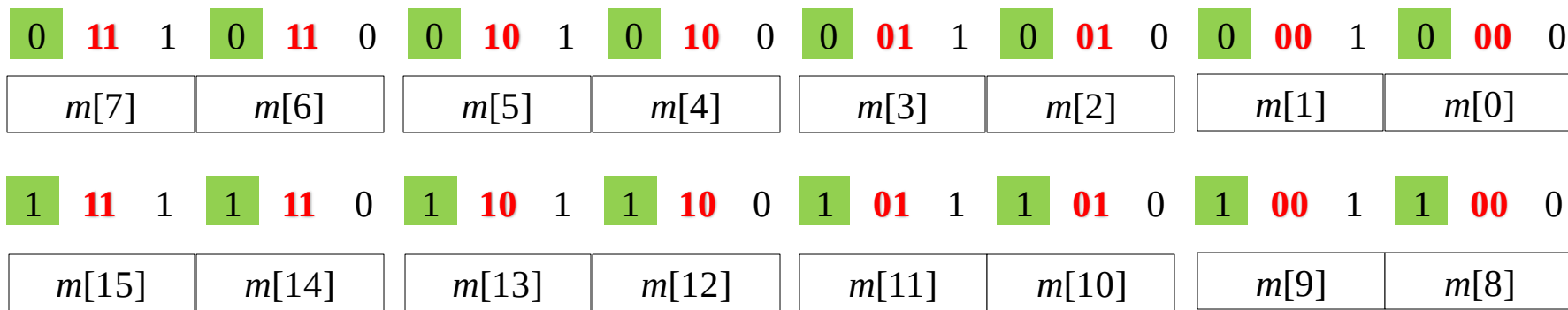
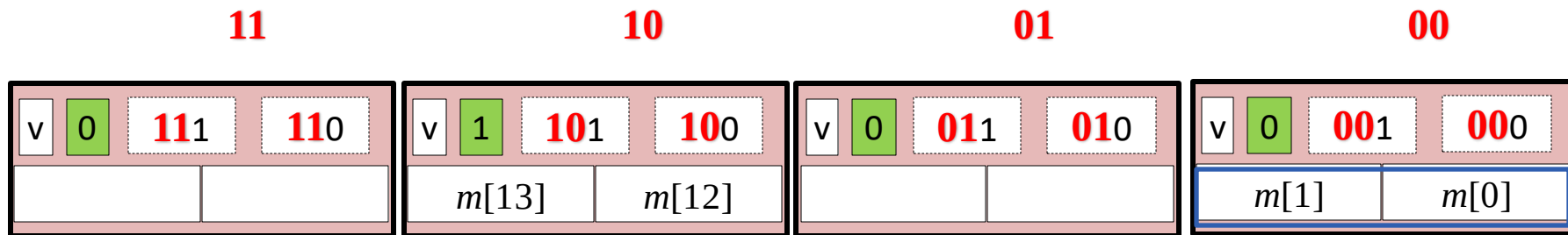


Main-Memory

Cache缓存示例

5. 读地址0的字

Cache



Main-Memory

6.直接映射高速缓存中的冲突不命中

- 两个向量点积函数
 - 具有良好的空间局部性
 - Cache命中率仍然较低
- 参数设置：
 - 高速缓存两个组，大小为32字节
 - 浮点数4字节
 - X加载到从0到32个字节连续内存
 - Y加载到从32开始的连续内存
 - 一个块为16字节

```
Float dotprod(float x[8], float y[8])
{
    float sum=0.0;
    int i;

    for(i=0;i<8;i++)
        sum+=x[i]*y[i];
    return sum;
}
```

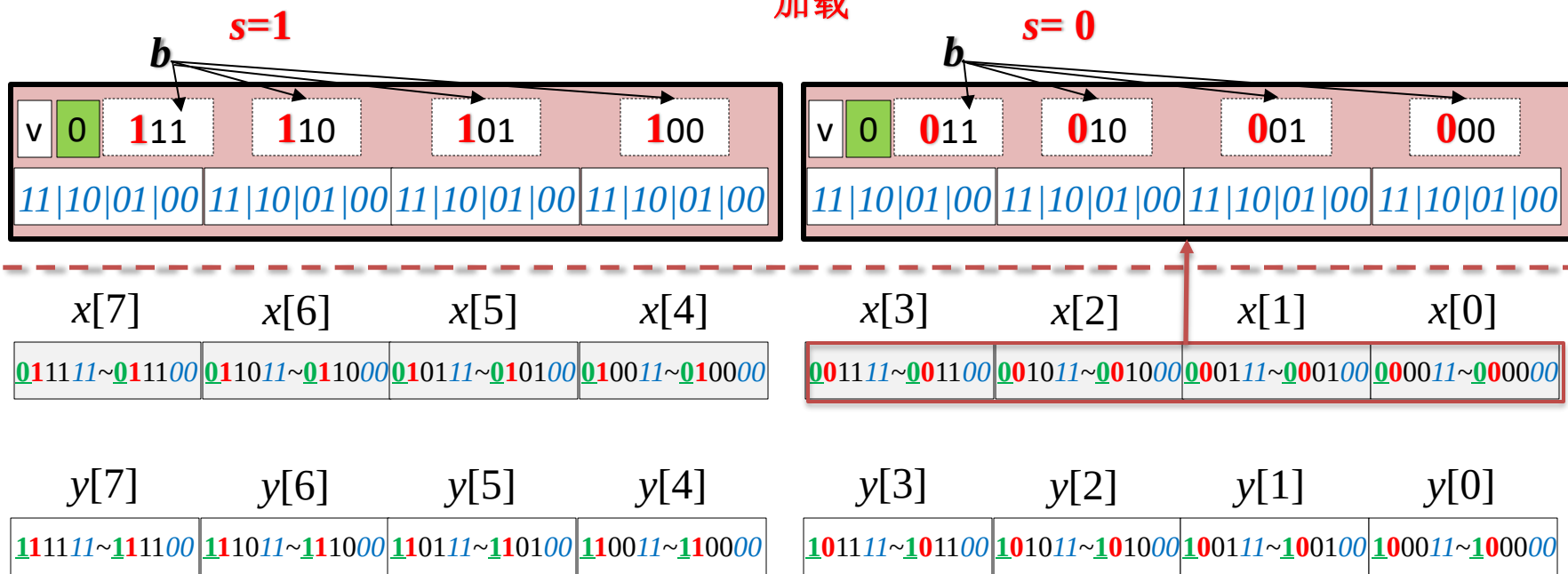
x				y			
元素	地址	组索引	二进制地址	元素	地址	组索引	二进制地址
x[0]	0	0	000000	y[0]	32	0	100000
x[1]	4	0	000100	y[1]	36	0	100100
x[2]	8	0	001000	y[2]	40	0	101000
x[3]	12	0	001100	y[3]	44	0	101100
x[4]	16	1	010000	y[4]	48	1	110000
x[5]	20	1	010100	y[5]	52	1	110100
x[6]	24	1	011000	y[6]	56	1	111000
x[7]	28	1	011100	y[7]	60	1	111100

内存与cache地址分布

sum+=x[0]*y[0];

↑
加载

Cache



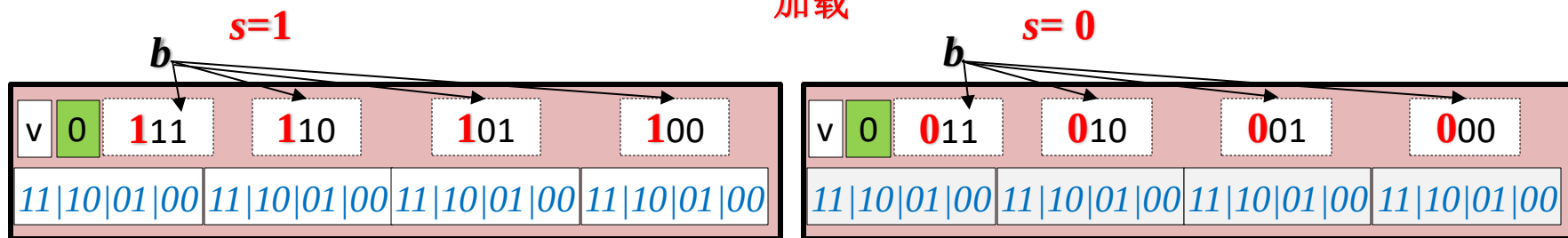
Main-Memory

内存与cache地址分布

sum+=x[0]*y[0];

↑
加载

Cache



x[7]

x[6]

x[5]

x[4]

x[3]

x[2]

x[1]

x[0]

011111~011100	011011~011000	010111~010100	010011~010000
---------------	---------------	---------------	---------------

001111~001100	001011~001000	000111~000100	000011~000000
---------------	---------------	---------------	---------------

y[7]

y[6]

y[5]

y[4]

y[3]

y[2]

y[1]

y[0]

111111~111100	111011~111000	110111~110100	110011~110000
---------------	---------------	---------------	---------------

101111~101100	101011~101000	100111~100100	100011~100000
---------------	---------------	---------------	---------------

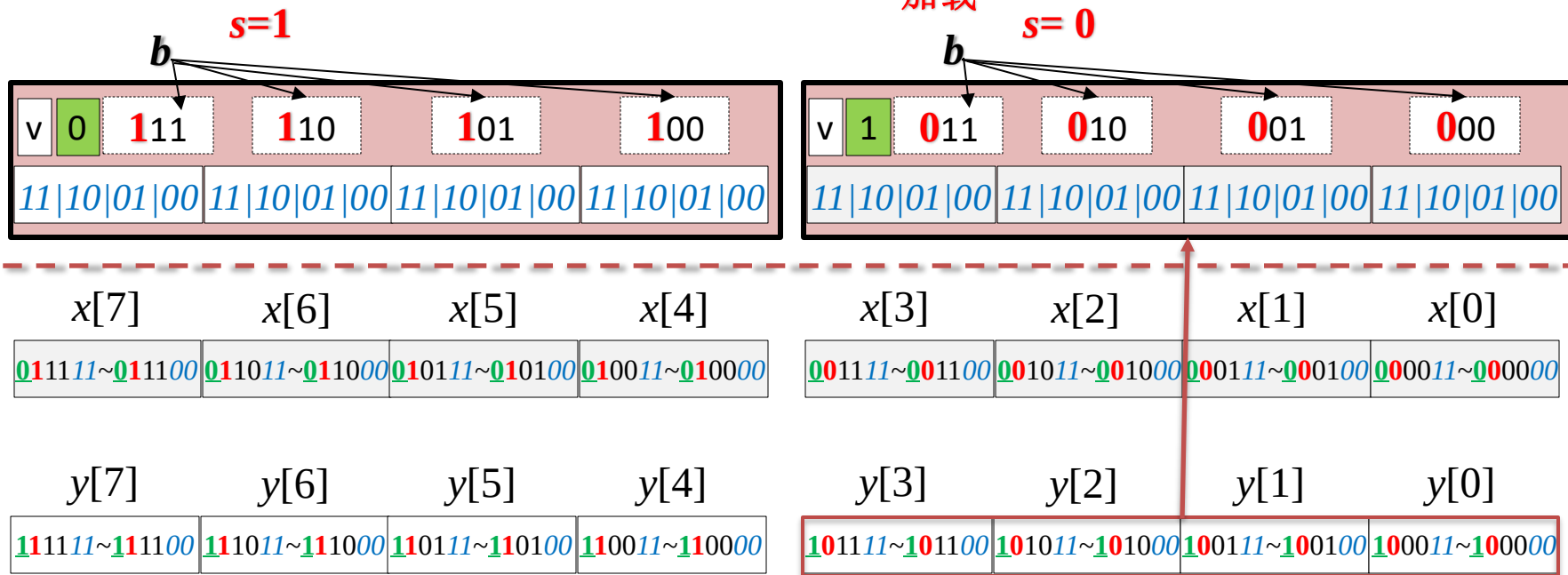
Main-Memory

内存与cache地址分布

sum+=x[0]*y[0];

加载

Cache



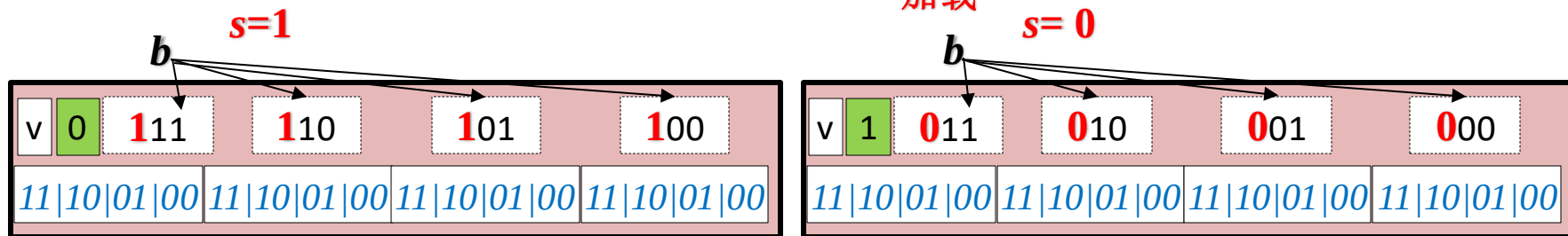
Main-Memory

内存与cache地址分布

sum+=x[0]*y[0];

↑
加载

Cache



x[7]

x[6]

x[5]

x[4]

x[3]

x[2]

x[1]

x[0]

011111~011100	011011~011000	010111~010100	010011~010000
---------------	---------------	---------------	---------------

001111~001100	001011~001000	000111~000100	000011~000000
---------------	---------------	---------------	---------------

y[7]

y[6]

y[5]

y[4]

y[3]

y[2]

y[1]

y[0]

011111~011100	011011~011000	010111~010100	010011~010000
---------------	---------------	---------------	---------------

001111~001100	001011~001000	000111~000100	000011~000000
---------------	---------------	---------------	---------------

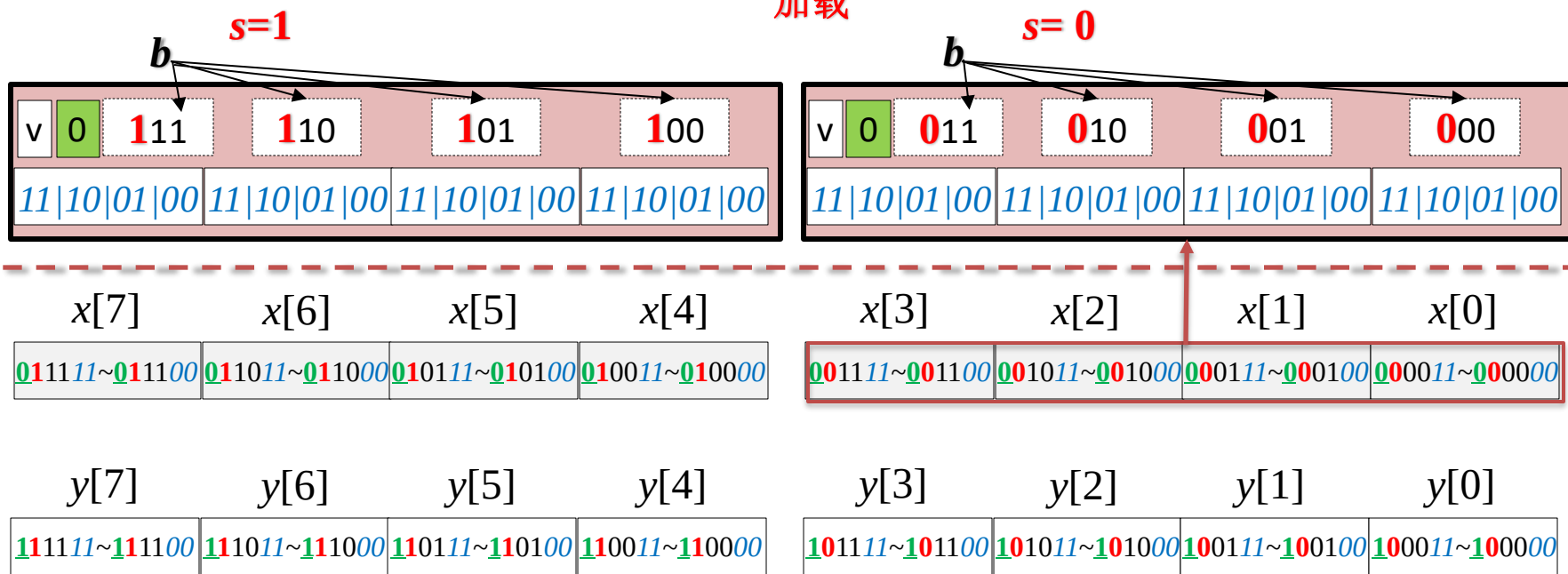
Main-Memory

内存与cache地址分布

sum+=x[1]*y[1];

↑
加载

Cache



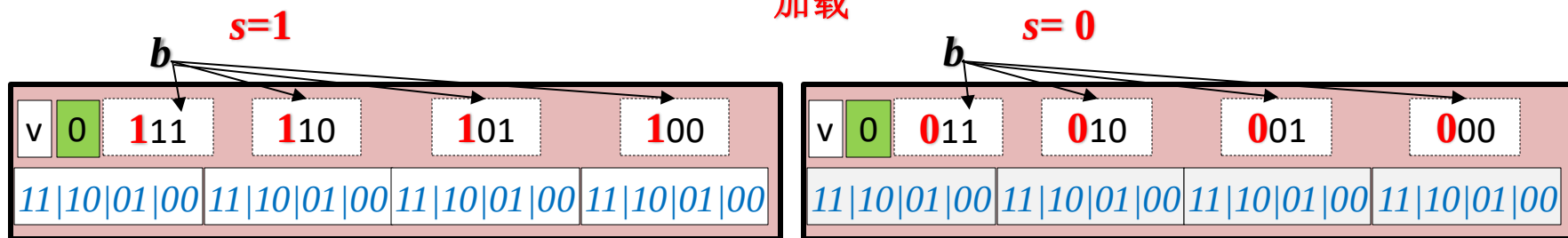
Main-Memory

内存与cache地址分布

sum+=x[1]*y[1];

↑
加载

Cache



x[7]

x[6]

x[5]

x[4]

x[3]

x[2]

x[1]

x[0]

011111~011100	011011~011000	010111~010100	010011~010000
---------------	---------------	---------------	---------------

001111~001100	001011~001000	000111~000100	000011~000000
---------------	---------------	---------------	---------------

y[7]

y[6]

y[5]

y[4]

y[3]

y[2]

y[1]

y[0]

111111~111100	111011~111000	110111~110100	110011~110000
---------------	---------------	---------------	---------------

101111~101100	101011~101000	100111~100100	100011~100000
---------------	---------------	---------------	---------------

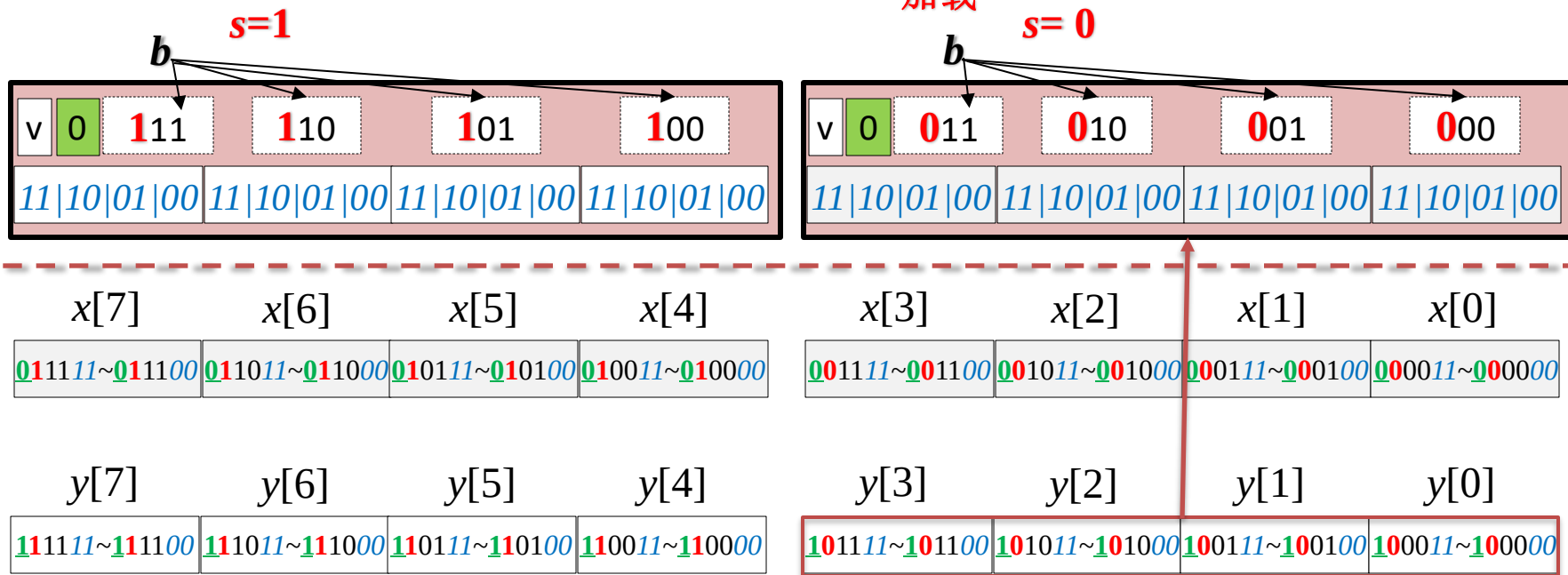
Main-Memory

内存与cache地址分布

sum+=x[1]*y[1];

加载

Cache



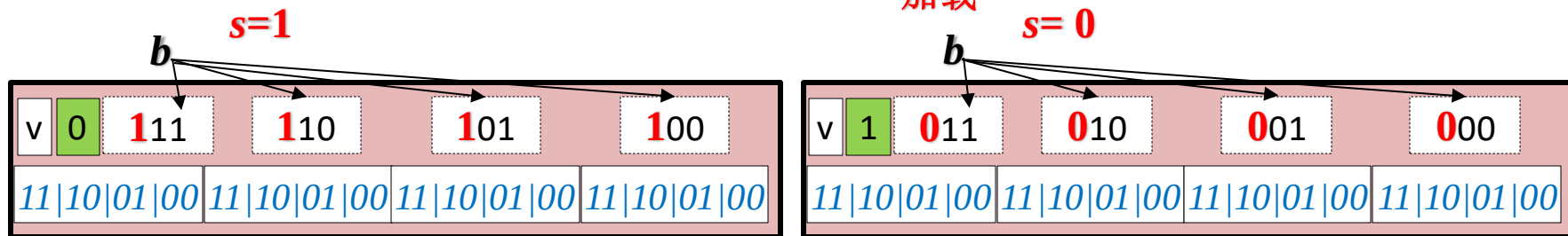
Main-Memory

内存与cache地址分布

sum+=x[1]*y[1];

↑
加载

Cache



x[7]

x[6]

x[5]

x[4]

x[3]

x[2]

x[1]

x[0]

011111~011100	011011~011000	010111~010100	010011~010000
---------------	---------------	---------------	---------------

001111~001100	001011~001000	000111~000100	000011~000000
---------------	---------------	---------------	---------------

y[7]

y[6]

y[5]

y[4]

y[3]

y[2]

y[1]

y[0]

111111~111100	111011~111000	110111~110100	110011~110000
---------------	---------------	---------------	---------------

101111~101100	101011~101000	100111~100100	100011~100000
---------------	---------------	---------------	---------------

Main-Memory

Cache thrashing原因分析

- 抖动(trash): 高速缓存反复地加载和驱逐相同的高速缓存块组
- 原因:
 - 循环迭代处理语句中 $x[i]$ 和 $y[i]$ 内存块对应相同的cache组
 - 读取 $x[i]$ 加载4个连续的浮点数到cache组
 - 读取 $y[i]$ 加载4个连续的浮点数到相同的cache组, 原 $x[i]$ 数据块被cache驱逐
 - 读取 $x[i+1]$ 需要重新加载4个连续的浮点数到cache组
 - 读取 $y[i+1]$ 重新驱逐已加载的4个连续的 $x[i+1]$ 对应的浮点数
 - 周而复始...
 - 导致每个 $x[i]$ 和 $y[i]$ 产生一个cache miss

Cache thrashing解决方案

- 在数组尾部增加B字节填充，使 $x[i]$ 和 $y[i]$ 内存块对应不同的cache组

x				y			
元素	地址	组索引	二进制地址	元素	地址	组索引	二进制地址
x[0]	0	0	000000	y[0]	48	1	100000
x[1]	4	0	000100	y[1]	52	1	100100
x[2]	8	0	001000	y[2]	56	1	101000
x[3]	12	0	001100	y[3]	60	1	101100
x[4]	16	1	010000	y[4]	64	0	110000
x[5]	20	1	010100	y[5]	68	0	110100
x[6]	24	1	011000	y[6]	72	0	111000
x[7]	28	1	011100	y[7]	76	0	111100

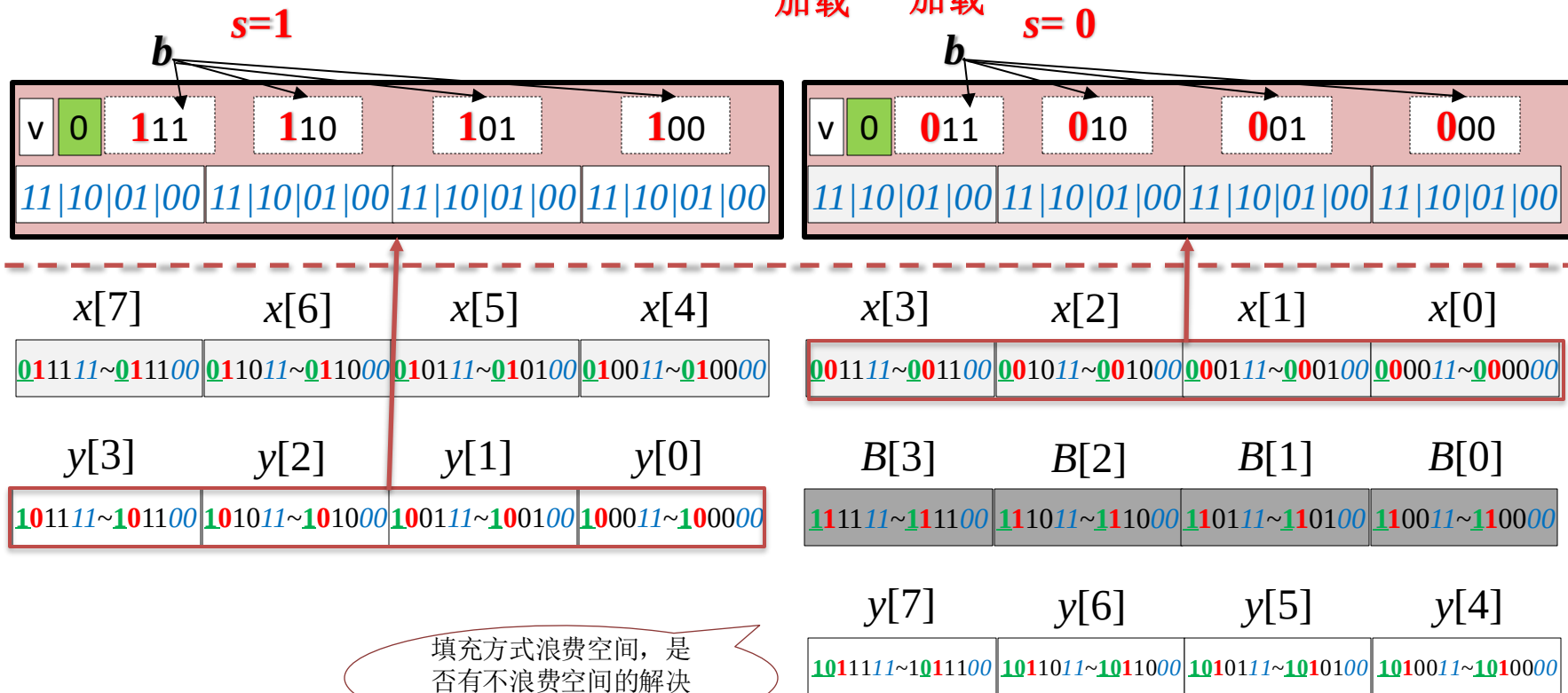
练习题6.10 填充后对x,y引用的命中率

- 每4个浮点数只加载一次，命中3次，命中率为75%

$sum += x[0] * y[0];$

加载 加载

Cache

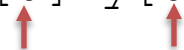


Main-Memory

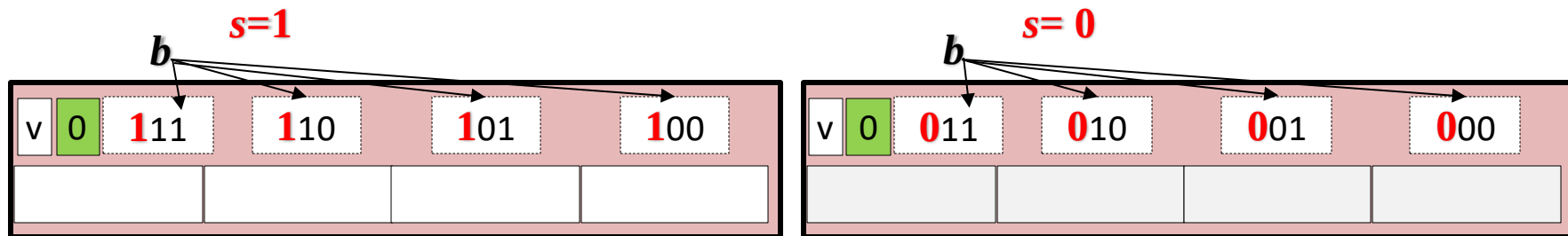
练习题6.10 填充后对x,y引用的命中率

- 每4个浮点数只加载一次，命中3次，命中率为75%

$\text{sum} += x[0] * y[0];$



Cache



$x[7]$ $x[6]$ $x[5]$ $x[4]$



$x[3]$ $x[2]$ $x[1]$ $x[0]$



$y[6]$ $y[5]$ $y[4]$ $y[3]$



$y[2]$ $y[1]$ $y[0]$



$y[7]$



Main-Memory

<https://www.archive.ece.cmu.edu/~ece740/f10/lib/exe/fetch.php?media=gaininginsightsintocachepartitioning.pdf>

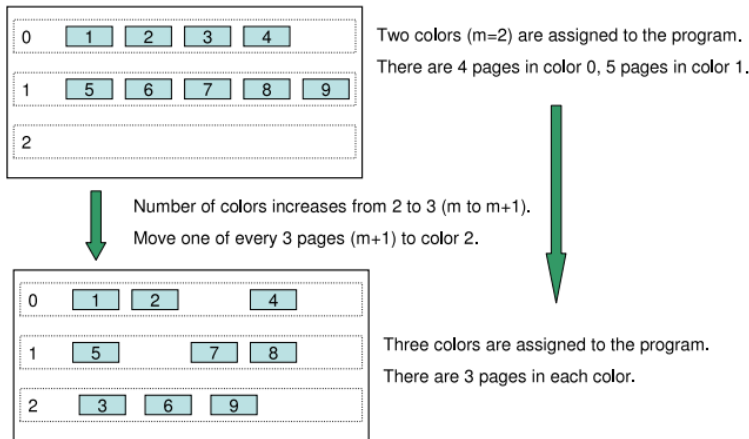


Figure 1: Page recoloring.

MCC-DB: Minimizing Cache Conflicts in Multi-core Processors for Databases

Rubao Lee^{1,2} Xiaoning Ding² Feng Chen² Qingda Lu^{2*} Xiaodong Zhang²

¹ Institute of Computing Technology
Chinese Academy of Sciences
Beijing, 100080, China

²Dept. of Computer Science & Engineering
The Ohio State University
Columbus, OH 43210, USA

{liu, dingxn, fchen, luq, zhang}@cse.ohio-state.edu

<http://web.cse.ohio-state.edu/hpcs/WWW/HTML/publications/papers/TR-09-4.pdf>

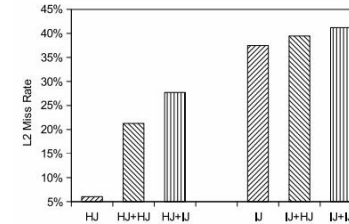


Figure 1: L2 miss rates when running a single or co-running various combinations of a hash join (HJ) and an index join (IJ).

International Conference on Web-Age Information Management
 WAIM 2011: [Web-Age Information Management](#) pp 480-492 | [Cite as](#)

W-Order Scan: Minimizing Cache Pollution by Application Software Level Cache Management for MMDB

Authors

Authors and affiliations

Yansong Zhang, Min Jiao, Zhanwei Wang, Shan Wang, Xuan Zhou

https://link.springer.com/chapter/10.1007%2F978-3-642-23535-1_41

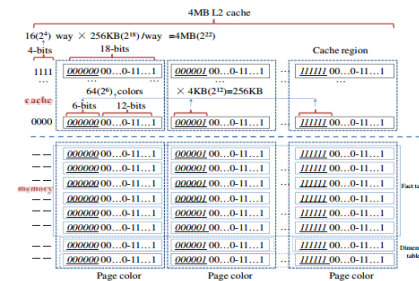


Fig. 2. Physical address layout for cache and main memory

练习题6.11

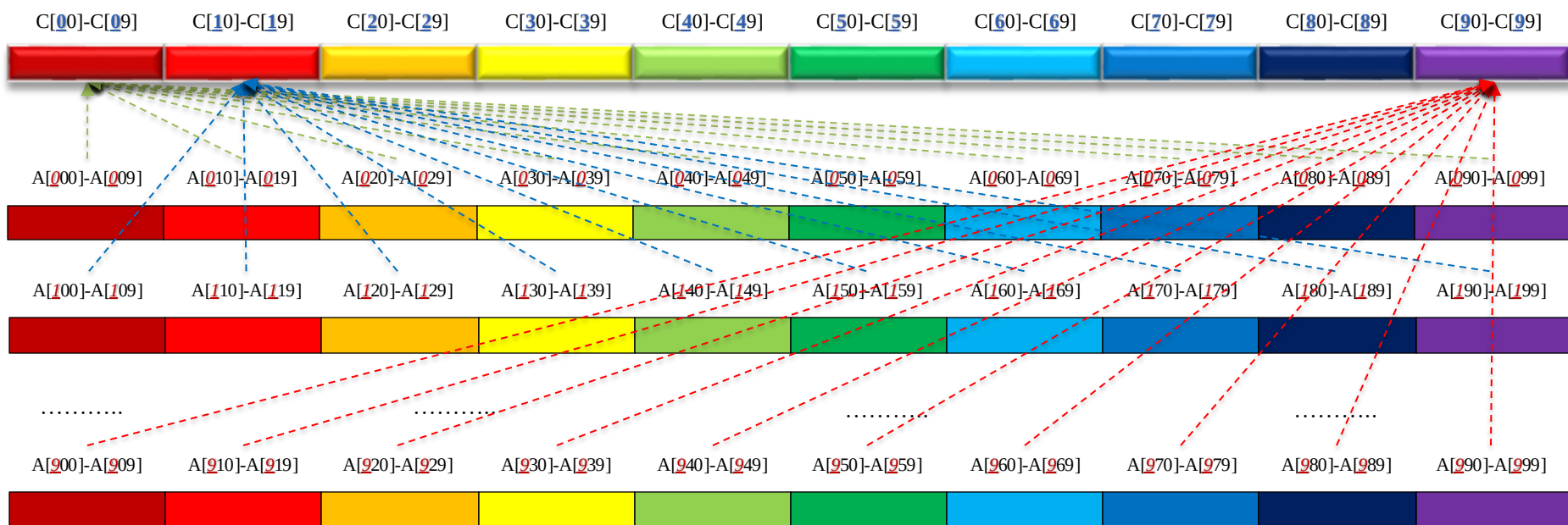
- 假设高速缓存使用地址的高 s 位作组索引，内存块连续的片(chunk)会被映射到同一个高速缓存组
 - **A.** 每个这样的连续的数组片中有多少个块？
 - 标记位数量 $t=m-(s+b)$ ，每个连续的数组片由 2^t 个块组成，数组头 2^t 个连续的块都会映射到组0，接下来的 2^t 个块会映射到组1，以此类推
 - **B.** 代码运行在高速缓存形式为 $(S,E,B,m)=(512,1,32,32)$ 系统上，任意时刻，存储在高速缓存中的数组块的最大数量是多少？

```
int array[4096];  
for (i=0; i<4096; i++)  
    sum+=array[i];
```

- 标记位数量 $t=m-(s+b)=32-(\log_2 512 + \log_2 32)=18$
- 数组中前 2^{18} 个块映射到组0，接下来的 2^{18} 个块映射到组1
- 数组长度为4096，由 $(4096*4)/32=512$ 个块组成，数组中所有的块都映射到组0，任何时刻，高速缓存至多保存一个数组块，即使数组能够全部容纳在cache中

高s位作组索引

- 假设数组A[0:999]，数据访问粒度为10个单元/块，cache能够缓存100个数组单元。



6.4.3 组相联高速缓存

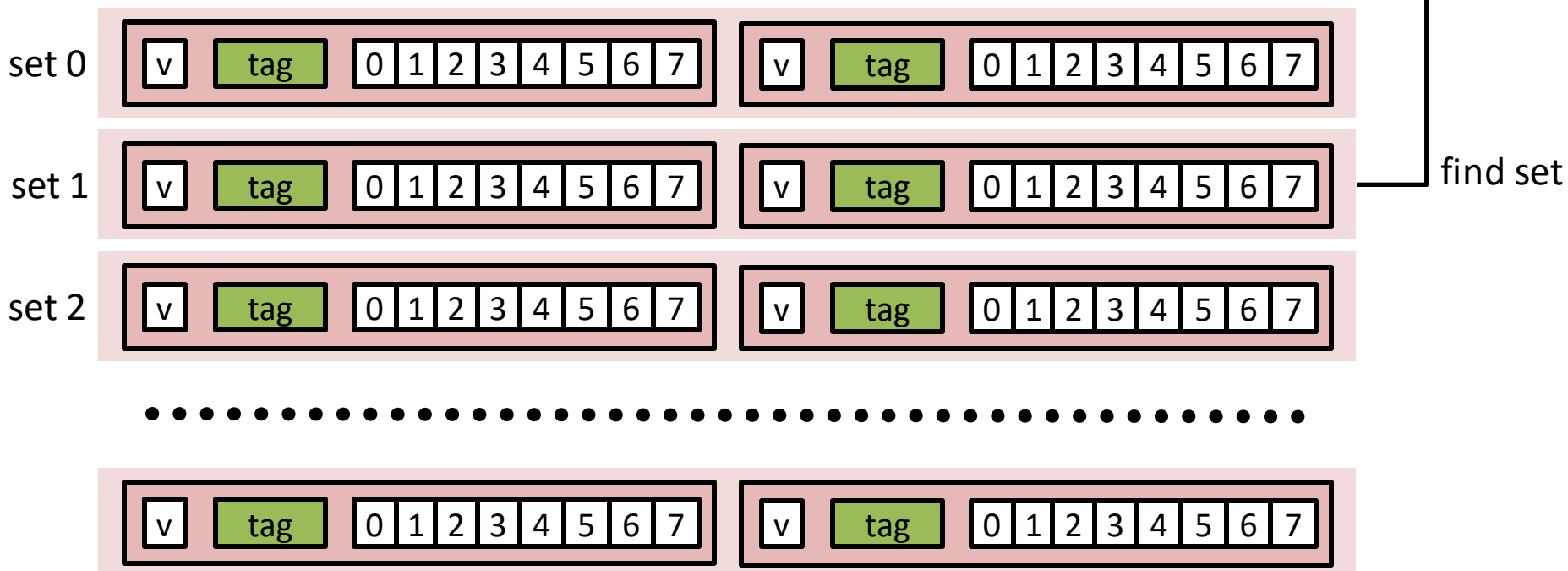
- 直接映射cache冲突不命中的根源为每个组只有一行($E=1$)
- 组相关高速缓存(set associative cache)每个组都保存多于一个的高速缓存行，即 $1 < E < C/B$
- 示例：2路组相联高速缓存

$E = 2$: Two lines per set

Assume: cache block size 8 bytes

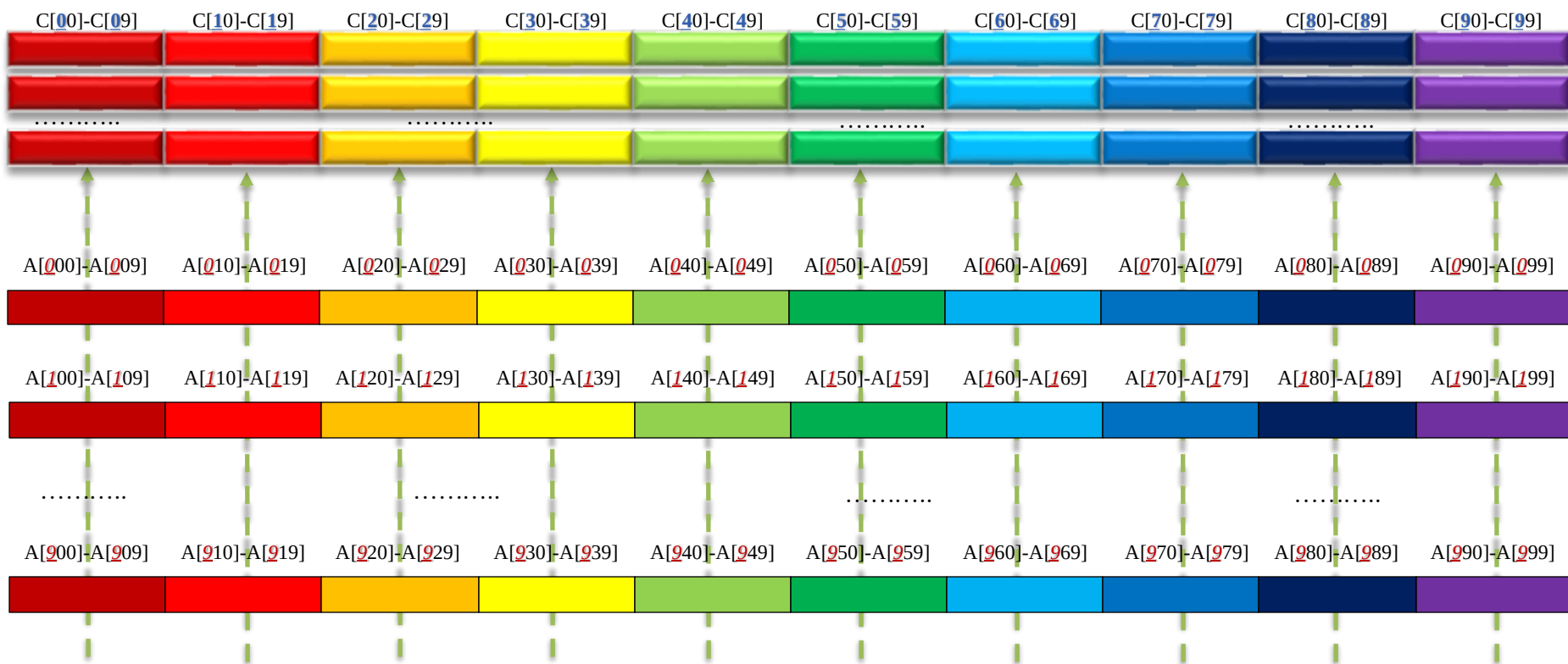
Address of short int:

t bits	0...01	100
--------	--------	-----



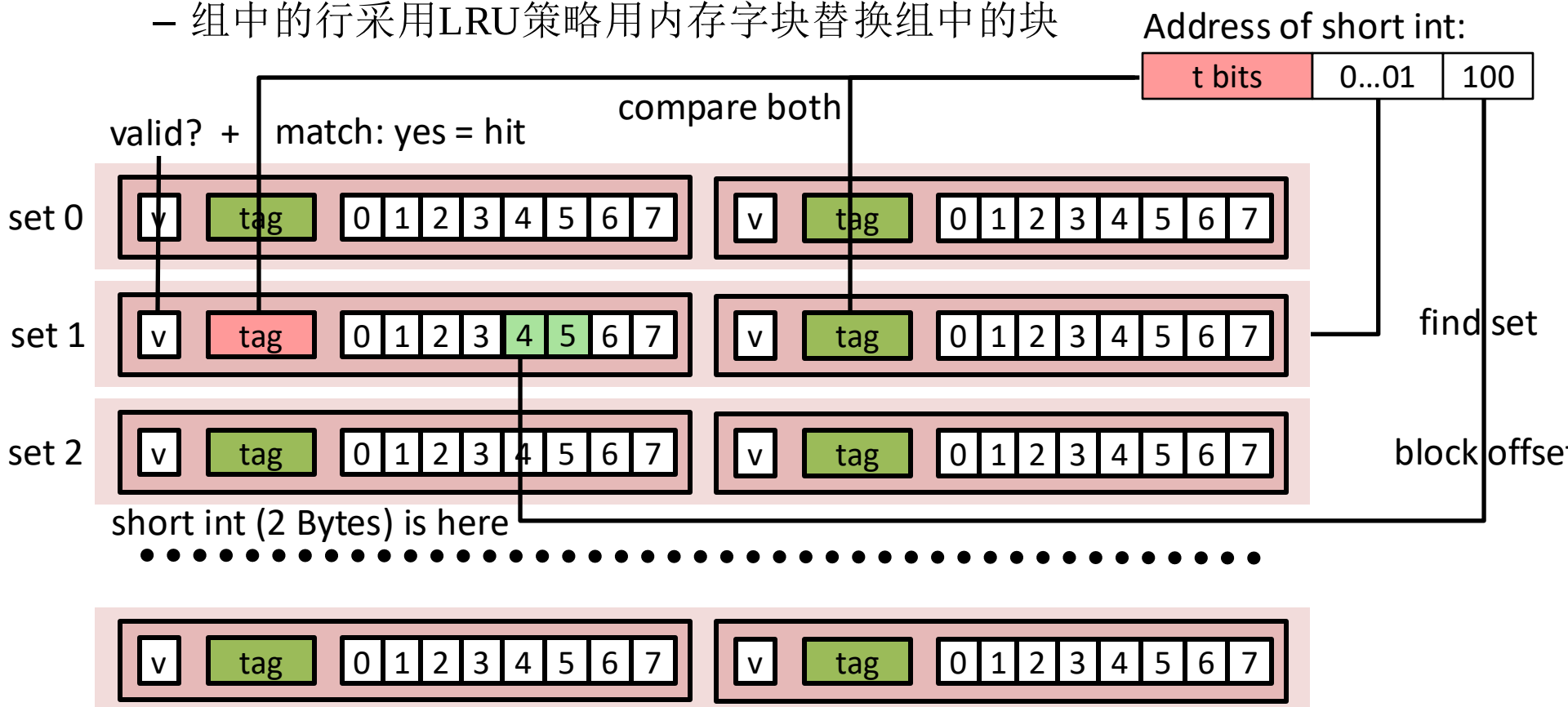
理解多路组相联cache结构

- 假设数组A[0:999]，数据访问粒度为10个单元/块，cache能够缓存100个数组单元。



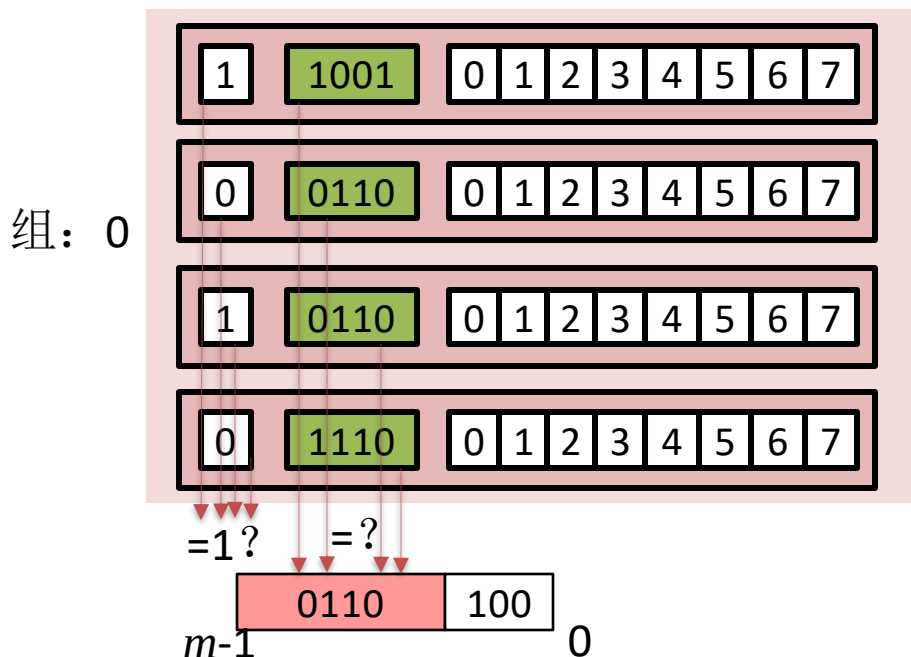
组相联高速缓存访问示例

- 1. 组相联高速缓存中的组选择
 - 组索引位标识cache组
- 2. 组相联高速缓存中的行匹配和字选择
 - 检查多个行的有效位和标记位，确定所请求的字是否在集合中
- 3. 组相联高速缓存中不命中时的行替换
 - 组中的行采用LRU策略用内存字块替换组中的块



6.4.4 全相联高速缓存

- 全相联高速缓存(full associative cache): 是由一个包含所有高速缓存行的组, 即 $E=C/B$ 组成
- 1.全关联高速缓存中的组待选择: 只有一个组, 没有组索引位
- 2.全相联高速缓存中的行匹配和字选择
 - 并行搜索相匹配的标记
- 缺点: 搜索多, 构造困难且昂贵, 适合做小的高速缓存, 如 TLB



练习题6.12 理解cache工作原理

- 假设：
 - 内存是字节寻址的
 - 内存访问是1字节的字，不是4字节的字
 - 地址的宽度为13位
 - Cache是2路组相联的($E=2$)，块大小为4字节($B=4$)，有8个组($S=8$)

	行0						行1					
组索引	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
0	09	1	86	30	3F	10	00	0	—	—	—	—
1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
2	EB	0	—	—	—	—	0B	0	—	—	—	—
3	06	0	—	—	—	—	32	1	12	08	7B	AD
4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
5	71	1	0B	DE	18	4B	6E	0	—	—	—	—
6	91	1	A0	B7	26	2D	F0	0	—	—	—	—
7	46	0	—	—	—	—	DE	1	12	C0	88	37

- 标出下列内容字段：

- CO:cache块偏移，CI:cache组索引；CT:cache标记 $\log_2 8$ $\log_2 4$

12	11	10	9	8	7	6	5	4	3	2	1	0
CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO

作业-5

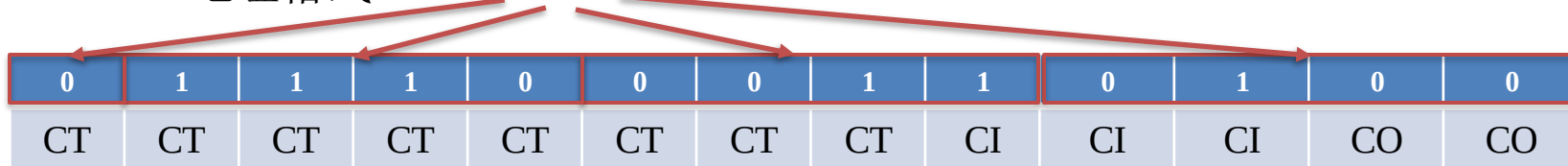


- P452
- 6.28
 - 目标：
 - 量化掌握cache工作原理

练习题6.13

- 程序引用0x0E34处1字节的字，指出访问的cache条目和十六进制表示的返回的cache字节值，指出是否发生cache miss，如果发生cache miss，用—表示返回的cache缓存字节

– A.地址格式：0x0E34



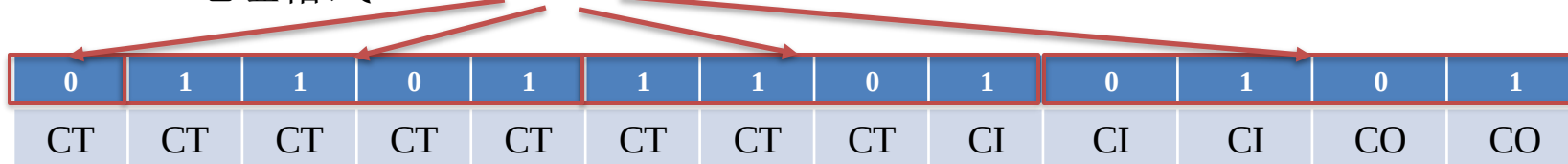
– B.内存引用：

			行0						行1					
		组索引	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
参数	值	0	09	1	86	30	3F	10	00	0	——	——	——	——
Cache块偏移(CO)	0x 0	1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
Cache组索引(CI)	0x 5	2	EB	0	——	——	——	——	0B	0	——	——	——	——
Cache标记(CT)	0x 71	3	06	0	——	——	——	——	32	1	12	08	7B	AD
Cache hits?	是	4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
返回cache字节	0x B	5	71	1	0B	DE	18	4B	6E	0	——	——	——	——
		6	91	1	A0	B7	26	2D	F0	0	——	——	——	——
		7	46	0	——	——	——	——	DE	1	12	C0	88	37

练习题6.14

- 程序引用0x0DD5处1字节的字，指出访问的cache条目和十六进制表示的返回的cache字节值，指出是否发生cache miss，如果发生cache miss，用—表示返回的cache缓存字节

– A.地址格式：0x0DD5



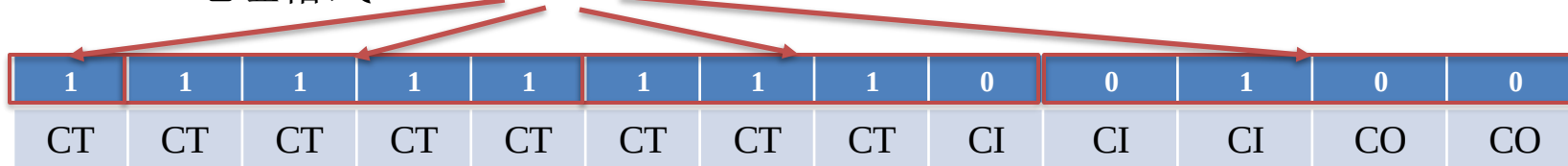
– B.内存引用：

			行0						行1					
		组索引	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
参数	值	0	09	1	86	30	3F	10	00	0	—	—	—	—
Cache块偏移(CO)	0x 1	1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
Cache组索引(CI)	0x 5	2	EB	0	—	—	—	—	0B	0	—	—	—	—
Cache标记(CT)	0x 6E	3	06	0	—	—	—	—	32	1	12	08	7B	AD
valid?	否	4	C7	1	96	78	07	C5	05	1	40	67	C2	3B
Cache hits?	否	5	71	1	0B	DE	18	4B	6E	0	—	—	—	—
返回cache字节	—	6	91	1	A0	B7	26	2D	F0	0	—	—	—	—
		7	46	0	—	—	—	—	DE	1	12	C0	88	37

练习题6.15

- 程序引用0x1FE4处1字节的字，指出访问的cache条目和十六进制表示的返回的cache字节值，指出是否发生cache miss，如果发生cache miss，用—表示返回的cache缓存字节

– A.地址格式：0x1FE4



– B.内存引用：

			行0						行1					
		组索引	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
参数	值	0	09	1	86	30	3F	10	00	0	——	——	——	——
Cache块偏移(CO)	0x 0	1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
Cache组索引(CI)	0x 1	2	EB	0	——	——	——	——	0B	0	——	——	——	——
Cache标记(CT)	0x FF	3	06	0	——	——	——	——	32	1	12	08	7B	AD
Cache hits?	否	4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
返回cache字节	—	5	71	1	0B	DE	18	4B	6E	0	——	——	——	——
		6	91	1	A0	B7	26	2D	F0	0	——	——	——	——
		7	46	0	——	——	——	——	DE	1	12	C0	88	37

练习题6.16

- 列出所有的在组3中会命中的十六进制内存地址

行0							行1					
组索引	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
3	06	0	—	—	—	—	32	1	12	08	7B	AD

- 组3包含一个有效行，标记为0x32，有效位为1，4个地址会命中，这些地址的有效形式为：

0	0	1	1	0	0	1	0	0	1	1		
CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO

- 4个十六进制地址为：

0	0	1	1	0	0	1	0	0	1	1	0	0
CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO

0	0	1	1	0	0	1	0	0	1	1	0	1
CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO

0	0	1	1	0	0	1	0	0	1	1	1	0
CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO

0	0	1	1	0	0	1	0	0	1	1	1	1
CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO

– 0x 064C, 0x 064D, 0x 064E, 0x 064F

作业-6

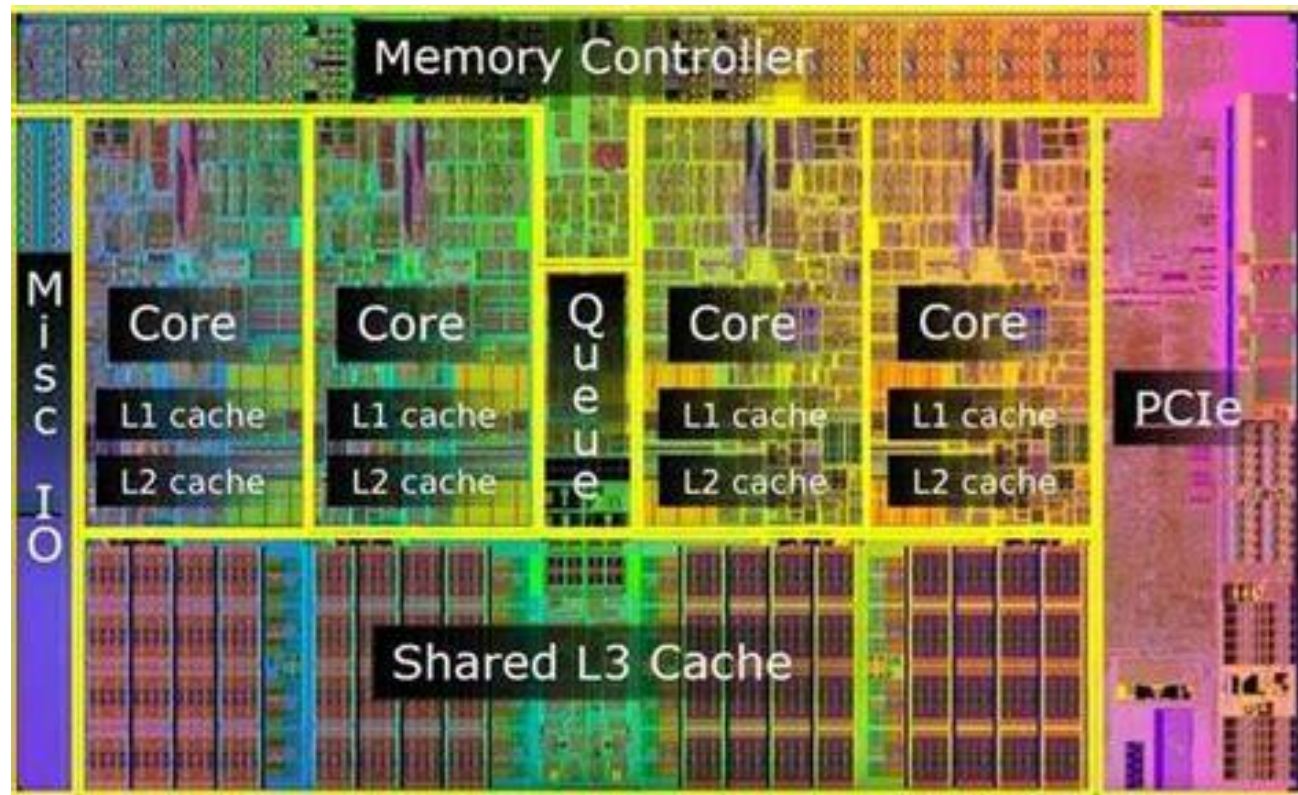


- P452
- 6.29, 6.30, 6.31, 6.32, 6.33
 - 目标:
 - 量化掌握cache工作原理

6.4.5 有关写的问题

- 读操作数据访问顺序
 - 寄存器 ← L1 cache ← L2 cache ← L3 cache ← main memory ← disk
- 写操作
 - 直写(write-through): 将修改的字w所在的块立即写回紧邻的下一层存储
 - 写回(write-back): 尽可能推迟更新, 当有替换算法要驱逐这个更新过的块时才将其写到紧邻的下一层存储
 - 提高局部性, 增加复杂性, 需要高速缓存行维护一个额外的修改位(dirty bit)标识修改
- 写不命中处理方法?
 - 写分配(Write-allocate): 加载低层块到缓存, 更新缓存块
 - 空间局部性好, 但每次不命中产生一个低层块传送到cache
 - 非写分配(No-write-allocate): 直接把字写到低一层存储
- 典型方案
 - 直写Write-through + 非写分配No-write-allocate
 - 写回Write-back + 写分配 Write-allocate
 - 复杂性可控, 利用空间局部性

6.4.6 一个真实的高速缓存层次解剖

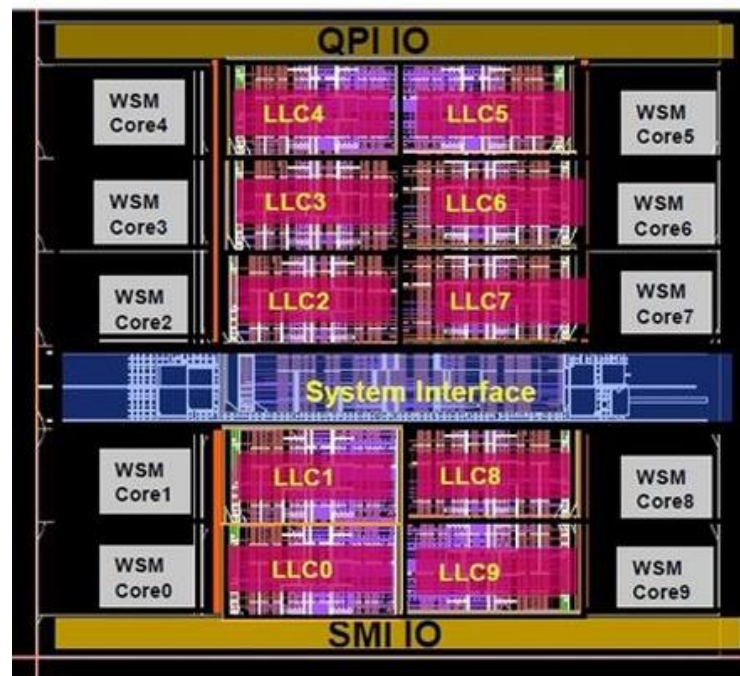


CPU案例

- 以Intel® Xeon® Processor E7-8867L（30M Cache，2.13 GHz，6.40 GT/s Intel® QPI 4通道，10核20线程）为例，如图4所示，处理器内集成了10个内核，能够支持20个物理线程，由10片共享L3 cache构成30MB的LLC，4个QPI（Quick Path Interconnect）通道实现芯片之间的高速互联。

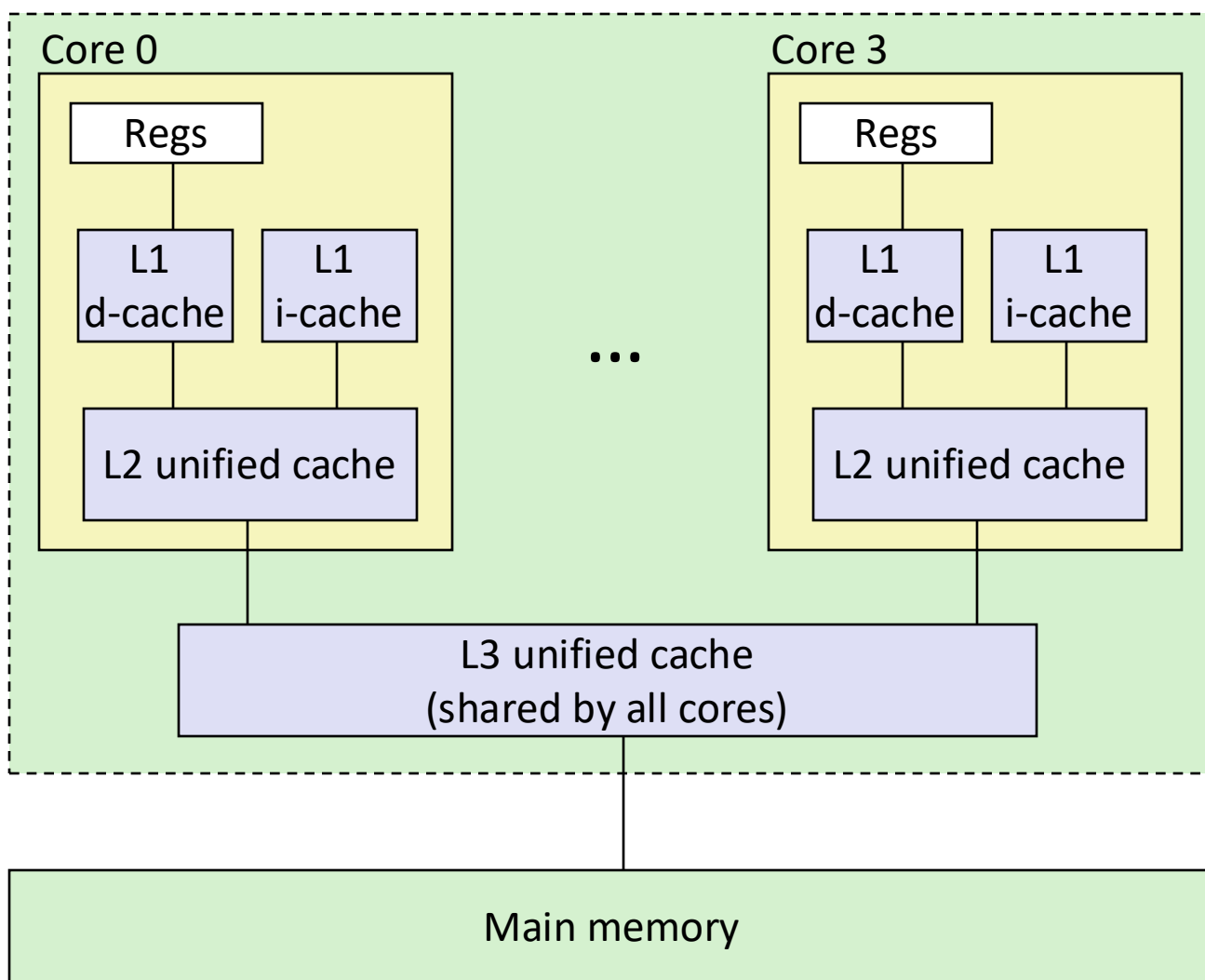
- L1 CACHE hit, ~4 cycles
- L2 CACHE hit, ~10 cycles
- L3 CACHE hit, line unshared ~40 cycles
- L3 CACHE hit, shared line in another core ~65 cycles
- L3 CACHE hit, modified in another core ~75 cycles remote
- remote L3 CACHE ~100-300 cycles

Cache带宽
计算？



6.4.6 一个真实的高速缓存层次解剖

Processor package



L1 i-cache and d-cache:

32 KB, 8-way,
Access: 4 cycles
Sets: 64

L2 unified cache:

256 KB, 8-way,
Access: 10 cycles
Sets: 512

L3 unified cache:

8 MB, 16-way,
Access: 40-75 cycles
Sets: 8192

Block size: 64 bytes for
all caches.

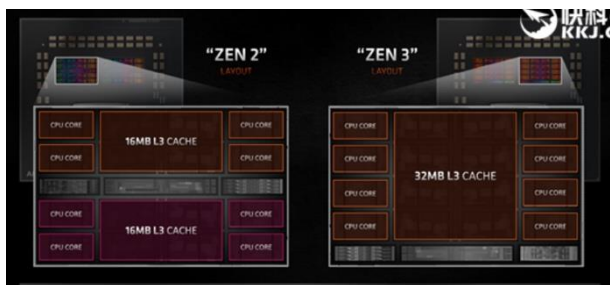
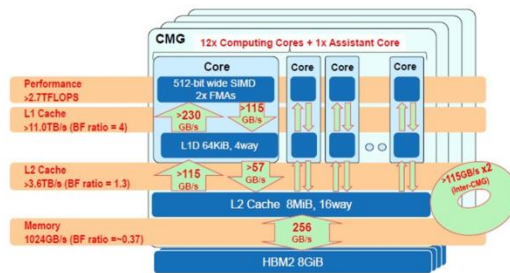
	set	block	way	cache size(Byte)	cache size(KB/MB)
L1	64	64	8	32768	32
L2	512	64	8	262144	256
L3	8192	64	16	8388608	8192

练习：CPU参数分析



AMD					
L1	128	64	4	32768	32
L2	2048	64	16	2097152	2048
L3	2048	64	64	8388608	8192
Intel					
L1	64	64	8	32768	32
L2	512	64	8	262144	256
L3	12288	64	20	15728640	15360

- | UPI | PCIe Gen4 x8/x16 | PCIe Gen4 x8/x16 | UPI | Dummy | Dummy | Dummy | PCIe Gen4 x8/x16 |
|----------|------------------|------------------|----------|----------|----------|----------|------------------|
| HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA |
| ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core |
| HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA |
| ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core |
| Ch-1 | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | Ch-1 |
| Ch-2 | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | Ch-2 |
| HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA |
| ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core |
| Ch-1 | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | HA/CA | Ch-1 |
| Ch-2 | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | Ch-2 |
| DMU/CBMA | UPI | Dummy | HA/CA | HA/CA | HA/CA | HA/CA | PCIe Gen4 x8/x16 |
| | Dummy | ICL Core | ICL Core | ICL Core | ICL Core | ICL Core | |

147

ARM, x86架构CPU配置信息

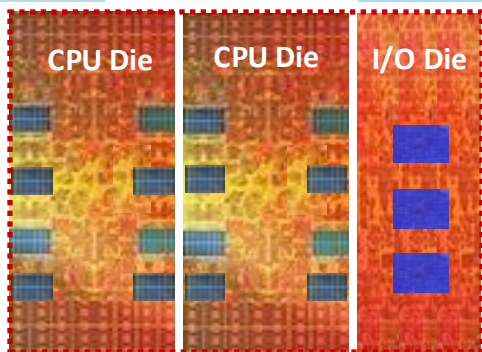
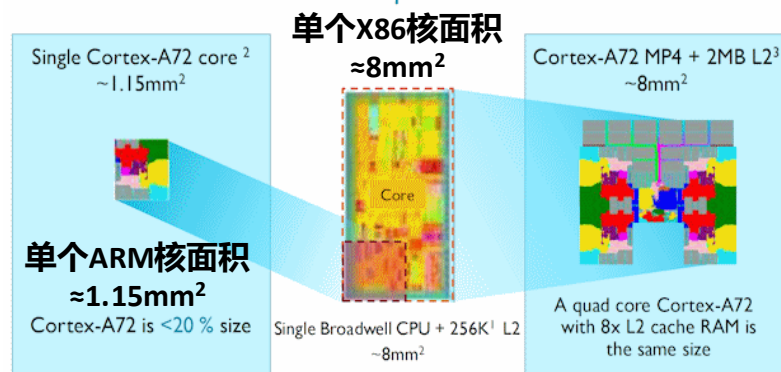
- ARM: 核心数量易扩展, 每核心cache容量中等
- AMD x86: 多模块设计, 核心数量易扩展, 每核心cache大
- Intel x86: 核心数量相对较少, L3共享cache容量大

CPU configuration	Huawei Kunpeng 920 7260 (ARM v8)	Intel(R) Xeon(R) Gold 6258R (CLX)	AMD EPYC 7742 (Rome Zen2)	Intel(R) Xeon(R) Platinum 8368 (ICX)	AMD EPYC 7763 (Milan Zen3)
Product family	2 nd Gen ARM v8	2 nd Gen Cascade Lake	2 nd Gen Rome Zen2	3 rd Gen Ice Lake	3 rd Gen Milan Zen3
Frequency	2.6GHz	2.70 GHz	2.25GHz	2.40 GHz	2.45GHz
Core count	64 core	Fully integrated 28 core	Multichip Module 64 Zen2 cores 8 x 8-core/die	Fully integrated 38 core	Multichip Module 64 Zen3 cores 8 x 8-core/die
Cache hierarchy	64KB L1 512KB L2 64MB L3 2 x 32MB 1MB/core	32KB L1 1MB L2 38.5MB L3 monolithic	32KB L1 512KB L2 256MB L3 8 x 32MB 16MB/4-core disaggregated	48KB L1 1.25MB L2 57MB L3 monolithic	32KB L1 512KB L2 256MB L3 8 x 32MB 32MB/8-core disaggregated
Memory	8 channels Per Socket Mem BW 190.7 GiB/s	6 channels Per Socket Mem BW 131.13 GB/s	8 channels Per Socket Mem BW 190.7 GB/s	8 channels Per Socket Mem BW ~200 GB/s	8 channels Per Socket Mem BW 204.8 GB/s

ARM CPU

- ARM: 使用精简指令集（RISC），大幅简化架构，仅保留所需要的指令，可以让整个处理器更为简化，拥有小体积、高效能的特性
- ARMv8架构支持64位操作，指令32位，寄存器64位，寻址能力64位
- 芯片的物理尺寸有限制，不能无限制的增加，ARM的众核横向扩展空间优势明显
- 鲲鹏CPU是多Die封装

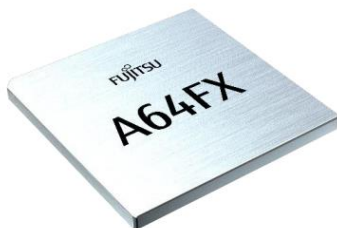
Cortex-A72: Ideal for dense compute environments



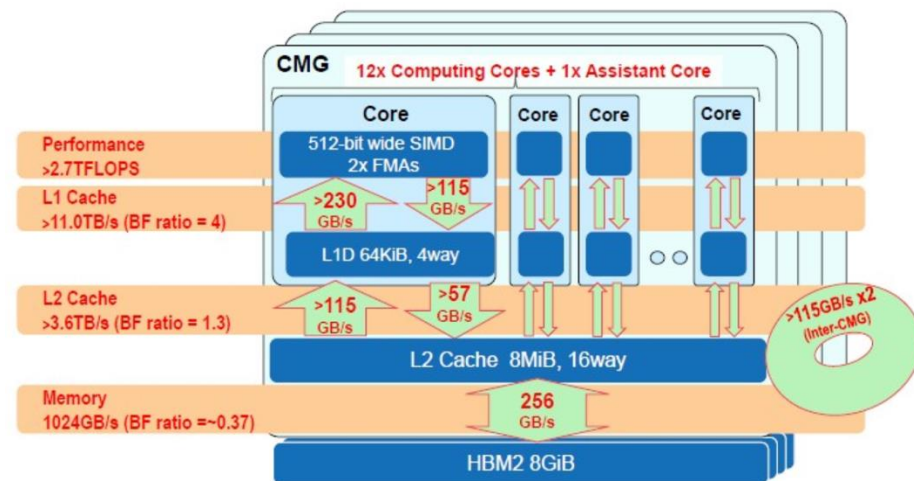
鲲鹏920芯片采用乐高架构封装而成，die内有ring通信，而die间需要通过SLLC接口进行通信

A64FX CPU

- 48核心
- 32GB HBM2高带宽内存，总带宽达到1TB/s

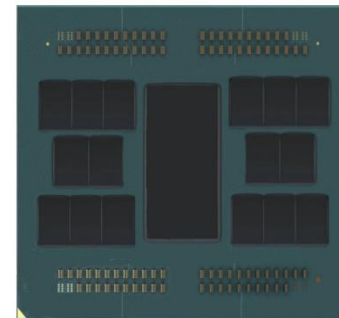
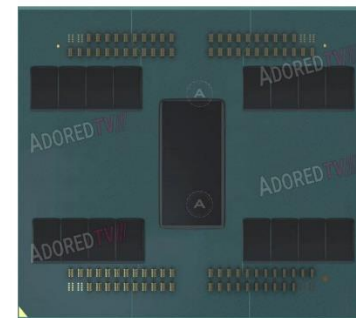
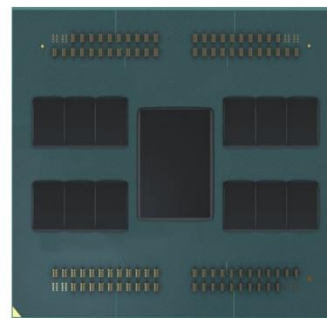
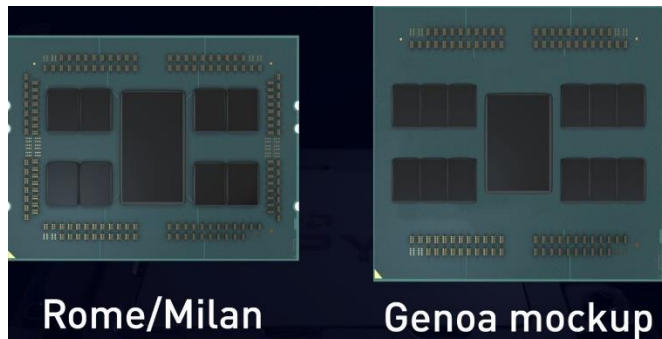
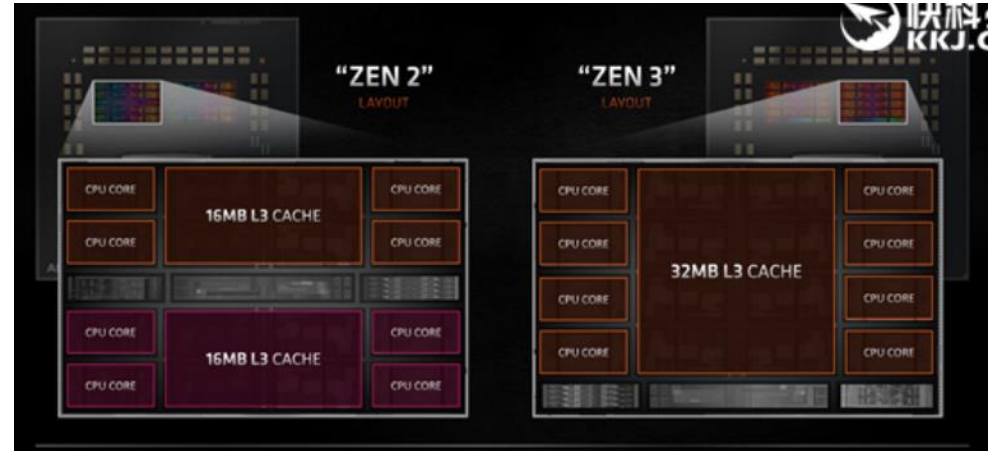
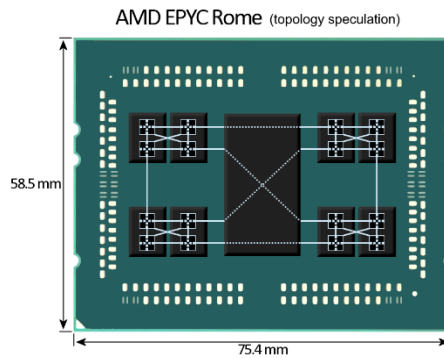
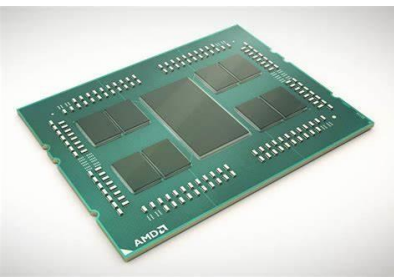


CPU specifications	
ISA	Arm v8.2 + SVE
Number of processor cores	48 compute cores, and 2 or 4 assistant cores *
Threads	48
Base Frequency	1.8GHz, 2.0GHz, 2.2GHz
Turbo Frequency	None (same as base frequency)
SIMD width	512bit
L1lcache size	3MiB (64KiB /core)
L1Dcache size	3MiB (64KiB /core)
L2 cache size	32MiB (8MiB x 4)
Cache-line size	256 bytes
Memory controller	4



AMD CPU

- 多模块CPU结构
- Zen2: 每4核心共享16MB L3 cache
- Zen3: 每8核心共享32MB L3 cache
- Zen4: 更多模块, 更多核心, 12-channel DDR5 memory controller



Intel CPU

- Intel Skylake芯片为一个die，片内通过mesh相互通信
- 较大的统一L3共享cache
- 各CPU各级cache访问延迟标识cache访问效率，cache容量与数据集大小决定综合性能



CHA – Caching & Home Agent
SF – Snoop Filter

Latency (cycles/ns)	Intel(R) Xeon(R) Gold 6258R (CLX)	AMD EPYC 7742 (Rome Zen2)	Intel(R) Xeon(R) Platinum 8368 (ICX)	AMD EPYC 7763 (Milan Zen3)
Local Socket Memory Latency (ns)	82	120	68	98
Remote Socket Memory Latency (ns)	146	252	122	282
L1 cache Latency(cycles/ns)	3.2/1.1	2.7/1.2	3.1/1.4	3.2/1.3
L2 cache Latency(cycles/ns)	11/3.8	8.7/3.9	8.6/3.8	11.9/4.9
L3 cache Latency(cycles/ns)	59/20	28/12	48/21	35/15
30MB buffer (ns)	26	117	21	16
50MB buffer (ns)	57	119	26	49
100MB buffer (ns)	76	119	49	76
500MB buffer (ns)	82	119	64	96

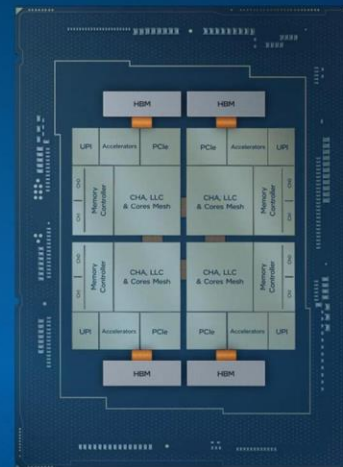
Intel CPU

- 更多核心
- 更大HBM2e缓存
- 内存通道数量未增长

Next Gen
Intel® Xeon® Scalable
Processors

+HBM

Up to
64 GB
HBM2e



	PURLEY	CEDAR ISLAND	WHITLEY	EAGLE STREAM
Platform / Processor Specification	Cascade Lake (2019) / Cascade Lake Refresh (2020)	Cooper Lake	Ice Lake	Sapphire Rapids
Core Count / CPU Socket	28 cores	28 cores	38 cores	56 cores
Socket Scalability (per node)	1S, 2S, 4S, 8S / 1S, 2S, 4S ⁽¹⁾	4S, 8S	1S, 2S	1S, 2S, 4S, 8S
Max TDP	205W	250W	270W ⁽²⁾	350W ⁽²⁾
Node controller support	Yes	Yes	No	Yes
Physical/Virtual Address Bits	46/48	46/48	52/57	52/57
Memory support (DDR4/DDR5)	DDR4	DDR4	DDR4	DDR5
# Memory channels	6	6	8	8
Memory max. speeds	2933 (1 DPC)	3200 (2 DPC)	3200 (2 DPC)	4800 (1 DPC) & 4400 (2 DPC)
High Bandwidth Memory (HBM)	No	No	No	Yes, 1TB/s BW, 64GB HBM2e per socket
# Intel® UPI links	UPI 1.0 (2, 3)	UPI 1.0 (6)	UPI 1.0 (2, 3)	UPI 2.0 (up to 4)
Intel® UPI speeds	Up to 10.4 GT/s	Up to 10.4 GT/s	Up to 11.2 GT/s	Up to 16 GT/s
PCIe Generation (I/O)	PCIe 3.0, 48 lanes (x16, x8, x4)	PCIe 3.0, 48 lanes (x16, x8, x4)	PCIe 4.0, 64 lanes (x16, x8, x4)	80 lanes, PCIe 5.0 (x16, x8, x4), PCIe 4.0 (x2)
Intel® Deep Learning Boost (AI Inference / Training)	AVX-512 (VNNI/INT8)	AVX-512 (VNNI/INT8 & BFloat16)	AVX-512 (VNNI/INT8)	AMX/TMUL (INT8 & BFloat16) & AVX-512 (VNNI/INT8)
Security – Intel® SGX	No	No	Yes	Enhanced
Crypto Instructions	Legacy	Legacy	Vector AES, SHA extensions, VPMADD52	Vector AES, SHA extensions, VPMADD52
Intel Optane memory support	Intel® Optane™ persistent memory module (Apache Pass)	Intel Optane Persistent Memory 200 Series (Barlow Pass) – validated on 4S only	Intel Optane Persistent Memory 200 Series (Barlow Pass)	Intel Optane Persistent Memory 300 Series (Crow Pass) – up to 2.6x 2R1W random-access BW ⁽³⁾
Compute Express Link (CXL)	No	No	No	Yes; spec 1.1, 4 x16 devices ⁽⁴⁾
Discrete FPGA accelerator	Yes (PCIe 3.0)	Yes (PCIe 3.0)	Yes (PCIe 4.0), UPI (POC only)	Yes (PCIe 5.0), CXL
SKU shelves	All / Gold6, Gold5, Silver & Bronze	Platinum, Gold SKUs All Barlow Pass capable	Targeted SKUs across all shelves (except Bronze)	All

6.4.7 高速缓存参数的性能影响

- 不命中率(Miss Rate)
 - 程序或程序一部分执行期间, 内存引用不命中的比率 (misses / accesses)
- 命中率(hit rate)
 - 命中的内存引用比率(hits / accesses)
 $\text{miss rate} = 1 - \text{hit rate}$
 - 不命中率典型取值:
 - 3-10% for L1
 - can be quite small (e.g., < 1%) for L2, depending on size, etc.
- 命中时间(Hit Time)
 - 从高速缓存传送一个字到CPU所需的时间
 - 包括组选择、行确认和字选择时间
- 不命中处罚(Miss Penalty)
 - 由于不命中所需要的额外时间
 - 典型时钟周期:
 - 4 clock cycle for L1
 - 10 clock cycles for L2
 - 50 clock cycles for L3
 - 200 clock cycles for main memory

高速缓存成本和性能的折中

• 1. 高速缓存大小的影响

- 较大的高速缓存：命中率提高；延迟提高
- 策略： $L1 < L2 < L3$
- 案例：
 - Skylake-SP (1MB L2 cache, 1.375MB L3 cache/core)
 - Skylake-S (256KB L2 cache, 2.5MB L3 cache/core)

• 2. 块大小的影响

- 较大的块：提高局部性，提高命中率；给定减少，损害时间局部性；块越大，不命中处

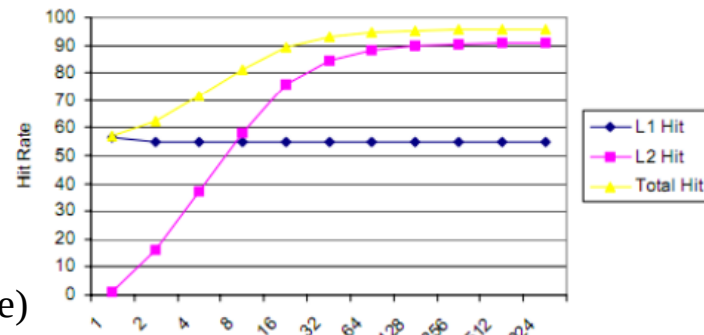
• 3. 相联度的影响

- 高速缓存行数E较大：
 - 优点：降低冲突不命中出现抖动的可能性
 - 缺点：成本较高，速度慢，每行额外的LRU状态中处罚

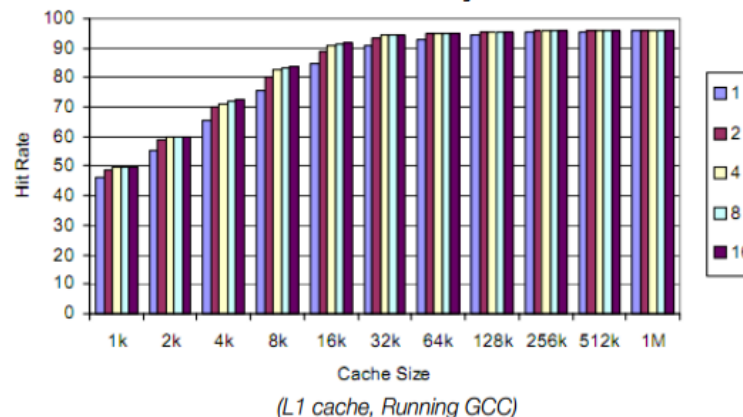
• 4. 写策略影响

- 直写：容易实现，可以使用独立的写缓冲区(write buffer)，读不命中不会触发内存写因而开销小
- 写回：传送少允许更多内存带宽用于执行DMA的I/O设备
- 存储层次向下，传送时间增加，减少传送更重要，写回设计越多

Hit Rates for Constant L1, Increasing L2



Hit Rate v. Associativity & Cache Size



6.5 编写高速缓存友好的代码

- 局部性好的代码有较好的cache命中率
 - 1) 让最常见的情况运行得快:
 - 80/20原则: 程序大部分时间花在少量核心函数上, 核心函数大部分时间花在少量循环上
 - 2) 尽量减少每个循环内部的缓存不命中数量:
 - 时间局部性: 对局部变量的反复引用, 编译器将其缓存在寄存器文件中
 - 空间局部性: 步长为1的引用模式, cache line一次加载多次访问
- 假设字长为4字节, cache line为4

```
Int sumarrayrows(int a[M][N])
{
    int I, j, sum=0;
    for(i=0; i<M; i++)
        for(j=0; j<N; j++)
            sum+=a[i][j];
    return sum;
}
```

$a[i][j]$	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	1[m]	2[h]	3[h]	4[h]	5[m]	6[h]	7[h]	8[h]
$i=1$	9[m]	10[h]	11[h]	12[h]	13[m]	14[h]	15[h]	16[h]
$i=2$	17[m]	18[h]	19[h]	20[h]	21[m]	22[h]	23[h]	24[h]
$i=3$	25[m]	26[h]	27[h]	28[h]	29[m]	30[h]	31[h]	32[h]

```
Int sumarraycols(int a[M][N])
{
    int I, j, sum=0;
    for(j=0; j<N; j++)
        for(i=0; i<M; i++)
            sum+=a[i][j];
    return sum;
}
```

$a[i][j]$	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	1[m]	5[m]	9[m]	13[m]	17[m]	21[m]	25[m]	29[m]
$i=1$	2[m]	6[m]	10[m]	14[m]	18[m]	22[m]	26[m]	30[m]
$i=2$	3[m]	7[m]	11[m]	15[m]	19[m]	23[m]	27[m]	31[m]
$i=3$	4[m]	8[m]	12[m]	16[m]	20[m]	24[m]	28[m]	32[m]

Virtual Denormalization via Array Index Reference for Main Memory OLAP (Extended Abstract)

Yansong Zhang^{##}, Xuan Zhou[#], Ying Zhang^{*}, Yu Zhang[†], Mingchuan Su[†], Shan Wang^{##}
[#] DEKE Lab at Renmin University, China
 zhou.xuan@outlook.com
[†] School of Information, Renmin University of China, China
 {zyszy, minchuan, swang}@ruc.edu.cn
^{##} National Survey Research Center at Renmin University of China, China
 zhangys_ruc@hotmail.com
^{*} Centrum Wiskunde & Informatica, Amsterdam, Netherlands
 y.zhang@cwi.nl

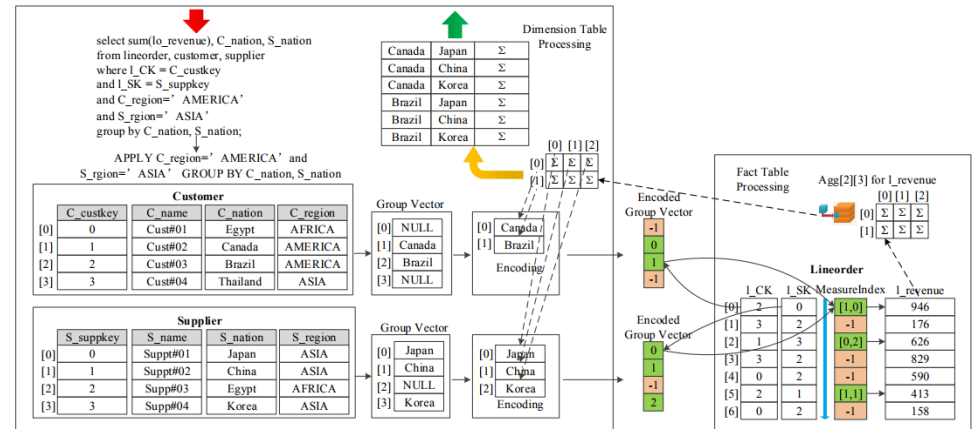
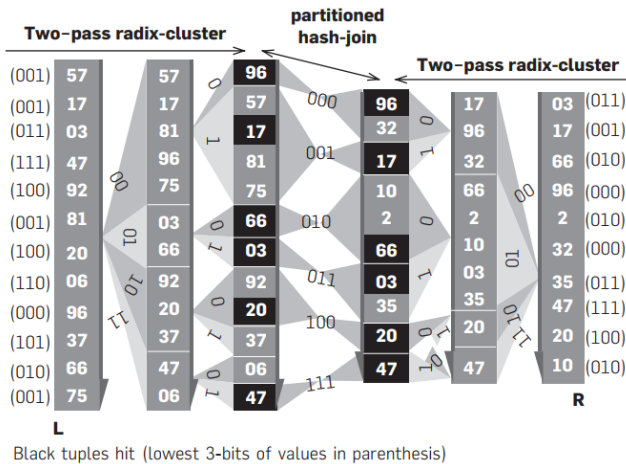
Breaking the Memory Wall in MonetDB

By Peter A. Boncz, Martin L. Kersten, and Stefan Manegold

<https://cse.buffalo.edu/~okennedy/courses/papers/monetdb.pdf>

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7498387>

Figure 3: Partitioned hash-join ($H = 8 \Leftrightarrow B = 3$).



An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory

Stefan Schuh, Xiao Chen, Jens Dittrich
 Information Systems Group
 Saarland University
<http://infosys.cs.uni-saarland.de>

<https://15721.courses.cs.cmu.edu/spring2018/papers/19-hashjoins/schuh-sigmod2016.pdf>

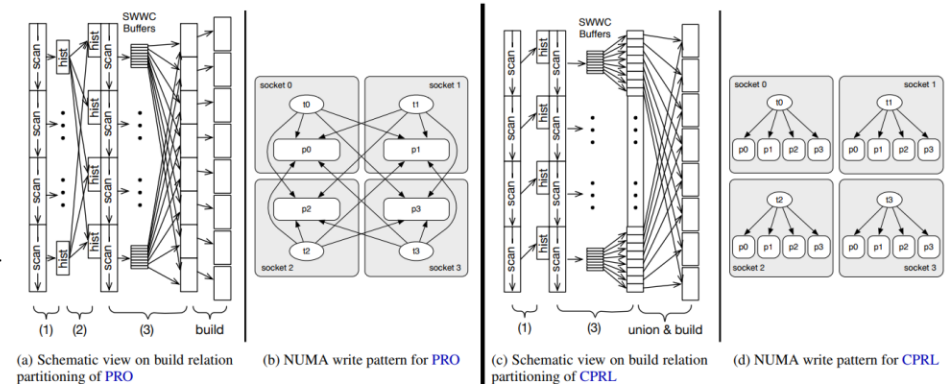
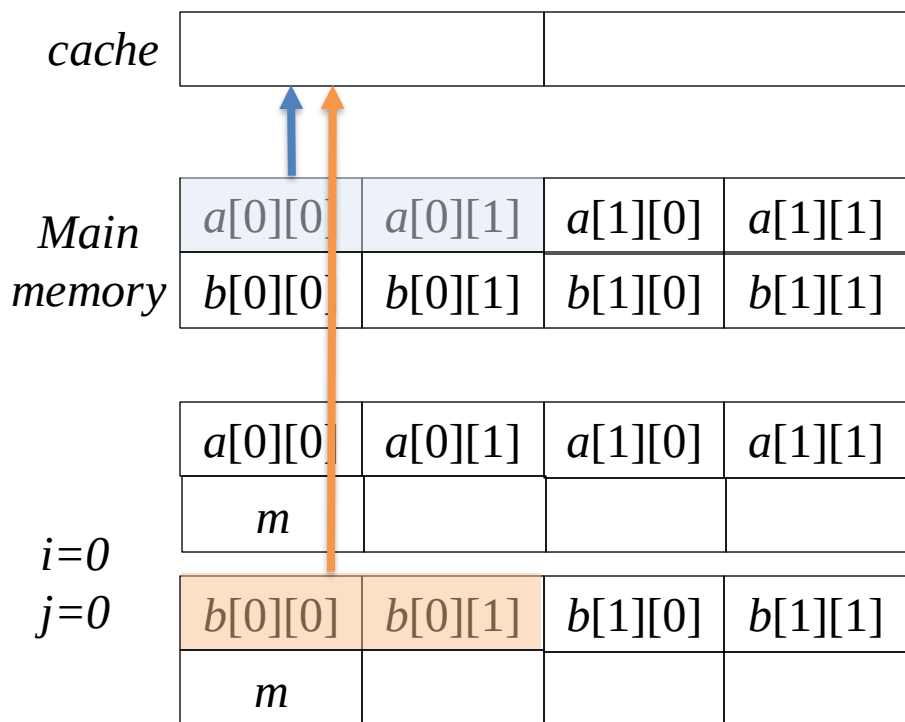


Figure 4: High-level schematic view and NUMA write pattern: PRO vs CPRL

练习题6.17 转置矩阵

- 假设：
 - `Sizeof(int)=4`
 - `Src`数组从地址0开始，`dst`数组从地址16开始
 - 只有一个L1 cache，直接映射、直写和写分配，块大小为8字节
 - Cache大小为16字节，开始为空
 - `Src`和`dst`数组访问分别是读和写不命中的唯一来源

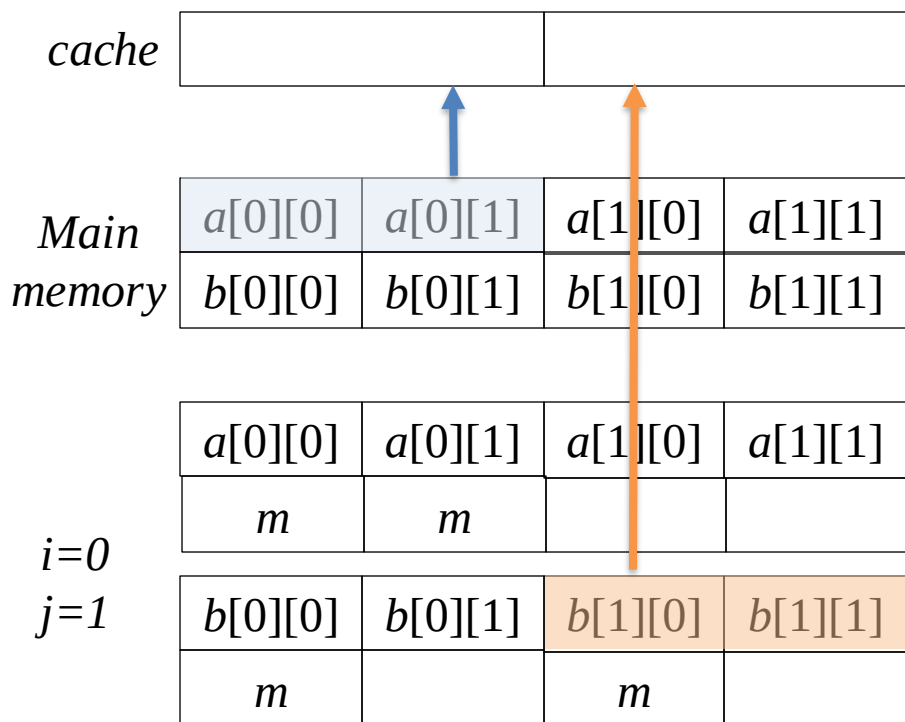
```
Typedef int array[2][2]
Void transpose1(array dst, array src)
{
    int i,j;
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            dst[j][i]=src[i][j];
        }
    }
}
```



练习题6.17 转置矩阵

- 假设：
 - `Sizeof(int)=4`
 - `Src`数组从地址0开始，`dst`数组从地址16开始
 - 只有一个L1 cache，直接映射、直写和写分配，块大小为8字节
 - Cache大小为16字节，开始为空
 - `Src`和`dst`数组访问分别是读和写不命中的唯一来源

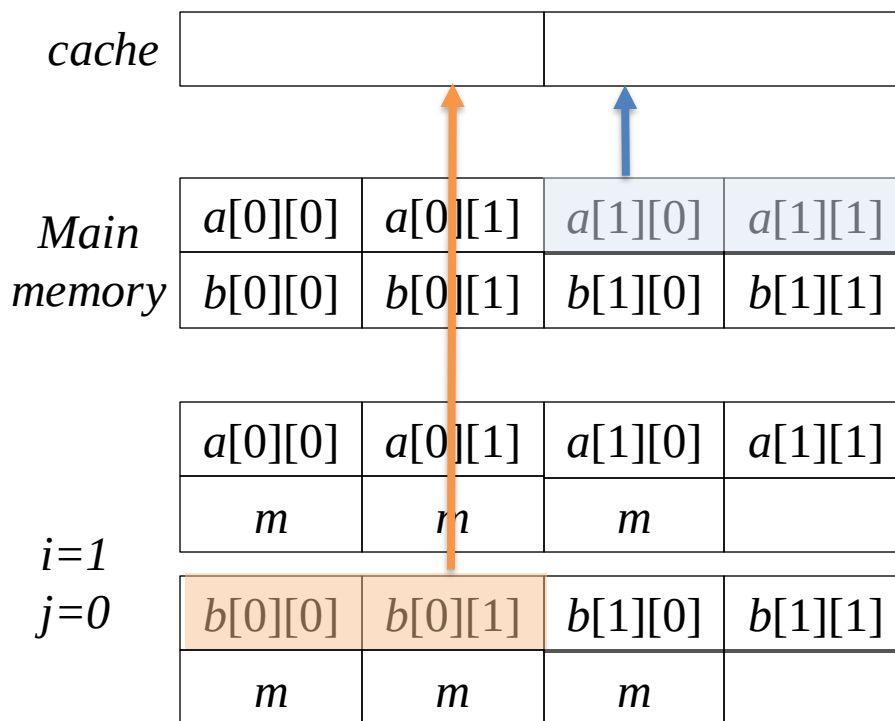
```
Typedef int array[2][2]
Void transpose1(array dst, array src)
{
    int i,j;
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            dst[j][i]=src[i][j];
        }
    }
}
```



练习题6.17 转置矩阵

- 假设：
 - `Sizeof(int)=4`
 - `Src`数组从地址0开始，`dst`数组从地址16开始
 - 只有一个L1 cache，直接映射、直写和写分配，块大小为8字节
 - Cache大小为16字节，开始为空
 - `Src`和`dst`数组访问分别是读和写不命中的唯一来源

```
typedef int array[2][2]
void transpose1(array dst, array src)
{
    int i, j;
    for (i=0; i<2; i++) {
        for (j=0; j<2; j++) {
            dst[j][i] = src[i][j];
        }
    }
}
```

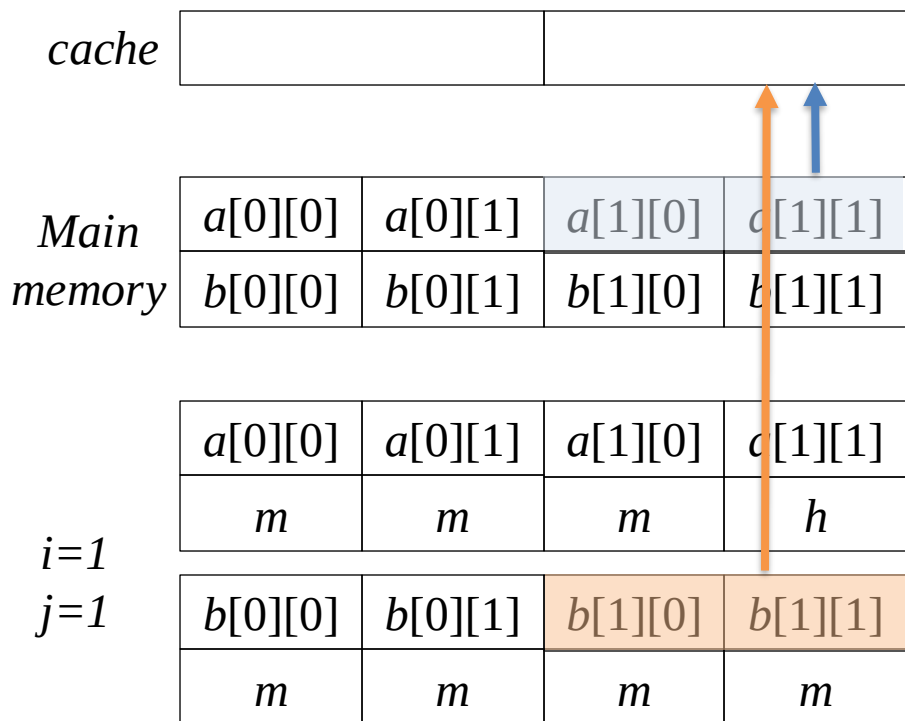


练习题6.17 转置矩阵

- 假设：
 - `Sizeof(int)=4`
 - `Src`数组从地址0开始，`dst`数组从地址16开始
 - 只有一个L1 cache，直接映射、直写和写分配，块大小为8字节
 - Cache大小为16字节，开始为空
 - `Src`和`dst`数组访问分别是读和写不命中的唯一来源

```

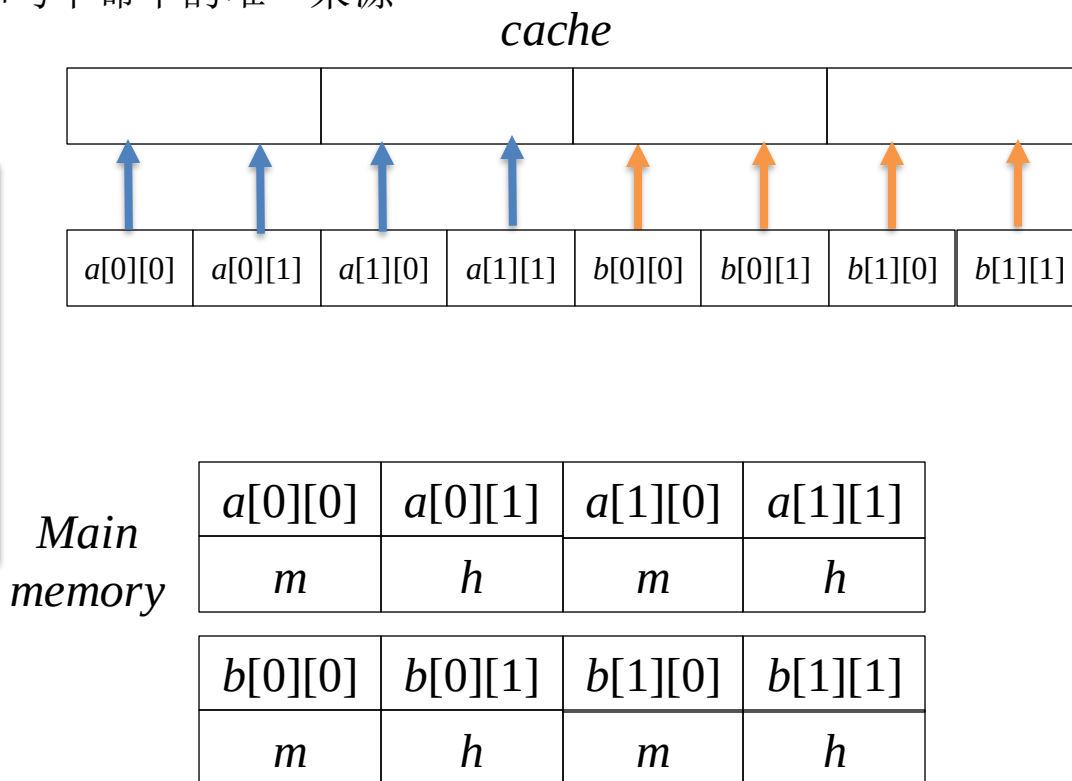
Typedef int array[2][2]
Void transpose1(array dst, array src)
{
    int i,j;
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            dst[j][i]=src[i][j];
        }
    }
}
    
```



练习题6.17 转置矩阵

- 假设：
 - `Sizeof(int)=4`
 - `Src`数组从地址0开始，`dst`数组从地址16开始
 - 只有一个L1 cache，直接映射、直写和写分配，块大小为8字节
 - Cache大小为**32**字节，开始为空
 - `Src`和`dst`数组访问分别是读和写不命中的唯一来源

```
Typedef int array[2][2]
Void transpose1(array dst, array src)
{
    int i,j;
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
            dst[j][i]=src[i][j];
        }
    }
}
```



作业-7



- P454
- 6.34
 - 目标：
 - 量化分析、估算代码的cache性能

练习题6.18 紧密循环cache命中率计算

- 硬件假设：
 - 块大小：16字节
 - Cache大小：1024字节
 - Sizeof(int)=4
 - Grid从内存地址0开始
 - Cache初始为空
 - 唯一内存访问是对数组grid元素的访问
 - 变量I,j,total_x,total_y存放在寄存器中
- 计算：
 - A. 读总数
 - B. Cache不命中的读总数
 - C. 不命中率

```
Struct algae_position{
    int x,y;
}
Struct algae_position grid[16][16];
Int total_x=0, total_y=0;
Int I,j;

For(i=0;i<16;i++){
    for(j=0;j<16;j++){
        total_x+=grid[i][j].x;
    }
}
For(i=0;i<16;i++){
    for(j=0;j<16;j++){
        total_y+=grid[i][j].y;
    }
}
```

Cache访问分析

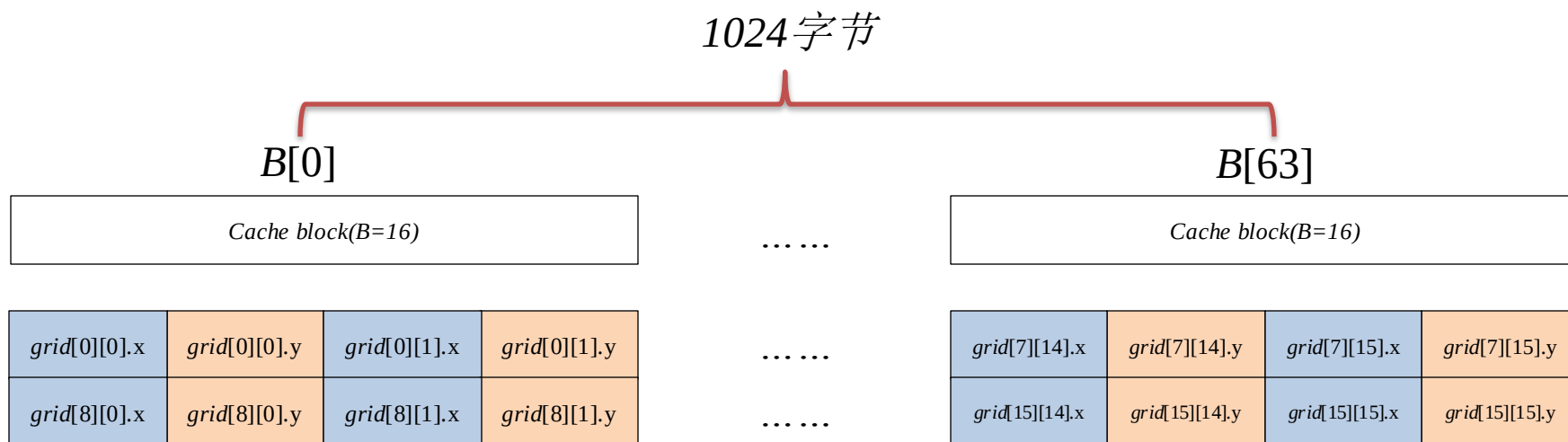
A. 读总数为:

– 每次读一个整数元素, 总数: $16*16*2=512$

B. 缓存不命中的读总数:

– 每个缓存块产生一个不命中, 每个循环产生 $64*2$ 个不命中, 两个循环共256次

C. 不命中率: $256/512=50\%$



练习题6.19

- A. 读总数为：512
- B. 缓存不命中的读总数：
- 每次j循环前1024字节产生8个不命中
 - 后1024字节与前半循环映射冲突，产生8个不命中(grid[0-7][i])
 - 16次循环共16*16=256次不命中(grid[8-15][i])
- C. 不命中率：256/512=50%

```
For (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        total_x+=grid[j][i].x;
        total_y+=grid[j][i].y;
    }
}
```

1024字节

B[0]

B[63]

Cache block(B=16)

Cache block(B=16)

....

....

....

grid[0][0].x grid[0][0].y grid[0][1].x grid[0][1].y

grid[8][0].x grid[8][0].y grid[8][1].x grid[8][1].y

grid[7][14].x grid[7][14].y grid[7][15].x grid[7][15].y

grid[15][14].x grid[15][14].y grid[15][15].x grid[15][15].y

练习题6.20

- A. 读总数为：512
- B. 缓存不命中的读总数：
 - 顺序步长为1访问，不命中为强制不命中
 - 不命中数为 $64 \times 2 = 128$
- C. 不命中率： $128 / 512 = 25\%$
- D. 缓存加倍，与数据相同时，强制不命中数量不会改变

```
For (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        total_x+=grid[i][j].x;
        total_y+=grid[i][j].y;
    }
}
```

1024字节

B[0]

B[63]

Cache block(B=16)

Cache block(B=16)

....

....

....

grid[0][0].x	grid[0][0].y	grid[0][1].x	grid[0][1].y
grid[8][0].x	grid[8][0].y	grid[8][1].x	grid[8][1].y

grid[7][14].x	grid[7][14].y	grid[7][15].x	grid[7][15].y
grid[15][14].x	grid[15][14].y	grid[15][15].x	grid[15][15].y

作业-7



- P455
- 6.37
 - 目标：
 - 量化分析、估算代码的cache性能

作业-8

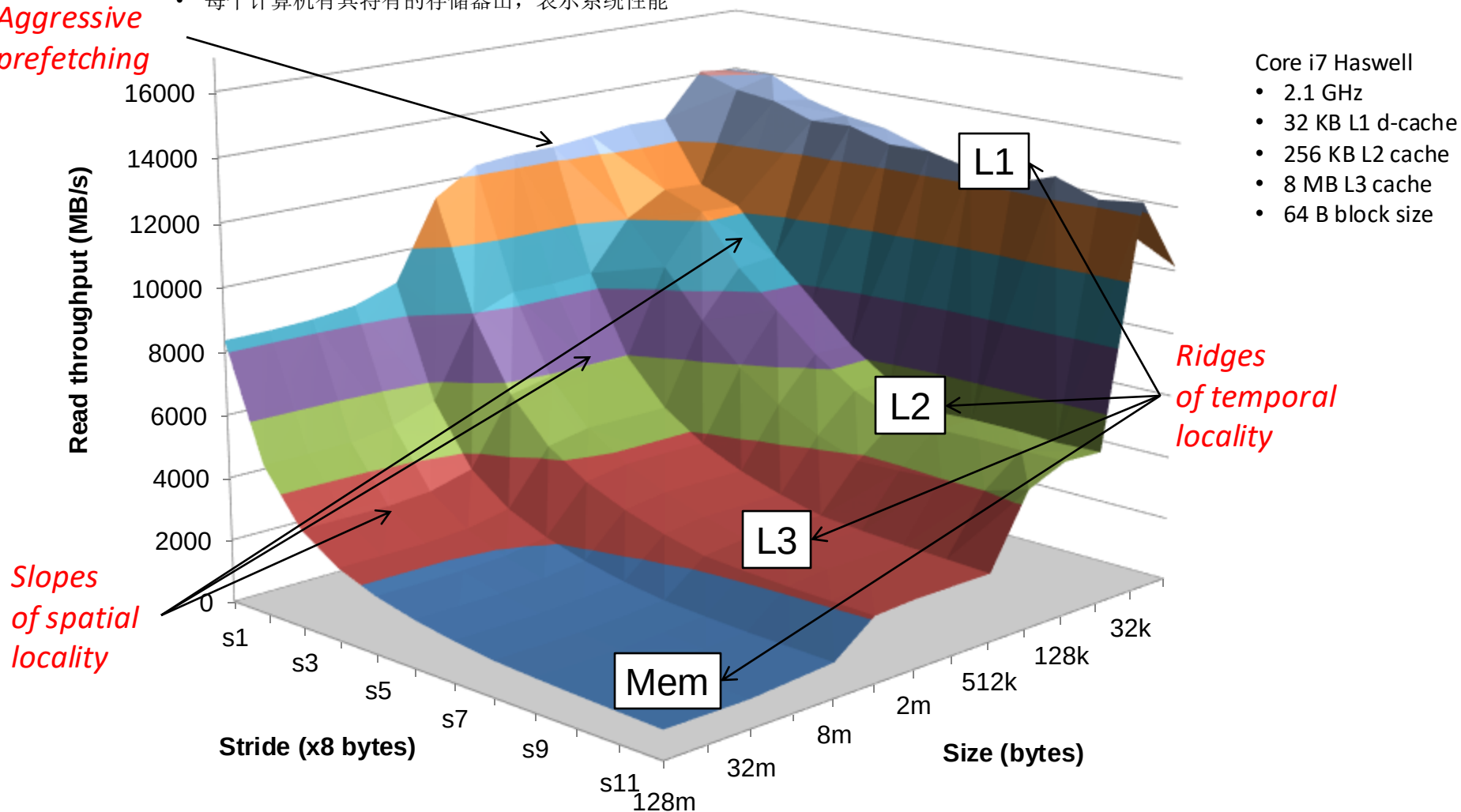


- P456
- 6.38-40
 - 目标：
 - 量化分析、估算代码的cache性能
 - 掌握通过实验及测试方法分析算法的cache性能
 - 编写三个相应的算法实现函数
 - 通过perf工具测试cache miss等相关的性能指标

6.6 高速缓存对程序性能的影响

- 6.6.1 存储器山
 - 读吞吐量(read throughput): 一个程序从存储读数据的速率, 有时称为读带宽(read bandwidth)
 - 存储器山: 读带宽的时间和空间局部性二维函数
 - 每个计算机有其特有的存储器山, 表示系统性能

Aggressive prefetching



存储器山测试函数

- 存储器山测试程序
 - 测量读序列读吞吐量函数
 - Stride步长扫描数组的头elems个元素，使用4*4展开
 - Size值越小，时间局部性越好，stride值越小空间局部性越好
 - test()函数包含elems和stride不同取值
 - 每一组elems和stride:
 - 1. 调用test()一次暖身(warm up the caches).
 - 2. 再次调用test()测试读吞吐量(read throughput(MB/s))

```
long data[MAXELEMS]; /* Global array to
traverse */

/* test - Iterate over first "elems" elements
of
*       array "data" with stride of "stride",
using
*       using 4x4 loop unrolling.
*/
int test(int elems, int stride) {
    long i, sx2=stride*2, sx3=stride*3,
    sx4=stride*4;
    long acc0 = 0, acc1 = 0, acc2 = 0, acc3 =
    0;
    long length = elems, limit = length - sx4;

    /* Combine 4 elements at a time */
    for (i = 0; i < limit; i += sx4) {
        acc0 = acc0 + data[i];
        acc1 = acc1 + data[i+stride];
        acc2 = acc2 + data[i+sx2];
        acc3 = acc3 + data[i+sx3];
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc0 = acc0 + data[i];
    }
    return ((acc0 + acc1) + (acc2 + acc3));
}
```

存储器山测试函数

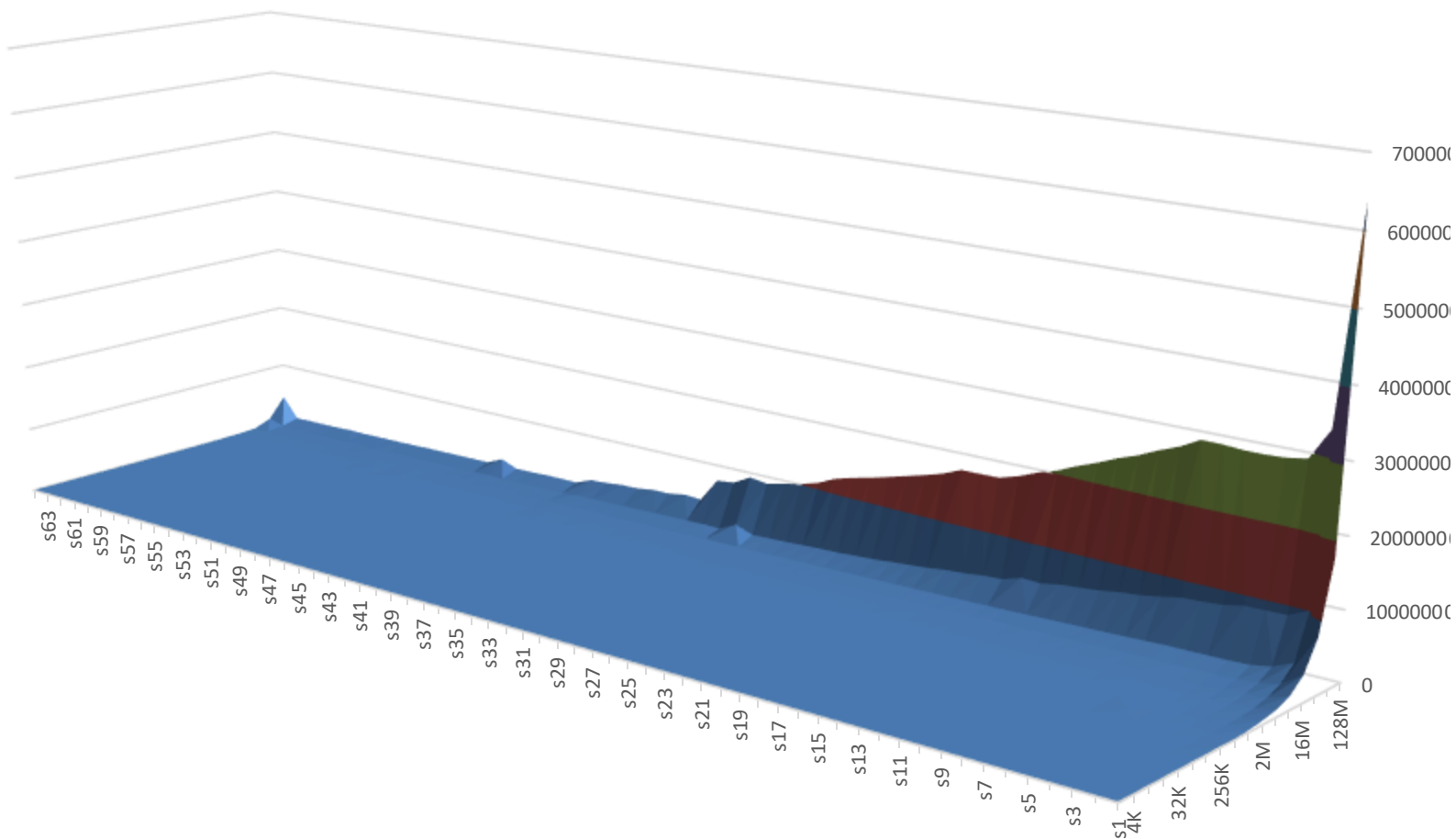
- 存储器山测试程序
 - 参考例程
 - 实验测试代码：以cycles为指标

```
/* The test function */
void test(int elems, int stride) {
    int i, result = 0;
    volatile int sink;
    for (i = 0; i < elems; i += stride)
        result += data[i];
    sink = result; /* So compiler doesn't optimize
away the loop */
}
/* Run test(elems, stride) and return read
throughput (MB/s) */
double run(int size, int stride, double Mhz)
{
    double cycles;
    int elems = size / sizeof(int);
    test(elems, stride); /* warm up the cache */
    cycles = fcyc2(test, elems, stride, 0); /*
call test(elems, stride) */
    return (size / stride) / (cycles / Mhz); /*
convert cycles to MB/s */
}
```

```
/* mountain.c - Generate the memory mountain. */
#define MINBYTES (1 << 10) /* Working set size
ranges from 1 KB */
#define MAXBYTES (1 << 23) /* ... up to 8 MB */
#define MAXSTRIDE 16 /* Strides range from 1 to
16 */
#define MAXELEMS MAXBYTES/sizeof(int)
int data[MAXELEMS]; /* The array we'll be
traversing */
int main()
{
    int size; Working set size in bytes) int stride;
    /* Stride (in array elements) */
    double Mhz; /* Clock frequency */
    init_data(data, MAXELEMS); /* Initialize each
element in data to 1 */
    Mhz = mhz(0); /* Estimate the clock frequency */
    for (size = MAXBYTES; size >= MINBYTES; size >>=
1) {
        for (stride = 1; stride <= MAXSTRIDE; stride++)
            printf("%.1f\t", run(size, stride, Mhz));
        printf("\n");
    }
    exit(0);
}
```

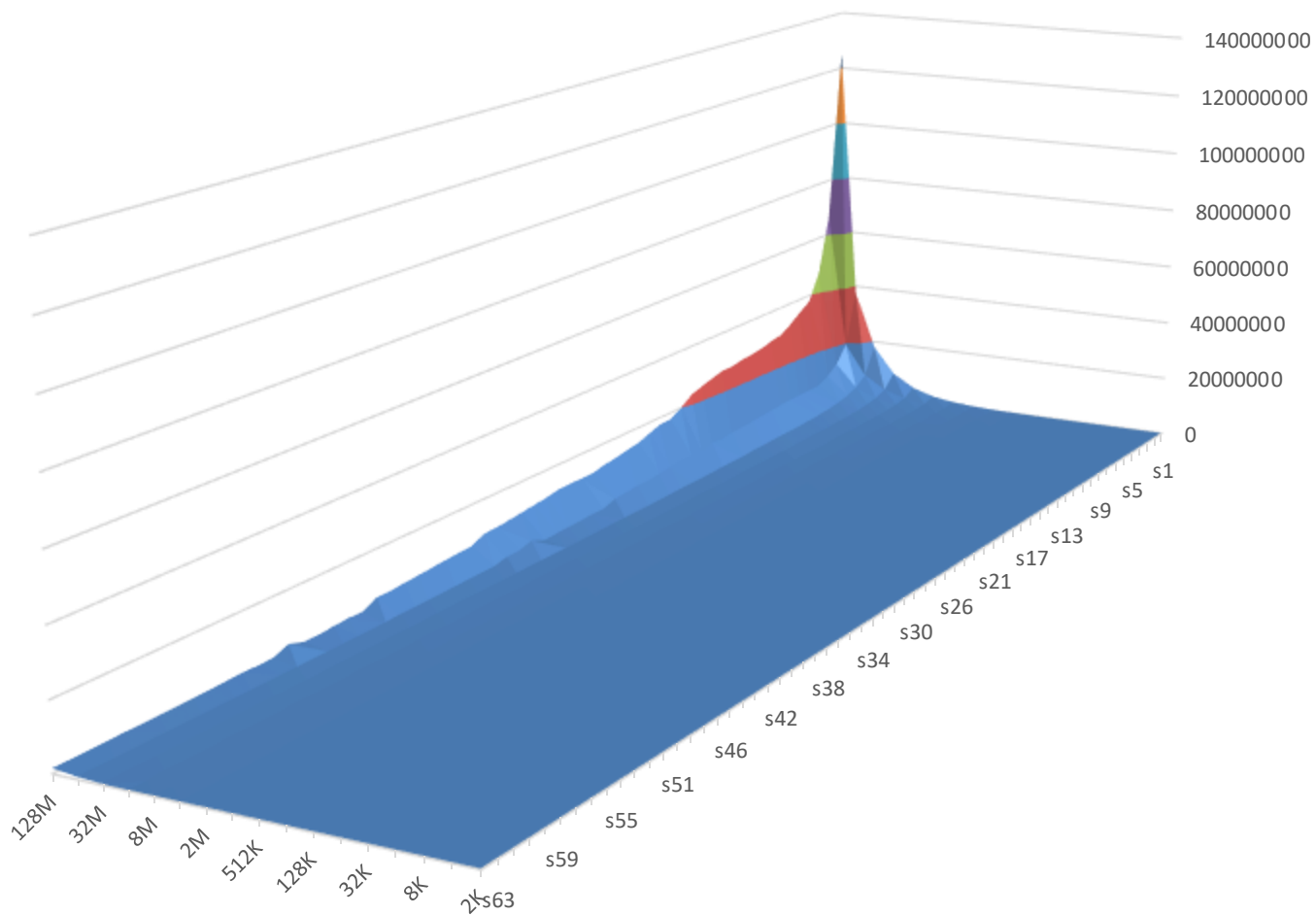
存储器山测试结果

- 无展开，

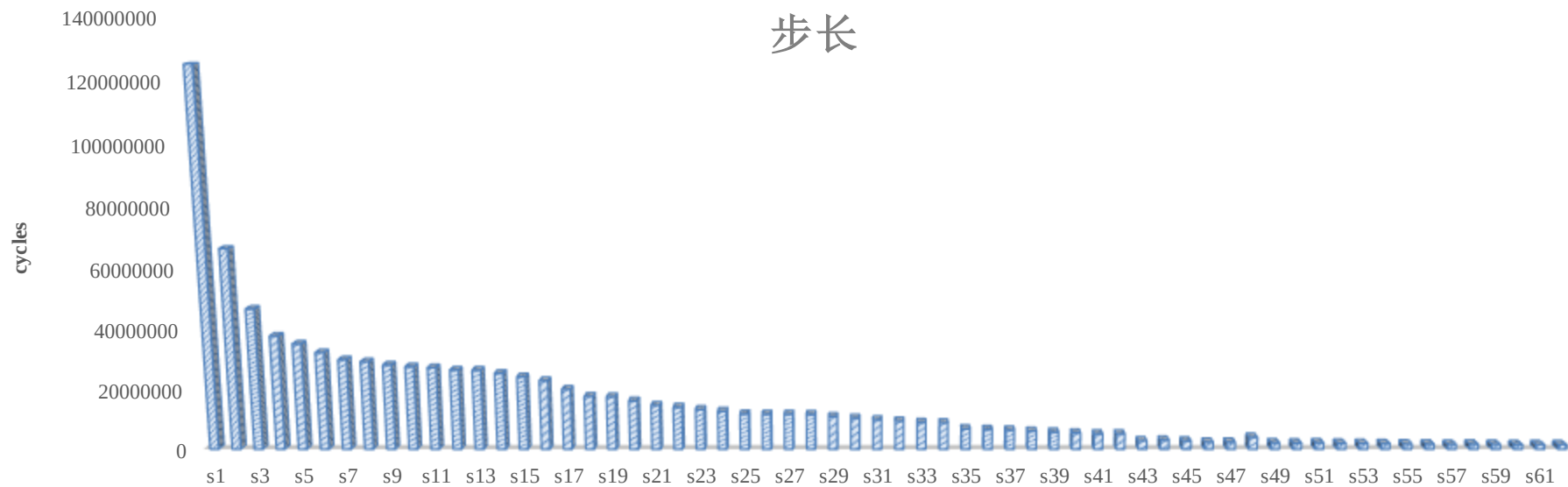
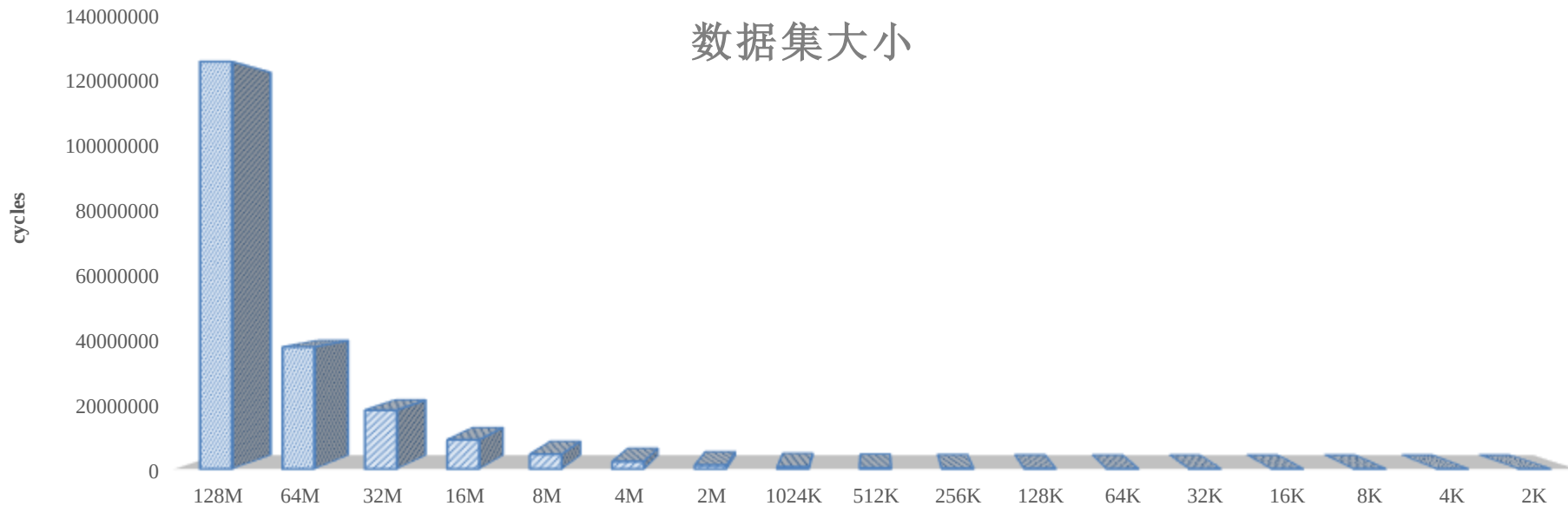


存储器山测试结果

- 4*4展开



工作集大小 vs. 步长



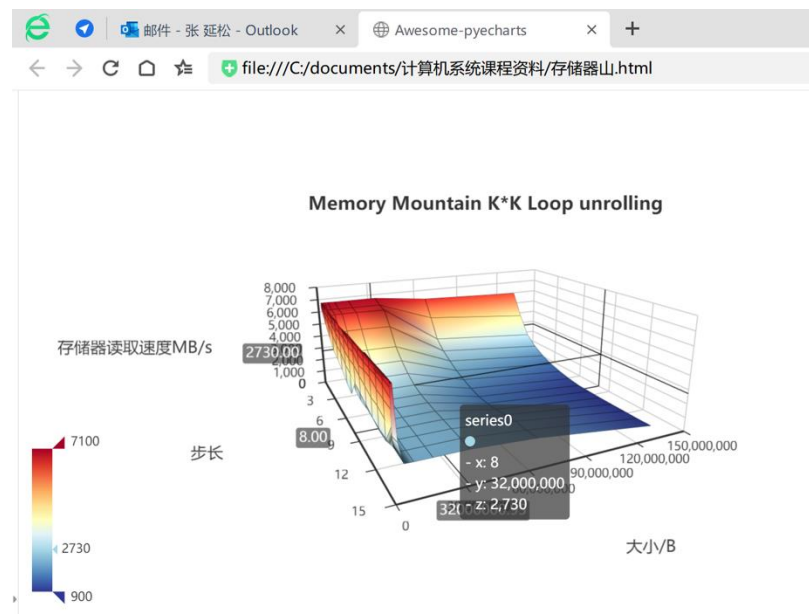
探索-9



• 实验重现

– 目标:

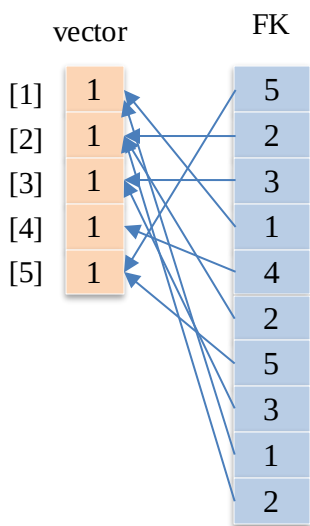
- 在自己的计算机进行内存山测试并生成内存山结果
- 结合自己计算机CPU硬件参数分析内存山的性能特征，确定内存山形状的主要影响因素
- 生成动态三维图查看
- 测试L1, L2, L3 cache大小、带宽、延迟性能指标



Cache locality evaluation——vector join

- 向量连接示例：

- Vector：代表主键表连接列，数组下标代表主键值
- FK：代表外键列，键值为所参照主键列数组下标
- 向量连接：外键列值映射为向量地址



- 局部性分析：

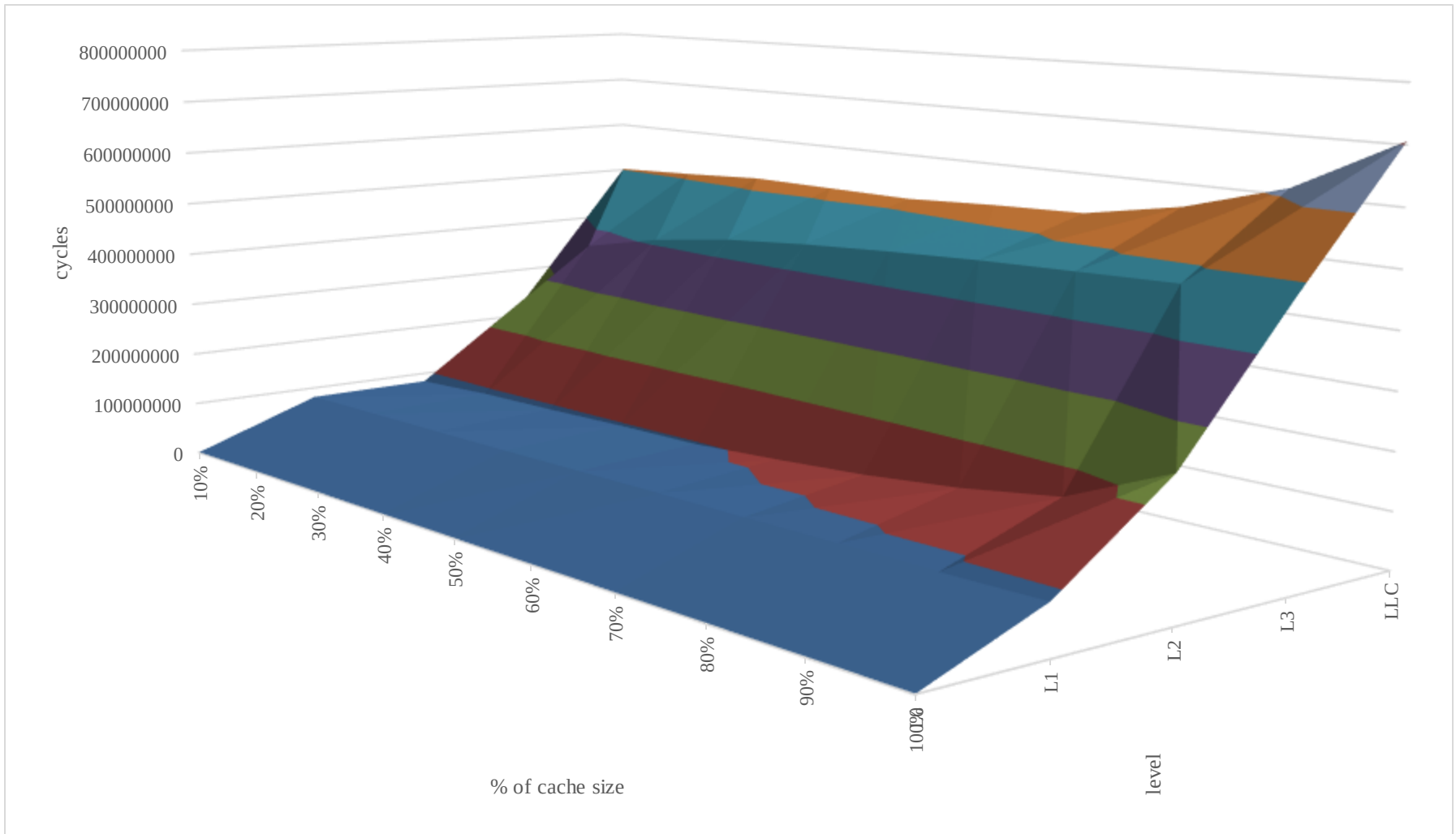
- FK列扫描操作：顺序列扫描操作，步长为1，空间局部性好
- Vector引用操作：随机访问vector向量单元，时间局部性取决于向量大小与cache大小之间的相对关系

- 两种cache访问模式：

- FK列扫描操作：弱局部性，数据一次性访问
- Vector引用访问：强局部性，数据重复访问
- 二种操作在cache中产生冲突——cache pollution
 - 弱局部性数据将强局部性数据驱逐出cache引起cache抖动
 - Vector和cache大小相同时cache性能亦降低

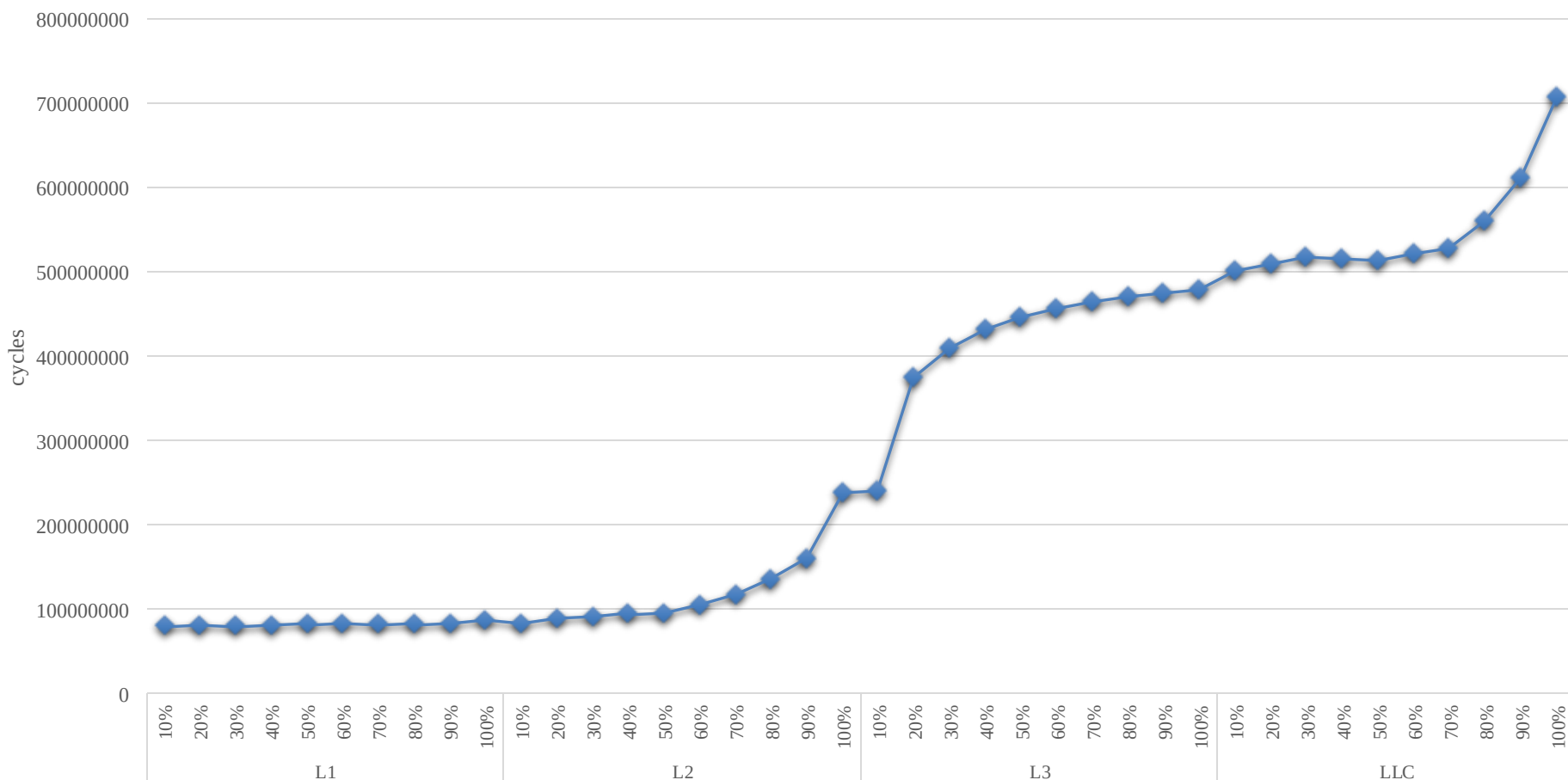
Cache locality evaluation——vector join

- 向量连接存储山



Cache locality evaluation——vector join

- 75%以上查询时间增长显著
- L1 cache大小向量连接性能最好，而且性能平稳



6.6.2 重新排列循环以提高空间局部性

- $n \times n$ 矩阵相乘问题: $C=AB$, 如果 $n=2$

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

- 其中:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

$$c_{22} = a_{21}b_{12} + a_{22}b_{22}$$

- 矩阵乘法实现: 3个嵌套循环, 算法时间复杂度为 $O(n^3)$

- 改变循环次序, 代码小改动 $\rightarrow$ 6个版本

- 算法假设:

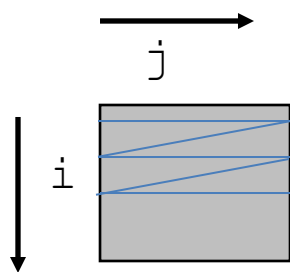
- 每个数组都是一个double类型的 $n \times n$ 的数组, $\text{sizeof}(\text{double})=8$
- 只有一个高速缓存, 其块大小为32字节 ($B=32$)
- 数组大小 n 很大, 以至于矩阵的一行都不能完全装进L1高速缓存
- 编译器将局部变量存储到寄存器中, 因此循环内对局部变量的引用不需要任何加载或存储指令

*Variable sum
held in register*

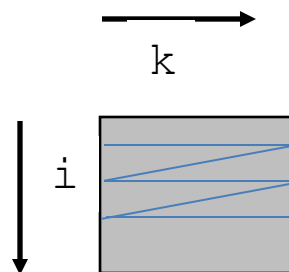
```
/* ijk */  
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        sum = 0.0;  
        for (k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

矩阵乘缓存不命中分析

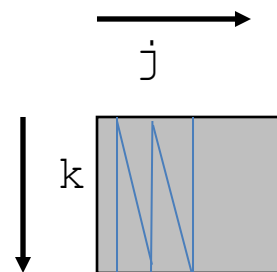
- 假设：
 - 块大小= 32B (容纳4个doubles元素)
 - 矩阵维(N)很大
 - Cache不足以容纳整个矩阵
- 分析方法：
 - 查看内循环访问模式



C



A



B

矩阵C在内存存储方式

- 矩阵C采用行模式数组存储
 - 每一行存储在连续的内存空间中
- 每次迭代扫描数组A的一行：
 - `for (i = 0; i < N; i++)`
 `sum += a[0][i];`
 - 访问连续的单元
 - 若块大小(B) > `sizeof(aij)` bytes, 访问具有空间局部性
 - 不命中率 = `sizeof(aij) / B`: n个数据共享一个cache miss
- 迭代时按列扫描:
 - `for (i = 0; i < n; i++)`
 `sum += a[i][0];`
 - 访问不连续的单元
 - 没有空间局部性
 - 不命中率 = 1 (i.e. 100%): 每个数据产生一个cache miss

矩阵乘 (ijk)

- A一次迭代, B n次迭代

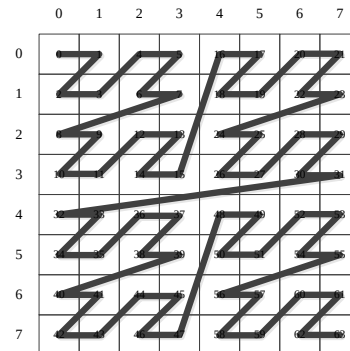


```
/* ijk */
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        sum = 0.0;
        for (k=0; k<n; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
}
```

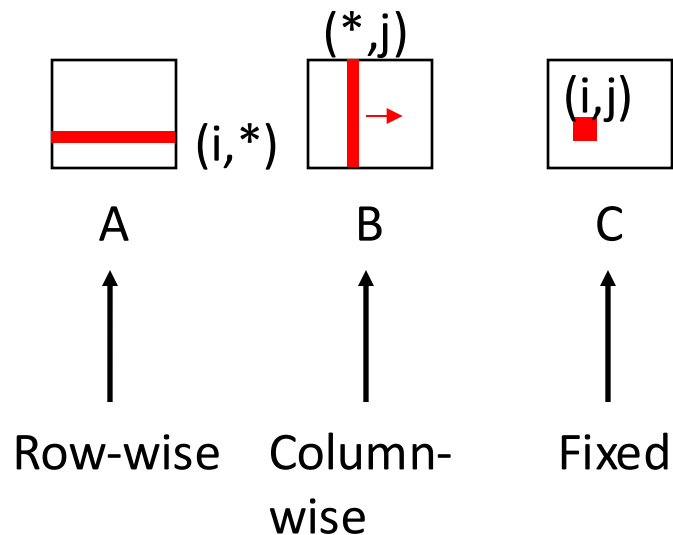
matmult/mm.c

理论分析: 如何解决局部性困境?

- Block column
- SIMD
- 矩阵转置
- 按局部性存储
- 专用矩阵乘法器



Inner loop:



Misses per inner loop iteration:

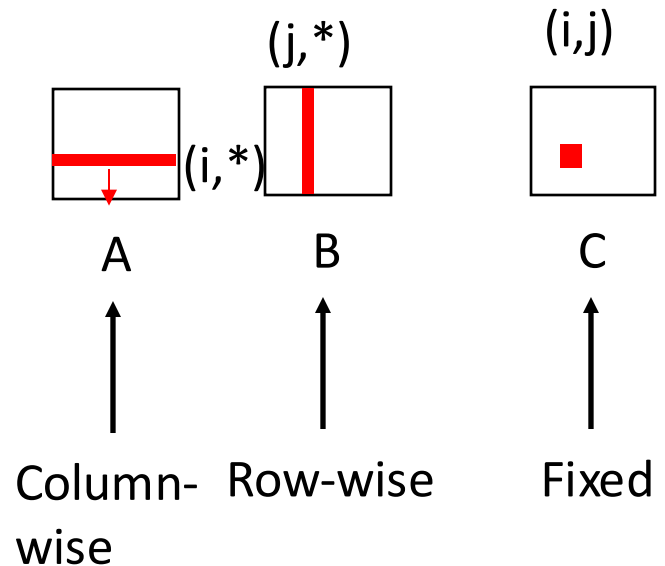
<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

矩阵乘 (jik)

```
/* jik */
for (j=0; j<n; j++) {
    for (i=0; i<n; i++) {
        sum = 0.0;
        for (k=0; k<n; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

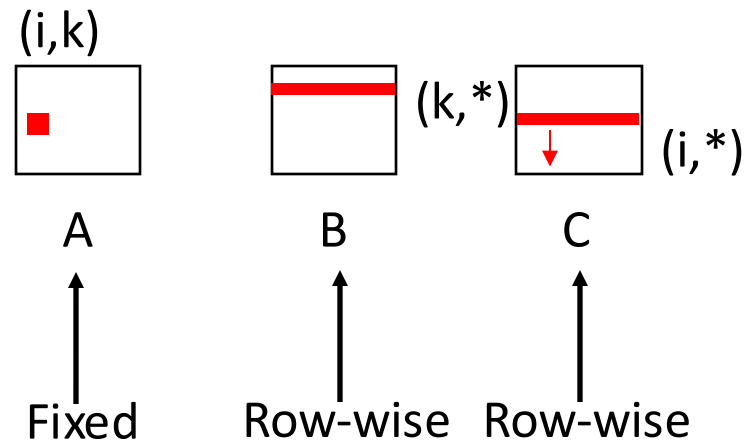
<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

矩阵乘 (kij)

```
/* kij */
for (k=0; k<n; k++) {
    for (i=0; i<n; i++) {
        r = a[i][k];
        for (j=0; j<n; j++)
            c[i][j] += r * b[k][j];
    }
}
```

matmult/mm.c

Inner loop:



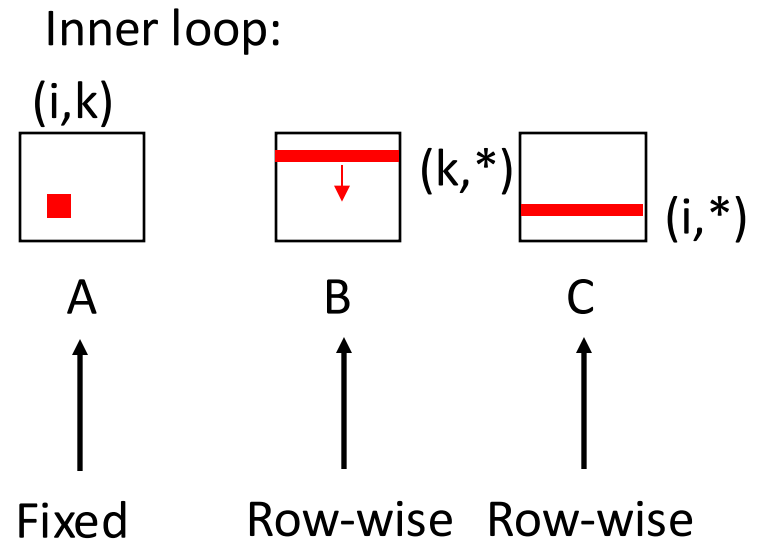
Misses per inner loop iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

矩阵乘 (ikj)

```
/* ikj */
for (i=0; i<n; i++) {
    for (k=0; k<n; k++) {
        r = a[i][k];
        for (j=0; j<n; j++)
            c[i][j] += r * b[k][j];
    }
}
```

matmult/mm.c



Misses per inner loop iteration:

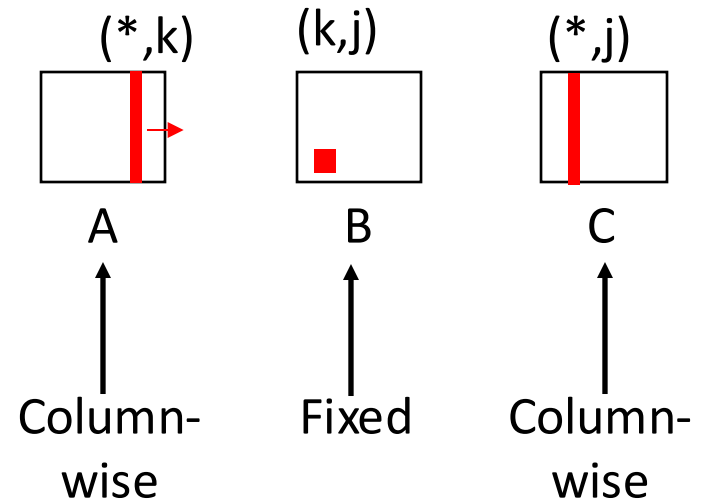
<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

矩阵乘 (jki)

```
/* jki */
for (j=0; j<n; j++) {
    for (k=0; k<n; k++) {
        r = b[k][j];
        for (i=0; i<n; i++)
            c[i][j] += a[i][k] * r;
    }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

A
1.0

B
0.0

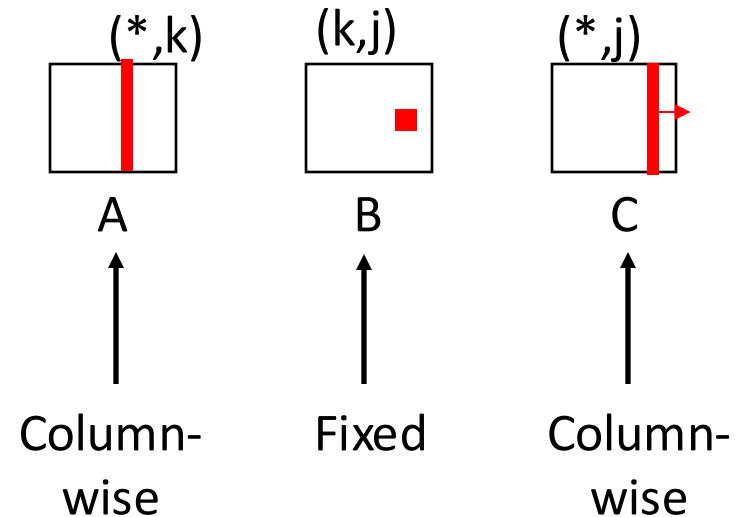
C
1.0

矩阵乘 (kji)

```
/* kji */
for (k=0; k<n; k++) {
    for (j=0; j<n; j++) {
        r = b[k][j];
        for (i=0; i<n; i++)
            c[i][j] += a[i][k] * r;
    }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

<u>A</u>	<u>B</u>	<u>C</u>
1.0	0.0	1.0

矩阵乘算法汇总

```
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        sum = 0.0;  
        for (k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

```
for (j=0; j<n; j++) {  
    for (k=0; k<n; k++) {  
        r = b[k][j];  
        for (i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

```
for (k=0; k<n; k++) {  
    for (i=0; i<n; i++) {  
        r = a[i][k];  
        for (j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

ijk (& jik):

- 加载: 2, 存储: 0
- A未命中: 0.25
- B未命中: 1
- C未命中: 0
- 未命中总数: 1.25

jki (& kji):

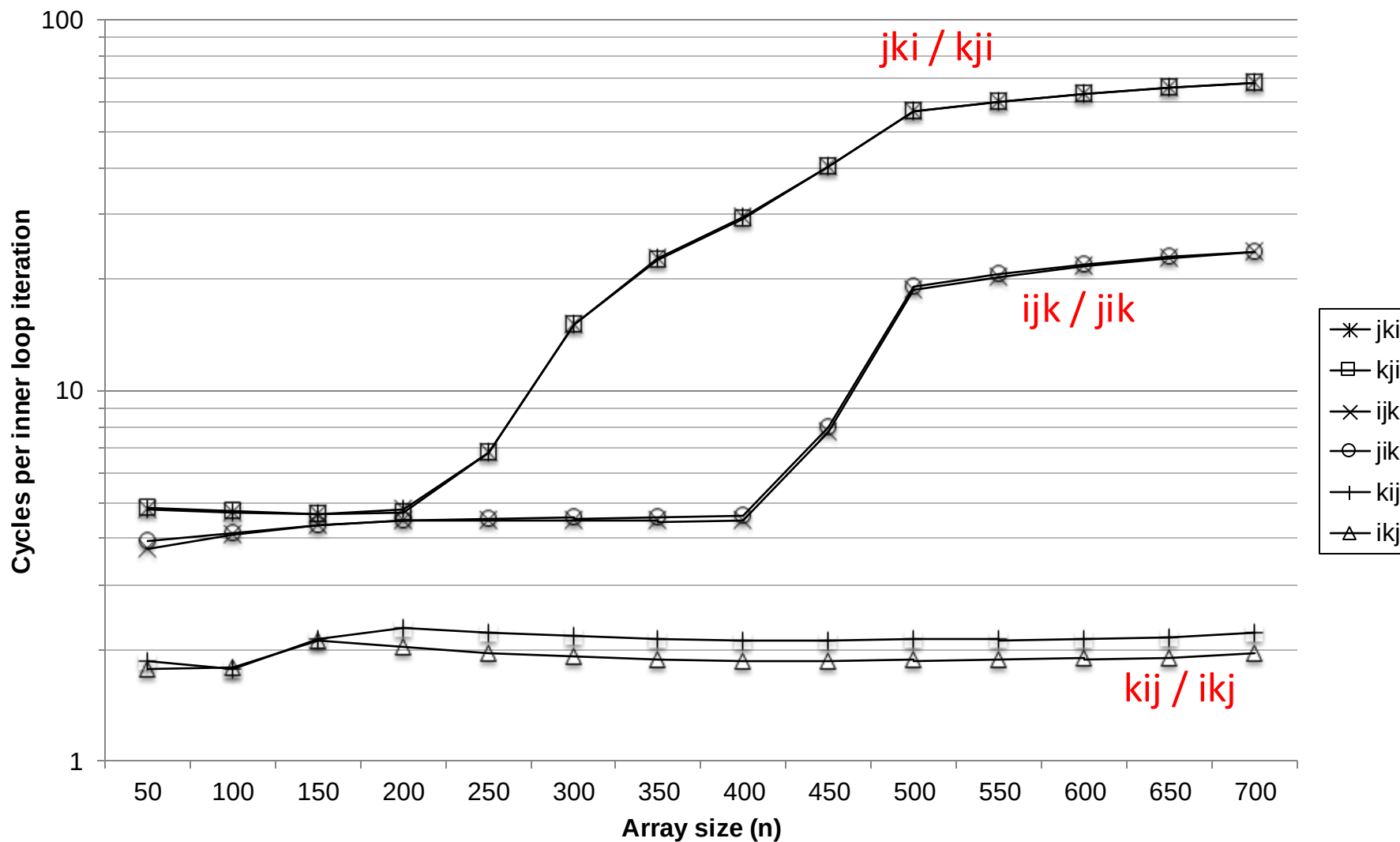
- 加载: 2, 存储: 1
- A未命中: 1
- B未命中: 0
- C未命中: 1
- 未命中总数: 2

kij (& ikj):

- 加载: 2, 存储: 1
- A未命中: 0
- B未命中: 0.25
- C未命中: 0.25
- 未命中总数: 0.5

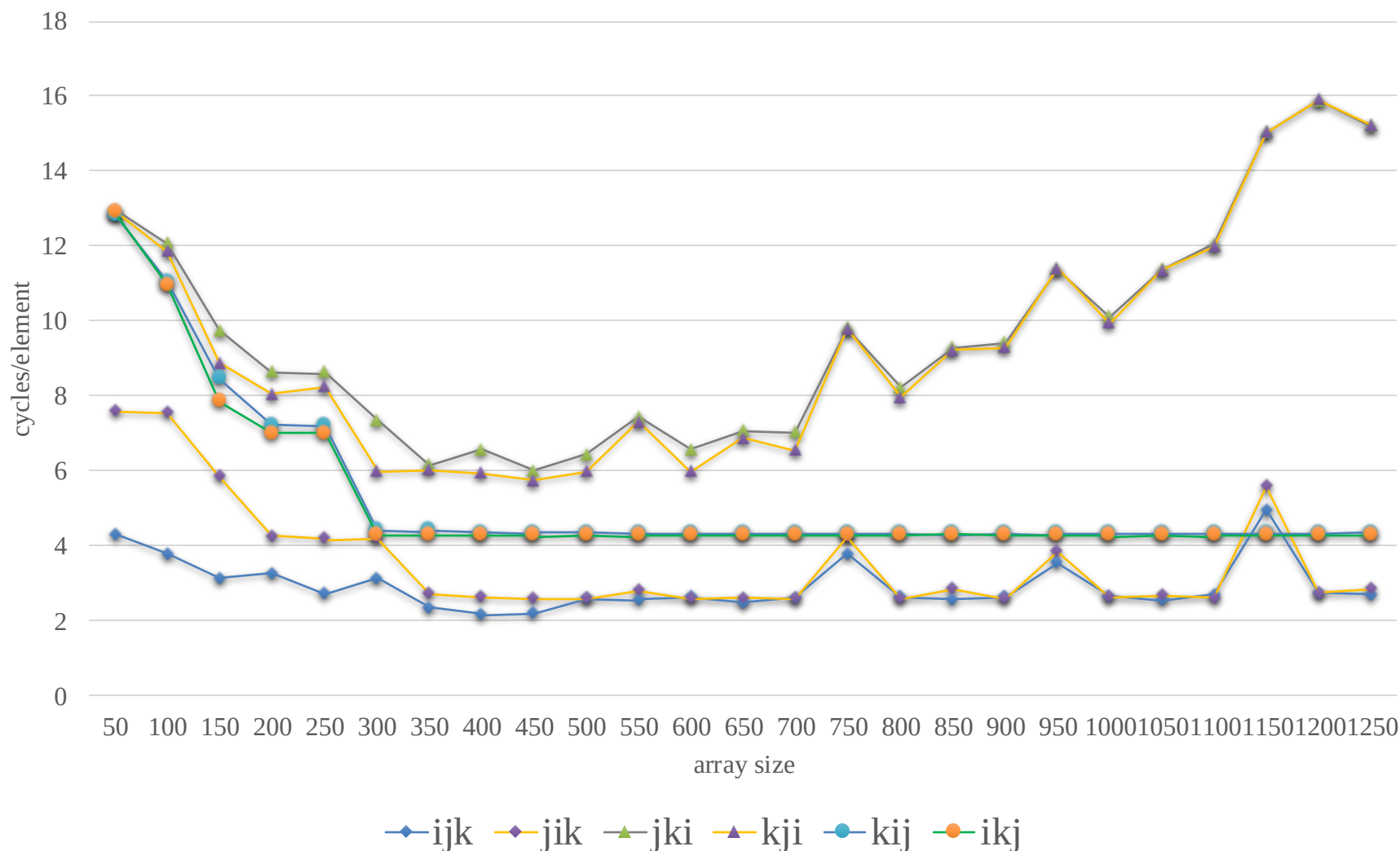
Core i7矩阵乘性能曲线

- 显著分为三类性能曲线



实测矩阵乘性能-long

- Intel Xeon E5-2699 v4@2.2GHz 22核心 CPU, 55MB L3 cache,



矩阵乘分析

- 基于指针的naïve矩阵乘法代码

```
c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            for (k = 0; k < n; k++)
                c[i*n + j] += a[i*n + k] * b[k*n + j];
}
```

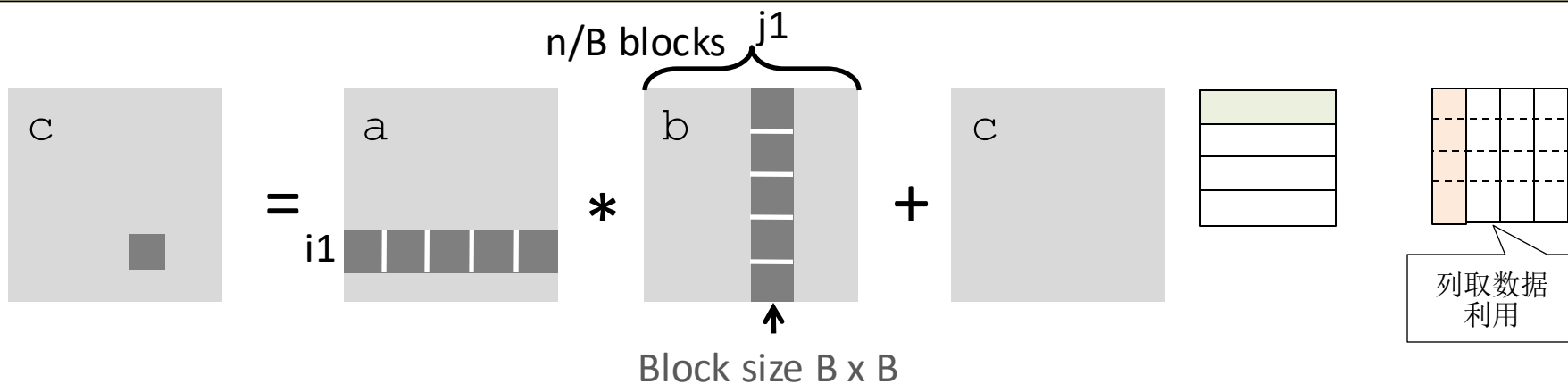


基于缓存块的矩阵乘

```
c = (double *) calloc(sizeof(double), n*n);

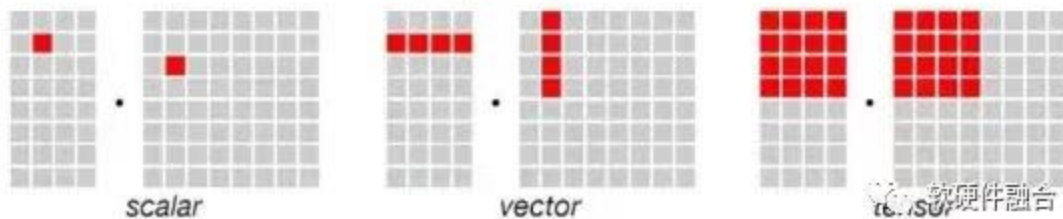
/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i+=B)
        for (j = 0; j < n; j+=B)
            for (k = 0; k < n; k+=B)
                /* B x B mini matrix multiplications */
                for (i1 = i; i1 < i+B && i1<n; i1++)
                    for (j1 = j; j1 < j+B && j1<n; j1++)
                        for (k1 = k; k1 < k+B && k1<n; k1++)
                            c[i1*n+j1] += a[i1*n + k1]*b[k1*n + j1];
}
```

matmult/bmm.c

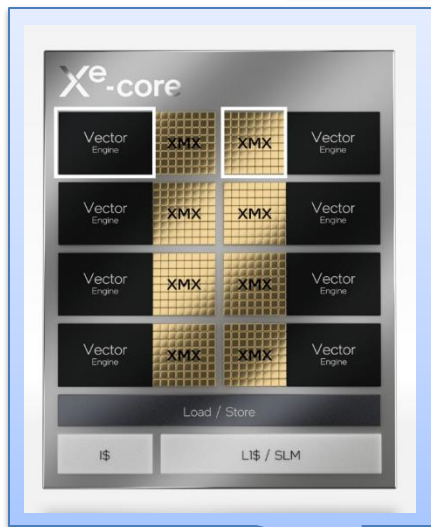


矩阵乘法加速技术

Compute Primitive

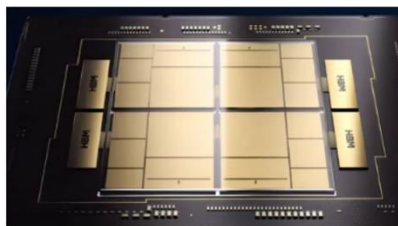


标量处理 → 向量处理 → 矩阵处理



英特尔通过AMX加速在第4代XeonSapphireRapidsCPU上提升AI稳定扩散性能

最近推出的英特尔第4代至强“SapphireRapids”CPU在云和数据中心领域得到了加速采用。英特尔付出额外努力的关键领域之一是其用于深度学习加速的硬件功能集，该功能集通过新的AMX(高级矩阵扩展)加速器得到提升。



性能测试结果

```
[nsrc@localhost ~]$ gcc -o 6.6.2 6.6.2.c -O1
```

```
[nsrc@localhost ~]$ 6.6.2
```

Matrix product (cycles)

M_size	ijk	jik	jki	kji	kij	ikj	mmm	mmm_block
50	4.19	7.49	18.43	18.40	18.41	18.40	10.22	8.79
100	3.81	7.15	17.00	15.87	15.26	14.62	8.07	6.45
150	2.98	5.33	12.44	11.53	10.54	10.04	5.97	4.75
200	3.27	4.18	10.42	10.48	10.08	10.02	6.60	4.94
250	2.89	4.16	10.50	10.53	9.36	6.14	4.09	3.02
300	2.22	2.57	6.86	6.92	6.17	6.13	4.40	3.10
350	2.34	2.61	6.51	6.51	6.17	6.13	4.11	3.06
400	2.32	2.55	6.90	7.01	6.17	6.13	4.47	3.11
450	2.17	2.56	6.68	6.73	6.16	6.13	4.22	3.03
500	2.64	2.59	6.91	7.02	6.16	6.12	4.59	3.10
550	4.07	4.86	8.99	9.11	6.16	6.13	5.45	2.98
600	2.68	2.59	7.06	7.10	6.16	6.13	4.62	3.11
650	2.49	2.61	7.57	7.58	6.15	6.12	4.33	3.06
700	2.66	2.58	7.45	7.52	6.16	6.13	4.61	3.11
750	3.82	4.30	10.19	10.18	6.16	6.13	5.34	3.06
800	2.68	2.57	8.42	8.51	6.16	6.12	4.64	3.12
850	2.60	2.86	9.70	9.73	6.16	6.12	4.40	3.09
900	2.67	2.60	9.63	9.66	6.15	6.17	4.63	3.11
950	3.59	3.89	11.20	11.21	6.16	6.12	5.10	2.99
1000	2.70	2.60	10.99	11.04	6.15	6.13	4.67	3.12

```
[nsrc@localhost ~]$ gcc -o 6.6.2 6.6.2.c -O2
```

```
[nsrc@localhost ~]$ 6.6.2
```

Matrix product (cycles)

M_size	ijk	jik	jki	kji	kij	ikj	mmm	mmm_block
50	4.14	7.60	12.99	12.93	12.92	12.84	5.38	6.87
100	3.85	7.40	11.99	11.73	11.04	10.92	3.96	5.43
150	3.16	5.64	9.65	8.67	7.87	7.35	2.76	3.67
200	3.16	4.20	8.65	8.04	7.15	7.01	3.54	3.82
250	2.85	4.17	8.44	8.00	7.17	7.01	3.31	3.31
300	2.34	2.63	6.31	5.89	4.43	4.33	2.42	2.40
350	2.35	2.65	5.67	5.54	4.42	4.36	2.55	2.41
400	2.27	2.62	6.50	5.99	4.36	4.29	2.44	2.42
450	2.21	2.60	6.04	5.76	4.35	4.29	2.36	2.35
500	2.62	2.60	6.46	5.99	4.35	4.29	2.72	2.40
550	4.45	4.92	8.84	8.92	4.35	4.29	4.48	2.32
600	2.65	2.60	6.61	6.03	4.34	4.29	2.75	2.42
650	2.47	2.62	7.16	6.95	4.34	4.29	2.59	2.38
700	2.64	2.58	7.16	6.68	4.34	4.29	2.74	2.39
750	3.81	4.28	9.95	9.91	4.34	4.29	3.87	2.38
800	2.66	2.58	8.21	7.92	4.34	4.29	2.76	2.43
850	2.58	2.85	9.40	9.33	4.34	4.29	2.67	2.40
900	2.65	2.60	9.69	9.56	4.35	4.29	2.75	2.40
950	3.58	3.88	11.42	11.42	4.33	4.29	3.64	2.31
1000	2.67	2.60	10.02	9.87	4.33	4.29	2.77	2.43

```
[nsrc@localhost ~]$ gcc -o 6.6.2 6.6.2.c -O3
```

```
[nsrc@localhost ~]$ 6.6.2
```

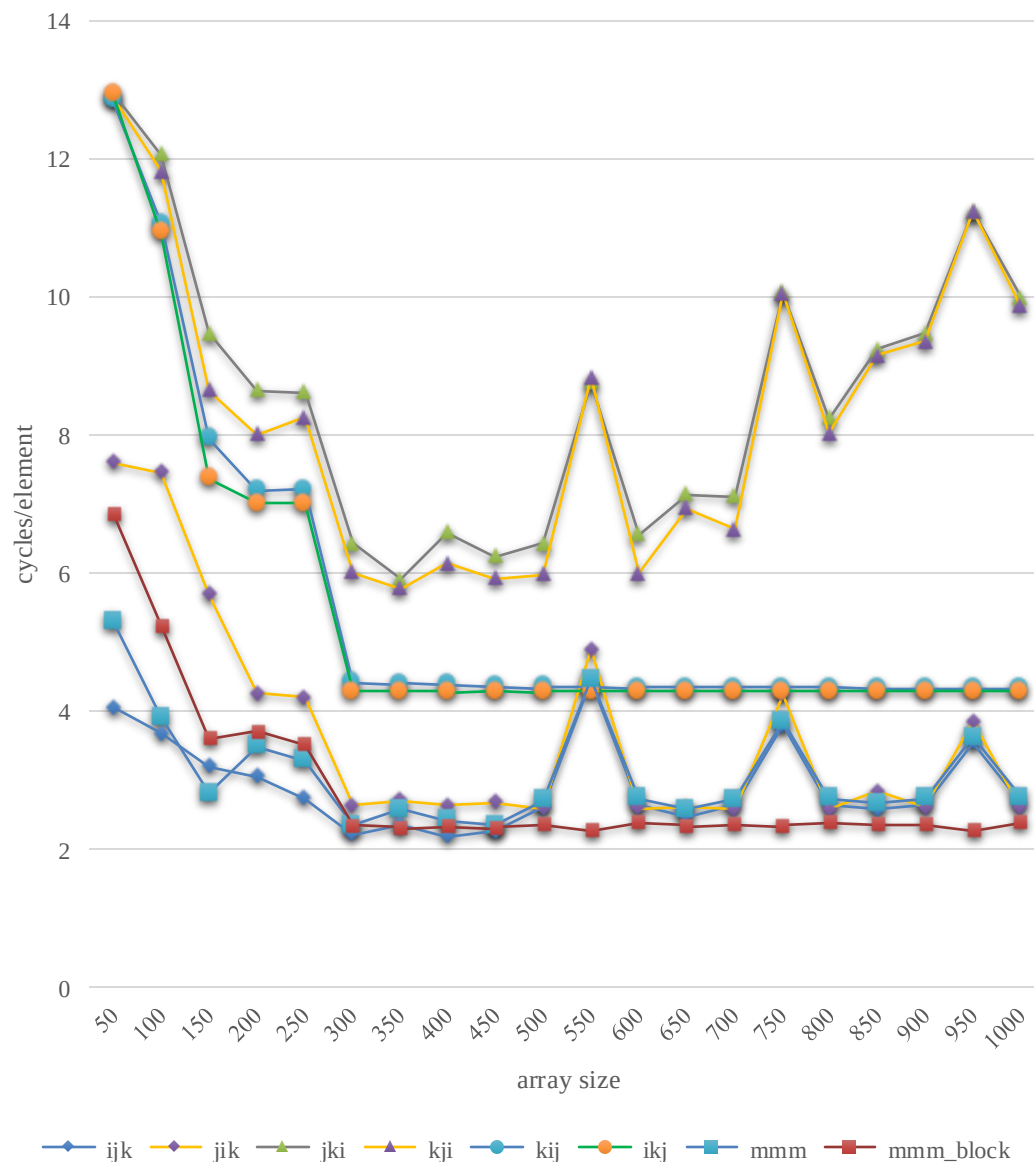
Matrix product (cycles)

M_size	ijk	jik	jki	kji	kij	ikj	mmm	mmm_block
50	3.85	7.60	13.00	12.90	12.91	12.84	5.40	6.66
100	3.69	7.40	11.98	11.78	11.06	10.89	3.91	5.23
150	2.98	5.68	9.67	8.74	7.88	7.36	2.68	3.65
200	3.21	4.22	8.66	8.02	7.17	7.01	3.55	3.75
250	2.86	4.16	8.43	8.00	7.17	7.01	3.32	3.36
300	2.33	2.59	6.26	5.82	4.40	4.29	2.39	2.36
350	2.33	2.63	5.62	5.48	4.37	4.28	2.53	2.36
400	2.25	2.60	6.43	5.95	4.37	4.29	2.44	2.39
450	2.21	2.59	6.03	5.76	4.36	4.29	2.36	2.31
500	2.62	2.59	6.45	5.99	4.36	4.34	2.75	2.37
550	4.47	4.93	7.96	7.90	4.35	4.28	4.50	2.29
600	2.66	2.60	6.60	6.03	4.37	4.35	2.78	2.42
650	2.50	2.64	7.27	7.05	4.39	4.33	2.61	2.38
700	2.67	2.61	7.20	6.67	4.34	4.29	2.74	2.37
750	3.82	4.28	9.99	9.96	4.34	4.29	3.88	2.36
800	2.66	2.58	8.06	7.76	4.34	4.28	2.76	2.40
850	2.59	2.85	9.42	9.35	4.34	4.29	2.67	2.38
900	2.66	2.61	9.39	9.27	4.34	4.28	2.76	2.36
950	3.59	3.87	11.44	11.44	4.33	4.29	3.64	2.28
1000	2.68	2.60	10.19	10.08	4.33	4.29	2.77	2.39



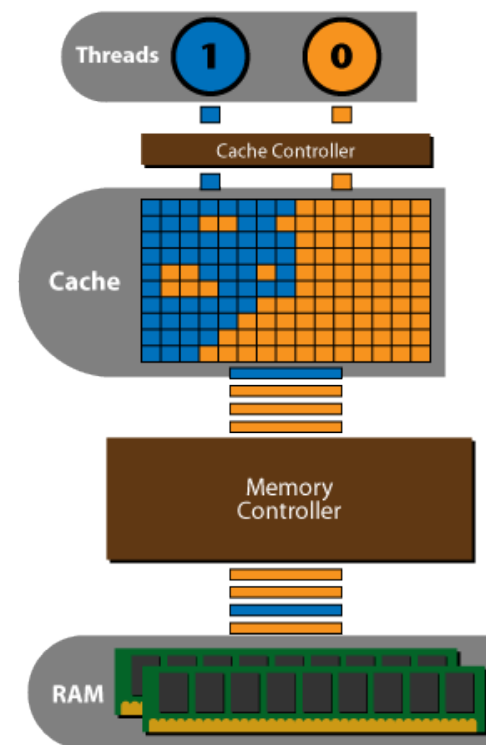
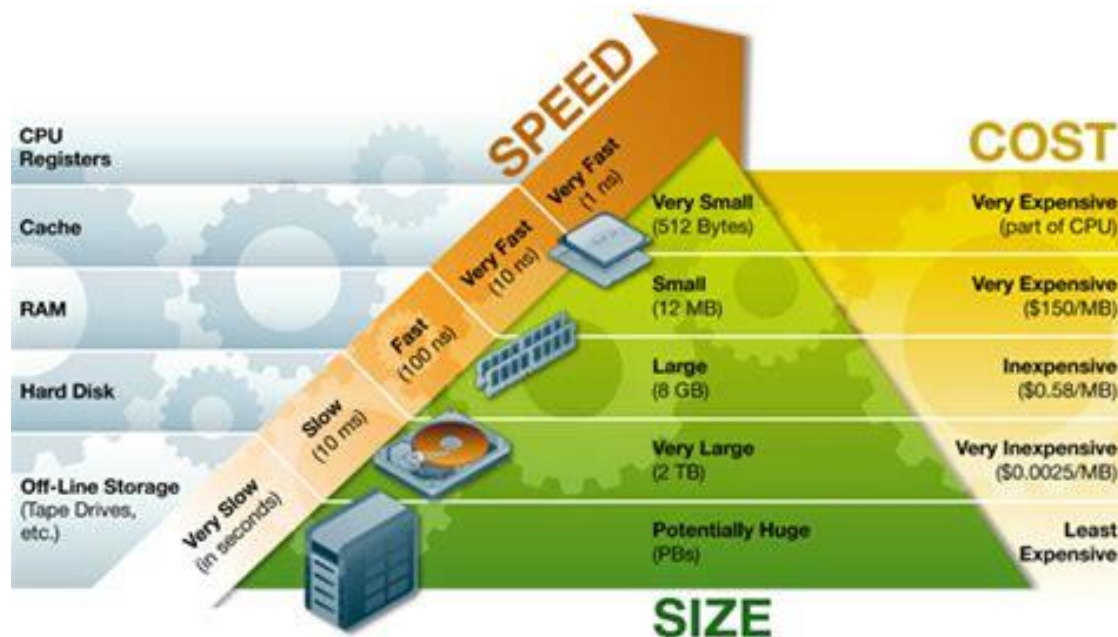
实验测试结果

M_size	ijk	jik	jki	kji	kij	ikj	mmm	mmm_block
50	4.05	7.6	12.95	12.9	12.84	12.92	5.3	6.86
100	3.68	7.46	12.05	11.81	11.06	10.93	3.91	5.24
150	3.2	5.68	9.47	8.64	7.95	7.36	2.8	3.61
200	3.05	4.26	8.64	8.01	7.19	7.01	3.48	3.7
250	2.73	4.2	8.63	8.25	7.2	7.01	3.28	3.51
300	2.22	2.64	6.43	6.01	4.42	4.28	2.34	2.35
350	2.36	2.7	5.91	5.77	4.4	4.28	2.57	2.32
400	2.18	2.64	6.58	6.14	4.39	4.28	2.42	2.33
450	2.25	2.68	6.23	5.92	4.36	4.29	2.35	2.32
500	2.6	2.59	6.42	5.97	4.35	4.28	2.7	2.36
550	4.43	4.89	8.75	8.83	4.35	4.28	4.45	2.27
600	2.63	2.59	6.56	5.99	4.34	4.28	2.74	2.39
650	2.47	2.6	7.13	6.93	4.34	4.28	2.57	2.35
700	2.63	2.58	7.1	6.63	4.34	4.28	2.72	2.36
750	3.79	4.24	10.06	10.04	4.34	4.28	3.85	2.35
800	2.64	2.57	8.24	8.02	4.34	4.28	2.74	2.39
850	2.58	2.84	9.23	9.15	4.33	4.28	2.66	2.36
900	2.63	2.61	9.46	9.35	4.33	4.28	2.73	2.36
950	3.55	3.85	11.23	11.23	4.33	4.28	3.62	2.27
1000	2.66	2.59	9.99	9.86	4.33	4.28	2.75	2.39



6.6.3 在程序中利用局部性

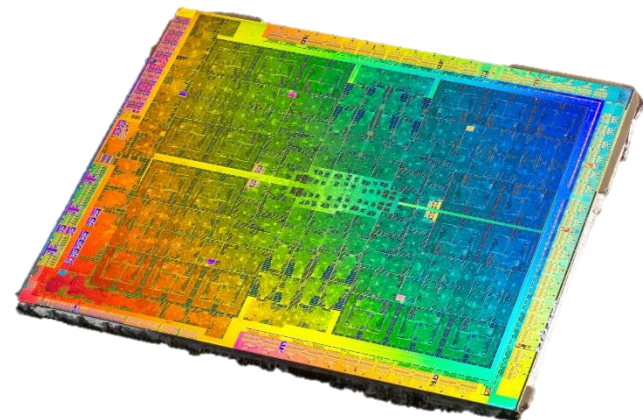
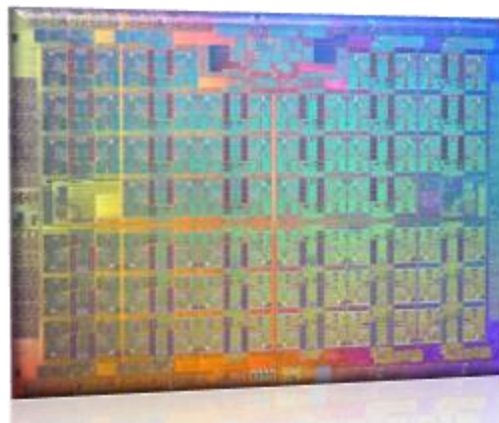
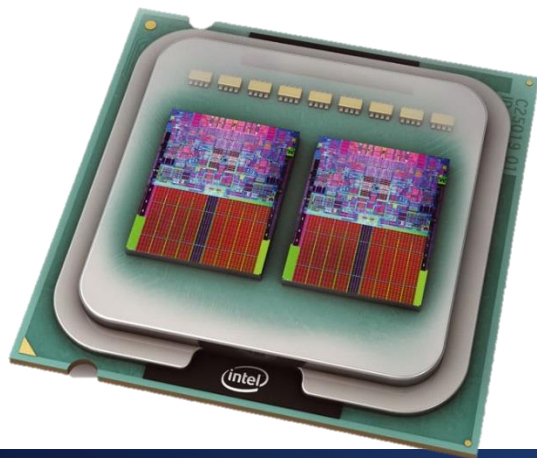
- 存储层次之间延迟性能达到数量级
 - 基本策略：良好局部性程序从cache中访问大部分数据
- 优化策略：
 - 关注内循环优化
 - 内存顺序访问、步长为1提高程序空间局部性
 - 从存储读的数据最大化使用



- P458
- 6.46
 - 目标：
 - 算法优化
 - 实现优化的算法，与naïve算法进行CPE性能对比
 - 通过perf的cache测试工具测试两个算法的关键cache性能指标
 - 说明算法优化的原理

6.7 小结

- DRAM和SRAM取决于芯片物理架构和成本/性能考虑
- CPU与DRAM之间performance gap加大使cache成为重要的优化手段
- 存储器层次结构强调局部性优化技术
 - 良好的时间局部性优化：内循环优化
 - 良好的空间局部性优化：步长为1顺序访问
 - Hot dataset → cache resident
- 不同处理器架构硬件特征及相应优化策略：
 - CPU: cache-centric优化策略
 - Xeon Phi: moderate in-core cache and big on-board cache
 - GPU: SIMT to overlap memory access latency





THANK YOU
谢谢